

Material Docente para el Examen

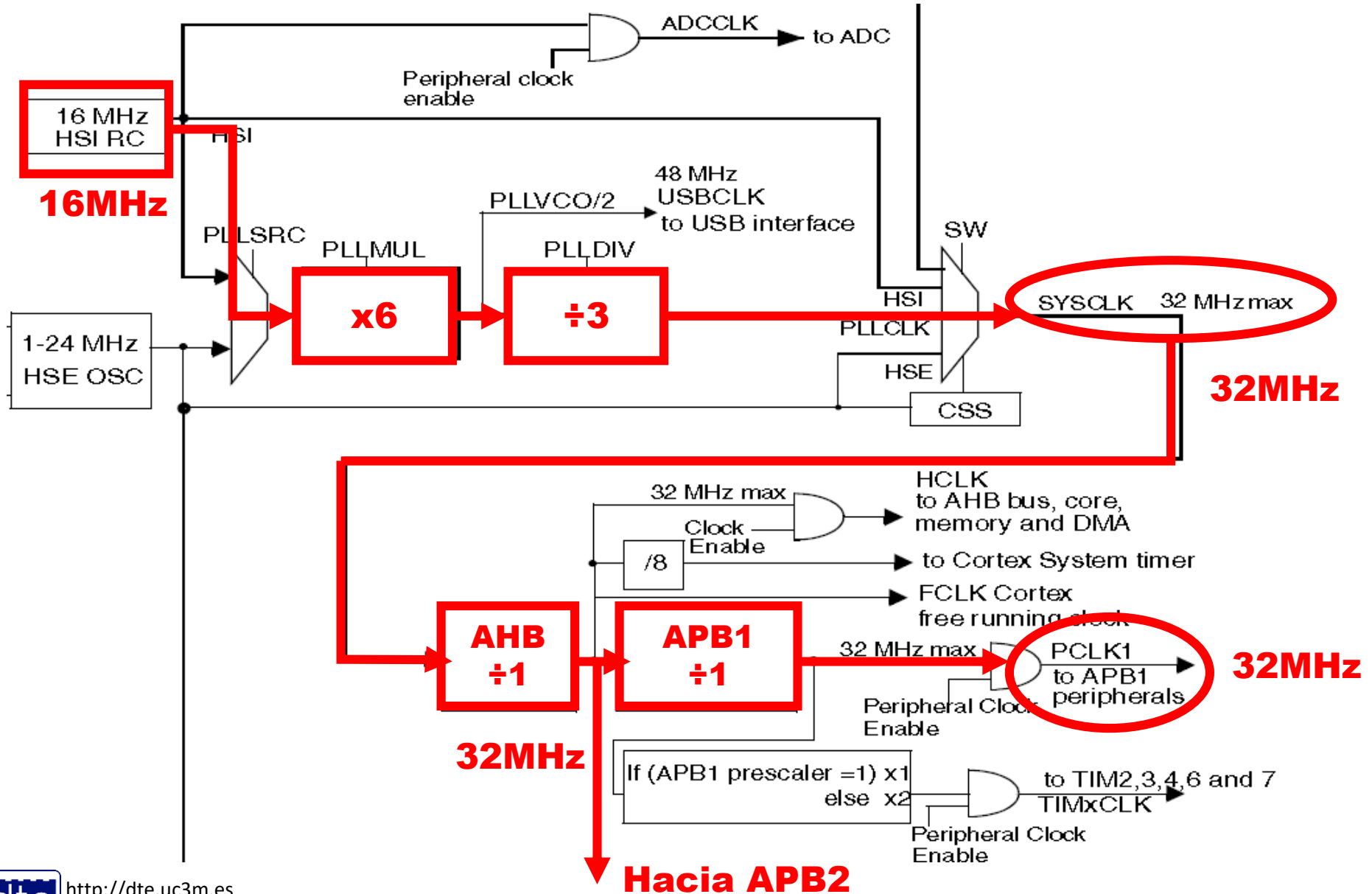
Sistemas Digitales Basados en Microprocesador

Universidad Carlos III de Madrid

Dpto. Tecnología Electrónica

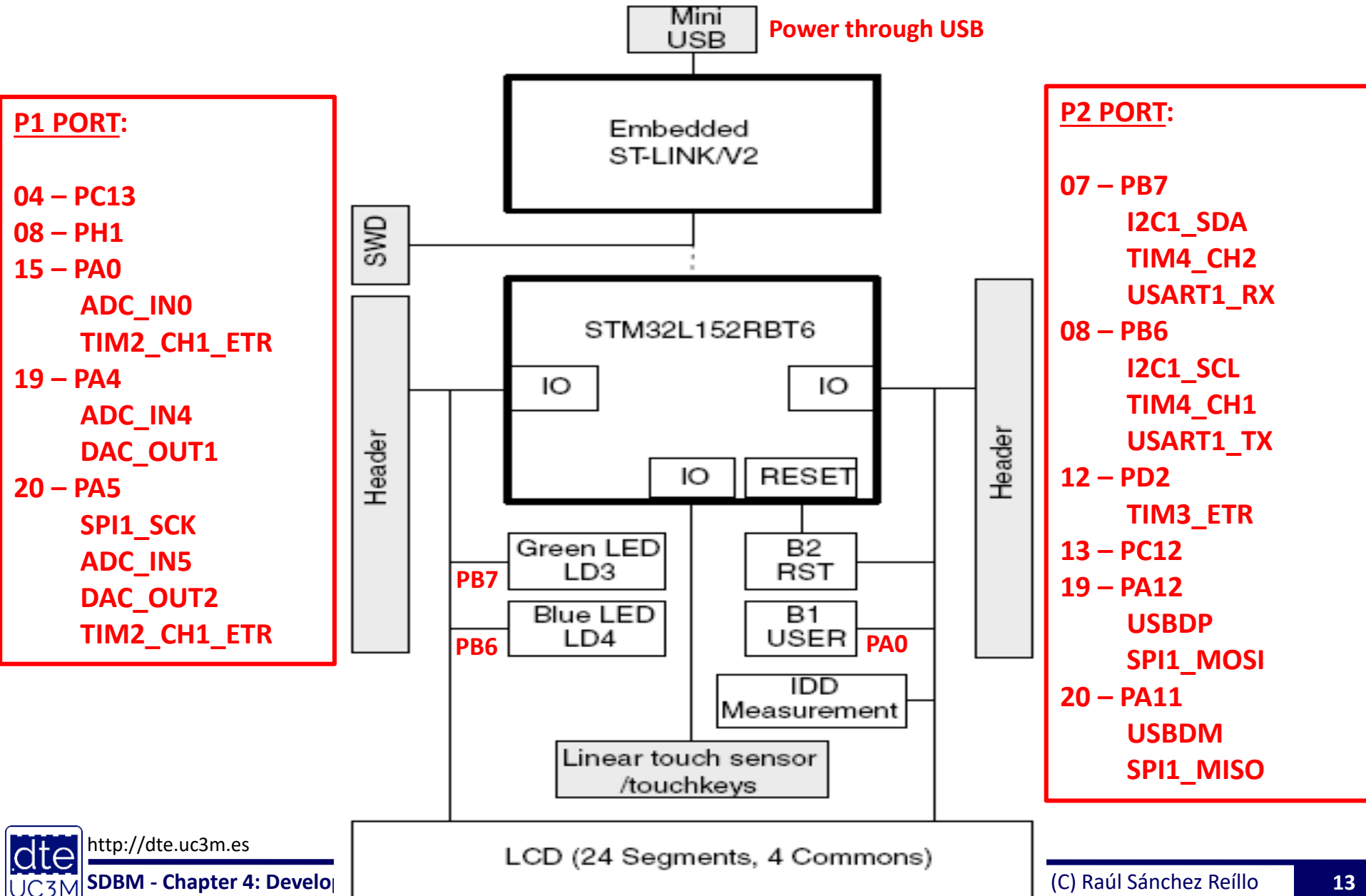
Clock selection for Exercises with STM32L-Discovery

Subsistema de Reloj durante el curso



Free GPIO in STM32L-Discovery

Block Diagram and Layout



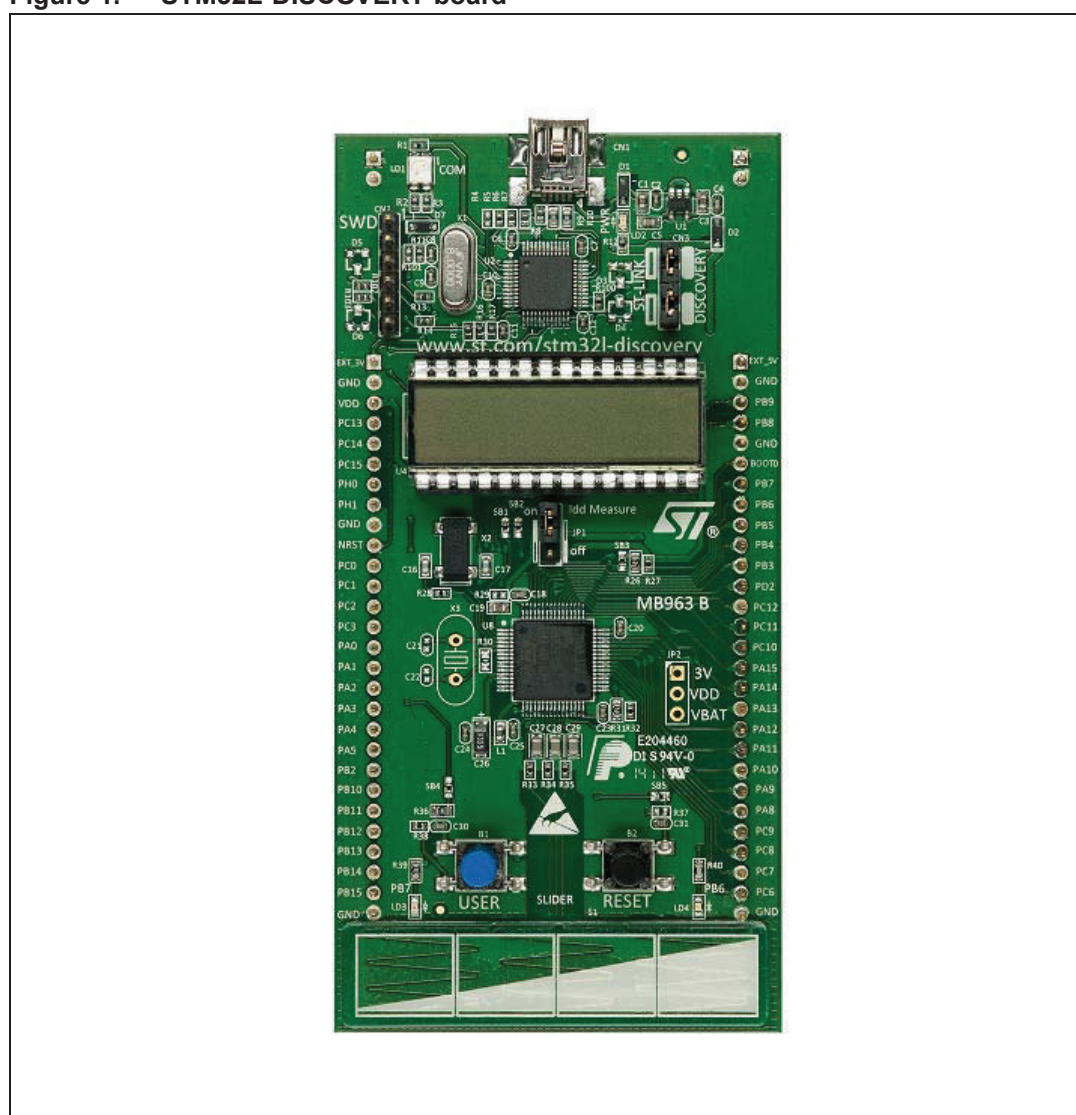
Datasheet STM32L-Discovery (reduced)

STM32L-DISCOVERY

Introduction

The STM32L-DISCOVERY helps you to discover the STM32L ultralow power features and to develop and share your applications. It is based on an STM32L152RBT6 and includes an ST-LINK/V2 embedded debug tool interface, LCD (24 segments, 4 commons), LEDs, pushbuttons, a linear touch sensor, and four touchkeys.

Figure 1. STM32L-DISCOVERY board



Contents

1	Conventions	4
2	Quick start	5
2.1	Getting started	5
2.2	System requirements	6
2.3	Development toolchain supporting the STM32L-DISCOVERY	6
2.4	Demonstration software	6
2.5	Order code	6
3	Features	7
4	Hardware and layout	8
4.1	STM32L152RBT6 microcontroller	11
4.2	Embedded ST-LINK/V2	13
4.2.1	Using the ST-LINK/V2 to program/debug the STM32L on board	14
4.2.2	Using the ST-LINK/V2 to program/debug an external STM32L application	15
4.3	Power supply and power selection	16
4.4	LEDs	16
4.5	Pushbuttons	17
4.6	Linear touch sensor/touchkeys	17
4.7	Built-in IDD measurement circuit	17
4.7.1	High I _{DD} range mode	18
4.7.2	Low I _{DD} range mode	18
4.7.3	I _{BIAS} current measurement procedure	19
4.8	Solder bridges	20
4.9	LCD (24 segments, 4 commons)	22
5	Extension connectors	24
6	Mechanical drawing	27
7	Electrical schematics	28

8	Revision history	34
---	------------------------	----

1 Conventions

Table 1 provides the definition of some conventions used in the present document.

Table 1. ON/OFF conventions

Convention	Definition
Jumper JP1 ON	Jumper placed between pin 2 and 3
Jumper JP1 OFF	Jumper placed between pin 1 and 2
Solder bridge SBx ON	SBx connections closed by solder
Solder bridge SBx OFF	SBx connections left open

2 Quick start

The STM32L-DISCOVERY is a low-cost and easy-to-use development kit to quickly evaluate and start a development with an STM32L ultralow power microcontroller.

Before installing and using the product, please accept the Evaluation Product License Agreement from www.st.com/stm32l-discovery.

For more information on the STM32L-DISCOVERY and for demonstration software visit www.st.com/stm32l-discovery.

2.1 Getting started

Follow the sequence below to configure the STM32L-DISCOVERY board and launch the DISCOVERY application:

1. Check jumper positions on the board: JP1 and CN3 must be ON (Discovery selected) (see [Figure 3 on page 9](#)).
2. Connect the STM32L-DISCOVERY board to a PC with a USB cable to power the board. Red LED LD2 (PWR) and LD1 (COM) are then lit up.
3. Function 1 is executed. Each click on user button B1 changes the executed function as described in [Table 2 on page 5](#).

A 4-LED bar shows the function being performed (1 to 4 bars can be switched ON).

Depending on the function selected, the voltage value, the linear touch sensor position, the touchkey status, or the STM32L current consumption is displayed on the LCD.

Table 2. Functions executed when clicking B1 button

Function	LED LD3/4	Bar status	Value displayed on LCD	Main function
1	LD3 and LD4 blink		Measured STM32L V _{DD} voltage	Voltage measurement
2	LD3 ON		Linear touch sensor position from 0 to 100%	Touch sensing
3	LD4 ON		Status of the 4 touchkeys	
4	LD3 and LD4 OFF		STM32L consumption measured in Run mode (4 MHz)	STM32L current consumption measurement
			STM32L consumption measured in Sleep mode (4 MHz)	
5			STM32L consumption measured in Run mode (32 KHz)	
			STM32L consumption measured in low power sleep mode (32 KHz)	
6			STM32L consumption measured in Stop mode, RTC ON	
			STM32L consumption measured in Stop mode, RTC OFF	
7			STM32L consumption measured in Standby mode	

To study or modify the DISCOVERY project related to this demonstration, visit www.st.com/stm32l-discovery and follow the tutorial. Discover the STM32L features, download and execute programs proposed in the list of projects. This site also contains examples from which you can develop your own applications.

2.2 System requirements

- Windows PC (2000, XP, Vista, 7)
- USB type A to Mini-B USB cable

2.3 Development toolchain supporting the STM32L-DISCOVERY

- Altium TASKING™ VX-Toolset
- Atollic TrueSTUDIO[?]
- IAR EWARM
- Keil™ MDK-ARM

2.4 Demonstration software

The demonstration software is preloaded in the board Flash memory. It uses the built-in I_{DD} measurement feature of the STM32L-DISCOVERY to automatically measure and display on the LCD the MCU consumption in Run and low power modes. It also allows to demonstrate touch sensing functionalities such as linear touch sensor or touchkeys.

The latest versions of this demonstration source code and associated documentation can be downloaded from www.st.com/stm32l-discovery.

2.5 Order code

To order the STM32L ultralow power discovery board, use the order code STM32L-DISCOVERY.

3 Features

The STM32L-DISCOVERY offers the following features:

- STM32L152RBT6 microcontroller, featuring 128 KB of Flash memory, 16 KB of RAM, 4 KB of data EPROM, in a 64-pin LQFP package
- On-board ST-LINK/V2 with selection mode switch to use the kit as a standalone ST-LINK/V2 (with SWD connector for programming and debugging)
- Board power supply: through USB bus or from an external 3.3 or 5 V supply voltage
- External application power supply: 3 V and 5 V
- I_{DD} current measurement
- LCD
 - DIP28 package
 - 24 segments, 4 commons
- Four LEDs:
 - LD1 (red/green) indicating USB communication
 - LD2 (red) indicating that 3.3 V power supply is ON
 - Two user LEDs, LD3 (green) and LD4 (blue)
- Two pushbuttons (user and reset)
- One linear touch sensor and four touchkeys
- Extension header for LQFP64 I/Os for quick connection to prototyping board and easy probing

4 Hardware and layout

The STM32L-DISCOVERY is designed around the STM32L152RBT6 microcontroller in a 64-pin LQFP package.

[Figure 2](#) illustrates the connections between the STM32L152RBT6 and its peripherals (ST-LINK/V2, pushbutton, LED, LCD, linear touch sensor, touchkeys, and connectors).

[Figure 3 on page 9](#) and [Figure 4 on page 10](#) help you to locate these features on the STM32L-DISCOVERY.

Figure 2. Hardware block diagram

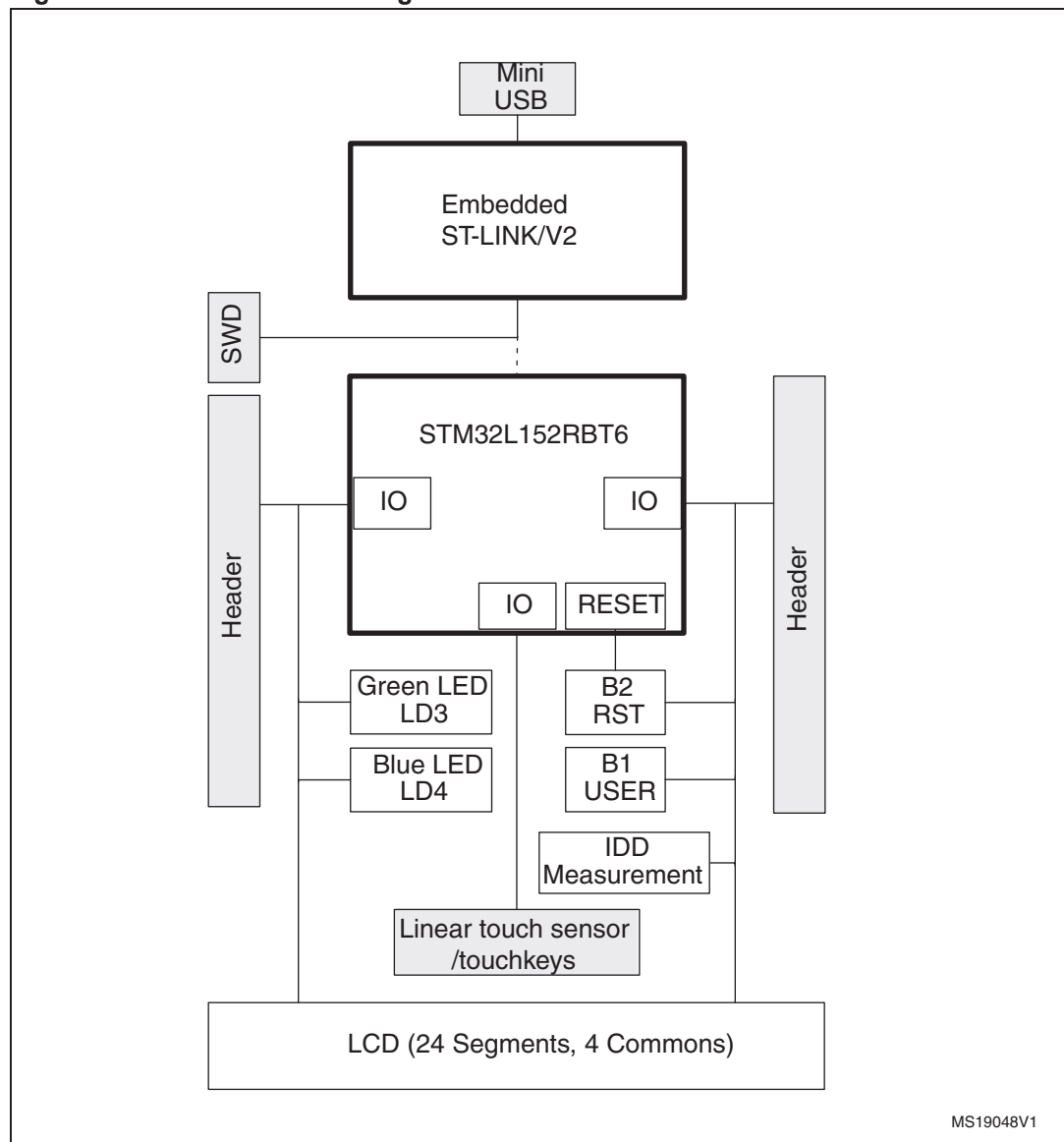
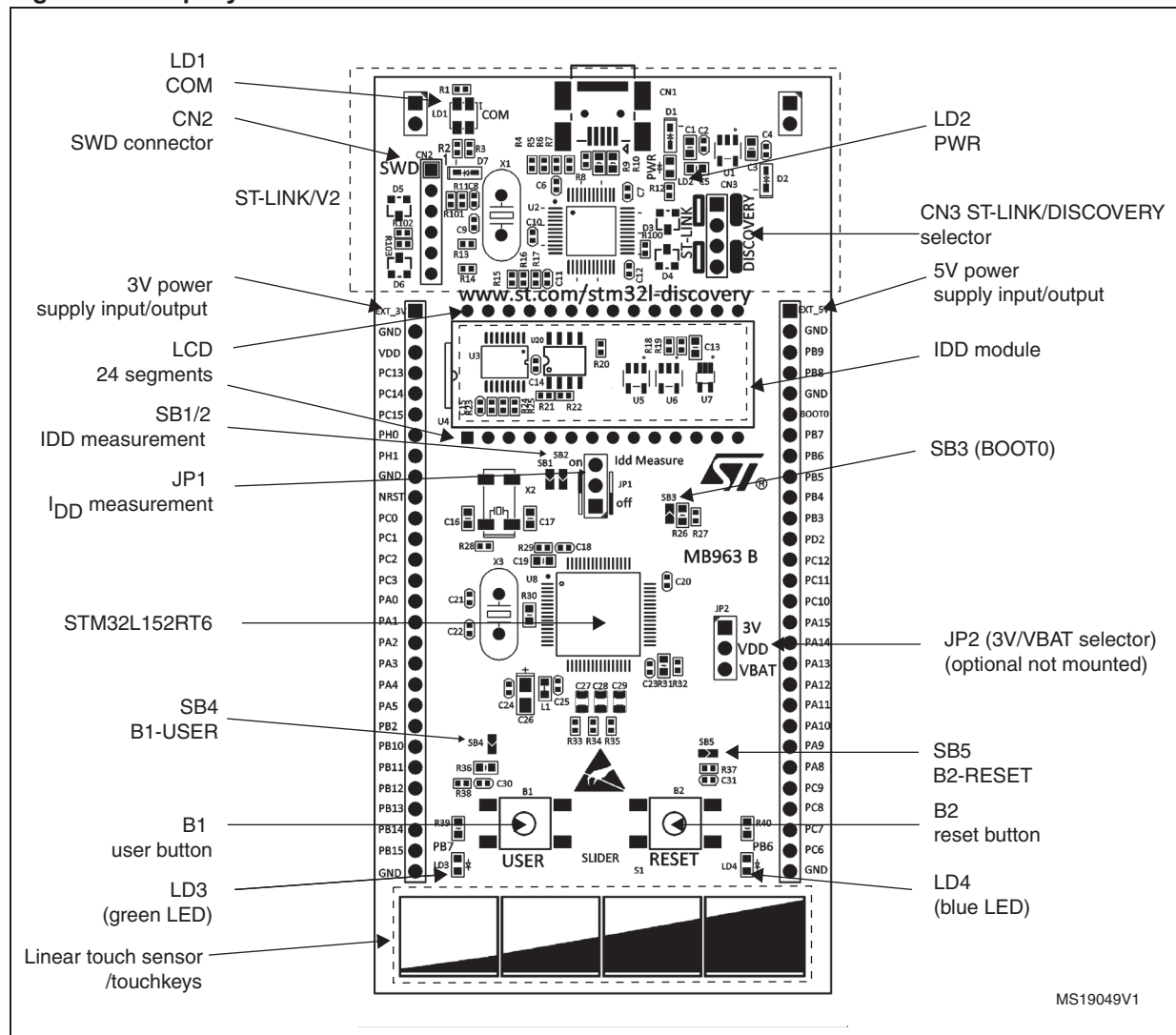
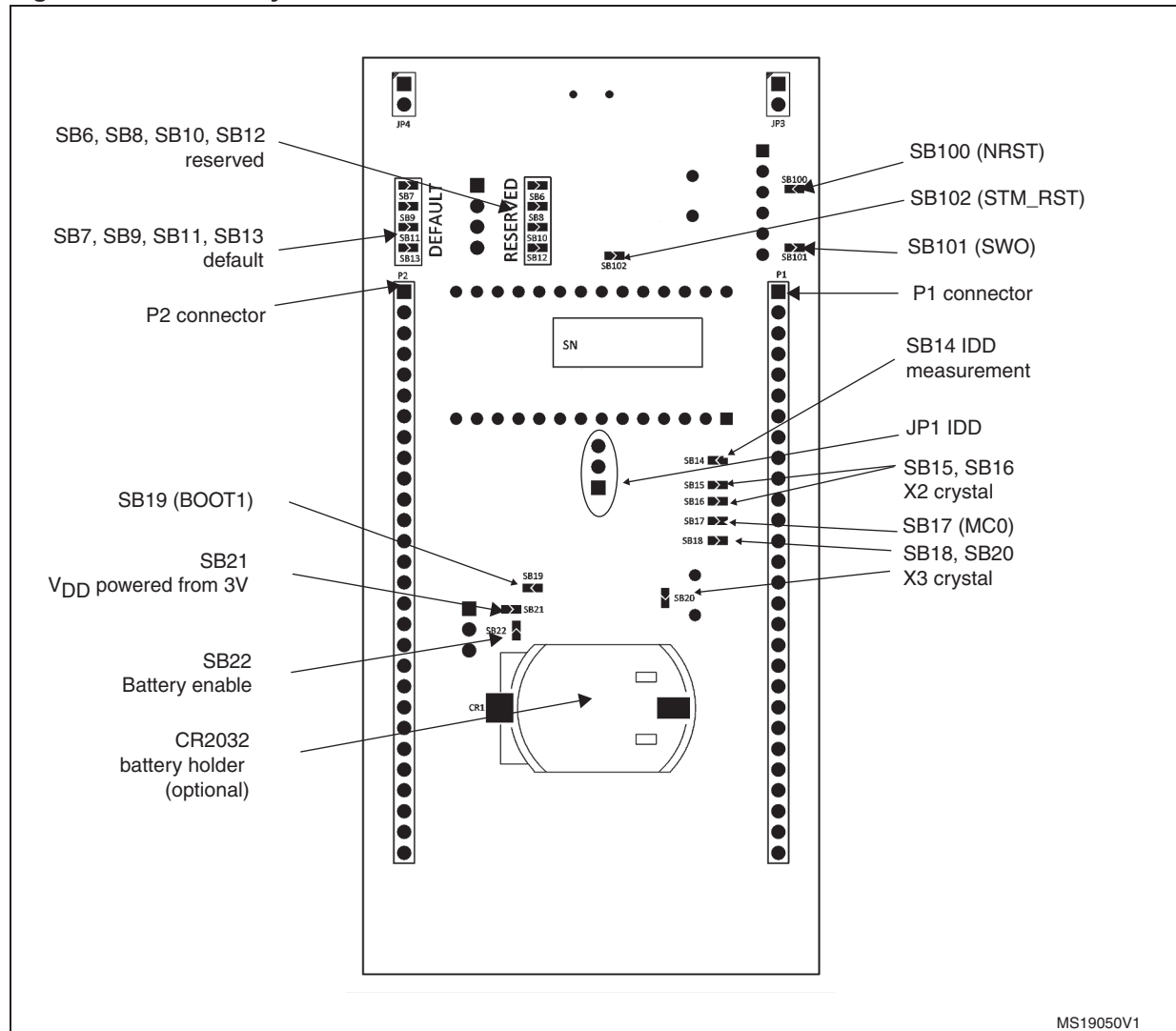


Figure 3. Top layout



1. Pin 1 of CN1, CN2, P1 and P2 connectors are identified by a square.

Figure 4. Bottom layout

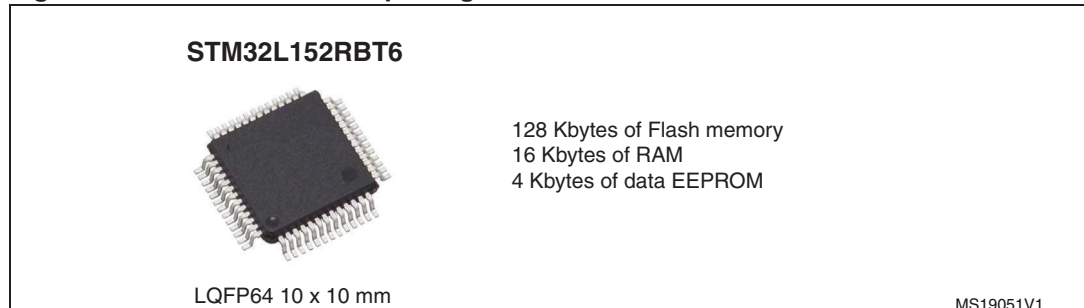


1. Pin 1 of CN1, CN2, P1 and P2 connectors are identified by a square.

4.1 STM32L152RBT6 microcontroller

This 32-bit ultralow power MCU has 128 KB Flash, 16 KB RAM, 4 KB data EEPROM, RTC, LCD, timers, USART, I2C, SPI, ADC, DAC and comparators.

Figure 5. STM32L152RBT6 package

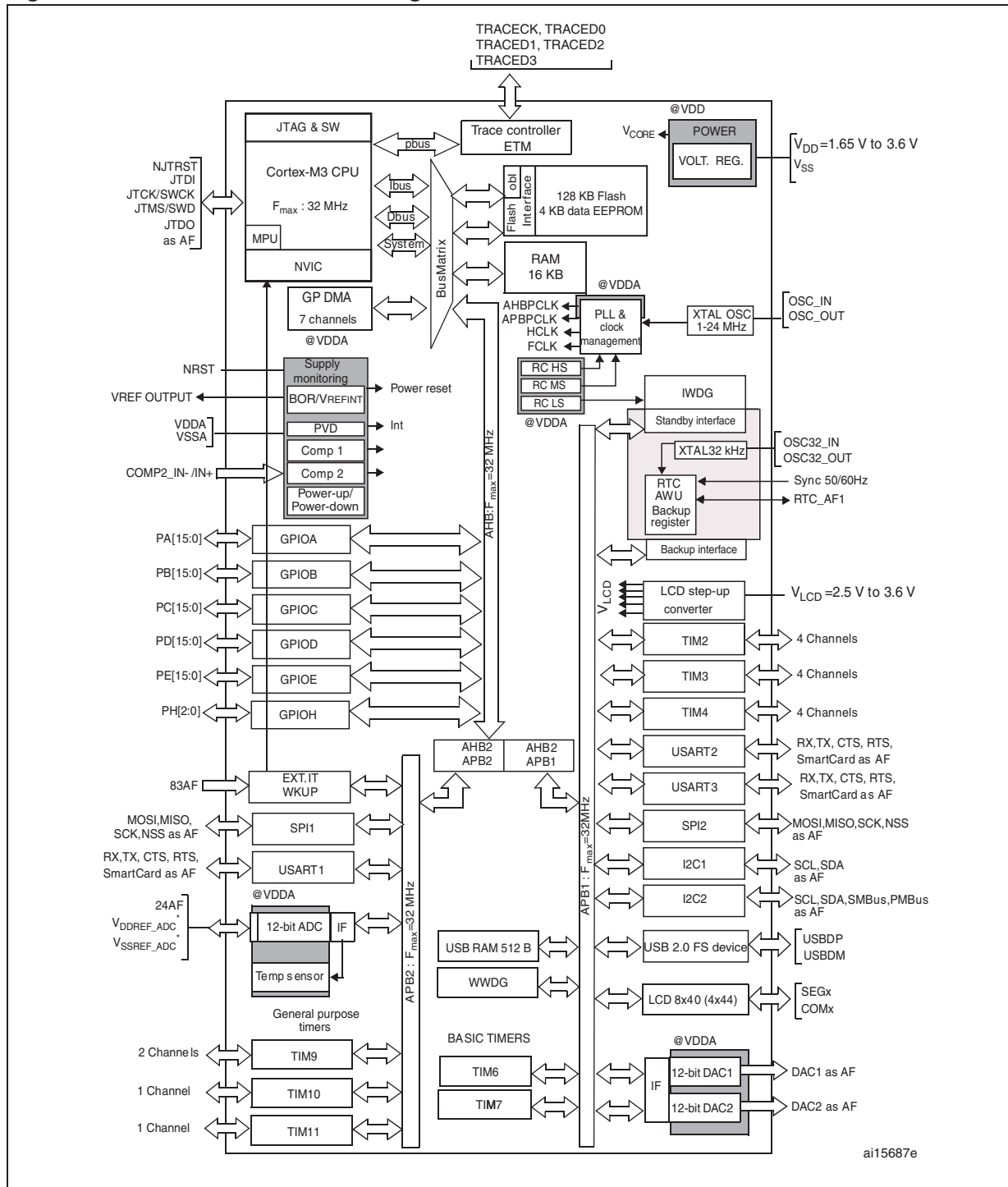


This device provides the following benefits:

- Ultralow power proprietary 130 nm technology:
Speed and power consumption independent of MCU power supply, and ultralow leakage
- Ultralow power design (clock gating, low-power Flash with power-off capability):
Reduced overall Run and Wait mode current consumption by turning off clocks of unused peripherals or Flash
- Sub 1 μ A hardware RTC and AWU system unit:
Ultralow power modes for applications requesting regular wake up
- Up to 6 low power modes:
Suitable for many applications from complete switch off to continuous monitoring at ultralow frequency
- Advanced and flexible clock system (multiple internal and external clock sources)
Switch and adjust frequency and clock sources on the fly depending on application needs
- Direct memory access on board (up to 12 DMA channels):
Autonomy for peripherals, independent from core; can switch off Flash memory and CPU (large current consumption contributors) while keeping peripherals active
- Ultralow power and ultrasafe features (POR, PDR, BOR, PVD) allowing integrated application safety and security
- Unique identifier to enhance user data confidentiality/reliability
- Ultrafast wakeup from lowest consumption low-power mode allowing fast switching from static and dynamic power modes
- Analog functional down to 1.8 V, and programming down to 1.65 V
- Full functionality over the complete V_{DD} range

For more information, refer to the STM32L152RBT6 datasheet on the ST website.

Figure 6. STM32L152RBT6 block diagram



4.2 Embedded ST-LINK/V2

The ST-LINK/V2 programming and debugging tool is integrated on the STM32L-DISCOVERY. The embedded ST-LINK/V2 can be used in 2 different ways according to the jumper states (see [Table 3 on page 13](#)):

- Program/debug the MCU on board,
- Program/debug an MCU in an external application board using a cable connected to SWD connector CN2.

The embedded ST-LINK/V2 supports only SWD for STM32 devices. For information about debugging and programming features refer to user manual UM1075 which describes in detail all the ST-LINK/V2 features.

Figure 7. Typical configuration

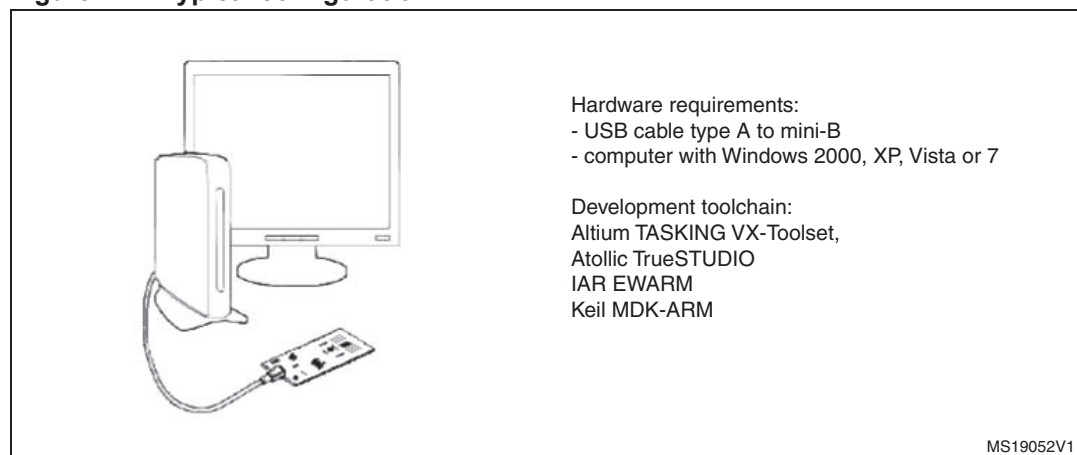


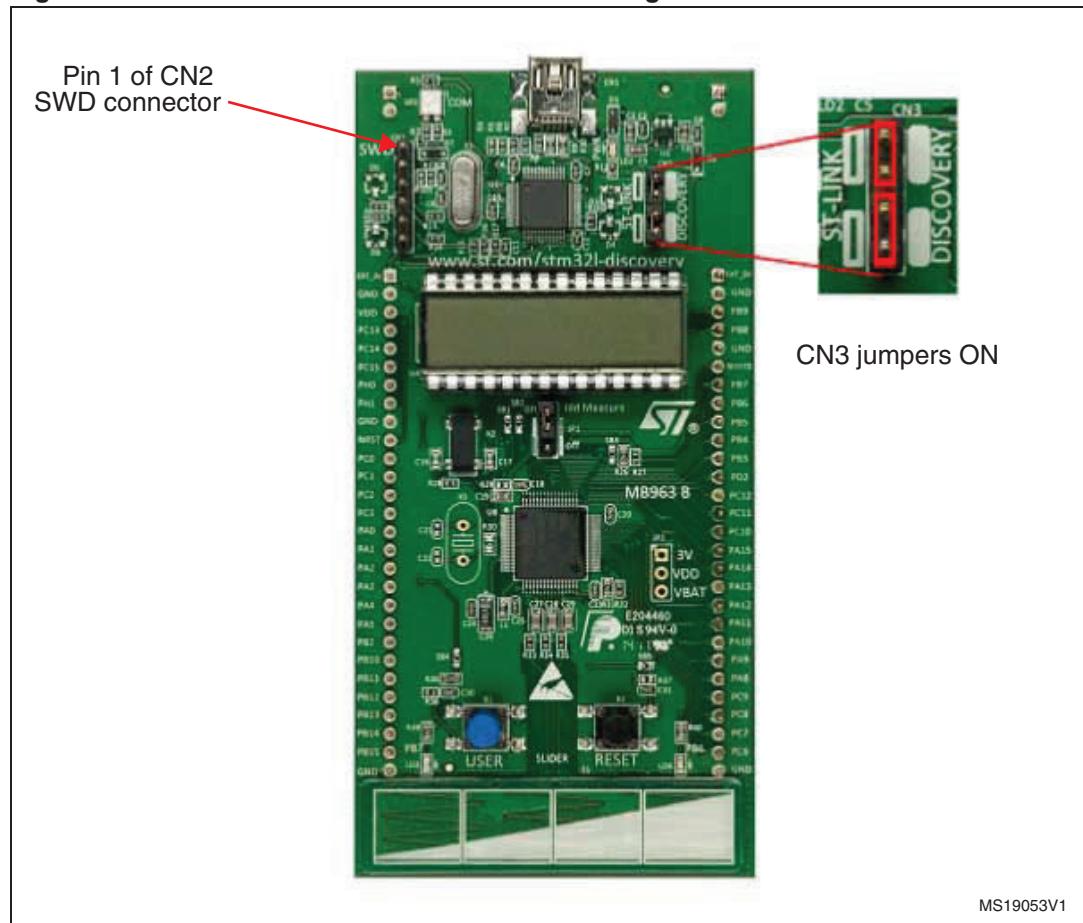
Table 3. Jumper states

Jumper state	Description
Both CN3 jumpers ON	ST-LINK/V2 functions enabled for on board programming (default)
Both CN3 jumpers OFF	ST-LINK/V2 functions enabled for external application through CN2 connector (SWD supported).

4.2.1 Using the ST-LINK/V2 to program/debug the STM32L on board

To program the STM32L on board, simply plug in the two jumpers on CN3, as shown in [Figure 8](#) in red, but do not use the CN2 connector as that could disturb communication with the STM32L152RBT6 of the STM32L-DISCOVERY.

Figure 8. STM32L-DISCOVERY connections image



4.2.2 Using the ST-LINK/V2 to program/debug an external STM32L application

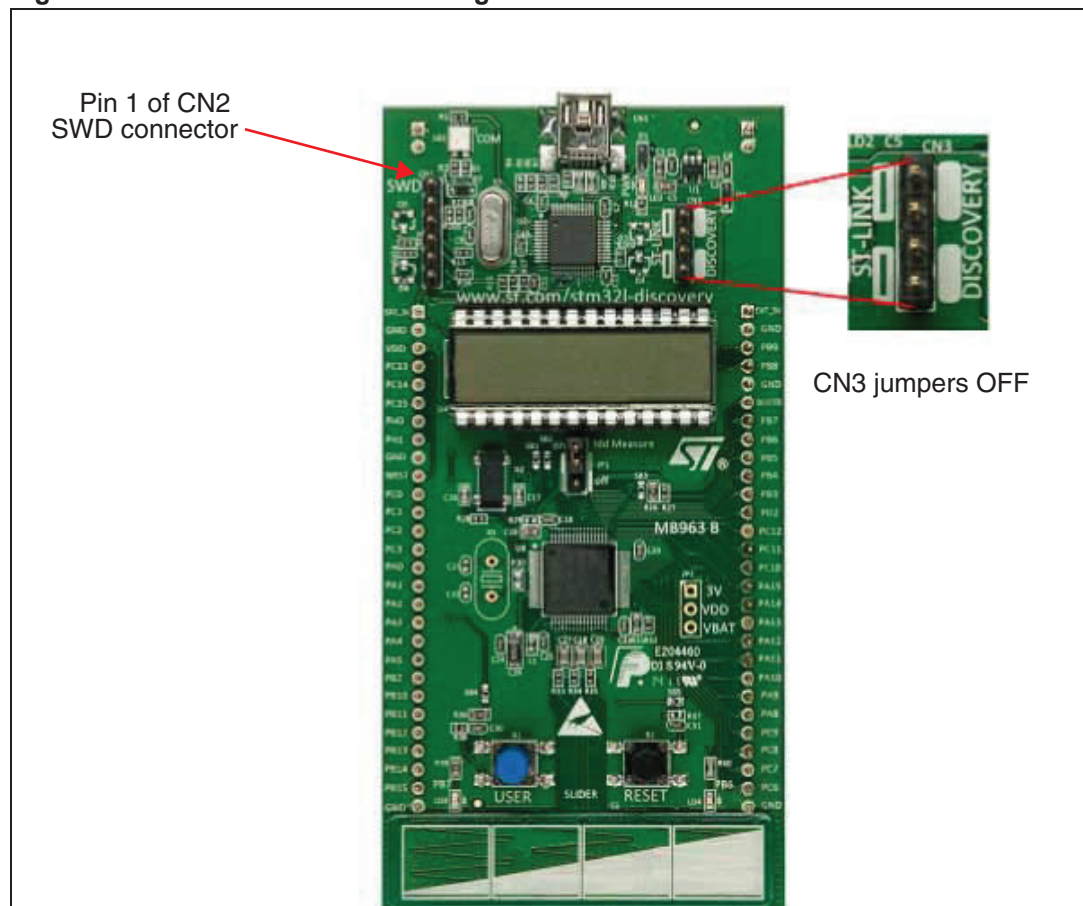
It is very easy to use the ST-LINK/V2 to program the STM32L on an external application. Simply remove the 2 jumpers from CN3 as shown in [Figure 9](#), and connect your application to the CN2 debug connector according to [Table 4](#).

Note: SB100 must be OFF if you use CN2 pin 5 in your external application.

Table 4. Debug connector CN2 (SWD)

Pin	CN2	Designation
1	VDD_TARGET	VDD from application
2	SWCLK	SWD clock
3	GND	Ground
4	SWDIO	SWD data input/output
5	NRST	RESET of target MCU
6	SWO	Reserved

Figure 9. ST-Link connections image



4.3 Power supply and power selection

The power supply is provided either by the host PC through the USB cable, or by an external 5 V or 3.3 V power supply.

The D1 and D2 protection diodes allow the EXT_5V and EXT_3V pins to be used independently as input or output power supplies (see [Figure 3 on page 9](#)):

- EXT_5V and EXT_3V can be used as output power supplies when the application board is connected to pins P1 and P2. In this case, the EXT_5V and EXT_3V pins deliver a 5 V or 3 V power supply and power consumption must be lower than 100 mA.
- EXT_5V and EXT_3V can also be used as input power supplies e.g. when the USB connector is not connected to the PC. In this case, the STM32L-DISCOVERY board must be powered by a power supply unit or by auxiliary equipment complying with standard EN-60950-1: 2006+A11/2009, and must be Safety Extra Low Voltage (SELV) with limited power capability.

Battery powered (optional)

In addition, the STM32L-DISCOVERY board has been designed to run from a CR2032 standalone battery (no connection with USB or other power supply is required).

By default, no battery holder is mounted on the board and SB21 and SB22 are configured in their default state (see [Table 5: Solder bridges on page 20](#)).

Follow the procedure below to power the STM32L-DISCOVERY from the battery:

1. Solder a B7410AP2L battery holder from LOTES on CR1.
2. Configure SB100 OFF.
3. Remove both jumpers from CN3 (see [Figure 9](#))
4. Select the battery as power supply. Two solutions are possible:
 - a) Solder bridge: Configure SB21 OFF, and SB22 ON. No header is required on JP2.
 - b) Jumper: Configure SB21 and SB22 OFF.
Solder a header on JP2, identical to JP1 on the top side, and set a jumper between VDD and VBAT to power the STM32L MCU.

Note: In this configuration, it is possible to power the STM32L from the board 3 V supply voltage by setting a jumper between VDD and 3V.

5. Plug the CR2032 battery into CR1 holder. You can now run the demonstration.

Warning: Wrong solder bridge configuration can damage board components.

4.4 LEDs

- LD1 COM: LD1 default status is red. LD1 turns to green to indicate that communications are in progress between the PC and the ST-LINK/V2.
- LD2 PWR: red LED indicates that the board is powered.
- User LD3: green LED is a user LED connected to the I/O PB7 of the STM32L152RBT6.
- User LD4: blue LED is a user LED connected to the I/O PB6 of the STM32L152RBT6.

4.5 Pushbuttons

- B1 USER: User pushbutton connected to the I/O PA0 of the STM32L152RBT6.
- B2 RESET: Pushbutton is used to RESET the STM32L152RBT6.

4.6 Linear touch sensor/touchkeys

To demonstrate touch sensing capabilities, the STM32L-DISCOVERY includes a linear touch sensor which can be used either as a 3-position linear touch sensor or as 4 touchkeys. Both functionalities are illustrated in the demonstration software (see [Table 2: Functions executed when clicking B1 button on page 5](#)).

3 pairs of I/O ports are assigned to the linear touch sensor/touchkeys. Each pair must belong to the same analog switch group:

- PA6, PA7 (group 2)
- PC4, PC5 (group 9)
- PB0, PB1 (group 3)

To minimize the noise, these pairs are dedicated to the linear touch sensor and the touchkeys and are not connected to external headers.

To design a touch sensing application, refer to the following documentation and firmware:

- For details concerning I/O ports, refer to the STM32L152RBT6 datasheet.
- For information on software development, see DISCOVERY application software on <http://www.st.com/stm32l-discovery>.
- For more detail concerning touch sensing application design and layout, refer to AN2869 -Guidelines for designing touch sensing applications.
- STM32 touch sensing library available from <http://www.st.com/stm32l-discovery>.

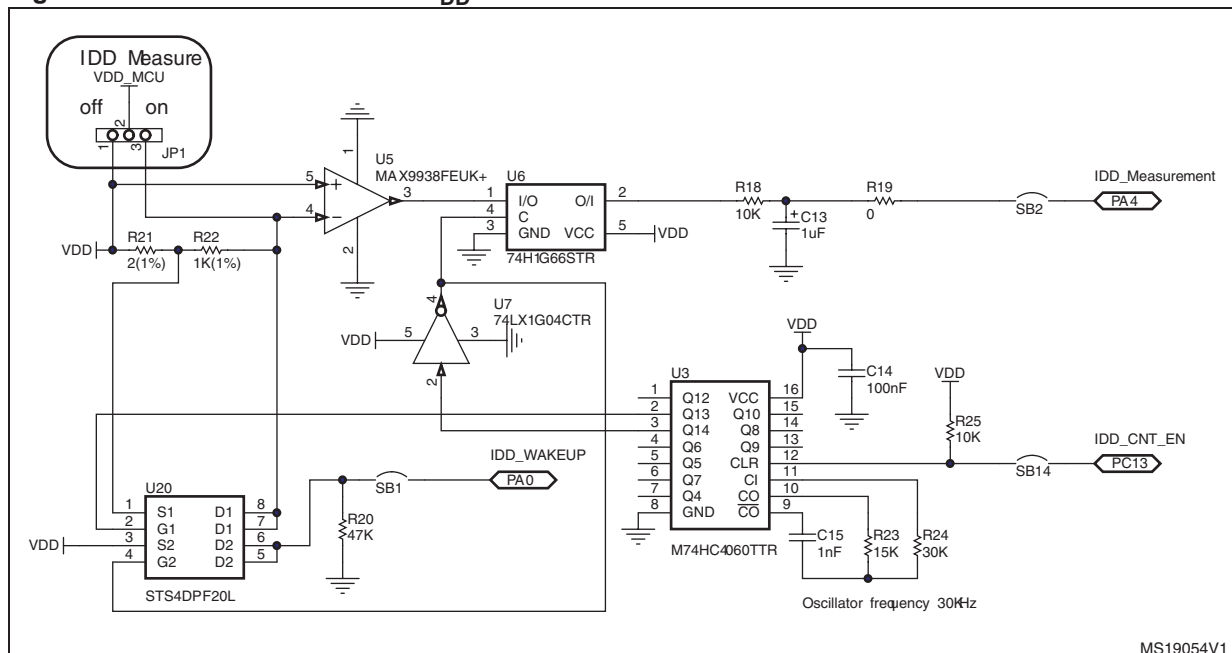
4.7 Built-in IDD measurement circuit

The STM32L-DISCOVERY built-in I_{DD} measurement circuit allows the consumption of the STM32L152RBT6 to be measured and displayed on the LCD Glass while the MCU is in Run or low power modes.

- JP1 ON: STM32L152RBT6 is powered through the I_{DD} measurement circuit (default).
- JP1 OFF: STM32L152RBT6 is directly powered, I_{DD} measurement circuit is bypassed.

Note: When jumper JP1 is removed the current consumption of the STM32L152RBT6 can be measured by connecting an ammeter between jumper pin 1 and pin 2 of JP1.

For I_{DD} measurement to be performed by the MCU itself, the circuit below is implemented on the STM32L-DISCOVERY. Solder bridges SB1, SB2 and SB14 must be closed and JP1 must be ON. The **low I_{DD} range** procedure (see [Section 4.7.2](#)) is recommended when the MCU is in low power mode and the I_{DD} current does not exceed 60 μ A. When the MCU operates in Run mode and can sink up to 30 mA, use the **high I_{DD} range** procedure (see [Section 4.7.1](#)).

Figure 10. STM32L-DISCOVERY I_{DD} measurement circuit

4.7.1 High I_{DD} range mode

In high I_{DD} range mode, the I_{DD} current is measured using the operational amplifier MAX9938FEUK+ (U5) connected to the 2 Ω shunt resistor (R21). In this case IDD_CNT_EN remains high during measurement, so R22 remains in short-circuit during the measurement because FET transistor 1 of U20 remains ON permanently.

4.7.2 Low I_{DD} range mode

In low I_{DD} range mode, the operational amplifier MAX9938FEUK+ (U5) is connected to the 1 K Ω shunt resistor (R22), controlled by FET transistor 1 of U20. In this case the counter 74HC4060 (U3) enabled by IDD_CNT_EN manages the measurement timing according to [Figure 11 on page 19](#).

Low I_{DD} range measurement principle

The principle used to measure the consumption current when the STM32L152RBT6 is in low I_{DD} range mode is as follows:

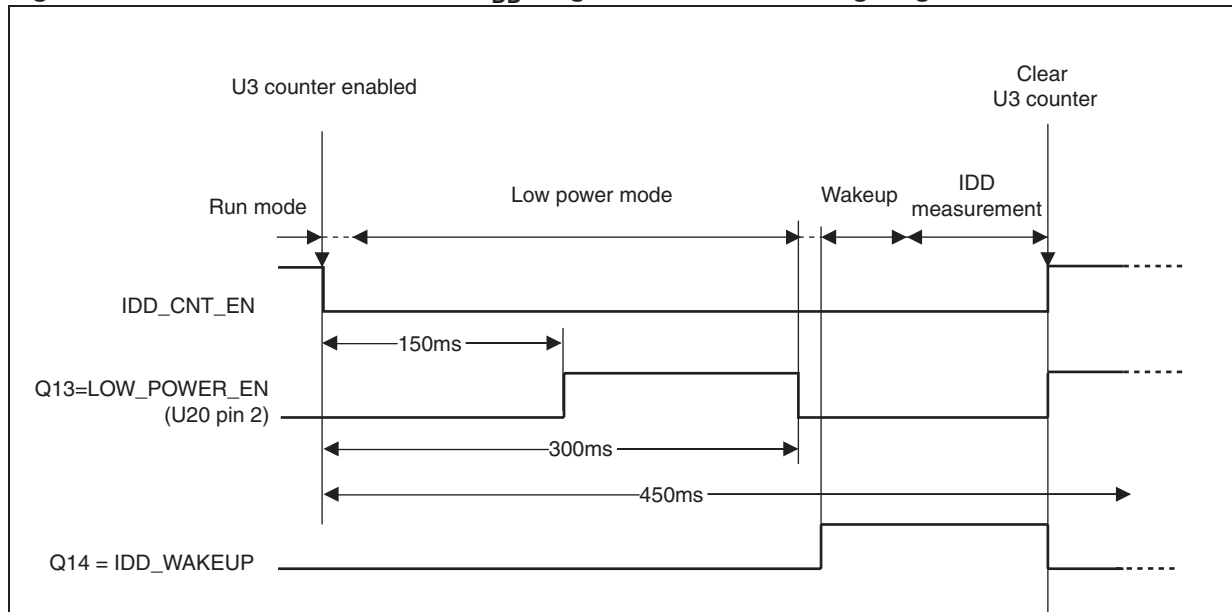
1. Configure ADC to measure voltage on the $IDD_Measurement$ pin.
2. Configure PA0 to serve as wakeup pin.
3. Enter low I_{DD} range mode after setting IDD_CNT_EN (PC13) signal low.
4. IDD_WAKEUP rising edge wakes up the MCU after around 300 ms.
5. Start ADC conversion as soon as possible after wakeup in order to measure the voltage corresponding to Low power mode on capacitor C13.
6. Reset the counter by programming IDD_CNT_EN high (in less than 150 ms after the wakeup) to avoid the R22 1 K Ω resistor being connected later in Run mode.

The measurement timing is given in [Figure 11](#). In low I_{DD} range mode, the 1 K Ω resistor is connected when FET transistor 1 of U20 goes OFF after entering low I_{DD} range mode. The

Q13 output of the counter allows connecting the 1 K Ω resistor when the current I_{DD} becomes very low.

Figure 11 shows how the counter and FET transistor 1 of U20 ensure that, 150 ms after IDD_CNT_EN falling edge, the shunt resistor R22 is connected between VDD_MCU and the power supply to reduce the measurement range to 60 μ A for the full scale. Then after another 150 ms required for current stabilization, R22 is shorted, the I_{DD} measurement is stored in C13, and the MCU is woken up. After wakeup the MCU can measure the I_{DD} current corresponding to the low power mode stored in C13.

Figure 11. STM32L-DISCOVERY low I_{DD} range measurement timing diagram



4.7.3 I_{BIAS} current measurement procedure

In Low I_{DD} range mode, the bias current of the operational amplifier input (U5 pin 4) is not negligible compared to I_{DD} current (typical I_{BIAS} is ~240 nA). To obtain a reliable STM32L152RBT6 I_{DD} measurement, it is mandatory to subtract the bias current from the low I_{DD} current value since this current is not sunk by the MCU. I_{BIAS} is measured during production test and stored in the MCU data EEPROM. The DISCOVER demonstration software, uses this value to display the correct I_{DD} .

The procedure for I_{BIAS} measurement implemented in the demonstration software is:

1. Power off the board (disconnect the USB cable).
2. Set JP1 OFF.
3. Push down B1 (USER button), power on the board from the USB.
4. Wait at least 1 second before releasing B1, the LCD displays the I_{BIAS} measurement.
5. Power off the board (disconnect the USB cable).
6. Set JP1 ON. The I_{BIAS} value is stored in data EEPROM. The bias current is then subtracted from the I_{DD} measured in I_{DD} range mode.

4.8 Solder bridges

Table 5. Solder bridges

Bridge	State ⁽¹⁾	Description
SB18,20 (X3 crystal) ⁽²⁾	ON	PH0, PH1 are connected to P1 (X3, C21, C22, R30 must not be fitted).
	OFF	X3, C21, C22 and R30 provide a clock as shown in Section 7: Electrical schematics . PH0, PH1 are disconnected from P1.
SB7,9,11,13 (DEFAULT)	ON	Reserved, do not modify.
SB6,8,10,12 (RESERVED)	OFF	Reserved, do not modify.
SB1,2,14 (IDD_Measurement)	ON	PA0, PA4, PC13 are used by the I _{DD} measurement. JP1 ON.
	OFF	PA0, PA4, PC13 are available and IDD module cannot be used JP1 OFF.
SB15,16 (X2 crystal)	OFF	X2, C16, C17 and R28 deliver a 32 KHz clock. PC14, PC15 are not connected to P1.
	ON	PC14, PC15 are only connected to P1. Do not remove X2, C16, C17, R28.
SB5 (B2-RESET)	ON	B2 Pushbutton is connected to the NRST pin of the STM32L152RBT6 MCU.
	OFF	B2 Pushbutton is not connected the NRST pin of the STM32L152RBT6 MCU.
SB4 (B1-USER)	ON	B1 Pushbutton is connected to PA0.
	OFF	B1 Pushbutton is not connected to PA0.
SB21 (VDD powered from 3 V)	ON	V _{DD} is powered from 3 V, SB22 must be OFF.
	OFF	V _{DD} is not powered from 3 V, SB22 must be ON.
SB22 (Battery enable)	OFF	V _{DD} is not powered by the CR2032 battery, SB21 must be ON.
	ON	V _{DD} is powered by the CR2032 battery, SB21 must be OFF.
SB100 (NRST)	ON	The NRST signal of the CN2 connector is connected to the NRST pin of the STM32L152RBT6 MCU.
	OFF	The NRST signal of the CN2 connector is not connected to the NRST pin of the STM32L152RBT6 MCU.
SB101 (SWO)	ON	The SWO signal of the CN2 connector is connected to PB3.
	OFF	The SWO signal is not connected.
SB102 (STM_RST)	OFF	No incidence on STM32F103C8T6 NRST signal.
	ON	STM32F103C8T6 NRST signal is connected to GND.

Table 5. Solder bridges (continued)

Bridge	State ⁽¹⁾	Description
SB3 (BOOT0)	ON	The BOOT0 signal of the STM32L152RBT6 MCU is held low through a 510 Ω pull-down resistor.
	OFF	The BOOT0 signal of the STM32L152RBT6 MCU is held high through a 10 K Ω pull-up resistor.
SB19 (BOOT1)	OFF	The BOOT1 signal of the STM32L152RBT6 MCU is held high through a 10 K Ω pull-up resistor.
	ON	The BOOT1 signal of the STM32L152RBT6 MCU is held low through a 510 Ω pull-down resistor.
SB17 (MCO) ⁽²⁾	OFF	STM32F103C8T6 MCO clock signal is not used.
	ON	STM32F103C8T6 MCO clock signal is connected to OSC_IN of the STM32L152RBT6 MCU.

1. Default SBx state is shown in bold.

2. SB17 and SB20 are OFF to allow the user to choose between MCO and X3 crystal for clock source.

4.9 LCD (24 segments, 4 commons)

This LCD allows the STM32L152RBT6 to display any information on six 14-segment digits and 4 bars, using all COMs. (See the LCD segment mapping in [Figure 17](#) and pin connections in [Table 6](#).)

Note: *This LCD also supports six 8-segment digits by only using COM0 and COM1. This configuration allows COM2 and COM3 to be used as I/O ports. In this case the 2 LCD pins must not be plugged into the LCD socket. To proceed with this configuration, remove the LCD carefully, slightly open the COM2 and COM3 pins (pin 13 and pin 14) of the LCD, then replug it in the socket.*

Characteristics overview:

- 24 segments and 4 commons
- Drive method: multiplexed 1/4 duty, 1/3 bias
- Operating voltage: 3 V
- Operating temperature: 0 to 50°C
- Connector: 28-pin DIL 2.54 mm pitch

Note: *When the LCD is plugged, all I/O ports listed in [Table 6](#) are unavailable. To use one of these as I/O, you must remove the LCD.*

Figure 12. LCD segment mapping

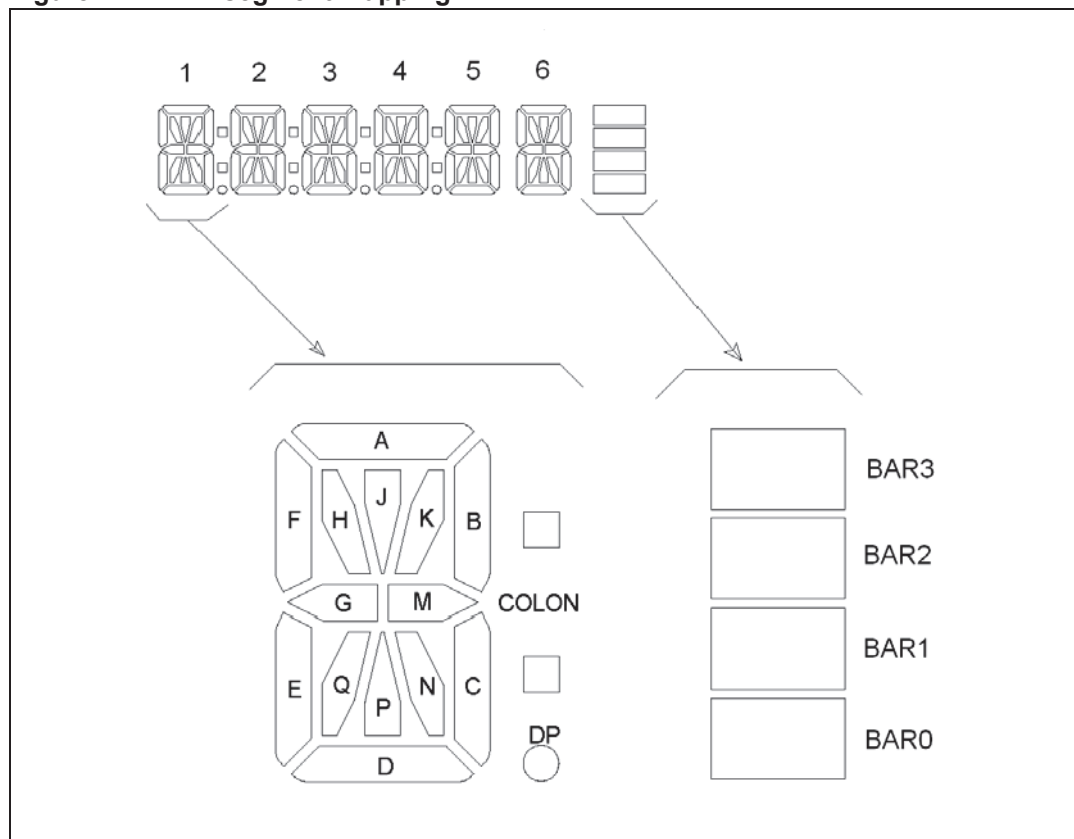


Table 6. LCD connections

STM32L152	LCD					
Name	Pin	COM3	COM2	COM1	COM0	Name
PA1	1	1N	1P	1D	1E	LCDSEG0
PA2	2	1DP	1COLON	1C	1M	LCDSEG1
PA3	3	2N	2P	2D	2E	LCDSEG2
PB3	4	2DP	2COLON	2C	2M	LCDSEG3
PB4	5	3N	3P	3D	3E	LCDSEG4
PB5	6	3DP	3COLON	3C	3M	LCDSEG5
PB10	7	4N	4P	4D	4E	LCDSEG6
PB11	8	4DP	4COLON	4C	4M	LCDSEG7
PB12	9	5N	5P	5D	5E	LCDSEG8
PB13	10	BAR2	BAR3	5C	5M	LCDSEG9
PB14	11	6N	6P	6D	6E	LCDSEG10
PB15	12	BAR0	BAR1	6C	6M	LCDSEG11
PB9	13	COM3				LCDCOM3
PA10	14		COM2			LCDCOM2
PA9	15			COM1		LCDCOM1
PA8	16				COM0	LCDCOM0
PA15	17	6J	6K	6A	6B	LCDSEG12
PB8	18	6H	6Q	6F	6G	LCDSEG13
PC0	19	5J	5K	5A	5B	LCDSEG14
PC1	20	5H	5Q	5F	5G	LCDSEG15
PC2	21	4J	4K	4A	4B	LCDSEG16
PC3	22	4H	4Q	4F	4G	LCDSEG17
PC6	23	3J	3K	3A	3B	LCDSEG18
PC7	24	3H	3Q	3F	3G	LCDSEG19
PC8	25	2J	2K	2A	2B	LCDSEG20
PC9	26	2H	2Q	2F	2G	LCDSEG21
PC10	27	1J	1K	1A	1B	LCDSEG22
PC11	28	1H	1Q	1F	1G	LCDSEG23

5 Extension connectors

The male headers P1 and P2 can connect the STM32L-DISCOVERY to a standard prototyping/wrapping board. STM32L152RBT6 GPIOs are available on these connectors. P1 and P2 can also be probed by an oscilloscope, logical analyzer or voltmeter.

Table 7. MCU pin description versus board function

MCU pin			Board function										
Main function	Alternate functions	LQFP 64 pin num.	LCD glass	Linear Touch Sensor	Push button	I _{DD}	LED	SWD	OSC	Free I/O	Power supply	P1	P2
-	-	-									EXT_3V	1	
-	-										EXT_5V		1
BOOT0	-	60											6
NRST	-	7						NRST				10	
PA0	WKUP1/USART2_CTS/ ADC_IN0/TIM2_CH1_ETR/ COMP1_INP	14			PA0	WAKE UP						15	
PA1	USART2_RTS/ADC_IN1/ TIM2_CH2/LCD_SEG0/ COMP1_INP	15	SEG0									16	
PA2	USART2_TX/ADC_IN2/ TIM2_CH3/TIM9_CH1/ LCD_SEG1/COMP1_INP	16	SEG1									17	
PA3	USART2_RX/ADC_IN3/ TIM2_CH4/TIM9_CH2/ LCD_SEG2/COMP1_INP	17	SEG2									18	
PA4	SPI1_NSS/USART2_CK/ ADC_IN4/DAC_OUT1/ COMP1_INP	20				Measurement						19	
PA5	SPI1_SCK/ADC_IN5/ DAC_OUT2/ TIM2_CH1_ETR/COMP1_INP	21								X		20	
PA6	SPI1_MISO/ADC_IN6/ TIM3_CH1/TIM1_BKIN/ LCD_SEG3/TIM10_CH1/ COMP1_INP	22		PA6									
PA7	SPI1_MOSI/ADC_IN7/ TIM3_CH2/TIM1_CH1N/ LCD_SEG4/TIM11_CH1/ COMP1_INP	23		PA7									
PA8	USART1_CK/MCO/ LCD_COM0	41	COM0										23
PA9	USART1_TX/LCD_COM1	42	COM1										22
PA10	USART1_RX/LCD_COM2	43	COM2										21
PA11	USART1_CTS/USBDM/ SPI1_MISO	44								X			20
PA12	USART1_RTS/USBDP/ SPI1_MOSI	45								X			19
JTMS/ SWDIO	PA13	46						SWD IO					18

Table 7. MCU pin description versus board function (continued)

MCU pin			Board function										
Main function	Alternate functions	LQFP 64 pin num.	LCD glass	Linear Touch Sensor	Push button	I _{DD}	LED	SWD	OSC	Free I/O	Power supply	P1	P2
JTCK/ SWCLK	PA14	49						SW CLK					17
JTDI	TIM2_CH1_ETR/PA15/ SPI1_NSS/LCD_SEG17	50	SEG12										16
PB0	ADC_IN8/TIM3_CH3/ LCD_SEG5/COMP1_INP/ VREF_OUT	26		PB0									
PB1	ADC_IN9/TIM3_CH4/ LCD_SEG6/COMP1_INP/ VREF_OUT	27		PB1									
PB2/BOOT1	-	28										21	
JTDO	TIM2_CH2/PB3/TRACES WO/SPI1_SCK/COMP2_I NM/LCD_SEG7	55	SEG3					SWO					11
JNTRST	TIM3_CH1/PB4/SPI1_MIS O/COMP2_INP/LCD_SEG 8	56	SEG4										10
PB5	I2C1_SMBAL/TIM3_CH2/ SPI1_MOSI/COMP2_INP/ LCD_SEG9	57	SEG5										9
PB6	I2C1_SCL/TIM4_CH1/ USART1_TX/LCD_SEG8	58					Blue						8
PB7	I2C1_SDA/TIM4_CH2/ USART1_RX/PVD_IN	59					Green						7
PB8	TIM4_CH3/I2C1_SCL/ LCD_SEG16/TIM10_CH1	61	SEG13										4
PB9	TIM4_CH4/I2C1_SDA/ LCD_COM3/TIM11_CH1	62	COM3										3
PB10	I2C2_SCL/USART3_TX/ TIM2_CH3/LCD_SEG10	29	SEG6									22	
PB11	I2C2_SDA/USART3_RX/ TIM2_CH4/LCD_SEG11	30	SEG7									23	
PB12	SPI2_NSS/I2C2_SMBAL/ USART3_CK/LCD_SEG1 2/ADC_IN18/COMP1_INP /TIM10_CH1	33	SEG8									24	
PB13	SPI2_SCK/USART3_CTS/ LCD_SEG13/ADC_IN19/ COMP1_INP/TIM9_CH1	34	SEG9									25	
PB14	SPI2_MISO/USART3_RT S/LCD_SEG14/ADC_IN20 /COMP1_INP/TIM9_CH2	35	SEG10									26	
PB15	SPI2_MOSI/TIM1_CH3N/ LCD_SEG15/ADC_IN21/ COMP1_INP/TIM11_CH1/ RTC_50_60Hz	36	SEG11									27	
PC0	ADC_IN10/LCD_SEG18/ COMP1_INP	8	SEG14									11	
PC1	ADC_IN11/LCD_SEG19/ COMP1_INP	9	SEG15									12	
PC2	ADC_IN12/LCD_SEG20/ COMP1_INP	10	SEG16									13	

Table 7. MCU pin description versus board function (continued)

MCU pin			Board function										
Main function	Alternate functions	LQFP 64 pin num.	LCD glass	Linear Touch Sensor	Push button	I _{DD}	LED	SWD	OSC	Free I/O	Power supply	P1	P2
PC3	ADC_IN13/LCD_SEG21/COMP1_INP	11	SEG17									14	
PC4	ADC_IN14/LCD_SEG22/COMP1_INP	24		PC4									
PC5	ADC_IN15/LCD_SEG23/COMP1_INP	25		PC5									
PC6	TIM3_CH1/LCD_SEG24	37	SEG18										27
PC7	TIM3_CH2/LCD_SEG25	38	SEG19										26
PC8	TIM3_CH3/LCD_SEG26	39	SEG20										25
PC9	TIM3_CH4/LCD_SEG27	40	SEG21										24
PC10	USART3_TX/LCD_SEG28/LCD_SEG40/LCD_COM4	51	SEG22										15
PC11	USART3_RX/LCD_SEG29/LCD_SEG41/LCD_COM5	52	SEG23										14
PC12	USART3_CK/LCD_SEG30/LCD_SEG42/LCD_COM6	53								X			13
PC13	RTC_AF1/WKUP2	2				CNT_EN						4	
PC14	OSC32_IN	3							OSC32_IN			5	
PC15	OSC32_OUT	4							OSC32_OUT			6	
PD2	TIM3_ETR/LCD_SEG31/LCD_SEG43/LCD_COM7	54								X			12
OSC_IN	PH0	5							OSC_IN			7	
OSC_OUT	PH1	6							OSC_OUT			8	
-	-	-									GND	2	2
-	-	-									GND	9	5
-	-	-									GND	28	28
-	-	-									VDD	3	

Alternative Functions in STM32L152 GPIO

Table 5. Alternate function input/output

Port name	Digital alternate function number															
	AFIO0	AFIO1	AFIO2	AFIO3	AFIO4	AFIO5	AFIO6	AFIO7	AFIO8	AFIO9	AFIO10	AFIO11	AFIO12	AFIO13	AFIO14	AFIO15
	Alternate function															
	SYSTEM	TIM2	TIM3/4	TIM9/10/11	I2C1/2	SPI1/2	N/A	USART 1/2/3	N/A	N/A	USBFS	LCD	N/A	N/A	RI	SYSTEM
BOOT0	BOOT0															
NRST	NRST															
PA0-WKUP1	WKUP1	TIM2_CH1_ETR						USART2_CTS							TIMx_IC1	EVENTOUT
PA1		TIM2_CH2						USART2_RTS				[SEG0]			TIMx_IC2	EVENTOUT
PA2		TIM2_CH3		TIM9_CH1				USART2_TX				[SEG1]			TIMx_IC3	EVENTOUT
PA3		TIM2_CH4		TIM9_CH2				USART2_RX				[SEG2]			TIMx_IC4	EVENTOUT
PA4						SPI1_NSS		USART2_CK							TIMx_IC1	EVENTOUT
PA5		TIM2_CH1_ETR				SPI1_SCK									TIMx_IC2	EVENTOUT
PA6			TIM3_CH1	TIM10_CH1		SPI1_MISO						[SEG3]			TIMx_IC3	EVENTOUT
PA7			TIM3_CH2	TIM11_CH1		SPI1_MOSI						[SEG4]			TIMx_IC4	EVENTOUT
PA8	MCO							USART1_CK				[COM0]			TIMx_IC1	EVENTOUT
PA9								USART1_TX				[COM1]			TIMx_IC2	EVENTOUT
PA10								USART1_RX				[COM2]			TIMx_IC3	EVENTOUT
PA11						SPI1_MISO		USART1_CTS			DM				TIMx_IC4	EVENTOUT
PA12						SPI1_MOSI		USART1_RTS			DP				TIMx_IC1	EVENTOUT
PA13	JTMS-SWDAT														TIMx_IC2	EVENTOUT
PA14	JTCK-SWCLK														TIMx_IC3	EVENTOUT

Table 5. Alternate function input/output (continued)

Port name	Digital alternate function number															
	AFIO0	AFIO1	AFIO2	AFIO3	AFIO4	AFIO5	AFIO6	AFIO7	AFIO8	AFIO9	AFIO10	AFIO11	AFIO12	AFIO13	AFIO14	AFIO15
	Alternate function															
	SYSTEM	TIM2	TIM3/4	TIM9/10/11	I2C1/2	SPI1/2	N/A	USART 1/2/3	N/A	N/A	USBFS	LCD	N/A	N/A	RI	SYSTEM
PA15	JTDI	TIM2_CH1_ETR				SPI1_NSS						SEG17			TIMx_IC4	EVENTOUT
PB0			TIM3_CH3									[SEG5]				EVENTOUT
PB1			TIM3_CH4									[SEG6]				EVENTOUT
PB2	BOOT1															EVENTOUT
PB3	JTDO	TIM2_CH2				SPI1_SCK						[SEG7]				EVENTOUT
PB4	JTRST		TIM3_CH1			SPI1_MISO						[SEG8]				EVENTOUT
PB5			TIM3_CH2		I2C1_SMB AI	SPI1_MOSI						[SEG9]				EVENTOUT
PB6			TIM4_CH1		I2C1_SCL			USART1_TX								EVENTOUT
PB7			TIM4_CH2		I2C1_SDA			USART1_RX								EVENTOUT
PB8			TIM4_CH3	TIM10_CH1	I2C1_SCL							SEG16				EVENTOUT
PB9			TIM4_CH4	TIM11_CH1	I2C1_SDA							[COM3]				EVENTOUT
PB10		TIM2_CH3			I2C2_SCL			USART3_TX				SEG10				EVENTOUT
PB11		TIM2_CH4			I2C2_SDA			USART3_RX				SEG11				EVENTOUT
PB12				TIM10_CH1	I2C2_SMB AI	SPI2_NSS		USART3_CK				SEG12				EVENTOUT
PB13				TIM9_CH1		SPI2_SCK		USART3_CTS				SEG13				EVENTOUT
PB14				TIM9_CH2		SPI2_MISO		USART3_RTS				SEG14				EVENTOUT
PB15	RTC 50/60 Hz			TIM11_CH1		SPI2_MOSI						SEG15				EVENTOUT
PC0												SEG18			TIMx_IC1	EVENTOUT

**Table 5. Alternate function input/output (continued)**

Port name	Digital alternate function number															
	AFIO0	AFIO1	AFIO2	AFIO3	AFIO4	AFIO5	AFIO6	AFIO7	AFIO8	AFIO9	AFIO10	AFIO11	AFIO12	AFIO13	AFIO14	AFIO15
	Alternate function															
	SYSTEM	TIM2	TIM3/4	TIM9/10/11	I2C1/2	SPI1/2	N/A	USART1/2/3	N/A	N/A	USBFS	LCD	N/A	N/A	RI	SYSTEM
PC1												SEG19			TIMx_IC2	EVENTOUT
PC2												SEG20			TIMx_IC3	EVENTOUT
PC3												SEG21			TIMx_IC4	EVENTOUT
PC4												SEG22			TIMx_IC1	EVENTOUT
PC5												SEG23			TIMx_IC2	EVENTOUT
PC6			TIM3_CH1									SEG24			TIMx_IC3	EVENTOUT
PC7			TIM3_CH2									SEG25			TIMx_IC4	EVENTOUT
PC8			TIM3_CH3									SEG26			TIMx_IC1	EVENTOUT
PC9			TIM3_CH4									SEG27			TIMx_IC2	EVENTOUT
PC10								USART3_TX				COM4 / SEG28 / SEG40			TIMx_IC3	EVENTOUT
PC11								USART3_RX				COM5 / SEG29 / SEG41			TIMx_IC4	EVENTOUT
PC12								USART3_CK				COM6 / SEG30 / SEG42			TIMx_IC1	EVENTOUT
PC13-RTC_AF1	RTC_AF1 / WKUP2														TIMx_IC2	EVENTOUT
PC14-OSC32_IN	OSC32_IN														TIMx_IC3	EVENTOUT
PC15-OSC32_OUT	OSC32_OUT														TIMx_IC4	EVENTOUT
PD0				TIM9_CH1		SPI2_NSS									TIMx_IC1	EVENTOUT
PD1						SPI2_SCK									TIMx_IC2	EVENTOUT

**Table 5. Alternate function input/output (continued)**

Port name	Digital alternate function number															
	AFIO0	AFIO1	AFIO2	AFIO3	AFIO4	AFIO5	AFIO6	AFIO7	AFIO8	AFIO9	AFIO10	AFIO11	AFIO12	AFIO13	AFIO14	AFIO15
	Alternate function															
	SYSTEM	TIM2	TIM3/4	TIM9/10/11	I2C1/2	SPI1/2	N/A	USART1/2/3	N/A	N/A	USBFS	LCD	N/A	N/A	RI	SYSTEM
PD2			TIM3_ETR									COM7 / SEG31 / SEG43			TIMx_IC3	EVENTOUT
PD3						SPI2_MISO		USART2_CTS							TIMx_IC4	EVENTOUT
PD4						SPI2_MOSI		USART2_RTS							TIMx_IC1	EVENTOUT
PD5								USART2_TX							TIMx_IC2	EVENTOUT
PD6								USART2_RX							TIMx_IC3	EVENTOUT
PD7				TIM9_CH2				USART2_CK							TIMx_IC4	EVENTOUT
PD8								USART3_TX				SEG28			TIMx_IC1	EVENTOUT
PD9								USART3_RX				SEG29			TIMx_IC2	EVENTOUT
PD10								USART3_CK				SEG30			TIMx_IC3	EVENTOUT
PD11								USART3_CTS				SEG31			TIMx_IC4	EVENTOUT
PD12			TIM4_CH1					USART3_RTS				SEG32			TIMx_IC1	EVENTOUT
PD13			TIM4_CH2									SEG33			TIMx_IC2	EVENTOUT
PD14			TIM4_CH3									SEG34			TIMx_IC3	EVENTOUT
PD15			TIM4_CH4									SEG35			TIMx_IC4	EVENTOUT
PE0			TIM4_ETR	TIM10_CH1								SEG36			TIMx_IC1	EVENTOUT
PE1				TIM11_CH1								SEG37			TIMx_IC2	EVENTOUT
PE2	TRACECK		TIM3_ETR									SEG 38			TIMx_IC3	EVENTOUT
PE3	TRACED0		TIM3_CH1									SEG 39			TIMx_IC4	EVENTOUT



Table 5. Alternate function input/output (continued)

Port name	Digital alternate function number															
	AFIO0	AFIO1	AFIO2	AFIO3	AFIO4	AFIO5	AFIO6	AFIO7	AFIO8	AFIO9	AFIO10	AFIO11	AFIO12	AFIO13	AFIO14	AFIO15
	Alternate function															
	SYSTEM	TIM2	TIM3/4	TIM9/10/11	I2C1/2	SPI1/2	N/A	USART 1/2/3	N/A	N/A	USBFS	LCD	N/A	N/A	RI	SYSTEM
PE4	TRACED1		TIM3_CH2												TIMx_IC1	EVENTOUT
PE5	TRACED2			TIM9_CH1*											TIMx_IC2	EVENTOUT
PE6	TRACED3 / WKUP3			TIM9_CH2*											TIMx_IC3	EVENTOUT
PE7															TIMx_IC4	EVENTOUT
PE8															TIMx_IC1	EVENTOUT
PE9		TIM2_CH1_ETR													TIMx_IC2	EVENTOUT
PE10		TIM2_CH2													TIMx_IC3	EVENTOUT
PE11		TIM2_CH3													TIMx_IC4	EVENTOUT
PE12		TIM2_CH4				SPI1_NSS									TIMx_IC1	EVENTOUT
PE13						SPI1_SCK									TIMx_IC2	EVENTOUT
PE14						SPI1_MISO									TIMx_IC3	EVENTOUT
PE15						SPI1_MOSI									TIMx_IC4	EVENTOUT
PH0-OSC_IN	OSC_IN															
PH1-OSC_OUT	OSC_OUT															
PH2																EVENTOUT

NVIC Registers

- La Gestión de las interrupciones por parte de la CPU se lleva a cabo utilizando el ***Nested Vectored Interrupt Controller (NVIC)***
 - Permite 45 fuentes de interrupción enmascarables
 - Puede permitir establecer niveles de prioridad programables (hasta 16)
 - Establece un sistema por defecto de fuentes de interrupción y su prioridad
 - En la Tabla 28 del Reference Manual están todas las fuentes.
 - La siguiente transparencia muestra una tabla con las fuentes de interés para el curso
- El uVision4 ya tiene unos nombres de RAI definidos para cada fuente de IRQ (ver tabla)



Posición	Prioridad	Acrónimo	Descripción	Nombre de la RAI
6	13	EXTI0	EXTI Line 0	EXTI0_IRQHandler
7	14	EXTI1	EXTI Line 1	EXTI1_IRQHandler
8	15	EXTI2	EXTI Line 2	EXTI2_IRQHandler
9	16	EXTI3	EXTI Line 3	EXTI3_IRQHandler
10	17	EXTI4	EXTI Line 4	EXTI4_IRQHandler
18	25	ADC1	ADC1 global IRQ	ADC1_IRQHandler
19	26	USB_HP	USB High Priority IRQ	USB_HP_IRQHandler
20	27	USB_LP	USB Low Priority IRQ	USB_LP_IRQHandler
21	28	DAC	D/A Converter	DAC_IRQHandler
23	30	EXTI9_5	EXTI Line[9:5]	EXTI9_5_IRQHandler
28	35	TIM2	TIM2 global IRQ	TIM2_IRQHandler
29	36	TIM3	TIM3 global IRQ	TIM3_IRQHandler
30	37	TIM4	TIM4 global IRQ	TIM4_IRQHandler
31	38	I2C1_EV	I ² C1 Event	I2C1_EV_IRQHandler
32	39	I2C1_ER	I ² C1 Error	I2C1_ER_IRQHandler
35	42	SPI1	SPI1 global IRQ	SPI1_IRQHandler
37	44	USART1	USART1 global IRQ	USART1_IRQHandler
40	47	EXTI15_10	EXTI Line[15:10]	EXTI15_10_IRQHandler
41	48	RTC_Alarm	RTC Alarms	RTC_Alarm_IRQHandler

- El NVIC se controla con los siguientes registros de 32 bits, donde cada bit se refiere a una de las posiciones de interrupción:
 - **NVIC->ISER[x] : Habilita la fuente de interrupción correspondiente al bit en el que se escriba un '1' del registro x**
 - Los bits a 0 no modifican el estado de enmascaramiento
 - ISER[0] cubre los bits para las primeras 32 fuentes de interrupción
 - ISER[1] cubre los bits para las fuentes 32 a 63, correspondiendo el bit 0 a la fuente 32
 - **NVIC->ICER[x] : Enmascara la fuente de interrupción correspondiente al bit en el que se escriba un '1' del registro x**
 - Los bits a 0 no modifican el estado de enmascaramiento
 - ICER[0] e ICER[1] análogo a ISER[x]

Name	Type	Reset	Description
ICTR	RO	-	<i>Interrupt Controller Type Register, ICTR</i>
NVIC_ISER0 - NVIC_ISER7	RW	0x00000000	Interrupt Set-Enable Registers
NVIC_ICER0 - NVIC_ICER7	RW	0x00000000	Interrupt Clear-Enable Registers
NVIC_ISPR0 - NVIC_ISPR7	RW	0x00000000	Interrupt Set-Pending Registers
NVIC_ICPR0 - NVIC_ICPR7	RW	0x00000000	Interrupt Clear-Pending Registers

STM32L1xx Reference Manual (reduced)



Introduction

This reference manual targets application developers. It provides complete information on how to use the STM32L151xx and STM32L152xx microcontroller memory and peripherals. The STM32L151xx and STM32L152xx will be referred to as STM32L15xxx throughout the document, unless otherwise specified.

The STM32L15xxx is a family of microcontrollers with different memory sizes, packages and peripherals.

For ordering information, mechanical and electrical device characteristics please refer to the *STM32L151xx and STM32L152xx* datasheet.

For information on programming, erasing and protection of the internal Flash memory please refer to the *STM32L15xxx Flash programming manual*.

For information on the ARM Cortex™-M3 core, please refer to the *Cortex™-M3 Technical Reference Manual*.

Related documents

Available from www.arm.com:

■ *Cortex™-M3 Technical Reference Manual*, available from:

http://infocenter.arm.com/help/topic/com.arm.doc.ddi0337g/DDI0337G_cortex_m3_r2p0_trm.pdf

Available from www.st.com:

■ *STM32L151xx STM32L152xx* datasheet

■ *STM32L15xxx Flash programming manual*

Contents

1	Documentation conventions	29
1.1	List of abbreviations for registers	29
1.2	Peripheral availability	29
2	Memory and bus architecture	30
2.1	System architecture	30
2.2	Memory organization	31
2.3	Memory map	31
2.3.1	Embedded SRAM	34
2.3.2	Bit banding	35
2.3.3	Embedded Flash memory	35
2.4	Boot configuration	41
3	Power control (PWR)	43
3.1	Power supplies	43
3.1.1	Independent A/D and DAC converter supply and reference voltage	44
3.1.2	Independent LCD supply	45
3.1.3	RTC and RTC backup registers	45
3.1.4	Voltage regulator	46
3.1.5	Dynamic voltage scaling management	46
3.1.6	Dynamic voltage scaling configuration	48
3.1.7	Voltage regulator and clock management when VDD drops below 2.0 V	49
3.1.8	Voltage regulator and clock management when modifying the VCORE range	49
3.2	Power supply supervisor	49
3.2.1	Power on reset (POR)/power down reset (PDR)	52
3.2.2	Brown out reset (BOR)	52
3.2.3	Programmable voltage detector (PVD)	54
3.2.4	Internal voltage reference (VREFINT)	54
3.3	Low-power modes	55
3.3.1	Behavior of clocks in low power modes	56
3.3.2	Slowing down system clocks	57
3.3.3	Peripheral clock gating	57

3.3.4	Low power run mode (LP run)	57
3.3.5	Sleep mode	58
3.3.6	Low power sleep mode (LP sleep)	59
3.3.7	Stop mode	61
3.3.8	Standby mode	63
3.3.9	Waking up the device from Stop and Standby modes using the RTC and comparators 64	
3.4	Power control registers	65
3.4.1	PWR power control register (PWR_CR)	66
3.4.2	PWR power control/status register (PWR_CSR)	68
3.4.3	PWR register map	69
4	Reset and clock control (RCC)	70
4.1	Reset	70
4.1.1	System reset	70
4.1.2	Power reset	71
4.1.3	RTC and backup registers reset	71
4.2	Clocks	72
4.2.1	HSE clock	74
4.2.2	HSI clock	75
4.2.3	MSI clock	75
4.2.4	PLL	76
4.2.5	LSE clock	77
4.2.6	LSI clock	77
4.2.7	System clock (SYSCLK) selection	78
4.2.8	System clock source frequency versus voltage range	78
4.2.9	Clock security system (CSS)	78
4.2.10	RTC and LCD clock	79
4.2.11	Watchdog clock	79
4.2.12	Clock-out capability	79
4.2.13	Internal/external clock measurement with TIM9/TIM10/TIM11	79
4.2.14	Clock-independent system clock sources for TIM9/TIM10/TIM11	81
4.3	RCC registers	82
4.3.1	Clock control register (RCC_CR)	82
4.3.2	Internal clock sources calibration register (RCC_ICSCR)	84
4.3.3	Clock configuration register (RCC_CFGR)	85
4.3.4	Clock interrupt register (RCC_CIR)	87

4.3.5	AHB peripheral reset register (RCC_AHBRSTR)	90
4.3.6	APB2 peripheral reset register (RCC_APB2RSTR)	91
4.3.7	APB1 peripheral reset register (RCC_APB1RSTR)	92
4.3.8	AHB peripheral clock enable register (RCC_AHBENR)	94
4.3.9	APB2 peripheral clock enable register (RCC_APB2ENR)	95
4.3.10	APB1 peripheral clock enable register (RCC_APB1ENR)	97
4.3.11	AHB peripheral clock enable in low power mode register (RCC_AHBLPENR)	99
4.3.12	APB2 peripheral clock enable in low power mode register (RCC_APB2LPENR)	100
4.3.13	APB1 peripheral clock enable in low power mode register (RCC_APB1LPENR)	101
4.3.14	Control/status register (RCC_CSR)	103
4.3.15	RCC register map	106
5	General-purpose I/Os (GPIO)	108
5.1	GPIO introduction	108
5.2	GPIO main features	108
5.3	GPIO functional description	108
5.3.1	General-purpose I/O (GPIO)	110
5.3.2	I/O pin multiplexer and mapping	111
5.3.3	I/O port control registers	113
5.3.4	I/O port data registers	113
5.3.5	I/O data bitwise handling	114
5.3.6	GPIO locking mechanism	114
5.3.7	I/O alternate function input/output	114
5.3.8	External interrupt/wakeup lines	115
5.3.9	Input configuration	115
5.3.10	Output configuration	116
5.3.11	Alternate function configuration	116
5.3.12	Analog configuration	117
5.3.13	Using the OSC32_IN/OSC32_OUT pins as GPIO PC14/PC15 port pins	118
5.3.14	Using the OSC_IN/OSC_OUT pins as GPIO PH0/PH1 port pins	118
5.3.15	Selection of RTC_AF1 alternate functions	118
5.4	GPIO registers	119
5.4.1	GPIO port mode register (GPIOx_MODER) (x = A..E and H)	119
5.4.2	GPIO port output type register (GPIOx_OTYPER) (x = A..E and H) ..	120

5.4.3	GPIO port output speed register (GPIOx_OSPEEDR) (x = A..E and H)	120
5.4.4	GPIO port pull-up/pull-down register (GPIOx_PUPDR) (x = A..E and H)	120
5.4.5	GPIO port input data register (GPIOx_IDR) (x = A..E and H)	121
5.4.6	GPIO port output data register (GPIOx_ODR) (x = A..E and H)	121
5.4.7	GPIO port bit set/reset register (GPIOx_BSRR) (x = A..E and H)	122
5.4.8	GPIO port configuration lock register (GPIOx_LCKR) (x = A..E and H)	122
5.4.9	GPIO alternate function low register (GPIOx_AFR1) (x = A..E and H)	123
5.4.10	GPIO alternate function high register (GPIOx_AFR2) (x = A..E and H)	124
5.4.11	GPIO register map	124

6	System configuration controller (SYSCFG) and routing interface (RI)	126
6.1	SYSCFG and RI introduction	126
6.2	RI main features	126
6.3	RI functional description	128
6.3.1	Special I/O configuration	128
6.3.2	Input capture routing	130
6.3.3	Reference voltage routing	131
6.4	RI registers	132
6.4.1	RI input capture register (RI_ICR)	132
6.4.2	RI analog switches control register (RI_ASCR1)	134
6.4.3	RI analog switch control register 2 (RI_ASCR2)	135
6.4.4	RI hysteresis control register (RI_HYSCR1)	136
6.4.5	RI Hysteresis control register (RI_HYSCR2)	136
6.4.6	RI Hysteresis control register (RI_HYSCR3)	137
6.4.7	Analog switch mode register (RI_ASMR1)	137
6.4.8	Channel mask register (RI_CMR1)	138
6.4.9	Channel identification for capture register (RI_CICR1)	138
6.4.10	Analog switch mode register (RI_ASMR2)	139
6.4.11	Channel mask register (RI_CMR2)	139
6.4.12	Channel identification for capture register (RI_CICR2)	140
6.4.13	Analog switch mode register (RI_ASMR3)	140
6.4.14	Channel mask register (RI_CMR3)	141
6.4.15	Channel identification for capture register (RI_CICR3)	141

6.4.16	RI register map	142
6.5	SYSCFG registers	144
6.5.1	SYSCFG memory remap register (SYSCFG_MEMRMP)	144
6.5.2	SYSCFG peripheral mode configuration register (SYSCFG_PMC)	144
6.5.3	SYSCFG external interrupt configuration register 1 (SYSCFG_EXTICR1)	145
6.5.4	SYSCFG external interrupt configuration register 2 (SYSCFG_EXTICR2)	145
6.5.5	SYSCFG external interrupt configuration register 3 (SYSCFG_EXTICR3)	146
6.5.6	SYSCFG external interrupt configuration register 4 (SYSCFG_EXTICR4)	146
6.5.7	SYSCFG register map	147
7	Interrupts and events	148
7.1	Nested vectored interrupt controller (NVIC)	148
7.1.1	SysTick calibration value register	148
7.1.2	Interrupt and exception vectors	148
7.2	External interrupt/event controller (EXTI)	150
7.2.1	Main features	150
7.2.2	Block diagram	151
7.2.3	Wakeup event management	151
7.2.4	Functional description	151
7.2.5	External interrupt/event line mapping	152
7.3	EXTI registers	154
7.3.1	EXTI interrupt mask register (EXTI_IMR)	154
7.3.2	EXTI event mask register (EXTI_EMR)	154
7.3.3	EXTI rising edge trigger selection register (EXTI_RTSTR)	155
7.3.4	Falling edge trigger selection register (EXTI_FTSR)	156
7.3.5	EXTI software interrupt event register (EXTI_SWIER)	156
7.3.6	EXTI pending register (EXTI_PR)	157
7.3.7	EXTI register map	157
8	DMA controller (DMA)	158
8.1	DMA introduction	158
8.2	DMA main features	158
8.3	DMA functional description	159
8.3.1	DMA transactions	159

8.3.2	Arbiter	160
8.3.3	DMA channels	160
8.3.4	Programmable data width, data alignment and endians	161
8.3.5	Error management	163
8.3.6	Interrupts	163
8.3.7	DMA request mapping	163
8.4	DMA registers	166
8.4.1	DMA interrupt status register (DMA_ISR)	166
8.4.2	DMA interrupt flag clear register (DMA_IFCR)	167
8.4.3	DMA channel x configuration register (DMA_CCRx) (x = 1..7, where x = channel number)	168
8.4.4	DMA channel x number of data register (DMA_CNDTRx) (x = 1..7), where x = channel number)	169
8.4.5	DMA channel x peripheral address register (DMA_CPARx) (x = 1..7), where x = channel number)	170
8.4.6	DMA channel x memory address register (DMA_CMARx) (x = 1..7), where x = channel number)	170
8.4.7	DMA register map	171
9	Analog-to-digital converter (ADC)	173
9.1	ADC introduction	173
9.2	ADC main features	173
9.3	ADC functional description	173
9.3.1	ADC power on-off control	175
9.3.2	ADC clock	176
9.3.3	Channel selection	176
9.3.4	Single conversion mode	177
9.3.5	Continuous conversion mode	177
9.3.6	Timing diagram	178
9.3.7	Analog watchdog	178
9.3.8	Scan mode	179
9.3.9	Injected channel management	180
9.3.10	Discontinuous mode	181
9.4	Data alignment	181
9.5	Channel-wise programmable sampling time	182
9.6	Conversion on external trigger	183
9.7	Aborting a conversion	185

9.7.1	Injected channels	185
9.7.2	Regular channels	185
9.8	Conversion resolution	185
9.9	Hardware freeze and delay insertion modes for slow conversions	185
9.9.1	Inserting a delay after each regular conversion	186
9.9.2	Inserting a delay after each sequence of auto-injected conversions	187
9.10	Power saving	188
9.11	Data management and overrun detection	190
9.11.1	Using the DMA	190
9.11.2	Managing a sequence of conversions without using the DMA	190
9.11.3	Conversions without reading all the data	191
9.11.4	Overrun detection	191
9.12	Temperature sensor	191
9.12.1	How to read the temperature	192
9.13	Internal reference voltage (V_{REFINT}) conversion	193
9.14	ADC interrupts	193
9.15	ADC registers	194
9.15.1	ADC status register (ADC_SR)	194
9.15.2	ADC control register 1 (ADC_CR1)	196
9.15.3	ADC control register 2 (ADC_CR2)	198
9.15.4	ADC sample time register 1 (ADC_SMPR1)	202
9.15.5	ADC sample time register 2 (ADC_SMPR2)	202
9.15.6	ADC sample time register 3 (ADC_SMPR3)	203
9.15.7	ADC injected channel data offset register x (ADC_JOFRx)(x=1..4)	204
9.15.8	ADC watchdog higher threshold register (ADC_HTR)	204
9.15.9	ADC watchdog lower threshold register (ADC_LTR)	204
9.15.10	ADC regular sequence register 1 (ADC_SQR1)	205
9.15.11	ADC regular sequence register 2 (ADC_SQR2)	205
9.15.12	ADC regular sequence register 3 (ADC_SQR3)	206
9.15.13	ADC regular sequence register 4 (ADC_SQR4)	207
9.15.14	ADC regular sequence register 5 (ADC_SQR5)	207
9.15.15	ADC injected sequence register (ADC_JSQR)	208
9.15.16	ADC injected data register x (ADC_JDRx) (x= 1..4)	208
9.15.17	ADC regular data register (ADC_DR)	209
9.15.18	ADC common status register (ADC_CSR)	209
9.15.19	ADC common control register (ADC_CCR)	210

	9.15.20	ADC register map	211
10		Digital-to-analog converter (DAC)	213
	10.1	DAC introduction	213
	10.2	DAC main features	213
	10.3	DAC functional description	215
	10.3.1	DAC channel enable	215
	10.3.2	DAC output buffer enable	215
	10.3.3	DAC data format	215
	10.3.4	DAC conversion	216
	10.3.5	DAC output voltage	217
	10.3.6	DAC trigger selection	217
	10.3.7	DMA request	217
	10.3.8	Noise generation	218
	10.3.9	Triangle-wave generation	219
	10.4	Dual DAC channel conversion	220
	10.4.1	Independent trigger without wave generation	220
	10.4.2	Independent trigger with single LFSR generation	221
	10.4.3	Independent trigger with different LFSR generation	221
	10.4.4	Independent trigger with single triangle generation	221
	10.4.5	Independent trigger with different triangle generation	222
	10.4.6	Simultaneous software start	222
	10.4.7	Simultaneous trigger without wave generation	222
	10.4.8	Simultaneous trigger with single LFSR generation	223
	10.4.9	Simultaneous trigger with different LFSR generation	223
	10.4.10	Simultaneous trigger with single triangle generation	223
	10.4.11	Simultaneous trigger with different triangle generation	224
	10.5	DAC registers	224
	10.5.1	DAC control register (DAC_CR)	224
	10.5.2	DAC software trigger register (DAC_SWTRIGR)	227
	10.5.3	DAC channel1 12-bit right-aligned data holding register (DAC_DHR12R1)	228
	10.5.4	DAC channel1 12-bit left aligned data holding register (DAC_DHR12L1)	228
	10.5.5	DAC channel1 8-bit right aligned data holding register (DAC_DHR8R1)	228
	10.5.6	DAC channel2 12-bit right aligned data holding register (DAC_DHR12R2)	229

10.5.7	DAC channel2 12-bit left aligned data holding register (DAC_DHR12L2)	229
10.5.8	DAC channel2 8-bit right-aligned data holding register (DAC_DHR8R2)	229
10.5.9	Dual DAC 12-bit right-aligned data holding register (DAC_DHR12RD)	230
10.5.10	DUAL DAC 12-bit left aligned data holding register (DAC_DHR12LD)	230
10.5.11	DUAL DAC 8-bit right aligned data holding register (DAC_DHR8RD)	231
10.5.12	DAC channel1 data output register (DAC_DOR1)	231
10.5.13	DAC channel2 data output register (DAC_DOR2)	231
10.5.14	DAC status register (DAC_SR)	232
10.5.15	DAC register map	233
11	Comparators (COMP)	234
11.1	Introduction	234
11.2	Main features	234
11.3	COMP clock	236
11.4	Comparator 1 (COMP1)	236
11.5	Comparator 2 (COMP2)	237
11.6	Comparators in Window mode	239
11.7	Low power modes	239
11.8	Interrupts	240
11.9	COMP registers	240
11.9.1	COMP comparator control and status register (COMP_CSR)	240
11.9.2	COMP register map	242
12	LCD controller (LCD)	243
12.1	Introduction	243
12.2	LCD main features	243
12.3	Glossary	244
12.4	LCD functional description	245
12.4.1	General description	245
12.4.2	Frequency generator	246
12.4.3	Common driver	247
12.4.4	Segment driver	251

12.4.5	Voltage generator	255
12.4.6	Double buffer memory	257
12.4.7	COM and SEG multiplexing	257
12.4.8	Flowchart	261
12.5	LCD registers	262
12.5.1	LCD control register (LCD_CR)	262
12.5.2	LCD frame control register (LCD_FCR)	263
12.5.3	LCD status register (LCD_SR)	265
12.5.4	LCD clear register (LCD_CLR)	266
12.5.5	LCD display memory (LCD_RAM)	268
12.5.6	LCD register map	268
13	General-purpose timers (TIM2 to TIM4)	270
13.1	TIM2 to TIM4 introduction	270
13.2	TIM2 to TIM4 main features	270
13.3	TIM2 to TIM4 functional description	271
13.3.1	Time-base unit	271
13.3.2	Counter modes	273
13.3.3	Clock selection	282
13.3.4	Capture/compare channels	285
13.3.5	Input capture mode	287
13.3.6	PWM input mode	288
13.3.7	Forced output mode	289
13.3.8	Output compare mode	289
13.3.9	PWM mode	290
13.3.10	One-pulse mode	293
13.3.11	Clearing the OCxREF signal on an external event	294
13.3.12	Encoder interface mode	295
13.3.13	Timer input XOR function	297
13.3.14	Timers and external trigger synchronization	297
13.3.15	Timer synchronization	300
13.3.16	Debug mode	305
13.4	TIMx registers	306
13.4.1	TIMx control register 1 (TIMx_CR1)	306
13.4.2	TIMx control register 2 (TIMx_CR2)	308
13.4.3	TIMx slave mode control register (TIMx_SMCR)	309
13.4.4	TIMx DMA/Interrupt enable register (TIMx_DIER)	311

13.4.5	TIMx status register (TIMx_SR)	312
13.4.6	TIMx event generation register (TIMx_EGR)	314
13.4.7	TIMx capture/compare mode register 1 (TIMx_CCMR1)	315
13.4.8	TIMx capture/compare mode register 2 (TIMx_CCMR2)	318
13.4.9	TIMx capture/compare enable register (TIMx_CCER)	319
13.4.10	TIMx counter (TIMx_CNT)	321
13.4.11	TIMx prescaler (TIMx_PSC)	321
13.4.12	TIMx auto-reload register (TIMx_ARR)	321
13.4.13	TIMx capture/compare register 1 (TIMx_CCR1)	322
13.4.14	TIMx capture/compare register 2 (TIMx_CCR2)	322
13.4.15	TIMx capture/compare register 3 (TIMx_CCR3)	323
13.4.16	TIMx capture/compare register 4 (TIMx_CCR4)	323
13.4.17	TIMx DMA control register (TIMx_DCR)	324
13.4.18	TIMx DMA address for full transfer (TIMx_DMAR)	324
13.4.19	TIMx register map	326
14	General-purpose timers (TIM9/10/11)	328
14.1	TIM9/10/11 introduction	328
14.2	TIM9/10/11 main features	328
14.2.1	TIM9 main features	328
14.2.2	TIM10 and TIM11 main features	329
14.3	TIM9/10/11 functional description	331
14.3.1	Time-base unit	331
14.3.2	Counter modes	332
14.3.3	Clock selection	335
14.3.4	Capture/compare channels	338
14.3.5	Input capture mode	339
14.3.6	PWM input mode (available for TIM9 only)	340
14.3.7	Forced output mode	341
14.3.8	Output compare mode	341
14.3.9	PWM mode	342
14.3.10	One-pulse mode (available for TIM9 only)	344
14.3.11	Timers and external trigger synchronization (available for TIM9 only) ..	345
14.3.12	Timer synchronization (available for TIM9 only)	347
14.3.13	Debug mode	352
14.4	TIM9/10/11 registers	353
14.4.1	TIMx control register 1 (TIMx_CR1)	353

14.4.2	TIMx control register 2 (TIMx_CR2) (available for TIM9 only)	354
14.4.3	TIMx slave mode control register (TIMx_SMCR)	355
14.4.4	TIMx Interrupt enable register (TIMx_DIER)	358
14.4.5	TIMx status register (TIMx_SR)	358
14.4.6	TIMx event generation register (TIMx_EGR)	360
14.4.7	TIMx capture/compare mode register 1 (TIMx_CCMR1)	361
14.4.8	TIMx capture/compare enable register (TIMx_CCER)	364
14.4.9	TIMx counter (TIMx_CNT)	365
14.4.10	TIMx prescaler (TIMx_PSC)	365
14.4.11	TIMx auto-reload register (TIMx_ARR)	366
14.4.12	TIMx capture/compare register 1 (TIMx_CCR1)	366
14.4.13	TIMx capture/compare register 2 (TIMx_CCR2) (available only for TIM9)	367
14.4.14	TIM9 option register 1 (TIM9_OR)	367
14.4.15	TIM10 option register 1 (TIM10_OR)	368
14.4.16	TIM11 option register 1 (TIM11_OR)	368
14.4.17	TIMx register map	368
15	Basic timers (TIM6&TIM7)	371
15.1	TIM6&TIM7 introduction	371
15.2	TIM6&TIM7 main features	371
15.3	TIM6&TIM7 functional description	372
15.3.1	Time-base unit	372
15.3.2	Counting mode	373
15.3.3	Clock source	376
15.3.4	Debug mode	376
15.4	TIM6&TIM7 registers	377
15.4.1	TIM6&TIM7 control register 1 (TIMx_CR1)	377
15.4.2	TIM6&TIM7 control register 2 (TIMx_CR2)	378
15.4.3	TIM6&TIM7 DMA/Interrupt enable register (TIMx_DIER)	378
15.4.4	TIM6&TIM7 status register (TIMx_SR)	379
15.4.5	TIM6&TIM7 event generation register (TIMx_EGR)	379
15.4.6	TIM6&TIM7 counter (TIMx_CNT)	379
15.4.7	TIM6&TIM7 prescaler (TIMx_PSC)	380
15.4.8	TIM6&TIM7 auto-reload register (TIMx_ARR)	380
15.4.9	TIM6&TIM7 register map	381

16	Independent watchdog (IWDG)	382
16.1	IWDG introduction	382
16.2	IWDG main features	382
16.3	IWDG functional description	382
16.3.1	Hardware watchdog	382
16.3.2	Register access protection	383
16.3.3	Debug mode	383
16.4	IWDG registers	384
16.4.1	Key register (IWDG_KR)	384
16.4.2	Prescaler register (IWDG_PR)	384
16.4.3	Reload register (IWDG_RLR)	385
16.4.4	Status register (IWDG_SR)	385
16.4.5	IWDG register map	386
17	Window watchdog (WWDG)	387
17.1	WWDG introduction	387
17.2	WWDG main features	387
17.3	WWDG functional description	387
17.4	How to program the watchdog timeout	389
17.5	Debug mode	389
17.6	WWDG registers	390
17.6.1	Control register (WWDG_CR)	390
17.6.2	Configuration register (WWDG_CFR)	391
17.6.3	Status register (WWDG_SR)	391
17.6.4	WWDG register map	392
18	Universal serial bus full-speed device interface (USB)	393
18.1	USB introduction	393
18.2	USB main features	393
18.3	USB functional description	393
18.3.1	Description of USB blocks	395
18.4	Programming considerations	396
18.4.1	Generic USB device programming	396
18.4.2	System and power-on reset	397
18.4.3	Double-buffered endpoints	403

18.4.4	Isochronous transfers	405
18.4.5	Suspend/Resume events	406
18.5	USB registers	408
18.5.1	Common registers	409
18.5.2	Endpoint-specific registers	416
18.5.3	Buffer descriptor table	420
18.5.4	USB register map	423
19	CRC calculation unit	425
19.1	CRC introduction	425
19.2	CRC main features	425
19.3	CRC functional description	425
19.4	CRC registers	426
19.4.1	Data register (CRC_DR)	426
19.4.2	Independent data register (CRC_IDR)	426
19.4.3	Control register (CRC_CR)	427
19.4.4	CRC register map	427
20	Real-time clock (RTC)	428
20.1	Introduction	428
20.2	RTC main features	429
20.3	RTC functional description	430
20.3.1	Clock and prescalers	430
20.3.2	Real-time clock and calendar	431
20.3.3	Programmable alarms	431
20.3.4	Periodic auto-wakeup	432
20.3.5	RTC initialization and configuration	432
20.3.6	Reading the calendar	434
20.3.7	Resetting the RTC	434
20.3.8	RTC reference clock detection	435
20.3.9	RTC digital calibration	435
20.3.10	Time-stamp function	436
20.3.11	Tamper detection	437
20.3.12	Calibration clock output	438
20.3.13	Alarm output	438
20.4	RTC and low power modes	438

20.5	RTC interrupts	439
20.6	RTC registers	440
20.6.1	RTC time register (RTC_TR)	440
20.6.2	RTC date register (RTC_DR)	441
20.6.3	RTC control register (RTC_CR)	442
20.6.4	RTC initialization and status register (RTC_ISR)	444
20.6.5	RTC prescaler register (RTC_PRER)	446
20.6.6	RTC wakeup timer register (RTC_WUTR)	447
20.6.7	RTC calibration register (RTC_CALIBR)	447
20.6.8	RTC alarm A register (RTC_ALRMAR)	448
20.6.9	RTC alarm B register (RTC_ALRMBR)	449
20.6.10	RTC write protection register (RTC_WPR)	450
20.6.11	RTC time stamp time register (RTC_TSTR)	451
20.6.12	RTC time stamp date register (RTC_TSDR)	451
20.6.13	RTC tamper and alternate function configuration register (RTC_TAFCR)	452
20.6.14	RTC backup registers (RTC_BKPxR)	453
20.6.15	Register map	453
21	Inter-integrated circuit (I²C) interface	455
21.1	I ² C introduction	455
21.2	I ² C main features	455
21.3	I ² C functional description	456
21.3.1	Mode selection	456
21.3.2	I2C slave mode	458
21.3.3	I2C master mode	460
21.3.4	Error conditions	466
21.3.5	SDA/SCL line control	467
21.3.6	SMBus	467
21.3.7	DMA requests	469
21.3.8	Packet error checking	471
21.4	I ² C interrupts	471
21.5	I ² C debug mode	473
21.6	I ² C registers	473
21.6.1	Control register 1 (I2C_CR1)	473
21.6.2	Control register 2 (I2C_CR2)	475
21.6.3	Own address register 1 (I2C_OAR1)	476

	21.6.4	Own address register 2 (I2C_OAR2)	477
	21.6.5	Data register (I2C_DR)	477
	21.6.6	Status register 1 (I2C_SR1)	478
	21.6.7	Status register 2 (I2C_SR2)	481
	21.6.8	Clock control register (I2C_CCR)	482
	21.6.9	TRISE register (I2C_TRISE)	483
	21.6.10	I2C register map	484
22		Serial peripheral interface (SPI)	485
	22.1	SPI introduction	485
	22.2	SPI main features	486
	22.2.1	SPI features	486
	22.3	SPI functional description	486
	22.3.1	General description	486
	22.3.2	Configuring the SPI in slave mode	490
	22.3.3	Configuring the SPI in master mode	492
	22.3.4	Configuring the SPI for Simplex communication	493
	22.3.5	Data transmission and reception procedures	493
	22.3.6	CRC calculation	500
	22.3.7	Status flags	502
	22.3.8	Disabling the SPI	503
	22.3.9	SPI communication using DMA (direct memory addressing)	504
	22.3.10	Error flags	505
	22.3.11	SPI interrupts	506
	22.4	SPI registers	507
	22.4.1	SPI control register 1 (SPI_CR1)	507
	22.4.2	SPI control register 2 (SPI_CR2)	509
	22.4.3	SPI status register (SPI_SR)	510
	22.4.4	SPI data register (SPI_DR)	511
	22.4.5	SPI CRC polynomial register (SPI_CRCPR)	511
	22.4.6	SPI RX CRC register (SPI_RXCRCR)	512
	22.4.7	SPI TX CRC register (SPI_TXCRCR)	512
	22.4.8	SPI register map	512
23		Universal synchronous asynchronous receiver transmitter (USART)	514
	23.1	USART introduction	514

23.2	USART main features	514
23.3	USART functional description	515
23.3.1	USART character description	518
23.3.2	Transmitter	519
23.3.3	Receiver	522
23.3.4	Fractional baud rate generation	526
23.3.5	USART receiver's tolerance to clock deviation	533
23.3.6	Multiprocessor communication	534
23.3.7	Parity control	536
23.3.8	LIN (local interconnection network) mode	537
23.3.9	USART synchronous mode	539
23.3.10	Single-wire half-duplex communication	541
23.3.11	Smartcard	542
23.3.12	IrDA SIR ENDEC block	544
23.3.13	Continuous communication using DMA	546
23.3.14	Hardware flow control	548
23.4	USART interrupts	550
23.5	USART mode configuration	551
23.6	USART registers	551
23.6.1	Status register (USART_SR)	551
23.6.2	Data register (USART_DR)	553
23.6.3	Baud rate register (USART_BRR)	553
23.6.4	Control register 1 (USART_CR1)	554
23.6.5	Control register 2 (USART_CR2)	557
23.6.6	Control register 3 (USART_CR3)	558
23.6.7	Guard time and prescaler register (USART_GTPR)	560
23.6.8	USART register map	561
24	Debug support (DBG)	562
24.1	Overview	562
24.2	Reference ARM documentation	563
24.3	SWJ debug port (serial wire and JTAG)	563
24.3.1	Mechanism to select the JTAG-DP or the SW-DP	564
24.4	Pinout and debug port pins	564
24.4.1	SWJ debug port pins	565
24.4.2	Flexible SWJ-DP pin assignment	565

24.4.3	Internal pull-up and pull-down on JTAG pins	566
24.4.4	Using serial wire and releasing the unused debug pins as GPIOs	567
24.5	STM32L15xxx JTAG TAP connection	567
24.6	ID codes and locking mechanism	568
24.6.1	MCU device ID code	568
24.6.2	Boundary scan TAP	569
24.6.3	Cortex-M3 TAP	569
24.6.4	Cortex-M3 JEDEC-106 ID code	569
24.7	JTAG debug port	569
24.8	SW debug port	571
24.8.1	SW protocol introduction	571
24.8.2	SW protocol sequence	571
24.8.3	SW-DP state machine (reset, idle states, ID code)	572
24.8.4	DP and AP read/write accesses	573
24.8.5	SW-DP registers	573
24.8.6	SW-AP registers	574
24.9	AHB-AP (AHB access port) - valid for both JTAG-DP and SW-DP	574
24.10	Core debug	575
24.11	Capability of the debugger host to connect under system reset	576
24.12	FPB (Flash patch breakpoint)	576
24.13	DWT (data watchpoint trigger)	576
24.14	ITM (instrumentation trace macrocell)	577
24.14.1	General description	577
24.14.2	Time stamp packets, synchronization and overflow packets	577
24.15	ETM (Embedded trace macrocell)	579
24.15.1	General description	579
24.15.2	Signal protocol, packet types	579
24.15.3	Main ETM registers	579
24.15.4	Configuration example	580
24.16	MCU debug component (DBGMCU)	580
24.16.1	Debug support for low-power modes	580
24.16.2	Debug support for timers, watchdog and I ² C	580
24.16.3	Debug MCU configuration register	581
24.16.4	Debug MCU APB1 freeze register (DBGMCU_APB1_FZ)	582
24.16.5	Debug MCU APB2 freeze register (DBGMCU_APB2_FZ)	584

24.17	TPIU (trace port interface unit)	585
24.17.1	Introduction	585
24.17.2	TRACE pin assignment	585
24.17.3	TPUI formatter	587
24.17.4	TPUI frame synchronization packets	588
24.17.5	Transmission of the synchronization frame packet	588
24.17.6	Synchronous mode	588
24.17.7	Asynchronous mode	589
24.17.8	TRACECLKIN connection inside the STM32L15xxx	589
24.17.9	TPIU registers	590
24.17.10	Example of configuration	591
24.18	DBG register map	591
25	Device electronic signature	592
25.1	Memory size register	592
25.1.1	Flash size register	592
25.2	Unique device ID registers (96 bits)	592
26	Revision history	594

List of tables

Table 1.	Register boundary addresses	32
Table 2.	Flash module organization	36
Table 3.	Number of wait states (WS) according to CPU clock (HCLK) frequency	37
Table 5.	Flash interface register map and reset values	41
Table 6.	Boot modes	41
Table 7.	Memory mapping vs. boot mode/physical remap	42
Table 8.	Performance versus VCORE ranges	47
Table 9.	Summary of low-power modes	55
Table 10.	Sleep-now	59
Table 11.	Sleep-on-exit	59
Table 12.	Sleep-now	61
Table 13.	Sleep-on-exit	61
Table 14.	Stop mode	62
Table 15.	Standby mode	63
Table 16.	PWR - register map and reset values	69
Table 17.	RCC register map and reset values	106
Table 18.	Port bit configuration table	110
Table 19.	Flexible SWJ-DP pin assignment	111
Table 20.	RTC_AF1 pin	118
Table 21.	GPIO register map and reset values	124
Table 22.	I/O groups and selection	128
Table 23.	Input capture mapping	130
Table 24.	Timer selection	131
Table 25.	Input capture selection	131
Table 26.	RI register map and reset values	142
Table 27.	SYSCFG register map and reset values	147
Table 28.	Vector table	148
Table 29.	External interrupt/event controller register map and reset values	157
Table 30.	Programmable data width & endian behavior (when bits PINC = MINC = 1)	162
Table 31.	DMA interrupt requests	163
Table 32.	Summary of DMA requests for each channel	165
Table 33.	DMA register map and reset values	171
Table 34.	ADC pins	175
Table 35.	Analog watchdog channel selection	179
Table 36.	Configuring the trigger edge detection	183
Table 37.	External trigger for regular channels	183
Table 38.	External trigger for injected channels	184
Table 39.	ADC interrupts	194
Table 40.	ADC global register map	211
Table 41.	ADC register map and reset values	211
Table 42.	ADC register map and reset values (common registers)	212
Table 43.	DAC pins	214
Table 44.	External triggers	217
Table 45.	DAC register map	233
Table 46.	Comparator behavior in the low power modes	239
Table 47.	COMP register map and reset values	242
Table 48.	Example of frame rate calculation	246

Table 49.	Blink frequency	255
Table 50.	Remapping capability	258
Table 51.	LCD register map and reset values	268
Table 52.	Counting direction versus encoder signals	296
Table 53.	TIMx internal trigger connection	311
Table 54.	Output control bit for standard OCx channels	320
Table 55.	TIM2 to TIM4 register map and reset values	326
Table 56.	TIMx internal trigger connection	357
Table 57.	Output control bit for standard OCx channels	365
Table 58.	TIM9 register map and reset values	368
Table 59.	TIM10/11 register map and reset values	370
Table 60.	TIM6&TIM7 register map and reset values	381
Table 61.	Min/max IWDG timeout period at 37 kHz (LSI)	383
Table 62.	IWDG register map and reset values	386
Table 63.	WWDG register map and reset values	392
Table 64.	Double-buffering buffer flag definition	404
Table 65.	Bulk double-buffering memory buffers usage	404
Table 66.	Isochronous memory buffers usage	406
Table 67.	Resume event detection	407
Table 68.	Reception status encoding	419
Table 69.	Endpoint type encoding	419
Table 70.	Endpoint kind meaning	419
Table 71.	Transmission status encoding	420
Table 72.	Definition of allocated buffer memory	423
Table 73.	USB register map and reset values	423
Table 74.	CRC calculation unit register map and reset values	427
Table 75.	Effect of low power modes on RTC	438
Table 76.	Interrupt control bits	439
Table 77.	RTC register map and reset values	453
Table 78.	SMBus vs. I2C	467
Table 79.	I2C Interrupt requests	472
Table 80.	I2C register map and reset values	484
Table 81.	SPI interrupt requests	506
Table 82.	SPI register map and reset values	513
Table 83.	Noise detection from sampled data	525
Table 84.	Error calculation for programmed baud rates at $f_{PCLK} = 8 \text{ MHz}$ or $f_{PCLK} = 12 \text{ MHz}$, oversampling by 16.	528
Table 85.	Error calculation for programmed baud rates at $f_{PCLK} = 8 \text{ MHz}$ or $f_{PCLK} = 12 \text{ MHz}$, oversampling by 8.	529
Table 86.	Error calculation for programmed baud rates at $f_{PCLK} = 16 \text{ MHz}$ or $f_{PCLK} = 24 \text{ MHz}$, oversampling by 16.	529
Table 87.	Error calculation for programmed baud rates at $f_{PCLK} = 16 \text{ MHz}$ or $f_{PCLK} = 24 \text{ MHz}$, oversampling by 8.	530
Table 88.	Error calculation for programmed baud rates at $f_{PCLK} = 1 \text{ MHz}$ or $f_{PCLK} = 8 \text{ MHz}$, oversampling by 16.	531
Table 89.	Error calculation for programmed baud rates at $f_{PCLK} = 1 \text{ MHz}$ or $f_{PCLK} = 8 \text{ MHz}$, oversampling by 8.	531
Table 90.	Error calculation for programmed baud rates at $f_{PCLK} = 16 \text{ MHz}$ or $f_{PCLK} = 32 \text{ MHz}$, oversampling by 16.	532
Table 91.	Error calculation for programmed baud rates at $f_{PCLK} = 16 \text{ MHz}$ or $f_{PCLK} = 32 \text{ MHz}$, oversampling by 8.	533
Table 92.	USART receiver's tolerance when DIV fraction is 0	534

Table 93.	USART receiver's tolerance when DIV_Fraction is different from 0	534
Table 94.	Frame formats	536
Table 95.	USART interrupt requests.	550
Table 96.	USART register map and reset values	561
Table 97.	SWJ debug port pins	565
Table 98.	Flexible SWJ-DP pin assignment	565
Table 99.	JTAG debug port data registers	569
Table 100.	32-bit debug port registers addressed through the shifted value A[3:2]	570
Table 101.	Packet request (8-bits)	571
Table 102.	ACK response (3 bits).	572
Table 103.	DATA transfer (33 bits).	572
Table 104.	SW-DP registers	573
Table 105.	Cortex-M3 AHB-AP registers	575
Table 106.	Core debug registers	575
Table 107.	Main ITM registers	578
Table 108.	Main ETM registers.	579
Table 109.	Asynchronous TRACE pin assignment.	585
Table 110.	Synchronous TRACE pin assignment	586
Table 111.	Flexible TRACE pin assignment	587
Table 112.	Important TPIU registers.	590
Table 113.	DBG register map and reset values	591
Table 114.	Document revision history	594

List of figures

Figure 1.	System architecture	30
Figure 2.	Power supply overview	44
Figure 3.	STM32L15xxx performance versus VDD and VCORE range.	48
Figure 4.	Power supply supervisors	51
Figure 5.	Power on reset/power down reset waveform	52
Figure 6.	BOR thresholds	53
Figure 7.	PVD thresholds	54
Figure 8.	Simplified diagram of the reset circuit.	71
Figure 9.	Clock tree	73
Figure 10.	HSE/ LSE clock sources	74
Figure 11.	System clock source frequency	78
Figure 12.	Using the TIM9/TIM10/TIM11 channel 1 input capture to measure frequencies	80
Figure 13.	Basic structure of a standard I/O port bit	109
Figure 14.	Basic structure of a five-volt tolerant I/O port bit.	109
Figure 15.	Selecting an alternate function	113
Figure 16.	Input floating/pull up/pull down configurations	115
Figure 17.	Output configuration	116
Figure 18.	Alternate function configuration	117
Figure 19.	High impedance-analog configuration	117
Figure 20.	Routing interface (RI) block diagram	127
Figure 21.	Internal reference voltage output	131
Figure 22.	External interrupt/event controller block diagram	151
Figure 23.	External interrupt/event GPIO mapping	153
Figure 24.	DMA block diagram in STM32L15xxx devices	159
Figure 25.	DMA request mapping	164
Figure 26.	ADC block diagram	174
Figure 27.	Timing diagram (normal mode, PDI=0).	178
Figure 28.	Analog watchdog's guarded area	179
Figure 29.	Injected conversion latency	180
Figure 30.	Right alignment of 12-bit data	182
Figure 31.	Left alignment of 12-bit data	182
Figure 32.	Left alignment of 6-bit data	182
Figure 33.	ADC freeze mode	186
Figure 34.	Continuous regular conversions with a delay	187
Figure 35.	Continuous conversions with a delay between each conversion	188
Figure 36.	Automatic power-down control: example 1	189
Figure 37.	Automatic power-down control: example 2	189
Figure 38.	Automatic power-down control: example 3	190
Figure 39.	Temperature sensor and VREFINT channel block diagram	192
Figure 40.	ADC flags and interrupts	193
Figure 41.	DAC channel block diagram	214
Figure 42.	Data registers in single DAC channel mode	215
Figure 43.	Data registers in dual DAC channel mode	216
Figure 44.	Timing diagram for conversion with trigger disabled $TEN = 0$	216
Figure 45.	DAC LFSR register calculation algorithm	218
Figure 46.	DAC conversion (SW trigger enabled) with LFSR wave generation.	219
Figure 47.	DAC triangle wave generation	219

Figure 48.	DAC conversion (SW trigger enabled) with triangle wave generation	220
Figure 49.	Comparator block diagram	235
Figure 50.	COMP1 interconnections	236
Figure 51.	COMP2 interconnections	237
Figure 52.	Redirecting the COMP2 output	238
Figure 53.	Comparators in Window mode	239
Figure 54.	LCD controller block diagram	245
Figure 55.	1/3 bias, 1/4 duty	248
Figure 56.	Static duty	249
Figure 57.	Static duty	249
Figure 58.	1/2 duty, 1/2 bias	250
Figure 59.	1/3 duty, 1/3 bias	252
Figure 60.	1/4 duty, 1/3 bias	253
Figure 61.	1/8 duty, 1/4 bias	254
Figure 62.	VLCD pin for 1/2 1/3 1/4 bias	256
Figure 63.	Deadtime	256
Figure 64.	SEG/COM mux feature example	260
Figure 65.	Flowchart example	261
Figure 66.	General-purpose timer block diagram	271
Figure 67.	Counter timing diagram with prescaler division change from 1 to 2	272
Figure 68.	Counter timing diagram with prescaler division change from 1 to 4	273
Figure 69.	Counter timing diagram, internal clock divided by 1	274
Figure 70.	Counter timing diagram, internal clock divided by 2	274
Figure 71.	Counter timing diagram, internal clock divided by 4	274
Figure 72.	Counter timing diagram, internal clock divided by N	275
Figure 73.	Counter timing diagram, Update event when ARPE=0 (TIMx_ARR not preloaded)	275
Figure 74.	Counter timing diagram, Update event when ARPE=1 (TIMx_ARR preloaded)	276
Figure 75.	Counter timing diagram, internal clock divided by 1	277
Figure 76.	Counter timing diagram, internal clock divided by 2	277
Figure 77.	Counter timing diagram, internal clock divided by 4	277
Figure 78.	Counter timing diagram, internal clock divided by N	278
Figure 79.	Counter timing diagram, Update event when repetition counter is not used	278
Figure 80.	Counter timing diagram, internal clock divided by 1, TIMx_ARR=0x6	279
Figure 81.	Counter timing diagram, internal clock divided by 2	280
Figure 82.	Counter timing diagram, internal clock divided by 4, TIMx_ARR=0x36	280
Figure 83.	Counter timing diagram, internal clock divided by N	280
Figure 84.	Counter timing diagram, Update event with ARPE=1 (counter underflow)	281
Figure 85.	Counter timing diagram, Update event with ARPE=1 (counter overflow)	281
Figure 86.	Control circuit in normal mode, internal clock divided by 1	282
Figure 87.	TI2 external clock connection example	283
Figure 88.	Control circuit in external clock mode 1	283
Figure 89.	External trigger input block	284
Figure 90.	Control circuit in external clock mode 2	284
Figure 91.	Capture/compare channel (example: channel 1 input stage)	285
Figure 92.	Capture/compare channel 1 main circuit	285
Figure 93.	Output stage of capture/compare channel (channel 1)	286
Figure 94.	PWM input mode timing	288
Figure 95.	Output compare mode, toggle on OC1	290
Figure 96.	Edge-aligned PWM waveforms (ARR=8)	291
Figure 97.	Center-aligned PWM waveforms (ARR=8)	292
Figure 98.	Example of one-pulse mode	293
Figure 99.	Clearing TIMx_OCxREF	295

Figure 100.	Example of counter operation in encoder interface mode.	296
Figure 101.	Example of encoder interface mode with IC1FP1 polarity inverted.	297
Figure 102.	Control circuit in reset mode.	298
Figure 103.	Control circuit in gated mode.	299
Figure 104.	Control circuit in trigger mode.	299
Figure 105.	Control circuit in external clock mode 2 + trigger mode.	300
Figure 106.	Master/Slave timer example.	301
Figure 107.	Gating TIM2 with OC1REF of TIM3.	302
Figure 108.	Gating TIM2 with Enable of TIM3.	303
Figure 109.	Triggering TIM2 with update of TIM3.	303
Figure 110.	Triggering TIM2 with Enable of TIM3.	304
Figure 111.	Triggering TIM3 and TIM2 with TIM3 TI1 input.	305
Figure 112.	General-purpose timer block diagram.	329
Figure 113.	General-purpose timer block diagram (TIM10).	330
Figure 114.	General-purpose timer block diagram (TIM11).	330
Figure 115.	Counter timing diagram with prescaler division change from 1 to 2.	332
Figure 116.	Counter timing diagram with prescaler division change from 1 to 4.	332
Figure 117.	Counter timing diagram, internal clock divided by 1.	333
Figure 118.	Counter timing diagram, internal clock divided by 2.	333
Figure 119.	Counter timing diagram, internal clock divided by 4.	334
Figure 120.	Counter timing diagram, internal clock divided by N.	334
Figure 121.	Counter timing diagram, update event when ARPE=0 (TIMx_ARR not preloaded).	334
Figure 122.	Counter timing diagram, update event when ARPE=1 (TIMx_ARR preloaded).	335
Figure 123.	Control circuit in normal mode, internal clock divided by 1.	336
Figure 124.	TI2 external clock connection example.	336
Figure 125.	External trigger input block.	337
Figure 126.	Control circuit in external clock mode 2.	337
Figure 127.	Capture/compare channel (example: channel 1 input stage).	338
Figure 128.	Capture/compare channel 1 main circuit.	338
Figure 129.	Output stage of capture/compare channel (channel 1).	339
Figure 130.	PWM input mode timing.	341
Figure 131.	Output compare mode, toggle on OC1.	342
Figure 132.	Edge-aligned PWM waveforms (ARR=8).	343
Figure 133.	Example of one-pulse mode.	344
Figure 134.	Control circuit in Reset mode.	346
Figure 135.	Control circuit in Gated mode.	346
Figure 136.	Control circuit in Trigger mode.	347
Figure 137.	Master/Slave timer example.	348
Figure 138.	Gating TIM2 with the OC1REF of TIM9.	349
Figure 139.	Gating TIM2 with the Enable of TIM9.	350
Figure 140.	Triggering TIM2 with the update of TIM9.	350
Figure 141.	Triggering TIM2 with the Enable of TIM9.	351
Figure 142.	Triggering TIM9 and TIM2 with TIM9's TI1 input.	352
Figure 143.	Basic timer block diagram.	371
Figure 144.	Counter timing diagram with prescaler division change from 1 to 2.	373
Figure 145.	Counter timing diagram with prescaler division change from 1 to 4.	373
Figure 146.	Counter timing diagram, internal clock divided by 1.	374
Figure 147.	Counter timing diagram, internal clock divided by 2.	374
Figure 148.	Counter timing diagram, internal clock divided by 4.	375
Figure 149.	Counter timing diagram, internal clock divided by N.	375
Figure 150.	Counter timing diagram, update event when ARPE = 0 (TIMx_ARR not preloaded).	375

Figure 151. Counter timing diagram, update event when ARPE=1 (TIMx_ARR preloaded).	376
Figure 152. Control circuit in normal mode, internal clock divided by 1.	376
Figure 153. Independent watchdog block diagram	383
Figure 154. Watchdog block diagram	388
Figure 155. Window watchdog timing diagram	389
Figure 156. USB peripheral block diagram	394
Figure 157. Packet buffer areas with examples of buffer description table locations	399
Figure 158. CRC calculation unit block diagram	425
Figure 159. RTC block diagram	430
Figure 160. I2C bus protocol	457
Figure 161. I2C block diagram.	457
Figure 162. Transfer sequence diagram for slave transmitter	459
Figure 163. Transfer sequence diagram for slave receiver	460
Figure 164. Transfer sequence diagram for master transmitter.	463
Figure 165. Transfer sequence diagram for master receiver.	465
Figure 166. I2C interrupt mapping diagram	472
Figure 167. SPI block diagram.	487
Figure 168. Single master/ single slave application.	488
Figure 169. Hardware/software slave select management	488
Figure 170. Data clock timing diagram	490
Figure 171. TXE/RXNE/BSY behavior in Master / full-duplex mode (BIDIMODE=0 and RXONLY=0) in the case of continuous transfers.	496
Figure 172. TXE/RXNE/BSY behavior in Slave / full-duplex mode (BIDIMODE=0, RXONLY=0) in the case of continuous transfers.	496
Figure 173. TXE/BSY behavior in Master transmit-only mode (BIDIMODE=0 and RXONLY=0) in the case of continuous transfers.	497
Figure 174. TXE/BSY in Slave transmit-only mode (BIDIMODE=0 and RXONLY=0) in the case of continuous transfers	498
Figure 175. RXNE behavior in receive-only mode (BIDIRMODE=0 and RXONLY=1) in the case of continuous transfers	499
Figure 176. TXE/BSY behavior when transmitting (BIDIRMODE=0 and RXONLY=0) in the case of discontinuous transfers.	500
Figure 177. Transmission using DMA	504
Figure 178. Reception using DMA.	505
Figure 179. USART block diagram	517
Figure 180. Word length programming	518
Figure 181. Configurable stop bits.	520
Figure 182. TC/TXE behavior when transmitting.	521
Figure 183. Start bit detection when oversampling by 16 or 8.	522
Figure 184. Data sampling when oversampling by 16.	525
Figure 185. Data sampling when oversampling by 8.	525
Figure 186. Mute mode using Idle line detection.	535
Figure 187. Mute mode using address mark detection	535
Figure 188. Break detection in LIN mode (11-bit break length - LBDL bit is set).	538
Figure 189. Break detection in LIN mode vs. Framing error detection.	539
Figure 190. USART example of synchronous transmission.	540
Figure 191. USART data clock timing diagram (M=0)	540
Figure 192. USART data clock timing diagram (M=1)	541
Figure 193. RX data setup/hold time	541
Figure 194. ISO 7816-3 asynchronous protocol	542
Figure 195. Parity error detection using the 1.5 stop bits	543

Figure 196. IrDA SIR ENDEC- block diagram	545
Figure 197. IrDA data modulation (3/16) -Normal mode	545
Figure 198. Transmission using DMA	547
Figure 199. Reception using DMA	548
Figure 200. Hardware flow control between 2 USARTs	548
Figure 201. RTS flow control	549
Figure 202. CTS flow control	549
Figure 203. USART interrupt mapping diagram	550
Figure 204. Block diagram of STM32L15xxx-level and Cortex-M3-level debug support	562
Figure 205. SWJ debug port	564
Figure 206. JTAG TAP connections	568
Figure 207. TPIU block diagram	585

1 Documentation conventions

1.1 List of abbreviations for registers

The following abbreviations are used in register descriptions:

read/write (rw)	Software can read and write to these bits.
read-only (r)	Software can only read these bits.
write-only (w)	Software can only write to this bit. Reading the bit returns the reset value.
read/clear (rc_w1)	Software can read as well as clear this bit by writing 1. Writing '0 has no effect on the bit value.
read/clear (rc_w0)	Software can read as well as clear this bit by writing 0. Writing 1 has no effect on the bit value.
read/clear by read (rc_r)	Software can read this bit. Reading this bit automatically clears it to '0. Writing '0 has no effect on the bit value.
read/set (rs)	Software can read as well as set this bit. Writing '0 has no effect on the bit value.
read-only write trigger (rt_w)	Software can read this bit. Writing '0 or '1 triggers an event but has no effect on the bit value.
toggle (t)	Software can only toggle this bit by writing '1. Writing '0 has no effect.
Reserved (Res.)	Reserved bit, must be kept at reset value.

1.2 Peripheral availability

For the peripherals available, and their number, across all STM32L15xxx sales types, please refer to the STM32L15xxx datasheet.

2 Memory and bus architecture

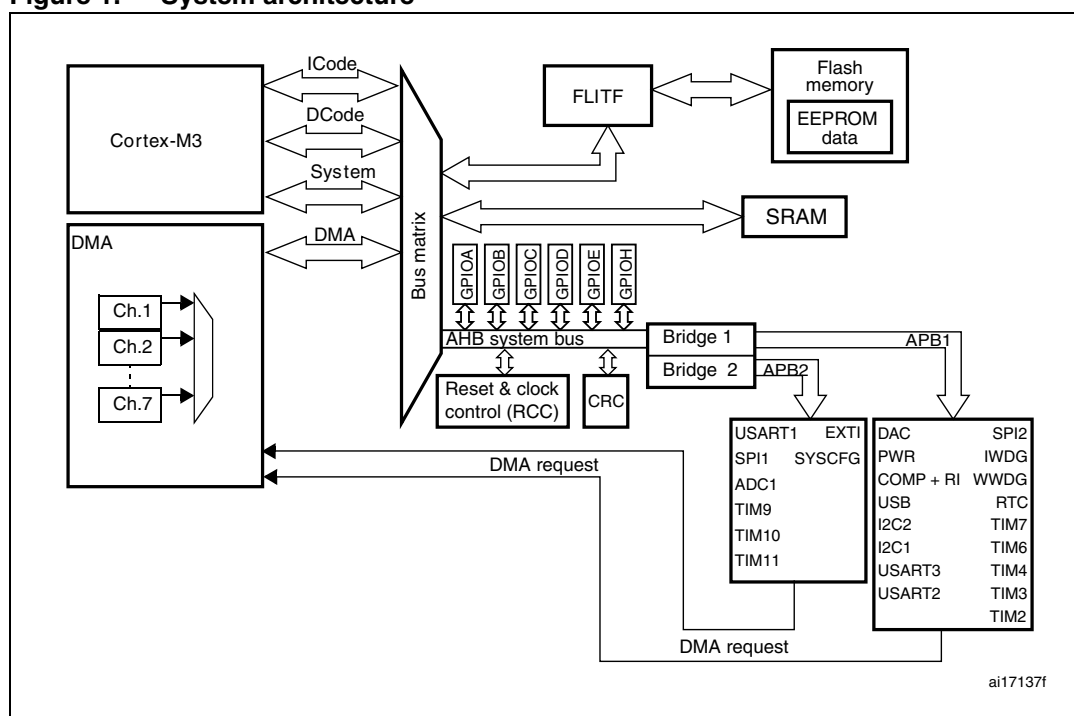
2.1 System architecture

The main system consists of a 32-bit multilayer AHB bus matrix that interconnects:

- Four masters:
 - Cortex-M3 I-bus, D-bus and S-bus
 - DMA
- Three slaves:
 - Internal Flash memory
 - Internal SRAM
 - AHB to APBx (APB1 or APB2), which connect all the APB peripherals

These are interconnected using the multilayer AHB bus architecture shown in [Figure 1](#):

Figure 1. System architecture



ICode bus

This bus connects the Instruction bus of the Cortex-M3 core to the BusMatrix. This bus is used by the core to fetch instructions. The target of this bus is a memory containing code (internal Flash memory or SRAM).

DCode bus

This bus connects the databus of the Cortex-M3 to the BusMatrix. This bus is used by the core for literal load and debug access. The target of this bus is a memory containing code or data (internal Flash memory or SRAM).

System bus

This bus connects the system bus of the Cortex-M3 core to a BusMatrix. This bus is used to access data located in a peripheral or in SRAM. Instructions may also be fetched on this bus (less efficient than ICode). The targets of this bus are the internal SRAM and the AHB/APB bridges.

DMA bus

This bus connects the AHB master interface of the DMA to the bus matrix which manages the access of the CPU DCode and DMA to the SRAM, Flash memory and peripherals.

Bus matrix

The bus matrix manages the access arbitration between the core system bus and the DMA master bus. The arbitration uses a round robin algorithm. The bus matrix is composed of four masters (ICode, DCode, System bus, DMA1 bus,) and three slaves (Flash interface, SRAM, FSMC, AES and AHB2APB bridges).

AHB peripherals are connected on the system bus through the bus matrix to allow DMA access.

AHB/APB bridges (APB)

The two AHB/APB bridges provide full synchronous connections between the AHB and the 2 APB buses. The two APB buses operates at full speed (up to 32 MHz).

Refer to [Table 1 on page 32](#) for the address mapping of the AHB and APB peripherals.

After each device reset, all peripheral clocks are disabled (except for the SRAM and Flash interface). Before using a peripheral, its clock should be enabled in the RCC_AHBENR, RCC_APB1ENR or RCC_APB2ENR register.

Note: When a 16- or 8-bit access is performed on an APB register, the access is transformed into a 32-bit access: the bridge duplicates the 16- or 8-bit data to feed the 32-bit vector.

2.2 Memory organization

Flash program memory, EEPROM data memory, SRAM data memory, registers and I/O ports are organized within the same linear 4 Gbyte address space.

The bytes are coded in memory in little endian format. The lowest numbered byte in a word is considered the word's least significant byte and the highest numbered byte, the most significant.

For the detailed mapping of peripheral registers, please refer to the related sections.

The addressable memory space is divided into 8 main blocks, each of 512 Mbytes.

All the memory areas that are not allocated to on-chip memories and peripherals are considered "Reserved"). Refer to the memory map figure in the STM32L15xxx datasheet.

2.3 Memory map

See the STM32L15xxx datasheet for a comprehensive diagram of the memory map. [Table 1](#) gives the boundary addresses of the peripherals available in all STM32L15xxx devices.

Table 1. Register boundary addresses

Boundary address	Peripheral	Bus	Register map
0xA000 0000 - 0xA000 0FFF	FSMC	AHB	Section 24.6.9: FSMC register map on page 651
0x5006 0000 - 0x5006 03FF	AES		Section 22.12.13: AES register map on page 564
0x4002 6000 - 0x4002 63FF	DMA		Section 8.4.7: DMA register map on page 171
0x4002 3C00 - 0x4002 3FFF	Flash memory interface		See Flash programming manual
0x4002 3800 - 0x4002 3BFF	RCC		Section 4.3.15: RCC register map on page 106
0x4002 3000 - 0x4002 33FF	CRC		Section 19.4.4: CRC register map on page 427
0x4002 1400 - 0x4002 17FF	GPIOH		Section 5.4.11: GPIO register map on page 124
0x4002 1000 - 0x4002 13FF	GPIOE		
0x4002 0C00 - 0x4002 0FFF	GPIOD		
0x4002 0800 - 0x4002 0BFF	GPIOC		
0x4002 0400 - 0x4002 07FF	GPIOB		
0x4002 0000 - 0x4002 03FF	GPIOA		
0x4001 3800 - 0x4001 3BFF	USART1	APB2	Section 23.6.8: USART register map on page 561
0x4001 3000 - 0x4001 33FF	SPI1		Section Table 82.: SPI register map and reset values on page 513
0x4001 2C00 - 0x4001 2FFF	SDIO		Section 28.9.16: SDIO register map on page 834
0x4001 2400 - 0x4001 27FF	ADC		Section 9.15.20: ADC register map on page 211
0x4001 1000 - 0x4001 13FF	TIM11		Section 14.4.17: TIMx register map on page 368
0x4001 0C00 - 0x4001 0FFF	TIM10		Section 14.4.17: TIMx register map on page 368
0x4001 0800 - 0x4001 0BFF	TIM9		Section 14.4.17: TIMx register map on page 368
0x4001 0400 - 0x4001 07FF	EXTI		Section 7.3.7: EXTI register map on page 157
0x4001 0000 - 0x4001 03FF	SYSCFG		Section 6.5.7: SYSCFG register map on page 147

Table 1. Register boundary addresses (continued)

Boundary address	Peripheral	Bus	Register map
0x4000 7C00 - 0x4000 7C03	COMP	APB1	Section 11.9.2: COMP register map on page 242
0x4000 7C04 - 0x4000 7C5B	RI		Section 6.5.7: SYSCFG register map on page 147
0x4000 7400 - 0x4000 77FF	DAC	APB1	Section 10.5.15: DAC register map on page 233
0x4000 7000 - 0x4000 73FF	PWR		Section 3.4.3: PWR register map on page 69
0x4000 6000 - 0x4000 63FF	USB device FS SRAM 512 bytes		Section 18.5.4: USB register map on page 423
0x4000 5C00 - 0x4000 5FFF	USB device FS		Section 18.5.4: USB register map on page 423
0x4000 5800 - 0x4000 5BFF	I2C2		Section 21.6.10: I2C register map on page 484
0x4000 5400 - 0x4000 57FF	I2C1		Section 21.6.10: I2C register map on page 484
0x4000 5000 - 0x4000 53FF	USART5		Section 23.6.8: USART register map on page 561
0x4000 4C00 - 0x4000 4FFF	USART4		Section 23.6.8: USART register map on page 561

Table 1. Register boundary addresses (continued)

Boundary address	Peripheral	Bus	Register map
0x4000 4800 - 0x4000 4BFF	USART3	APB1	Section 23.6.8: USART register map on page 561
0x4000 4400 - 0x4000 47FF	USART2		Section 23.6.8: USART register map on page 561
0x4000 3800 - 0x3C00 3FFF	SPI3		Section Table 82.: SPI register map and reset values on page 513
0x4000 3800 - 0x4000 3BFF	SPI2		Section Table 82.: SPI register map and reset values on page 513
0x4000 3000 - 0x4000 33FF	IWDG		Section 16.4.5: IWDG register map on page 386
0x4000 2C00 - 0x4000 2FFF	WWDG		Section 17.6.4: WWDG register map on page 392
0x4000 2800 - 0x4000 2BFF	RTC		Section 20.6.15: Register map on page 453
0x4000 2400 - 0x4000 27FF	LCD		Section 12.5.6: LCD register map on page 268
0x4000 1400 - 0x4000 17FF	TIM7		Section 15.4.9: TIM6&TIM7 register map on page 381
0x4000 1000 - 0x4000 13FF	TIM6		Section 15.4.9: TIM6&TIM7 register map on page 381
0x4000 0C00 - 0x4000 0FFF	TIM5 (32-bits)		Section 13.4.19: TIMx register map on page 326
0x4000 0800 - 0x4000 0BFF	TIM4		Section 13.4.19: TIMx register map on page 326
0x4000 0400 - 0x4000 07FF	TIM3		Section 13.4.19: TIMx register map on page 326
0x4000 0000 - 0x4000 03FF	TIM2		Section 13.4.19: TIMx register map on page 326

2.3.1 Embedded SRAM

The STM32L15xxx features 16 Kbytes of SRAM. It can be accessed as bytes, half-words (16 bits) or full words (32 bits). The SRAM start address is 0x2000 0000.

Read and write access at CPU speed with 0 wait states.

The CPU can access the SRAM through the system bus or through the I-Code/D-Code bus when boot in SRAM is selected or when physical remap is selected (see [Section 6.5.1: SYSCFG memory remap register \(SYSCFG_MEMRMP\)](#) register in the SYSCFG controller). To get the best SRAM execution performance, physical remap must be selected (boot or software selection).

2.3.2 Bit banding

The Cortex-M3 memory map includes two bit-band regions. These regions map each word in an alias region of memory to a bit in a bit-band region of memory. Writing to a word in the alias region has the same effect as a read-modify-write operation on the targeted bit in the bit-band region.

In the STM32L15xxx both the peripheral registers and the SRAM are mapped in a bit-band region. This allows single bit-band write and read operations to be performed. These operations are only available for cortex-M3 accesses, not from other bus masters (e.g. DMA).

A mapping formula shows how to reference each word in the alias region to a corresponding bit in the bit-band region. The mapping formula is:

$$bit_word_addr = bit_band_base + (byte_offset \times 32) + (bit_number \times 4)$$

where:

bit_word_addr is the address of the word in the alias memory region that maps to the targeted bit

bit_band_base is the start address of the alias region

byte_offset is the number of the byte in the bit-band region, that contains the targeted bit

bit_number is the bit position (0-7) of the targeted bit

Example:

The following example shows how to map bit 2 of the byte located at SRAM address 0x2000 0300 in the alias region:

$$0x2200\ 6008 = 0x2200\ 0000 + (0x300 \times 32) + (2 \times 4)$$

Writing to address 0x2200 6008 has the same effect as a read-modify-write operation on bit 2 of the byte at SRAM address 0x2000 0300.

Reading address 0x22006008 returns the value (0x01 or 0x00) of bit 2 of the byte at SRAM address 0x20000300 (0x01: bit set; 0x00: bit reset).

For more information on bit-banding, please refer to the *Cortex-M3 Technical Reference Manual*.

2.3.3 Embedded Flash memory

- Up to 132 Kbytes of Flash memory
- Memory organization (dual banks):
 - 128 Kbytes of Flash program memory and 412 Kbytes of data EEPROM
 - 4 Kbytes of system memory and 32 bytes of option bytes

Flash memory interface (FLITF) features:

- Flash memory read operations: read access is performed by 64 or 32 bits
- Flash memory program/erase operations
- Read/write protection
- Write access is performed by 32 bits
- Option byte loader reset
- Low power mode:
 - Flash memory in Power down mode when the STM32L15xxx is in the Standby mode or the Stop mode
 - Flash memory can be placed in Power down or Idle mode when the STM32L15xxx is in the Sleep mode
 - Flash memory can be placed in Power down or Idle mode when the STM32L15xxx is in the Run mode

Flash module organization

The memory is organized as a main program memory block, a data memory block of 512 double words, and an information block. [Table 2](#) shows the Flash memory organization.

The program memory block is divided into 32 sectors of 4 Kbytes each, and each sector is further split up into 16 pages of 256 bytes each. The sector is the write protection granularity. In total, the program memory block contains 512 pages.

Table 2. Flash module organization

Block	Flash memory addresses	Size	Name	Description
Program memory	0x0800 0000 - 0x0800 00FF	256 bytes	Page 0	Sector 0
	0x0800 0100 - 0x0800 01FF	256 bytes	Page 1	
	0x0800 0200 - 0x0800 02FF	256 bytes	Page 2	
	0x0800 0300 - 0x0800 03FF	256 bytes	Page 3	
	0x0800 0400 - 0x0800 07FF	1 Kbytes	Page 4 to 7	
	0x0800 0800 - 0x0800 0BFF	1 Kbytes	Page 8 to 11	
	0x0800 0C00 - 0x0800 0FFF	1 Kbytes	Page 12 to 15	
	0x0800 1000 - 0x0800 1FFF	4 Kbytes	Page 16 to 31	Sector 1
	0x0800 2000 - 0x0800 2FFF	4 Kbytes	Page 32 to 47	Sector 2
	0x0800 3000 - 0x0800 3FFF	4 Kbytes	Page 48 to 63	Sector 3
	.	.	.	.
	.	.	.	.
	.	.	.	.
	0x0801 E000 - 0x0801 EFFF	4 Kbytes	Page 478 to 495	Sector 30
	0x0801 F000 - 0x0801 FFFF	4 Kbytes	Page 496 to 511	Sector 31
Data memory	0x0808 0000 - 0x0808 0FFF	4096 bytes	DATA	Data memory

Table 2. Flash module organization (continued)

Block	Flash memory addresses	Size	Name	Description
Information Block	0x1FF0 0000 - 0x1FF0 00FF	256 bytes	Page 0	System memory
	0x1FF0 0100 - 0x1FF0 01FF	256 bytes	Page 1	
	0x1FF0 0200 - 0x1FF0 02FF	256 bytes	Page 2	
	0x1FF0 0300 - 0x1FF0 03FF	256 bytes	Page 3	
	.	.	.	
	.	.	.	
	0x1FF0 0F00 - 0x1FF0 0FFF	256 bytes	Page 15	
	0x1FF8 0000 - 0x1FF8 001F	32 bytes	OPTB	Option bytes block

Reading the Flash memory

Relation between CPU clock frequency and Flash memory read time

The Flash memory is read by 64 bits or 32 bits.

64-bit access is configured by setting the ACC64 bit in the Flash access control register (FLASH_ACR). This access mode accelerates the execution of program operations. Prefetch is useful when the Flash memory cannot be accessed for a CPU cycle. In this case, the number of wait states (LATENCY) must be correctly programmed in the Flash access control register (FLASH_ACR) according to the frequency of the Cortex-M3 clock and the supply voltage of the device. [Table 3](#) shows the correspondence between wait states and core clock frequency.

Table 3. Number of wait states (WS) according to CPU clock (HCLK) frequency

HCLK frequency (MHz)			Wait states (LATENCY)
Voltage range 1.65 V to 3.6 V		Voltage range 2.0 V to 3.6 V	
V _{CORE} = 1.2 V	V _{CORE} = 1.5 V	V _{CORE} = 1.8 V	
0 < f _{HCLK} ≤ 2 MHz	0 < f _{HCLK} ≤ 8 MHz	0 < f _{HCLK} ≤ 16 MHz	0 WS (1 HCLK cycle)
2 < f _{HCLK} ≤ 4 MHz _{HCLK}	8 < f _{HCLK} ≤ 16 MHz	16 < f _{HCLK} ≤ 32 MHz	1 WS (2 HCLK cycles)

It is also possible to access the Flash memory by 32 bits. This is done by clearing the ACC64 bit in FLASH_ACR. In this case, prefetch has to be disabled. 32-bit access reduces the consumption, so it is used when the CPU frequency is low. In this case, the number of wait states must be 0.

After reset, the used clock is the MSI (2 MHz) with 0 WS configured in the FLASH_ACR register. 32-bit access is enabled and prefetch is disabled.

ST strongly recommends to use the following software sequences to tune the number of wait states needed to access the Flash memory with the CPU frequency.

Increasing the CPU frequency (in the same voltage range)

- Program the 64-bit access by setting the ACC64 bit in *Flash access control register (FLASH_ACR)*
- Check that 64-bit access is taken into account by reading FLASH_ACR
- Program 1 WS to the LATENCY bit in FLASH_ACR
- Check that the new number of WS is taken into account by reading FLASH_ACR
- Modify the CPU clock source by writing to the SW bits in the *Clock configuration register (RCC_CFGR)*
- If needed, modify the CPU clock prescaler by writing to the HPRE bits in RCC_CFGR
- Check that the new CPU clock source or/and the new CPU clock prescaler value is/are taken into account by reading the clock source status (SWS bits) or/and the AHB prescaler value (HPRE bits), respectively, in the RCC_CFGR register

Decreasing the CPU frequency (in the same voltage range)

- Modify the CPU clock source by writing to the SW bits in the *Clock configuration register (RCC_CFGR)*
- If needed, modify the CPU clock prescaler by writing to the HPRE bits in RCC_CFGR
- Check that the new CPU clock source or/and the new CPU clock prescaler value is/are taken into account by reading the clock source status (SWS bits) or/and the AHB prescaler value (HPRE bits), respectively, in the RCC_CFGR register
- Program the new number of WS to the LATENCY bit in *Flash access control register (FLASH_ACR)*
- Check that the new number of WS is taken into account by reading FLASH_ACR
- Program the 32-bit access by clearing ACC64 in FLASH_ACR
- Check that 32-bit access is taken into account by reading FLASH_ACR

Instruction prefetch when Flash access is 64 bits

Each Flash memory read operation provides 64 bits from either two 32-bit instructions or four 16-bit instructions. So, in case of a sequential code, at least 2 CPU cycles are needed to read the previous instruction line. Prefetch on the ICode bus can be used to read the next sequential instruction line from the Flash memory while the current instruction line is being requested by the CPU. Prefetch is enabled by setting the PRFTEN bit in the FLASH_ACR register. This feature is useful if at least one wait state is needed to access the Flash memory.

[Table 4](#) shows the supported ACC64, LATENCY and PRFTEN configurations.

Table 4. Allowed configuration in FLASH_ACR

LATENCY	ACC64 = 0		ACC64 = 1	
	PRFTEN = 0	PRFTEN = 1	PRFTEN = 0	PRFTEN = 1
0	OK	-	OK	OK
1	-	-	OK	OK

Note: The [Flash access control register \(FLASH_ACR\)](#) is used to control the Flash state in run/sleep modes, the prefetch status and access time depending on the CPU frequency. The table above provides the bit map for this register.

For complete information on Flash memory operations, please refer to the [STM32L15xxx Flash programming manual \(PM0062\)](#). For details on register configurations, refer to the following section ([Flash access control register \(FLASH_ACR\) on page 40](#)).

Flash access control register (FLASH_ACR)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																											RUN_PD	SLEEP_PD	ACC64	PRFTEN	LATENCY
																											rw	rw	rw	rw	rw

Bits 31:5 Reserved, must be kept cleared.

Bit 4 **RUN_PD:** Flash mode during Run

This bit can be written only when it is unlocked by writing to FLASH_PDKEYR.

This bit determines whether the Flash memory is in Power down mode or Idle mode when the STM32L15xxx is in Run mode.

The Flash memory can be placed in Power down mode only when the code is executed from RAM).

0: Flash in Idle mode

1: Flash in Power down mode

Bit 3 **SLEEP_PD:** Flash mode during Sleep

This bit is used to have the Flash memory in Power down mode or Idle mode when the STM32L15xxx is in Sleep mode.

0: Flash in Idle mode

1: Flash in Power down mode

Bit 2 **ACC64:** 64-bit access

This bit is used to read data from the Flash memory 64 bits or 32 bits at a time. 32-bit access is used to decrease the Flash memory consumption. On the contrary, 64-bit access is used to improve the performance. In this case it is useful to enable prefetch.

0: 32-bit access

1: 64-bit access

Note: 32-bit access is a low power mode. It is used only at low frequencies, that is with 0 wait state of latency and prefetch off.

Note: This bit cannot be written at the same time as the LATENCY and PRFTEN bits.

Bit 1 **PRFTEN:** Prefetch enable

0: prefetch disabled

1: prefetch enabled

Note: Prefetch can be enabled only when ACC64 is set.

This bit can be set or cleared only if ACC64 is set.

Bit 0 **LATENCY:** Latency

This bit represents the ratio of the CPU clock period to the Flash access time.

0: zero wait state

1: one wait state

Note: Latency can be set only when ACC64 is set.

This bit can be set or cleared only if ACC64 is set.

Flash interface register map

The following table summarizes the Flash memory registers.

Table 5. Flash interface register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x00	FLASH_ACR	Reserved																												RUN_PD	SLEEP_PD	Acc64	PRFTEN	LATENCY0
	Reset value: 0x0000 0000																													0	0	0	0	0

2.4 Boot configuration

Due to its fixed memory map, the code area starts from address 0x0000 0000 (accessed through the ICode/DCode buses) while the data area (SRAM) starts from address 0x2000 0000 (accessed through the system bus). The Cortex-M3 CPU always fetches the reset vector from the ICode bus, which implies to have the boot space available only in the code area (typically, Flash memory). STM32L15xxx microcontrollers implement a special mechanism to be able to boot from other memory than the Flash (like internal SRAM).

In the STM32L15xxx, 3 different boot modes can be selected through the BOOT[1:0] pins as shown in [Table 6](#).

Table 6. Boot modes

Boot mode selection pins		Boot mode	Aliasing
BOOT1	BOOT0		
x	0	Main Flash memory	Main Flash memory is selected as the boot space
0	1	System memory	System memory is selected as the boot space
1	1	Embedded SRAM	Embedded SRAM is selected as the boot space

The values on the BOOT pins are latched on the 4th rising edge of SYSCLK after a reset. It is up to the user to set the BOOT1 and BOOT0 pins after reset to select the required boot mode.

BOOT0 is a dedicated pin while BOOT1 is shared with a GPIO pin. Once BOOT1 has been sampled, the corresponding GPIO pin is free and can be used by the application.

The BOOT pins are also resampled when exiting the Standby mode. Consequently they must be kept in the required Boot mode configuration in Standby mode. After this startup delay has elapsed, the CPU fetches the top-of-stack value from address 0x0000 0000, then starts code execution from the boot memory starting from 0x0000 0004.

Note: When booting from SRAM, in the application initialization code, you have to relocate the vector table in SRAM using the NVIC exception table and offset register.

Physical remap

When the boot pins are configured as desired, the application software can modify the memory accessible in the code area (code can thus be executed through the ICode/DCode

in place of the System bus). This modification is performed by programming the [SYSCFG memory remap register \(SYSCFG_MEMRMP\)](#) in the SYSCFG controller.

The following memory can then be remapped:

- Main Flash memory
- System memory
- Embedded SRAM

Table 7. Memory mapping vs. boot mode/physical remap

Addresses	Boot/Remap in main Flash memory	Boot/Remap in embedded SRAM	Boot/Remap in System memory
0x2000 0000 - 0x2000 3FFF	SRAM	SRAM	SRAM
0x1FF0 0000 - 0x1FF0 0FFF	System memory	System memory	System memory
0x0802 0000 - 0x0FFF FFFF	Reserved	Reserved	Reserved
0x0800 0000 - 0x0801 FFFF	Flash memory	Flash memory	Flash memory
0x0002 0000 - 0x07FF FFFF	Reserved	Reserved	Reserved
0x0000 0000 - 0x0001 FFFF	Flash (128 KB) Aliased	SRAM Aliased	System memory (4 KB) Aliased

Note: Even when aliased in the boot memory space, the related memory is still accessible at its original memory space.

Embedded boot loader

The embedded boot loader is used to reprogram the Flash memory through one of the following interfaces: USART1 or USART2. This program is located in the system memory and is programmed by ST during production.

5 General-purpose I/Os (GPIO)

5.1 GPIO introduction

Each general-purpose I/O port has four 32-bit configuration registers (GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR and GPIOx_PUPDR), two 32-bit data registers (GPIOx_IDR and GPIOx_ODR), a 32-bit set/reset register (GPIOx_BSRR), a 32-bit locking register (GPIOx_LCKR) and two 32-bit alternate function selection register (GPIOx_AFRH and GPIOx_AFRL).

5.2 GPIO main features

- Up to 16 I/Os under control
- Output states: push-pull or open drain + pull-up/down
- Output data from output data register (GPIOx_ODR) or peripheral (alternate function output)
- Speed selection for each I/O
- Input states: floating, pull-up/down, analog
- Input data to input data register (GPIOx_IDR) or peripheral (alternate function input)
- Bit set and reset register (GPIOx_BSRR) for bitwise write access to GPIOx_ODR
- Locking mechanism (GPIOx_LCKR) provided to freeze the I/O configuration
- Analog function
- Alternate function input/output selection registers (at most 16 AFs per I/O)
- Fast toggle capable of changing every two clock cycles
- Highly flexible pin multiplexing allows the use of I/O pins as GPIOs or as one of several peripheral functions

5.3 GPIO functional description

Subject to the specific hardware characteristics of each I/O port listed in the datasheet, each port bit of the general-purpose I/O (GPIO) ports can be individually configured by software in several modes:

- Input floating
- Input pull-up
- Input-pull-down
- Analog
- Output open-drain with pull-up or pull-down capability
- Output push-pull with pull-up or pull-down capability
- Alternate function push-pull with pull-up or pull-down capability
- Alternate function open-drain with pull-up or pull-down capability

Each I/O port bit is freely programmable, however the I/O port registers have to be accessed as 32-bit words, half-words or bytes. The purpose of the GPIOx_BSRR register is to allow atomic read/modify accesses to any of the GPIO registers. In this way, there is no risk of an IRQ occurring between the read and the modify access.

Figure 13 and Figure 14 show the basic structures of a standard and a 5 V tolerant I/O port bit, respectively. Table 21 gives the possible port bit configurations.

Figure 13. Basic structure of a standard I/O port bit

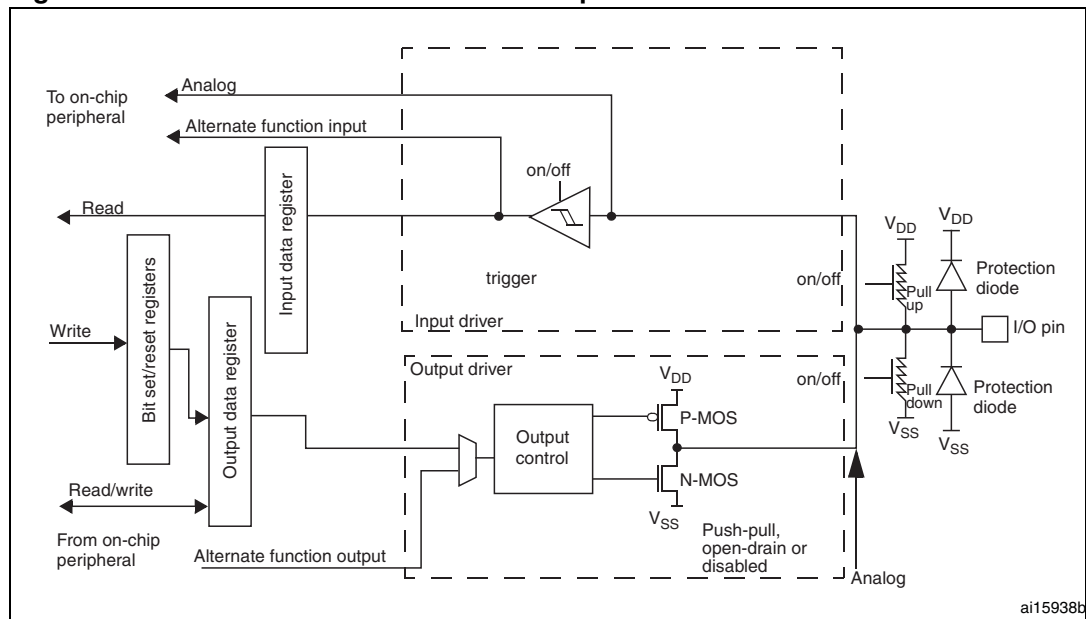
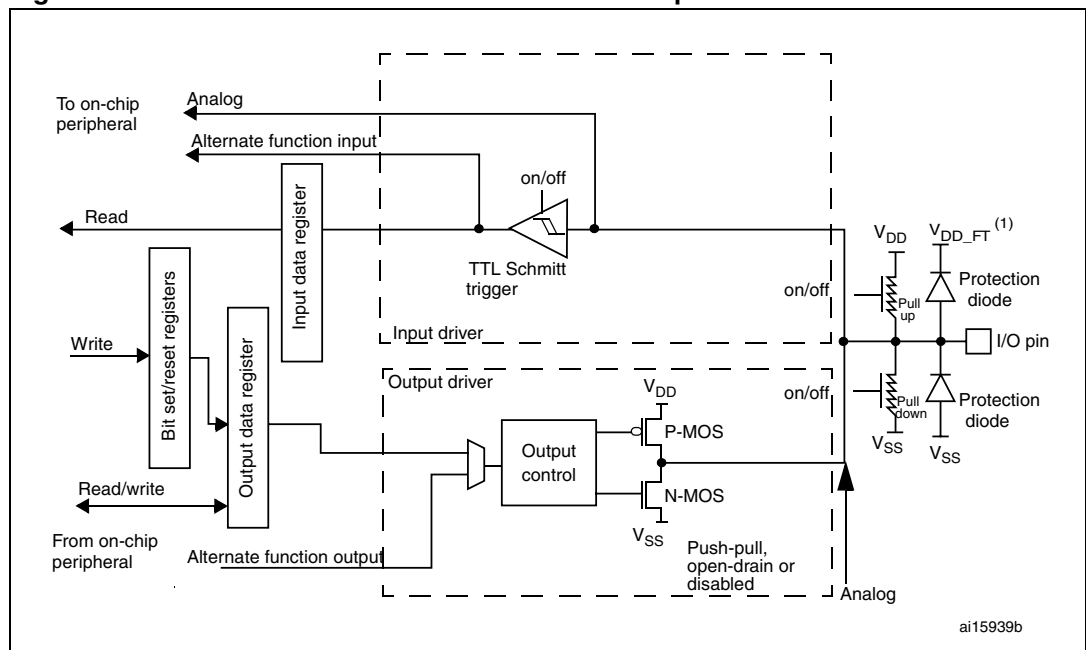


Figure 14. Basic structure of a five-volt tolerant I/O port bit



1. V_{DD_FT} is a potential specific to five-volt tolerant I/Os and different from V_{DD} .

Table 18. Port bit configuration table⁽¹⁾

MODER(i) [1:0]	OTYPER(i)	OSPEEDR(i) [B:A]		PUPDR(i) [1:0]		I/O configuration	
01	0	SPEED [B:A]		0	0	GP output	PP
	0			0	1	GP output	PP + PU
	0			1	0	GP output	PP + PD
	0			1	1	Reserved	
	1			0	0	GP output	OD
	1			0	1	GP output	OD + PU
	1			1	0	GP output	OD + PD
	1			1	1	Reserved (GP output OD)	
10	0	SPEED [B:A]		0	0	AF	PP
	0			0	1	AF	PP + PU
	0			1	0	AF	PP + PD
	0			1	1	Reserved	
	1			0	0	AF	OD
	1			0	1	AF	OD + PU
	1			1	0	AF	OD + PD
	1			1	1	Reserved	
00	x	x	x	0	0	Input	Floating
	x	x	x	0	1	Input	PU
	x	x	x	1	0	Input	PD
	x	x	x	1	1	Reserved (input floating)	
11	x	x	x	0	0	Input/output	Analog
	x	x	x	0	1	Reserved	
	x	x	x	1	0		
	x	x	x	1	1		

1. GP = general-purpose, PP = push-pull, PU = pull-up, PD = pull-down, OD = open-drain, AF = alternate function.

5.3.1 General-purpose I/O (GPIO)

During and just after reset, the alternate functions are not active and the I/O ports are configured in input floating mode.

The JTAG pins are in input pull-up/pull-down after reset:

- PA15: JTDI in pull-up
- PA14: JTCK in pull-down
- PA13: JTMS in pull-up
- PB4: NJTRST in pull-up

When the pin is configured as output, the value written to the output data register (GPIOx_ODR) is output on the I/O pin. It is possible to use the output driver in push-pull mode or open-drain mode (only the N-MOS is activated when 0 is output).

The input data register (GPIOx_IDR) captures the data present on the I/O pin at every AHB clock cycle.

All GPIO pins have weak internal pull-up and pull-down resistors, which can be activated or not depending on the value in the GPIOx_PUPDR register.

5.3.2 I/O pin multiplexer and mapping

The STM32L15xxx I/O pins are connected to onboard peripherals/modules through a multiplexer that allows only one peripheral's alternate function (AF) connected to an I/O pin at a time. In this way, there can be no conflict between peripherals sharing the same I/O pin.

Each I/O pin has a multiplexer with sixteen alternate function inputs (AF0 to AF15) that can be configured through the GPIOx_AFRL (for pin 0 to 7) and GPIOx_AFRH (for pin 8 to 15) registers:

- After reset all I/Os are connected to the system's alternate function 0 (AF0)
- The peripherals' alternate functions are mapped from AF1 to AF14
- Cortex-M3 EVENTOUT is mapped on AF15

This structure is shown in [Figure 15](#) below.

In addition to this flexible I/O multiplexing architecture, each peripheral has alternate functions mapped onto different I/O pins to optimize the number of peripherals available in smaller packages.

To use an I/O in a given configuration, you have to proceed as follows:

- **System function:** you have to connect the I/O to AF0 and configure it depending on the function used:
 - JTAG/SWD, after each device reset these pins are assigned as dedicated pins immediately usable by the debugger host (not controlled by the GPIO controller)
 - RTC_AF1: refer to [Table 20: RTC_AF1 pin](#) for more details about this pin configuration
 - RTC_50Hz: this pin should be configured in Input floating mode
 - MCO: this pin has to be configured in alternate function mode.

Note: You can disable some or all of the JTAG/SWD pins and so release the associated pins for GPIO usage.

For more details please refer to [Section 4.2.12: Clock-out capability](#).

Table 19. Flexible SWJ-DP pin assignment

Available debug ports	SWJ I/O pin assigned				
	PA13 / JTMS / SWDIO	PA14 / JTCK / SWCLK	PA15 / JTDI	PB3 / JTDO	PB4 / NJTRST
Full SWJ (JTAG-DP + SW-DP) - Reset state	X	X	X	X	X

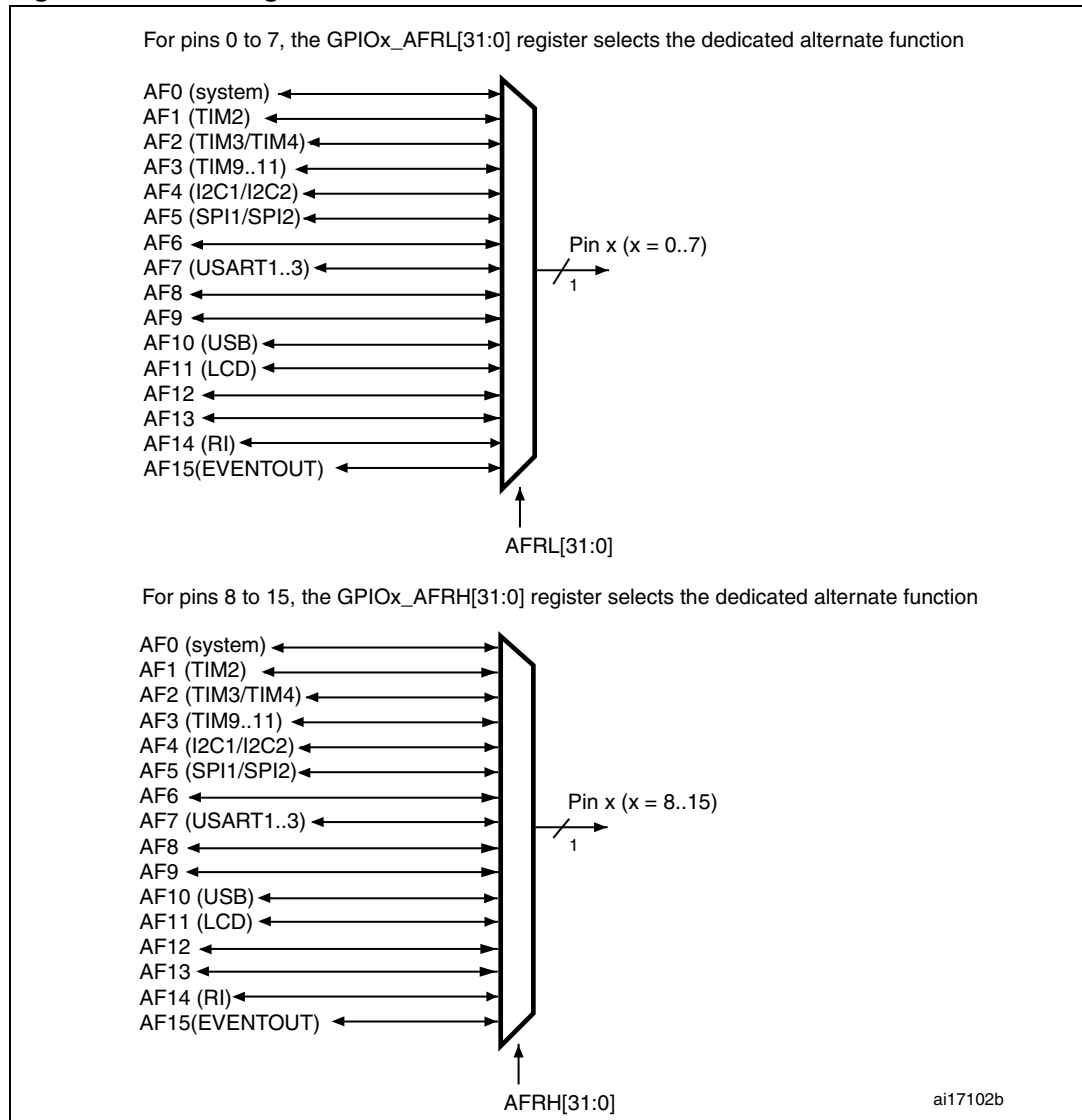
Table 19. Flexible SWJ-DP pin assignment (continued)

Available debug ports	SWJ I/O pin assigned				
	PA13 / JTMS / SWDIO	PA14 / JTCK / SWCLK	PA15 / JTDI	PB3 / JTDO	PB4 / NJTRST
Full SWJ (JTAG-DP + SW-DP) but without NJTRST	X	X	X	X	
JTAG-DP Disabled and SW-DP Enabled	X	X			
JTAG-DP Disabled and SW-DP Disabled	Released				

- **GPIO:** configure the desired I/O as output, input or analog in the GPIOx_MODER register.
- **Peripheral's alternate function:**
For the ADC and DAC, configure the desired I/O as analog in the GPIOx_MODER register.
For other peripherals:
 - Configure the desired I/O as an alternate function in the GPIOx_MODER register
 - Select the type, pull-up/pull-down and output speed via the GPIOx_OTYPER, GPIOx_PUPDR and GPIOx_OSPEEDER registers, respectively
 - Connect the I/O to the desired AFx in the GPIOx_AFRL or GPIOx_AFRH register
- **EVENTOUT:** you can configure the I/O pin used to output the Cortex-M3 EVENTOUT signal by connecting it to AF15

Note: EVENTOUT is not mapped onto the following I/O pins: PH0, PH1 and PH2.

Please refer to the “Alternate function mapping” table in the STM32L15xxx datasheets for the detailed mapping of the system and peripherals’ alternate function I/O pins.

Figure 15. Selecting an alternate function

5.3.3 I/O port control registers

Each of the GPIOs has four 32-bit memory-mapped control registers (GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR, GPIOx_PUPDR) to configure up to 16 I/Os. The GPIOx_MODER register is used to select the I/O direction (input, output, AF, analog). The GPIOx_OTYPER and GPIOx_OSPEEDR registers are used to select the output type (push-pull or open-drain) and speed (the I/O speed pins are directly connected to the corresponding GPIOx_OSPEEDR register bits whatever the I/O direction). The GPIOx_PUPDR register is used to select the pull-up/pull-down whatever the I/O direction.

5.3.4 I/O port data registers

Each GPIO has two 16-bit memory-mapped data registers: input and output data registers (GPIOx_IDR and GPIOx_ODR). GPIOx_ODR stores the data to be output, it is read/write

accessible. The data input through the I/O are stored into the input data register (GPIOx_IDR), a read-only register.

See [Section 5.4.5: GPIO port input data register \(GPIOx_IDR\) \(x = A..E and H\)](#) and [Section 5.4.6: GPIO port output data register \(GPIOx_ODR\) \(x = A..E and H\)](#) for the register descriptions.

5.3.5 I/O data bitwise handling

The bit set reset register (GPIOx_BSRR) is a 32-bit register which allows the application to set and reset each individual bit in the output data register (GPIOx_ODR). The bit set reset register has twice the size of GPIOx_ODR.

To each bit in GPIOx_ODR, correspond two control bits in GPIOx_BSRR: BSRR(i) and BSRR(i+SIZE). When written to 1, bit BSRR(i) **sets** the corresponding ODR(i) bit. When written to 1, bit BSRR(i+SIZE) **resets** the ODR(i) corresponding bit.

Writing any bit to 0 in GPIOx_BSRR does not have any effect on the corresponding bit in GPIOx_ODR. If there is an attempt to both set and reset a bit in GPIOx_BSRR, the set action takes priority.

Using the GPIOx_BSRR register to change the values of individual bits in GPIOx_ODR is a “one-shot” effect that does not lock the GPIOx_ODR bits. The GPIOx_ODR bits can always be accessed directly. The GPIOx_BSRR register provides a way of performing atomic bitwise handling.

There is no need for the software to disable interrupts when programming the GPIOx_ODR at bit level: it is possible to modify one or more bits in a single atomic AHB write access.

5.3.6 GPIO locking mechanism

It is possible to freeze the GPIO control registers by applying a specific write sequence to the GPIOx_LCKR register. The frozen registers are GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR, GPIOx_PUPDR, GPIOx_AFRL and GPIOx_AFRH.

To write the GPIOx_LCKR register, a specific write / read sequence has to be applied. When the right LOCK sequence is applied to bit 16 in this register, the value of LCKR[15:0] is used to lock the configuration of the I/Os (during the write sequence the LCKR[15:0] value must be the same). When the LOCK sequence has been applied to a port bit, the value of the port bit can no longer be modified until the next reset. Each GPIOx_LCKR bit freezes the corresponding bit in the control registers (GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR, GPIOx_PUPDR, GPIOx_AFRL and GPIOx_AFRH).

The LOCK sequence (refer to [Section 5.4.8: GPIO port configuration lock register \(GPIOx_LCKR\) \(x = A..E and H\)](#)) can only be performed using a word (32-bit long) access to the GPIOx_LCKR register due to the fact that GPIOx_LCKR bit 16 has to be set at the same time as the [15:0] bits.

For more details please refer to LCKR register description in [Section 5.4.8: GPIO port configuration lock register \(GPIOx_LCKR\) \(x = A..E and H\)](#).

5.3.7 I/O alternate function input/output

Two registers are provided to select one out of the sixteen alternate function inputs/outputs available for each I/O. With these registers, you can connect an alternate function to some other pin as required by your application.

This means that a number of possible peripheral functions are multiplexed on each GPIO using the GPIOx_AFRL and GPIOx_AFRH alternate function registers. The application can thus select any one of the possible functions for each I/O. The AF selection signal being common to the alternate function input and alternate function output, a single channel is selected for the alternate function input/output of one I/O.

To know which functions are multiplexed on each GPIO pin, refer to the STM32L15xxx datasheets.

Note: The application is allowed to select one of the possible peripheral functions for each I/O at a time.

5.3.8 External interrupt/wakeup lines

All ports have external interrupt capability. To use external interrupt lines, the port must be configured in input mode, refer to [Section 7.2: External interrupt/event controller \(EXTI\)](#) and [Section 7.2.3: Wakeup event management](#).

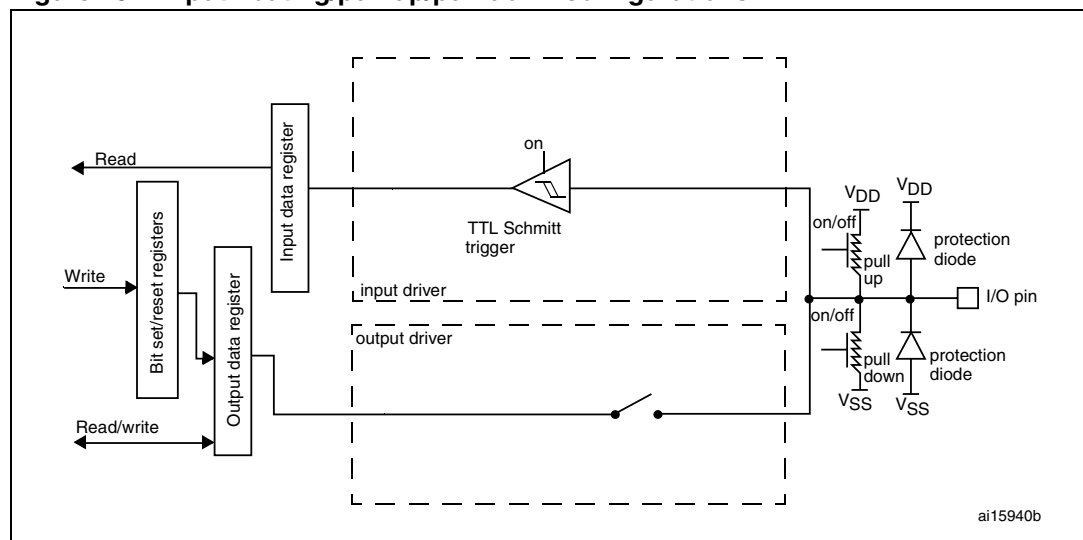
5.3.9 Input configuration

When the I/O port is programmed as Input:

- the output buffer is disabled
- the Schmitt trigger input is activated
- the pull-up and pull-down resistors are activated depending on the value in the GPIOx_PUPDR register
- The data present on the I/O pin are sampled into the input data register every AHB clock cycle
- A read access to the input data register provides the I/O State

[Figure 16](#) shows the input configuration of the I/O port bit.

Figure 16. Input floating/pull up/pull down configurations



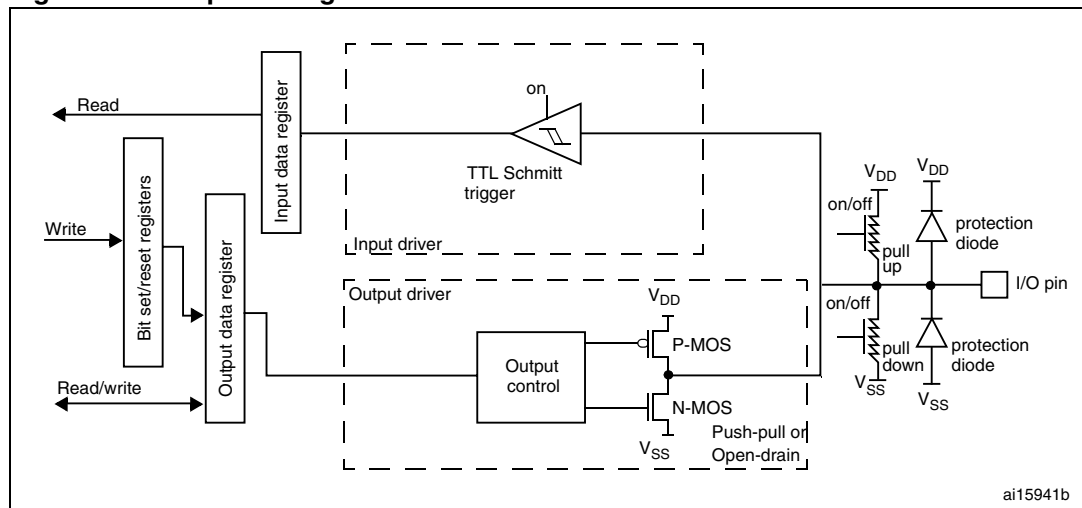
5.3.10 Output configuration

When the I/O port is programmed as output:

- The output buffer is enabled:
 - Open drain mode: A “0” in the Output register activates the N-MOS whereas a “1” in the Output register leaves the port in Hi-Z (the P-MOS is never activated)
 - Push-pull mode: A “0” in the Output register activates the N-MOS whereas a “1” in the Output register activates the P-MOS
- The Schmitt trigger input is activated
- The weak pull-up and pull-down resistors are activated or not depending on the value in the GPIOx_PUPDR register
- The data present on the I/O pin are sampled into the input data register every AHB clock cycle
- A read access to the input data register gets the I/O state
- A read access to the output data register gets the last written value in Push-pull mode

Figure 17 shows the output configuration of the I/O port bit.

Figure 17. Output configuration



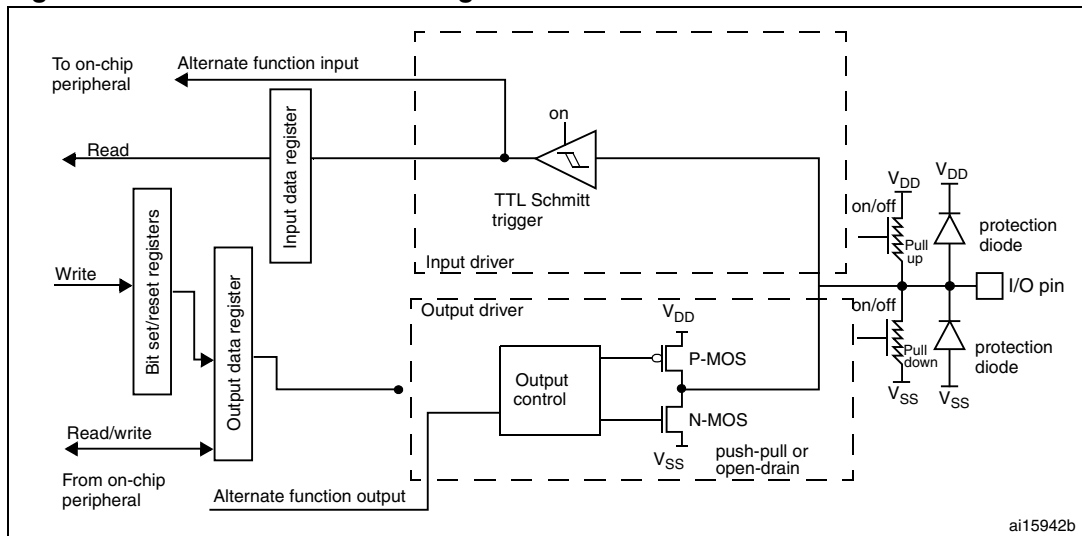
5.3.11 Alternate function configuration

When the I/O port is programmed as alternate function:

- The output buffer is turned on in open-drain or push-pull configuration
- The output buffer is driven by the signal coming from the peripheral (alternate function out)
- The Schmitt trigger input is activated
- The weak pull-up and pull-down resistors are activated or not depending on the value in the GPIOx_PUPDR register
- The data present on the I/O pin are sampled into the input data register every AHB clock cycle
- A read access to the input data register gets the I/O state
- A read access to the output data register gets the last value written in push-pull mode

Figure 18 shows the Alternate function configuration of the I/O port bit.

Figure 18. Alternate function configuration



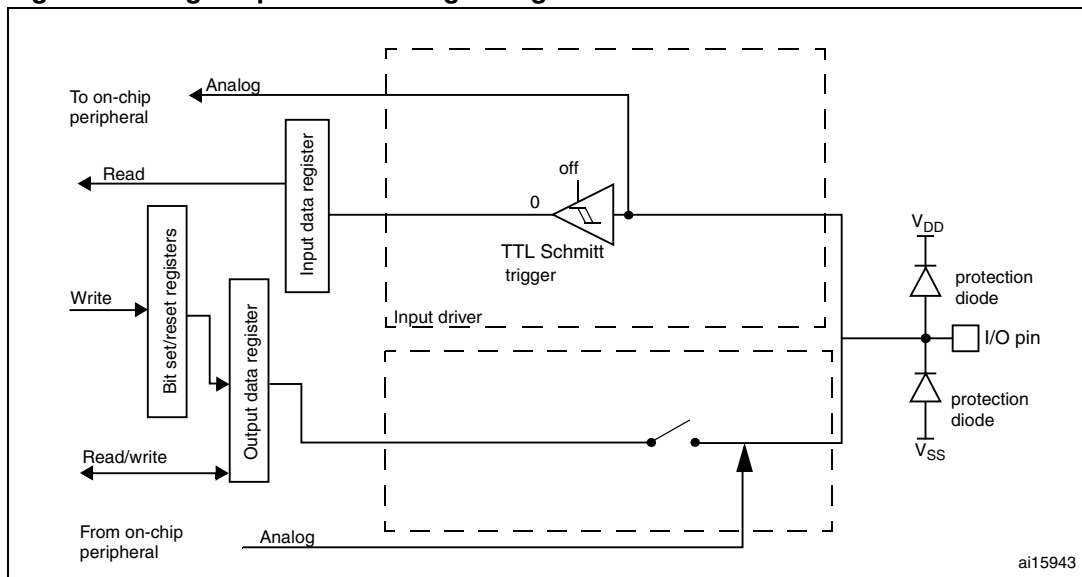
5.3.12 Analog configuration

When the I/O port is programmed as analog configuration:

- The output buffer is disabled
- The Schmitt trigger input is deactivated, providing zero consumption for every analog value of the I/O pin. The output of the Schmitt trigger is forced to a constant value (0).
- The weak pull-up and pull-down resistors are disabled
- Read access to the input data register gets the value "0"

Figure 19 shows the high-impedance, analog-input configuration of the I/O port bit.

Figure 19. High impedance-analog configuration



5.3.13 Using the OSC32_IN/OSC32_OUT pins as GPIO PC14/PC15 port pins

The LSE oscillator pins OSC32_IN and OSC32_OUT can be used as general-purpose PC14 and PC15 I/Os, respectively, when the LSE oscillator is off (after reset, the LSE oscillator is off). The PC14 and PC15 I/Os are only configured as OSC32_IN and OSC32_OUT LSE oscillator pins when the LSE oscillator is ON (by setting the LSEON bit in the RCC_CSR register). The LSE has priority over the GPIO function.

Note: 1 The PC14/PC15 GPIO functionality is lost when the V_{CORE} domain is powered off (by the device entering the standby mode). In this case the I/Os are set in analog input mode.

5.3.14 Using the OSC_IN/OSC_OUT pins as GPIO PH0/PH1 port pins

The HSE oscillator pins OSC_IN/OSC_OUT can be used as general-purpose PH0/PH1 I/Os, respectively, when the HSE oscillator is off. (after reset, the HSE oscillator is off). The PH0/PH1 I/Os are only configured as OSC_IN/OSC_OUT HSE oscillator pins when the HSE oscillator is ON (by setting the HSEON bit in the RCC_CR register). The HSE has priority over the GPIO function.

5.3.15 Selection of RTC_AF1 alternate functions

The STM32L15xxx features:

- Two GPIO pins, which can be used as wakeup pins (WKUP1 and WKUP3).
- One GPIO pin, which can be used as a wakeup pin (WKUP2), for the detection of a tamper or time-stamp event, or to output RTC AFO_ALARM or AFO_CALIB.

The RTC_AF1 pin (PC13) can be used for the following purposes:

- Wakeup pin 2 (WKUP2): this feature is enabled by setting the EWUP2 in the PWR_CSR register.
- RTC AFO_ALARM output: this output can be RTC Alarm A, RTC Alarm B or RTC Wakeup depending on the OSEL[1:0] bits in the RTC_CR register.
- RTC AFO_CALIB output: this feature is enabled by setting the COE[23] bit in the RTC_CR register.
- RTC AFI_TAMPER1: Tamper event detection
- Time-stamp event detection

The selection of the RTC AFO_ALARM output is performed through the RTC_TAFCR register as follows: ALARMOUTTYPE is used to select whether the RTC AFO_ALARM output is configured in push-pull or open-drain mode.

The output mechanism follows the priority order shown in [Table 20](#).

Table 20. RTC_AF1 pin ⁽¹⁾

Pin configuration and function	AFO_ALARM enabled	AFO_CALIB enabled	Tamper enabled	Time-stamp enabled	EWUP2 enabled	ALARMOUTTYPE AFO_ALARM configuration
Alarm out output OD	1	0	Don't care	Don't care	Don't care	0
Alarm out output PP	1	0	Don't care	Don't care	Don't care	1

Table 20. RTC_AF1 pin (continued)⁽¹⁾

Pin configuration and function	AFO_ALARM enabled	AFO_CALIB enabled	Tamper enabled	Time-stamp enabled	EWUP2 enabled	ALARMOUTTYPE AFO_ALARM configuration
Calibration out output PP	0	1	Don't care	Don't care	Don't care	Don't care
TAMPER input floating	0	0	1	0	Don't care	Don't care
TIMESTAMP and TAMPER input floating	0	0	1	1	Don't care	Don't care
TIMESTAMP input floating	0	0	0	1	Don't care	Don't care
Wakeup Pin 2	0	0	0	0	1	Don't care
Standard GPIO	0	0	0	0	0	Don't care

1. OD: open drain; PP: push-pull.

5.4 GPIO registers

This section gives a detailed description of the GPIO registers.

For a summary of register bits, register address offsets and reset values, refer to [Table 21](#).

The peripheral registers have to be accessed by words (32-bit).

5.4.1 GPIO port mode register (GPIOx_MODER) (x = A..E and H)

Address offset: 0x00

Reset values:

- 0xA800 0000 for port A
- 0x0000 0280 for port B
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y:2y+1 **MODERy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O direction mode.

00: Input (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

5.4.2 GPIO port output type register (GPIOx_OTYPER) (x = A..E and H)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, always read as 0.

Bits 15:0 **OTy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the output type of the I/O port.

0: Output push-pull (reset state)

1: Output open-drain

5.4.3 GPIO port output speed register (GPIOx_OSPEEDR) (x = A..E and H)

Address offset: 0x08

Reset values:

- 0x0000 00C0 for port B
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OSPEEDR15[1:0]		OSPEEDR14[1:0]		OSPEEDR13[1:0]		OSPEEDR12[1:0]		OSPEEDR11[1:0]		OSPEEDR10[1:0]		OSPEEDR9[1:0]		OSPEEDR8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OSPEEDR7[1:0]		OSPEEDR6[1:0]		OSPEEDR5[1:0]		OSPEEDR4[1:0]		OSPEEDR3[1:0]		OSPEEDR2[1:0]		OSPEEDR1[1:0]		OSPEEDR0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y:2y+1 **OSPEEDRy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O output speed.

00: 400 kHz Very low speed

01: 2 MHz Low speed

10: 10 MHz Medium speed

11: 40 MHz High speed on 50 pF (50 MHz output max speed on 30 pF)

5.4.4 GPIO port pull-up/pull-down register (GPIOx_PUPDR) (x = A..E and H)

Address offset: 0x0C

Reset values:

- 0x6400 0000 for port A
- 0x0000 0100 for port B
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y:2y+1 **PUPDRy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O pull-up or pull-down

00: No pull-up, pull-down

01: Pull-up

10: Pull-down

11: Reserved

5.4.5 GPIO port input data register (GPIOx_IDR) (x = A..E and H)

Address offset: 0x10

Reset value: 0x0000 xxxx (where x means undefined)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, always read as 0.

Bits 15:0 **IDRy[15:0]**: Port input data (y = 0..15)

These bits are read-only and can be accessed in word mode only. They contain the input value of the corresponding I/O port.

5.4.6 GPIO port output data register (GPIOx_ODR) (x = A..E and H)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, always read as 0.

Bits 15:0 **ODRy[15:0]**: Port output data (y = 0..15)

These bits can be read and written by software.

Note: For atomic bit set/reset, the ODR bits can be individually set and reset by writing to the GPIOx_BSRR register (x = A..E and H).

5.4.7 GPIO port bit set/reset register (GPIOx_BSRR) (x = A..E and H)

Address offset: 0x18

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **BRy**: Port x reset bit y (y = 0..15)

These bits are write-only and can be accessed in word, half-word or byte mode. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODRx bit

1: Resets the corresponding ODRx bit

Note: If both BSx and BRx are set, BSx has priority.

Bits 15:0 **BSy**: Port x set bit y (y = 0..15)

These bits are write-only and can be accessed in word, half-word or byte mode. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODRx bit

1: Sets the corresponding ODRx bit

5.4.8 GPIO port configuration lock register (GPIOx_LCKR) (x = A..E and H)

This register is used to lock the configuration of the port bits when a correct write sequence is applied to bit 16 (LCKK). The value of bits [15:0] is used to lock the configuration of the GPIO. During the write sequence, the value of LCKR[15:0] must not change. When the LOCK sequence has been applied on a port bit, the value of this port bit can no longer be modified until the next reset.

Note: A specific write sequence is used to write to the GPIOx_LCKR register. Only word access (32-bit long) is allowed during this write sequence.

Each lock bit freezes a specific configuration register (control and alternate function registers).

Address offset: 0x1C

Reset value: 0x0000 0000

Access: 32-bit word only, read/write register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															LCKK
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LCK15	LCK14	LCK13	LCK12	LCK11	LCK10	LCK9	LCK8	LCK7	LCK6	LCK5	LCK4	LCK3	LCK2	LCK1	LCK0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:17 Reserved

Bit 16 **LCKK[16]**: Lock key

This bit can be read any time. It can only be modified using the lock key write sequence.

0: Port configuration lock key not active

1: Port configuration lock key active. The GPIOx_LCKR register is locked until an MCU reset occurs.

LOCK key write sequence:

WR LCKR[16] = '1' + LCKR[15:0]

WR LCKR[16] = '0' + LCKR[15:0]

WR LCKR[16] = '1' + LCKR[15:0]

RD LCKR

RD LCKR[16] = '1' (this read operation is optional but it confirms that the lock is active)

Note: During the LOCK key write sequence, the value of LCK[15:0] must not change.

Any error in the lock sequence aborts the lock.

After the first lock sequence on any bit of the port, any read access on the LCKK bit will return '1' until the next CPU reset.

Bits 15:0 **LCKy**: Port x lock bit y (y = 0..15)

These bits are read/write but can only be written when the LCKK bit is '0'.

0: Port configuration not locked

1: Port configuration locked

5.4.9 GPIO alternate function low register (GPIOx_AFRL) (x = A..E and H)

Address offset: 0x20

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRL7[3:0]				AFRL6[3:0]				AFRL5[3:0]				AFRL4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRL3[3:0]				AFRL2[3:0]				AFRL1[3:0]				AFRL0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFRLy**: Alternate function selection for port x bit y (y = 0..7)

These bits are written by software to configure alternate function I/Os

AFRLy selection:

0000: AF0

1000: AF8

0001: AF1

1001: AF9

0010: AF2

1010: AF10

0011: AF3

1011: AF11

0100: AF4

1100: AF12

0101: AF5

1101: AF13

0110: AF6

1110: AF14

0111: AF7

1111: AF15

5.4.10 GPIO alternate function high register (GPIOx_AFRH) (x = A..E and H)

Address offset: 0x24

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRH15[3:0]				AFRH14[3:0]				AFRH13[3:0]				AFRH12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRH11[3:0]				AFRH10[3:0]				AFRH9[3:0]				AFRH8[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFRHy**: Alternate function selection for port x bit y (y = 8..15)

These bits are written by software to configure alternate function I/Os

AFRHy selection:

0000: AF0	1000: AF8
0001: AF1	1001: AF9
0010: AF2	1010: AF10
0011: AF3	1011: AF11
0100: AF4	1100: AF12
0101: AF5	1101: AF13
0110: AF6	1110: AF14
0111: AF7	1111: AF15

5.4.11 GPIO register map

Table 21. GPIO register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	GPIOA_MODER	Moder15[1:0]	Moder14[1:0]	Moder13[1:0]	Moder12[1:0]	Moder11[1:0]	Moder10[1:0]	Moder9[1:0]	Moder8[1:0]	Moder7[1:0]	Moder6[1:0]	Moder5[1:0]	Moder4[1:0]	Moder3[1:0]	Moder2[1:0]	Moder1[1:0]	Moder0[1:0]																
	Reset value	1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00	GPIOB_MODER	Moder15[1:0]	Moder14[1:0]	Moder13[1:0]	Moder12[1:0]	Moder11[1:0]	Moder10[1:0]	Moder9[1:0]	Moder8[1:0]	Moder7[1:0]	Moder6[1:0]	Moder5[1:0]	Moder4[1:0]	Moder3[1:0]	Moder2[1:0]	Moder1[1:0]	Moder0[1:0]																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0
0x00	GPIOx_MODER (where x = C..F)	Moder15[1:0]	Moder14[1:0]	Moder13[1:0]	Moder12[1:0]	Moder11[1:0]	Moder10[1:0]	Moder9[1:0]	Moder8[1:0]	Moder7[1:0]	Moder6[1:0]	Moder5[1:0]	Moder4[1:0]	Moder3[1:0]	Moder2[1:0]	Moder1[1:0]	Moder0[1:0]																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x04	GPIOx_OTYPER (where x = A..E and H)	Reserved																OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
	Reset value																	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0

Table 21. GPIO register map and reset values (continued)

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x08	GPIOx_OSPEEDER (where x = A..E and H except B)	OSPEEDR15[1:0]		OSPEEDR14[1:0]		OSPEEDR13[1:0]		OSPEEDR12[1:0]		OSPEEDR11[1:0]		OSPEEDR10[1:0]		OSPEEDR9[1:0]		OSPEEDR8[1:0]		OSPEEDR7[1:0]		OSPEEDR6[1:0]		OSPEEDR5[1:0]		OSPEEDR4[1:0]		OSPEEDR3[1:0]		OSPEEDR2[1:0]		OSPEEDR1[1:0]		OSPEEDR0[1:0]		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x08	GPIOB_OSPEEDER	OSPEEDR15[1:0]		OSPEEDR14[1:0]		OSPEEDR13[1:0]		OSPEEDR12[1:0]		OSPEEDR11[1:0]		OSPEEDR10[1:0]		OSPEEDR9[1:0]		OSPEEDR8[1:0]		OSPEEDR7[1:0]		OSPEEDR6[1:0]		OSPEEDR5[1:0]		OSPEEDR4[1:0]		OSPEEDR3[1:0]		OSPEEDR2[1:0]		OSPEEDR1[1:0]		OSPEEDR0[1:0]		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0C	GPIOA_PUPDR	PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]		PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]		
	Reset value	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0C	GPIOB_PUPDR	PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]		PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
0x0C	GPIOx_PUPDR (where x = C..F)	PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]		PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x10	GPIOx_IDR (where x = A..E and H)	Reserved																IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0	
	Reset value																																	
0x14	GPIOx_ODR (where x = A..E and H)	Reserved																ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0	
	Reset value																																	
0x18	GPIOx_BSRR (where x = A..E and H)	BSR15	BSR14	BSR13	BSR12	BSR11	BSR10	BSR9	BSR8	BSR7	BSR6	BSR5	BSR4	BSR3	BSR2	BSR1	BSR0	BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x1C	GPIOx_LCKR (where x = A..E and H)	Reserved																LCK15	LCK14	LCK13	LCK12	LCK11	LCK10	LCK9	LCK8	LCK7	LCK6	LCK5	LCK4	LCK3	LCK2	LCK1	LCK0	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x20	GPIOx_AFR1 (where x = A..E and H)	AFR17[3:0]			AFR16[3:0]			AFR15[3:0]			AFR14[3:0]			AFR13[3:0]			AFR12[3:0]			AFR11[3:0]			AFR10[3:0]			AFR9[3:0]			AFR8[3:0]			AFR7[3:0]		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x24	GPIOx_AFRH (where x = A..E and H)	AFRH15[3:0]			AFRH14[3:0]			AFRH13[3:0]			AFRH12[3:0]			AFRH11[3:0]			AFRH10[3:0]			AFRH9[3:0]			AFRH8[3:0]			AFRH7[3:0]			AFRH6[3:0]			AFRH5[3:0]		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Refer to [Table 1: Register boundary addresses](#) for the register boundary addresses. The following tables give the GPIO register map and the reset values.

6 System configuration controller (SYSCFG) and routing interface (RI)

6.1 SYSCFG and RI introduction

The system configuration controller is mainly used to remap the memory accessible in the code area, and manage the external interrupt line connection to the GPIOs.

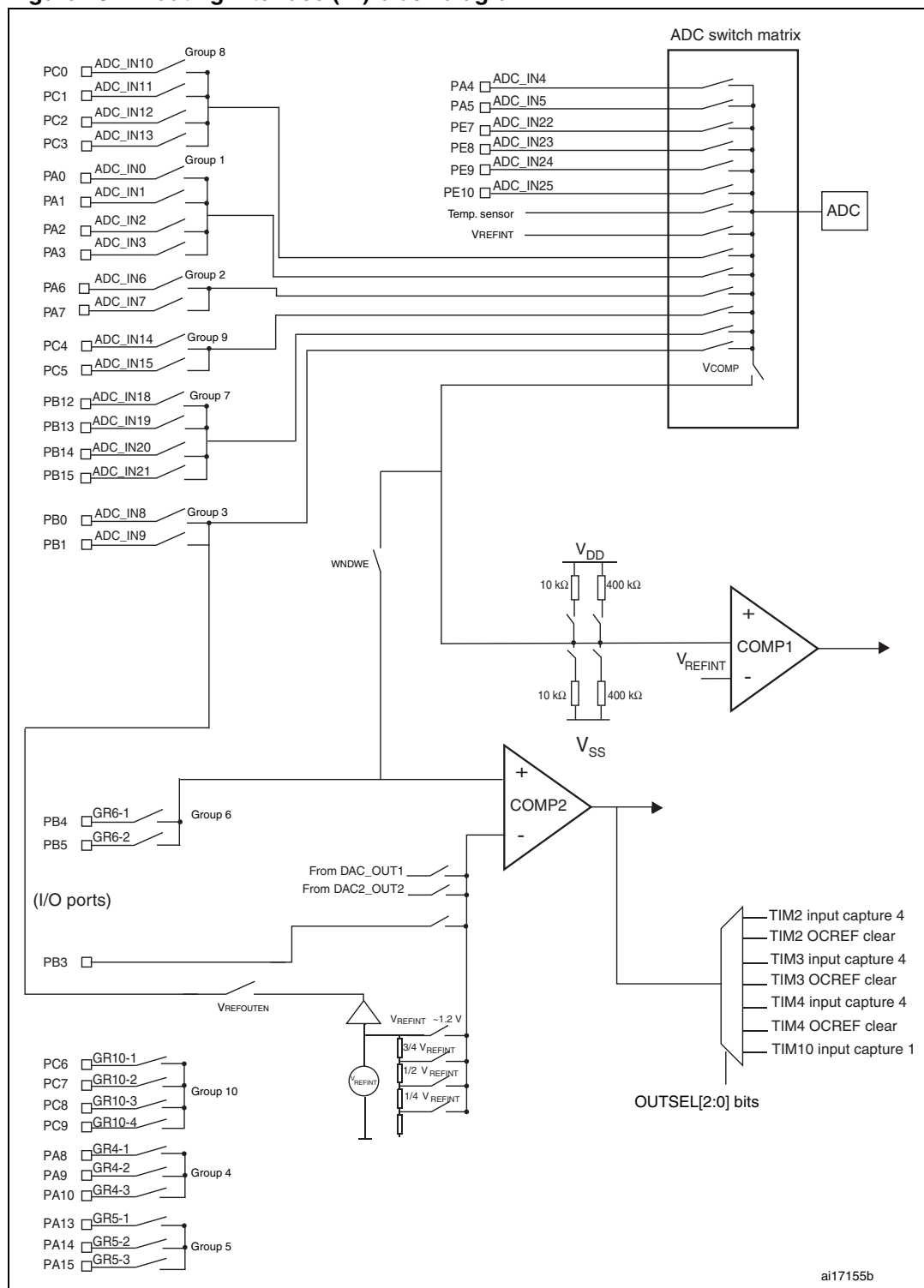
The routing interface provides high flexibility by allowing the software routing of I/Os toward the input captures of the STM32L15xxx's three high-end timers (TIM2, TIM3 and TIM4). The STM32L15xxx's ADC has an analog input matrix that is usually managed by a specific ADC interface. With the routing interface, it is possible to connect several I/O analog pins to a given channel of the ADC matrix by managing the analog switches of each I/O.

6.2 RI main features

- TIM2/TIM3/TIM4's input captures 1,2,3 and four routing selections from selectable I/Os
- Routing of internal reference voltage V_{REFINT} to selectable I/Os for all packages
- 24 external I/Os + 2 internal nodes (internal reference voltage + temperature sensor) can be used for data acquisition purposes in conjunction with the ADC interface
- Input and output routing of COMP1 and COMP2

Note: The RI registers can be accessed only when the comparator interface clock is enabled by setting the COMPEN bit in the RCC_APB1ENR register. Refer to [Section 4.3.10 on page 97](#).

Figure 20. Routing interface (RI) block diagram



Note: The internal reference voltage and temperature sensor cannot be used as COMP1 non-inverting input.

6.3 RI functional description

6.3.1 Special I/O configuration

Two matrices of switches control the routing of I/Os toward analog blocks (that is the ADC or the comparator): I/O switches and ADC switches (refer to [Figure 20: Routing interface \(RI\) block diagram](#)).

- When I/Os are used for analog purposes other than data acquisition, the I/O and ADC switch matrices have to be controlled by the RI_ASCR1 and RI_ASCR2 registers. These registers are then used to close or open switches by software: closing switches sets the corresponding bits whereas opening switches clears them.
- When I/Os are used as ADC inputs for data acquisition purposes, the I/O and ADC switch matrices are directly controlled by the ADC interface. The corresponding bits in the RI_ASCR1 and RI_ASCR2 registers must be kept cleared (switches open).

Up to 6 I/Os are connected directly and independently to the ADC through the pad resistor: these fast channels are capable of fast acquisition (1 megasample/s at the maximum ADC acquisition time). Other ADC channels do not exceed 800 kilosamples/s.

When the I/Os are programmed in input mode by standard I/O port registers, the Schmitt trigger and the hysteresis are enabled by default. In this mode, the RI_ASCRx and RI_HYSCR registers can be used to simultaneously close the corresponding I/O analog switch and disable the Schmitt trigger hysteresis. It is therefore possible to read the corresponding port with a trigger level of $V_{DDIO}/2$.

Only 6 groups (18 I/Os) are multiplexed to the ADC by the I/O analog switches. With the 8 or 6 fast channels, this makes 24 I/Os available for data acquisition and COMP1 comparison versus V_{REFINT} .

Note: For all I/Os used as comparator inputs, the I/O port configuration must be kept in analog mode.

[Table 22](#) shows the grouping of I/Os, the control register bits used to configure them as analog inputs or outputs (irrespective of standard I/O port programming), and the associated ADC channel number.

Table 22. I/O groups and selection

Group numbering		GPIO port	Analog ADC channel	I/O + ADC analog switch	I/O functions
Group 1	GR1-1	PA0	CH0	RI_ASCR1->CH0	COMP1_INP
	GR1-2	PA1	CH1	RI_ASCR1->CH1	
	GR1-3	PA2	CH2	RI_ASCR1->CH2	
	GR1-4	PA3	CH3	RI_ASCR1->CH3	
Group 2	GR2-1	PA6	CH6	RI_ASCR1->CH6	COMP1_INP
	GR2-2	PA7	CH7	RI_ASCR1->CH7	
Group 3	GR3-1	PB0	CH8	RI_ASCR1->CH8	COMP1_INP/VREF_OUT
	GR3-2	PB1	CH9	RI_ASCR1->CH9	

Table 22. I/O groups and selection (continued)

Group numbering		GPIO port	Analog ADC channel	I/O + ADC analog switch	I/O functions
Group 4	GR4-1	PA8	NA	RI_ASCR2->GR4-1	
	GR4-2	PA9		RI_ASCR2->GR4-2	
	GR4-3	PA10		RI_ASCR2->GR4-3	
Group 5	GR5-1	PA13	NA	RI_ASCR2->GR5-1	
	GR5-2	PA14		RI_ASCR2->GR5-2	
	GR5-3	PA15		RI_ASCR2->GR5-3	
Group 6	GR6-1	PB4	NA	RI_ASCR2->GR6-1	COMP2_INP
	GR6-2	PB5		RI_ASCR2->GR6-2	
Group 7	GR7-1	PB12	CH18	RI_ASCR1->CH18	COMP1_INP
	GR7-2	PB13	CH19	RI_ASCR1->CH19	
	GR7-3	PB14	CH20	RI_ASCR1->CH20	
	GR7-4	PB15	CH21	RI_ASCR1->CH21	
Group 8	GR8-1	PC0	CH10	RI_ASCR1->CH10	COMP1_INP
	GR8-2	PC1	CH11	RI_ASCR1->CH11	
	GR8-3	PC2	CH12	RI_ASCR1->CH12	
	GR8-4	PC3	CH13	RI_ASCR1->CH13	
Group 9	GR9-1	PC4	CH14	RI_ASCR1->CH14	COMP1_INP
	GR9-2	PC5	CH15	RI_ASCR1->CH15	
Group 10	GR10-1	PC6	NA	RI_ASCR2->GR10-1	
	GR10-2	PC7		RI_ASCR2->GR10-2	
	GR10-3	PC8		RI_ASCR2->GR10-3	
	GR10-4	PC9		RI_ASCR2->GR10-4	
Fast channels		PA4	CH4	RI_ASWCR->CH4	COMP1_INP/DAC1
		PA5	CH5	RI_ASWCR->CH5	COMP1_INP/DAC2
		PE7	CH22	RI_ASWCR->CH22	COMP1_INP
		PE8	CH23	RI_ASWCR->CH23	COMP1_INP
		PE9	CH24	RI_ASWCR->CH24	COMP1_INP
		PE10	CH25	RI_ASWCR->CH25	COMP1_INP
NA		PB3	NA		COMP2_INN
NA		PB7	NA		PVD_IN/COMP2_INP

6.3.2 Input capture routing

By default (at reset), the four input captures of the three general-purpose timers (TIM2, TIM3, TIM4) are connected to the I/O port specified in the STM32L15xxx datasheet's "pin descriptions" table.

The I/O routing can be changed by programming register RI_ICR as indicated below:

- The input capture 1 of TIM2, TIM3 and TIM4 can be rerouted from any I/O by configuring the IC1IOS[3:0] bits in RI_ICR.
- The input capture 2 of TIM2, TIM3 and TIM4 can be rerouted from any I/O by configuring the IC2IOS[3:0] bits in RI_ICR.
- The input capture 3 of TIM2, TIM3 and TIM4 can be rerouted from any I/O by configuring the IC3IOS[3:0] bits in RI_ICR.
- The input capture 4 of TIM2, TIM3 and TIM4 can be rerouted from any I/O by configuring the IC4IOS[3:0] bits in RI_ICR.

Refer to the following table for I/O routing to the input capture timers.

This capability can be applied on only one out of the three timers at a time by configuring TIM[1:0] in RI_ICR. When TIM[1:0] = 00 none of the three timers are affected by the I/O routing: the defaults connections are enabled.

Moreover, when a timer is selected, I/O routing can be enabled for one or more input captures by configuring the IC1, IC2, IC3 and IC4 bits in RI_ICR.

Refer to [Table 23](#) for the I/O correspondence and to [Table 24](#) for the timer selection.

Table 23. Input capture mapping

IC1IOS / IC2IOS / IC3IOS / IC4IOS	TIMx IC1 / TIMx IC2 / TIMx IC3 / TIMx IC4
0000	PA0 / PA1 / PA2 / PA3
0001	PA4 / PA5 / PA6 / PA7
0010	PA8 / PA9 / PA10 / PA11
0011	PA12 / PA13 / PA14 / PA15
0100	PC0 / PC1 / PC2 / PC3
0101	PC4 / PC5 / PC6 / PC7
0110	PC8 / PC9 / PC10 / PC11
0111	PC12 / PC13 / PC14 / PC15
1000	PD0 / PD1 / PD2 / PD3
1001	PD4 / PD5 / PD6 / PD7
1010	PD8 / PD9 / PD10 / PD11
1011	PD12 / PD13 / PD14 / PD15
1100	PE0 / PE1 / PE2 / PE3
1101	PE4 / PE5 / PE6 / PE7
1110	PE8 / PE9 / PE10 / PE11
1111	PE12 / PE13 / PE14 / PE15

Note: 1 The I/O should be configured in alternate function mode (AF14).

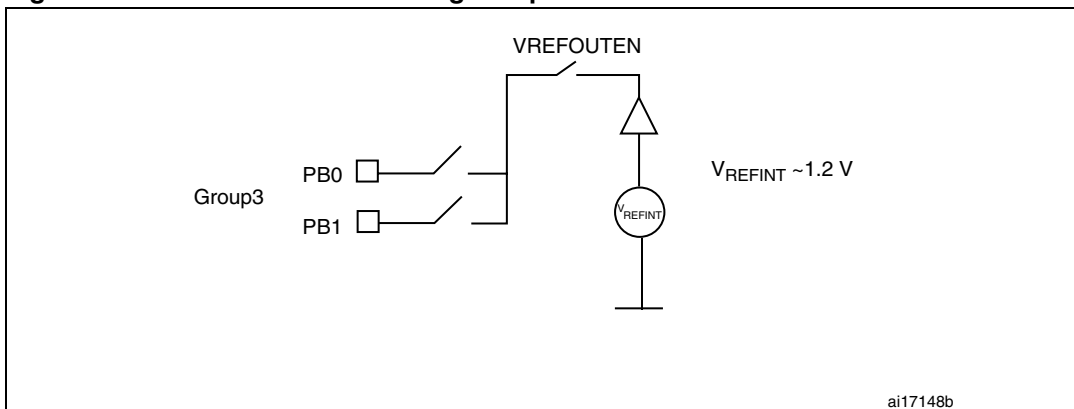
Table 24. Timer selection

TIM[1:0]	Selected timer
00	No timer selected, default routing on all timers
01	TIM2 selected
10	TIM3 selected
11	TIM4 selected

Table 25. Input capture selection

IC4 / IC3 / IC2 / IC1	Selected input capture
0	IC deselected, default routing on the input capture (AF)
1	Input capture routing follows Table 24

6.3.3 Reference voltage routing

Figure 21. Internal reference voltage output

The V_{REFINT} output can be routed to any I/O in group 3 by following this procedure:

1. Set the VREFOUTEN bit in COMP_CSR.
2. Close the analog switch of all I/Os in group 3 by setting CH8 or CH9 in RI_ASCR1.

6.4 RI registers

The peripheral registers have to be accessed by words (32-bit).

6.4.1 RI input capture register (RI_ICR)

The RI_ICR register is used to select the routing of 4 full ports to the input captures of TIM2, TIM3 and TIM4.

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved										IC4	IC3	IC2	IC1	TIM[1:0]	
										rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IC4IOS[3:0]				IC3IOS[3:0]				IC2IOS[3:0]				IC1IOS[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:22 Reserved, must be kept cleared.

Bit 21 **IC4**: This bit is set and cleared by software to select the standard AF or the large routing capability on the input capture 4 of the timer selected by TIM[1:0] (bits 17:16).

0: AF on IC4

1: Multiple port routing capability according to IC4IOS[3:0] (bits 15:12)

Bit 20 **IC3**: This bit is set and cleared by software to select the standard AF or the large routing capability on the input capture 3 of the timer selected by TIM[1:0] (bits 17:16).

0: AF on IC3

1: Multiple port routing capability according to IC3IOS[3:0] (bits 11:8)

Bit 19 **IC2**: This bit is set and cleared by software to select the standard AF or the large routing capability on the input capture 2 of the timer selected by TIM[1:0] (bits 17:16).

0: AF on IC2

1: Multiple port routing capability according to IC2IOS[3:0] (bits 7:4)

Bit 18 **IC1**: This bit is set and cleared by software to select the standard AF or the large routing capability on the input capture 1 of the timer selected by TIM[1:0] (bits 17:16).

0: AF on IC1

1: Multiple port routing capability according to IC1IOS[3:0] (bits 3:0)

Bits 17:16 **TIM[1:0]**: Timer select bits

These bits are set and cleared by software. They are used to select one out of three timers or none.

00: non timer selected

01: TIM2 selected

10: TIM3 selected

11: TIM4 selected

Bits 15:12 **IC4IOS[3:0]**: Input capture 4 select bits

These bits are set and cleared by software. They select the input port to be routed to the IC4 of the selected timer (see bits 16:17).

0000: PA3	1000: PD3
0001: PA7	1001: PD7
0010: PA11	1010: PD11
0011: PA15	1011: PD15
0100: PC3	1100: PE3
0101: PC7	1101: PE7
0110: PC11	1110: PE11
0111: PC15	1111: PE15

Bits 11:8 **IC3IOS[3:0]**: Input capture 3 select bits

These bits are set and cleared by software. They select the input port to be routed toward the IC3 of the selected timer (see bits 16:17).

0000: PA2	1000: PD2
0001: PA6	1001: PD6
0010: PA10	1010: PD10
0011: PA14	1011: PD14
0100: PC2	1100: PE2
0101: PC6	1101: PE6
0110: PC10	1110: PE10
0111: PC14	1111: PE14

Bits 7:4 **IC2IOS[3:0]**: Input capture 2 select bits

These bits are set and cleared by software. They select the input port to be routed toward the IC2 of the selected timer (see bits 16:17).

0000: PA1	1000: PD1
0001: PA5	1001: PD5
0010: PA9	1010: PD9
0011: PA13	1011: PD13
0100: PC1	1100: PE1
0101: PC5	1101: PE5
0110: PC9	1110: PE9
0111: PC13	1111: PE13

Bits 3:0 **IC1IOS[3:0]**: Input capture 1 select bits

These bits are set and cleared by software. They select the input port to be routed toward the IC1 of the selected timer (see bits 16:17).

0000: PA0	1000: PD0
0001: PA4	1001: PD4
0010: PA8	1010: PD8
0011: PA12	1011: PD12
0100: PC0	1100: PE0
0101: PC4	1101: PE4
0110: PC8	1110: PE8
0111: PC12	1111: PE12

Note: 1 The standard AFs dedicated to TIM2 are:

IC4-> PA3, PB11 or PE12

IC3-> PA2, PB10 or PE11

IC2-> PA1, PB3 or PE10

IC1-> PA0, PA5, PA15 or PE9

2 The standard AFs dedicated to TIM3 are:

IC4-> PB1 or PC9

IC3-> PB0 or PC8
 IC2-> PA7, PC7, PB5 or PE4
 IC1-> PA6, PC6, PB4 or PE3

3 The standard AFs dedicated to TIM4 are:

IC4-> PD15 or PB9
 IC3-> PD14 or PB8
 IC2-> PD13 or PB7
 IC1-> PD12 or PB6

6.4.2 RI analog switches control register (RI_ASCR1)

The RI_ASCR1 register is used to configure the analog switches of the I/Os linked to the ADC. These I/Os are pointed to by the ADC channel number.

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SCM	Reserved				VCOMP	CH25	CH24	CH23	CH22	CH21	CH20	CH19	CH18	Reserved	
rw					rw	rw	rw	rw	rw	rw	rw	rw	rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CH15	CH14	CH13	CH12	CH11	CH10	CH9	CH8	CH7	CH6	CH5	CH4	CH3	CH2	CH1	CH0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **SCM**: Switch control mode

This bit is set and cleared by software to control the analog ADC switches according to the state of the analog I/O switches controlled by bits [25:18] and [15:0].

0: ADC analog switches open or controlled by the ADC interface

1: ADC analog switches closed if the corresponding I/O switch is also closed

Bits 30:27 Reserved

Bit 26 **VCOMP**: ADC analog switch selection for internal node to comparator 1

This bit is set and cleared by software to control the VCOMP ADC analog switch. See [Figure 50 on page 236](#).

0: Analog switch open

1: Analog switch closed

Bits 25:22 **CH[25:22]**: Analog I/O switch control of channels CH[25:22]

These bits are set and cleared by software to control the analog switches. If the I/O is used as an ADC input, the switch must be left open to allow the ADC to control it.

0: Analog switch open

1: Analog switch closed

Bits 21:18 **CH[21:18]** Analog switch control

These bits are set and cleared by software to control the analog switches.

0: Analog switch open

1: Analog switch closed

Bits 17:16 Reserved

Bits 15:0 **CH[15:0]**: Analog I/O switch control of channels CH[15:0]

These bits are set and cleared by software to control the analog switches. If the I/O is used as an ADC input, the switch must be left open to allow the ADC to control it.

0: Analog switch open

1: Analog switch closed

- Note:**
- 1 The ADC_IN16 and ADC_IN17 channels are internal and controlled only by the ADC interface for data acquisition purposes.
 - 2 The ADC_IN4, ADC_IN5, ADC_IN22, ADC_IN23, ADC_IN24 and ADC_IN25 channels are directly connected to the ADC through a resistor, no need to close external I/O analog switches.
 - 3 When the SCM bit is low, the CH bits are used to connect groups of I/Os together by analog switches, independently of the ADC.
 - 4 When the SCM bit is high, the CH bits are used to connect several I/Os together through the ADC switch matrix in order to allow a possible wakeup by COMP1 if the VCOMP bit is high.

6.4.3 RI analog switch control register 2 (RI_ASCR2)

The RI_ASCR2 register is used to configure the analog switches of groups of I/Os not linked to the ADC. In this way, predefined groups of I/Os can be connected together.

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				GR4-3	GR4-2	GR4-1	GR5-3	GR5-2	GR5-1	GR6-2	GR6-1	GR10-4	GR10-3	GR10-2	GR10-1
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:12 Reserved, must be kept cleared.

Bit 11:0 **GRx-x**: GRx-x analog switch control

These bits are set and cleared by software to control the analog switches independently. Refer to [Table 22: I/O groups and selection on page 128](#) from the ADC interface.

0: Analog switch open

1: Analog switch closed

6.4.4 RI hysteresis control register (RI_HYSCR1)

The RI_HYSCR1 register is used to enable/disable the hysteresis of the input Schmitt trigger of ports A and B.

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PB[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PA[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 **PB[15:0]**: Port B hysteresis control on/off

These bits are set and cleared by software to control the Schmitt trigger hysteresis of the Port B[15:0].

0: Hysteresis on

1: Hysteresis off

Bits 15:0 **PA[15:0]**: Port A hysteresis control on/off

These bits are set and cleared by software to control the Schmitt trigger hysteresis of the Port A[15:0].

0: Hysteresis on

1: Hysteresis off

6.4.5 RI Hysteresis control register (RI_HYSCR2)

RI_HYSCR2 register allows to enable/disable hysteresis of input Schmitt trigger of ports C and D.

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PD[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 **PD[15:0]**: Port D hysteresis control on/off

These bits are set and cleared by software to control the Schmitt trigger hysteresis of the Port D[15:0].

0: Hysteresis on

1: Hysteresis off

Bits 15:0 **PC[15:0]**: Port C hysteresis control on/off

These bits are set and cleared by software to control the Schmitt trigger hysteresis of the Port C[15:0].

0: Hysteresis on

1: Hysteresis off

6.4.6 RI Hysteresis control register (RI_HYSCR3)

The RI_HYSCR3 register is used to enable/disable the hysteresis of the input Schmitt trigger of the entire port E.

Address offset: 0x18

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PE[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 27:22 Reserved, must be kept cleared.

Bits 15:0 **PE[15:0]**: Port E hysteresis control on/off

These bits are set and cleared by software to control the Schmitt trigger hysteresis of the Port E[15:0].

0: Hysteresis on

1: Hysteresis off

6.4.7 Analog switch mode register (RI_ASMR1)

The RI_ASMR1 register is used to select if analog switches of port A are to be controlled by the timer OC or through the ADC interface or RI_ASCRx registers.

Address offset: 0x20

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PA[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept cleared.

Bits 15:0 **PA[15:0]: Port A analog switch mode selection**

These bits are set and cleared by software to select the mode of controlling the analog switches for Port A.

- 0: ADC interface or RI_ASCRx controlled
- 1: Timer controlled

6.4.8 Channel mask register (RI_CM1)

RI_CM1 is used to mask a port A channel designated as a timer input capture (after acquisition completion to avoid triggering multiple detections).

Address offset: 0x24

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PA[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept cleared.

Bits 15:0 **PA[15:0]: Port A channel masking**

These bits are set and cleared by software to mask the ZI input of Port A.

- 0: Masked
- 1: Not masked

6.4.9 Channel identification for capture register (RI_CICR1)

The RI_CICR1 register is used when analog switches are controlled by a timer OC. RI_CICR1 allows a channel to be identified for timer input capture.

Address offset: 0x28

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PA[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept cleared.

Bits 15:0 **PA[15:0]**: Port A channel identification for capture

These bits are set and cleared by software to identify the sampling capacitor I/Os on Port A.

0: Channel I/O

1: Sampling capacitor I/O

6.4.10 Analog switch mode register (RI_ASMR2)

The RI_ASMR2 register is used to select if analog switches of port B are to be controlled by the timer OC or through the ADC interface or RI_ASCRx registers.

Address offset: 0x2C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PB[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept cleared.

Bits 15:0 **PB[15:0]**: Port B analog switch mode selection

These bits are set and cleared by software to select the mode of controlling the analog switches for Port B.

0: ADC interface or RI_ASCRx controlled

1: Timer controlled

6.4.11 Channel mask register (RI_CMR2)

RI_CMR2 is used to mask a port B channel designated as a timer input capture (after acquisition completion to avoid triggering multiple detections)

Address offset: 0x30

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PB[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept cleared.

Bits 15:0 **PB[15:0]**: *Port B channel masking*

These bits are set and cleared by software to mask ZI input of Port B.

0: Masked

1: Not masked

6.4.12 Channel identification for capture register (RI_CICR2)

The RI_CICR2 register is used when analog switches are controlled by a timer OC. RI_CICR2 allows a port B channel to be identified for timer input capture.

Address offset: 0x34

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PB[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept cleared.

Bits 15:0 **PB[15:0]**: *Port B channel identification for capture*

These bits are set and cleared by software to identify the sampling capacitor I/Os on Port B.

0: Channel I/O

1: Sampling capacitor I/O

6.4.13 Analog switch mode register (RI_ASMR3)

The RI_ASMR3 register is used to select if analog switches of port C are to be controlled by the timer OC or through the ADC interface or RI_ASCRx registers.

Address offset: 0x38

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept cleared.

Bits 15:0 **PC[15:0]: Port C analog switch mode selection**

These bits are set and cleared by software to select the mode of controlling the analog switches for Port C.

- 0: ADC interface or RI_ASCRx controlled
- 1: Timer controlled

6.4.14 Channel mask register (RI_CM3)

RI_CM3 is used to mask a port C channel designated as a timer input capture (after acquisition completion to avoid triggering multiple detections)

Address offset: 0x3C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept cleared.

Bits 15:0 **PC[15:0]: Port C channel masking**

These bits are set and cleared by software to mask ZI input of Port C.

- 0: Masked
- 1: Not masked

6.4.15 Channel identification for capture register (RI_CICR3)

The RI_CICR3 register is used when analog switches are controlled by a timer OC. RI_CICR3 allows a port C channel to be identified for timer input capture.

Address offset: 0x40

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept cleared.

Bits 15:0 **PC[15:0]**: Port C channel identification for capture

These bits are set and cleared by software to identify the sampling capacitor I/Os on Port C.

0: Channel I/O

1: Sampling capacitor I/O

6.4.16 RI register map

Table 26 summarizes the RI registers.

Table 26. RI register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0									
0x04	RI_ICR	Reserved										IC 4	IC 3	IC 2	IC 1	TIM[1:0]		IC4IOS[3:0]			IC3IOS[3:0]			IC2IOS[3:0]			IC1IOS[3:0]															
	Reset value											0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
0x08	RI_ASCR1	SCM	Reserved				VCOMP	CH25:22				CH[21:18]				Reserved		CH[15:6]						Reserved		CH[3:0]																
	Reset value	0					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0								
0x0C	RI_ASCR2	Reserved														GR4[3:1]			GR5[3:1]			GR6 [2:1]		GR10[4:1]																		
	Reset value															0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x10	RI_HYSCR1	PB[15:0]														PA[15:0]																										
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0								
0x14	RI_HYSCR2	PD[15:0]														PC[15:0]																										
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0								
0x18	RI_HYSCR3	Reserved														PE[15:0]																										
	Reset value															0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x20	RI_ASMR1	Reserved														PA[15:0]																										
	Reset value															0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x24	RI_CMR1	Reserved														PA[15:0]																										
	Reset value															0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x28	RI_CICR1	Reserved														PA[15:0]																										
	Reset value															0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x2C	RI_ASMR2	Reserved														PB[15:0]																										
	Reset value															0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x30	RI_CMR2	Reserved														PB[15:0]																										
	Reset value															0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x34	RI_CICR2	Reserved														PB[15:0]																										
	Reset value															0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 26. RI register map and reset values (continued)

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x38	RI_ASMR3	Reserved																PC[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x3C	RI_CMR3	Reserved																PC[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x40	RI_CICR3	Reserved																PC[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x44	RI_ASMR4	Reserved																PF[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x48	RI_CMR4	Reserved																PF[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x4C	RI_CICR4	Reserved																PF[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x50	RI_ASMR5	Reserved																PG[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x54	RI_CMR5	Reserved																PG[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x58	RI_CICR5	Reserved																PG[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

6.5 SYSCFG registers

The peripheral registers have to be accessed by words (32-bit).

6.5.1 SYSCFG memory remap register (SYSCFG_MEMRMP)

This register is used for specific configurations on memory remap:

- Two bits are used to configure the type of memory accessible at address 0x0000 0000. These bits are used to select the physical remap by software and so, bypass the BOOT pins.
- After reset these bits take the value selected by the BOOT pins.

Note: This register is not reset through the SYSCFGRST bit in the RCC_APB2RSTR register.

Address offset: 0x00

Reset value: 0x0000 000X (X is the memory mode selected by the BOOT pins)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														MEM_MODE	
														rw	rw

Bits 31:2 Reserved

Bits 1:0 **MEM_MODE**: Memory mapping selection

Set and cleared by software. This bit controls the memory's internal mapping at address 0x0000 0000. After reset these bits take on the memory mapping selected by the BOOT pins.

- 00: Main Flash memory mapped at 0x0000 0000
- 01: System Flash memory mapped at 0x0000 0000
- 10: Reserved
- 11: SRAM mapped at 0x0000 0000

6.5.2 SYSCFG peripheral mode configuration register (SYSCFG_PMC)

An internal pull-up resistor (1.5 kΩ) can be connected by software on the USB data + (DP) line. This internal pull-up resistor is enabled if the USB is not in power-down mode and if the USB_PU bit is set.

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														USB_PU	
															rw

Bits 31:1 Reserved

Bit 0 **USB_PU** *USB pull-up enable on DP line*

Set and cleared by software. This bit controls the internal pull-up (1.5 kΩ) on the USB DP line.

0: no pull-up on the USB DP line (even if USB is not in power down mode)

1: internal pull-up is connected on USB DP line (only if USB is not in power down mode)

6.5.3 SYSCFG external interrupt configuration register 1 (SYSCFG_EXTICR1)

Address offset: 0x08

Reset value: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI3[3:0]				EXTI2[3:0]				EXTI1[3:0]				EXTI0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x = 0 to 3)

These bits are written by software to select the source input for the EXTIx external interrupt.

0000: PA[x] pin

0001: PB[x] pin

0010: PC[x] pin

0011: PD[x] pin

0100: PE[x] pin

0101: PH[x] (only PH[2:0])

PH[3] is not used.

6.5.4 SYSCFG external interrupt configuration register 2 (SYSCFG_EXTICR2)

Address offset: 0x0C

Reset value: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI7[3:0]				EXTI6[3:0]				EXTI5[3:0]				EXTI4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x = 4 to 7)

These bits are written by software to select the source input for the EXTIx external interrupt.

0000: PA[x] pin

0001: PB[x] pin

0010: PC[x] pin

0011: PD[x] pin

0100: PE[x] pin

PH[7:4] are not used.

6.5.5 SYSCFG external interrupt configuration register 3 (SYSCFG_EXTICR3)

Address offset: 0x10

Reset value: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI11[3:0]				EXTI10[3:0]				EXTI9[3:0]				EXTI8[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x = 8 to 11)

These bits are written by software to select the source input for the EXTIx external interrupt.

0000: PA[x] pin

0001: PB[x] pin

0010: PC[x] pin

0011: PD[x] pin

0100: PE[x] pin

PH[11:8] are not used.

6.5.6 SYSCFG external interrupt configuration register 4 (SYSCFG_EXTICR4)

Address offset: 0x14

Reset value: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI15[3:0]				EXTI14[3:0]				EXTI13[3:0]				EXTI12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x = 12 to 15)

These bits are written by software to select the source input for the EXTIx external interrupt.

0000: PA[x] pin

0001: PB[x] pin

0010: PC[x] pin

0011: PD[x] pin

0100: PE[x] pin

PH[15:12] are not used.

6.5.7 SYSCFG register map

The following table gives the SYSCFG register map and the reset values.

Table 27. SYSCFG register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	SYSCFG_MEMRMP	Reserved																														MEM_MODE	
	Reset value																															x	x
0x04	SYSCFG_PMC	Reserved																														USB_PU	
	Reset value																															0	
0x08	SYSCFG_EXTICR1	Reserved															EXTI3[3:0]			EXTI2[3:0]			EXTI1[3:0]			EXTI0[3:0]							
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0C	SYSCFG_EXTICR2	Reserved															EXTI7[3:0]			EXTI6[3:0]			EXTI5[3:0]			EXTI4[3:0]							
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x10	SYSCFG_EXTICR3	Reserved															EXTI11[3:0]			EXTI10[3:0]			EXTI9[3:0]			EXTI8[3:0]							
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x14	SYSCFG_EXTICR4	Reserved															EXTI15[3:0]			EXTI14[3:0]			EXTI13[3:0]			EXTI12[3:0]							
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Refer to [Table: “Register boundary addresses”](#).

7 Interrupts and events

This Section applies to the whole STM32L15xxx family, unless otherwise specified.

7.1 Nested vectored interrupt controller (NVIC)

Features

- 45 maskable interrupt channels (not including the 16 interrupt lines of Cortex™-M3)
- 16 programmable priority levels (4 bits of interrupt priority are used)
- Low-latency exception and interrupt handling
- Power management control
- Implementation of system control registers

The NVIC and the processor core interface are closely coupled, which enables low-latency interrupt processing and efficient processing of late arriving interrupts.

All interrupts including the core exceptions are managed by the NVIC. For more information on exceptions and NVIC programming see Chapter 5 “Exceptions & Chap 8 Nested Vectored Interrupt Controller” in the ARM *Cortex™-M3 Technical Reference Manual*.

7.1.1 SysTick calibration value register

The SysTick calibration value is fixed to 4000, which gives a reference time base of 1 ms with the SysTick clock set to 4 MHz (max HCLK/8).

7.1.2 Interrupt and exception vectors

[Table 28](#) is the vector table for STM32L15xxx devices.

Table 28. Vector table

Position	Priority	Type of priority	Acronym	Description	Address
	-	-	-	Reserved	0x0000_0000
	-3	fixed	Reset	Reset	0x0000_0004
	-2	fixed	NMI_Handler	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.	0x0000_0008
	-1	fixed	HardFault_Handler	All class of fault	0x0000_000C
	0	settable	MemManage_Handler	Memory management	0x0000_0010
	1	settable	BusFault_Handler	Pre-fetch fault, memory access fault	0x0000_0014
	2	settable	UsageFault_Handler	Undefined instruction or illegal state	0x0000_0018
	-	-	-	Reserved	0x0000_001C - 0x0000_002B
	3	settable	SVC_Handler	System service call via SWI instruction	0x0000_002C

Table 28. Vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
	4	settable	DebugMon_Handler	Debug Monitor	0x0000_0030
	-	-	-	Reserved	0x0000_0034
	5	settable	PendSV_Handler	Pendable request for system service	0x0000_0038
	6	settable	SysTick_Handler	System tick timer	0x0000_003C
0	7	settable	WWDG	Window Watchdog interrupt	0x0000_0040
1	8	settable	PVD	PVD through EXTI Line detection interrupt	0x0000_0044
2	9	settable	TAMPER_STAMP	Tamper and TimeStamp through EXTI line interrupts	0x0000_0048
3	10	settable	RTC_WKUP	RTC Wakeup through EXTI line interrupt	0x0000_004C
4	11	settable	FLASH	Flash global interrupt	0x0000_0050
5	12	settable	RCC	RCC global interrupt	0x0000_0054
6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000_0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000_005C
8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000_0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000_0064
10	17	settable	EXTI4	EXTI Line4 interrupt	0x0000_0068
11	18	settable	DMA1_Channel1	DMA1 Channel1 global interrupt	0x0000_006C
12	19	settable	DMA1_Channel2	DMA1 Channel2 global interrupt	0x0000_0070
13	20	settable	DMA1_Channel3	DMA1 Channel3 global interrupt	0x0000_0074
14	21	settable	DMA1_Channel4	DMA1 Channel4 global interrupt	0x0000_0078
15	22	settable	DMA1_Channel5	DMA1 Channel5 global interrupt	0x0000_007C
16	23	settable	DMA1_Channel6	DMA1 Channel6 global interrupt	0x0000_0080
17	24	settable	DMA1_Channel7	DMA1 Channel7 global interrupt	0x0000_0084
18	25	settable	ADC1	ADC1 global interrupt	0x0000_0088
19	26	settable	USB_HP	USB High priority interrupt	0x0000_008C
20	27	settable	USB_LP	USB Low priority interrupt	0x0000_0090
21	28	settable	DAC	DAC interrupt	0x0000_0094
22	29	settable	COMP	Comparator wakeup through EXTI line (21 and 22) interrupt	0x0000_0098
23	30	settable	EXTI9_5	EXTI Line[9:5] interrupts	0x0000_009C
24	31	settable	LCD	LCD global interrupt	0x0000_00A0
25	32	settable	TIM9	TIM9 global interrupt	0x0000_00A4
26	33	settable	TIM10	TIM10 global interrupt	0x0000_00A8

Table 28. Vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
27	34	settable	TIM11	TIM11 global interrupt	0x0000_00AC
28	35	settable	TIM2	TIM2 global interrupt	0x0000_00B0
29	36	settable	TIM3	TIM3 global interrupt	0x0000_00B4
30	37	settable	TIM4	TIM4 global interrupt	0x0000_00B8
31	38	settable	I2C1_EV	I ² C1 event interrupt	0x0000_00BC
32	39	settable	I2C1_ER	I ² C1 error interrupt	0x0000_00C0
33	40	settable	I2C2_EV	I ² C2 event interrupt	0x0000_00C4
34	41	settable	I2C2_ER	I ² C2 error interrupt	0x0000_00C8
35	42	settable	SPI1	SPI1 global interrupt	0x0000_00CC
36	43	settable	SPI2	SPI2 global interrupt	0x0000_00D0
37	44	settable	USART1	USART1 global interrupt	0x0000_00D4
38	45	settable	USART2	USART2 global interrupt	0x0000_00D8
39	46	settable	USART3	USART3 global interrupt	0x0000_00DC
40	47	settable	EXTI15_10	EXTI Line[15:10] interrupts	0x0000_00E0
41	48	settable	RTC_Alarm	RTC Alarms (A and B) through EXTI line interrupt	0x0000_00E4
42	49	settable	USB_FS_WKUP	USB Device FS Wakeup through EXTI line interrupt	0x0000_00E8
43	50	settable	TIM6	TIM6 global interrupt	0x0000_00EC
44	51	settable	TIM7	TIM7 global interrupt	0x0000_00F0

7.2 External interrupt/event controller (EXTI)

The external interrupt/event controller consists of up to 23 edge detectors for generating event/interrupt requests. Each input line can be independently configured to select the type (pulse or pending) and the corresponding trigger event (rising edge, falling edge or both). Each line can also be masked independently. A pending register maintains the status line of the interrupt requests.

7.2.1 Main features

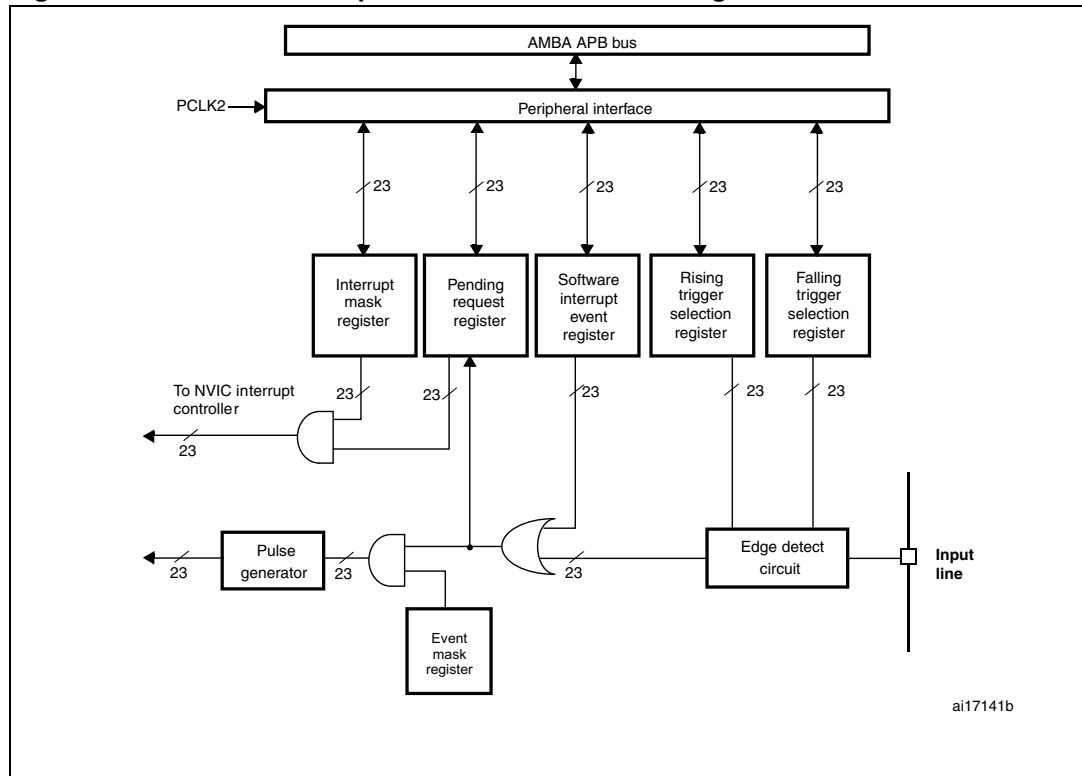
The main features of the EXTI controller are the following:

- Independent trigger and mask on each interrupt/event line
- Dedicated status bit for each interrupt line
- Generation of up to 23 software event/interrupt requests
- Detection of external signals with a pulse width lower than the APB2 clock period. Refer to the electrical characteristics section of the STM32L15xxx datasheet for details on this parameter.

7.2.2 Block diagram

The block diagram is shown in [Figure 22](#).

Figure 22. External interrupt/event controller block diagram



7.2.3 Wakeup event management

The STM32L15xxx is able to handle external or internal events in order to wake up the core (WFE). The wakeup event can be generated by either:

- enabling an interrupt in the peripheral control register but not in the NVIC, and enabling the SEVONPEND bit in the Cortex-M3 system control register. When the MCU resumes from WFE, the peripheral interrupt pending bit and the peripheral NVIC IRQ channel pending bit (in the NVIC interrupt clear pending register) have to be cleared.
- or configuring an external or internal EXTI line in event mode. When the CPU resumes from WFE, it is not necessary to clear the peripheral interrupt pending bit or the NVIC IRQ channel pending bit as the pending bit corresponding to the event line is not set.

To use an external line as a wakeup event, refer to [Section 7.2.4: Functional description](#).

7.2.4 Functional description

To generate the interrupt, the interrupt line should be configured and enabled. This is done by programming the two trigger registers with the desired edge detection and by enabling the interrupt request by writing a '1' to the corresponding bit in the interrupt mask register. When the selected edge occurs on the external interrupt line, an interrupt request is generated. The pending bit corresponding to the interrupt line is also set. This request is reset by writing a '1' into the pending register.

To generate the event, the event line should be configured and enabled. This is done by programming the two trigger registers with the desired edge detection and by enabling the event request by writing a '1 to the corresponding bit in the event mask register. When the selected edge occurs on the event line, an event pulse is generated. The pending bit corresponding to the event line is not set

An interrupt/event request can also be generated by software by writing a '1 into the software interrupt/event register.

Hardware interrupt selection

To configure the 23 lines as interrupt sources, use the following procedure:

- Configure the mask bits of the 23 Interrupt lines (EXTI_IMR)
- Configure the Trigger Selection bits of the Interrupt lines (EXTI_RTISR and EXTI_FTSR)
- Configure the enable and mask bits that control the NVIC IRQ channel mapped to the external interrupt controller (EXTI) so that an interrupt coming from any one of the 23 lines can be correctly acknowledged.

Hardware event selection

To configure the 23 lines as event sources, use the following procedure:

- Configure the mask bits of the 23 Event lines (EXTI_EMR)
- Configure the Trigger Selection bits of the Event lines (EXTI_RTISR and EXTI_FTSR)

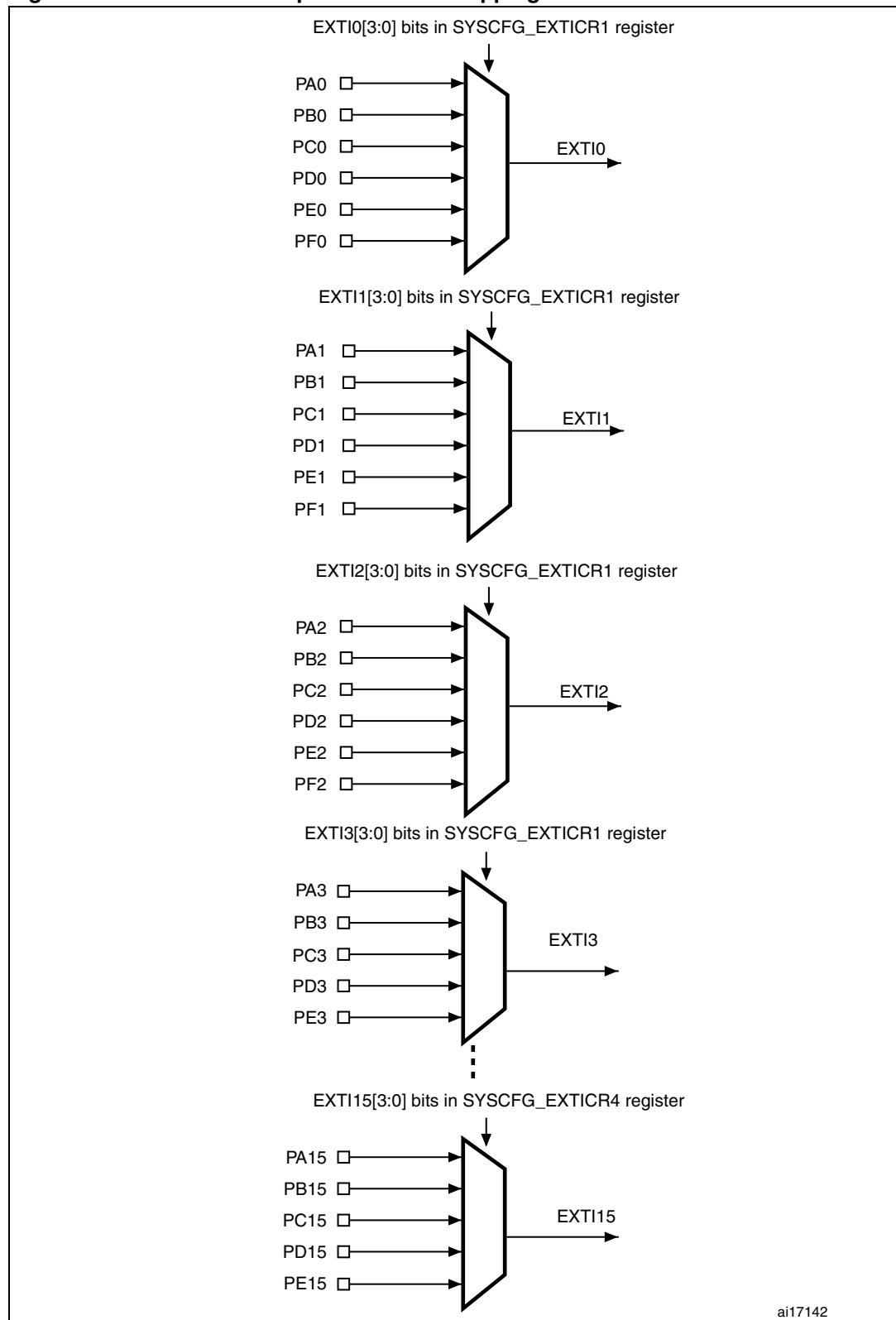
Software interrupt/event selection

The 23 lines can be configured as software interrupt/event lines. The following is the procedure to generate a software interrupt.

- Configure the mask bits of the 23 Interrupt/Event lines (EXTI_IMR, EXTI_EMR)
- Set the required bit in the software interrupt register (EXTI_SWIER)

7.2.5 External interrupt/event line mapping

The 83 GPIOs are connected to the 16 external interrupt/event lines in the following manner:

Figure 23. External interrupt/event GPIO mapping

The six other EXTI lines are connected as follows:

- EXTI line 16 is connected to the PVD output
- EXTI line 17 is connected to the RTC Alarm event
- EXTI line 18 is connected to the USB Device FS wakeup event
- EXTI line 19 is connected to the RTC Tamper and TimeStamp events
- EXTI line 20 is connected to the RTC Wakeup event
- EXTI line 21 is connected to the Comparator 1 wakeup event and EXTI line 22 is connected to the Comparator 2 wakeup event

7.3 EXTI registers

Refer to [Section 1.1](#) for a list of abbreviations used in register descriptions.

The peripheral registers have to be accessed by words (32-bit).

7.3.1 EXTI interrupt mask register (EXTI_IMR)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									MR22	MR21	MR20	MR19	MR18	MR17	MR16
									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value (0).

Bits 22:0 **MRx**: Interrupt mask on line x

0: Interrupt request from Line x is masked

1: Interrupt request from Line x is not masked

7.3.2 EXTI event mask register (EXTI_EMR)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									MR22	MR21	MR20	MR19	MR18	MR17	MR16
									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value (0).

Bits 22:0 **MRx**: Event mask on line x

0: Event request from Line x is masked

1: Event request from Line x is not masked

7.3.3 EXTI rising edge trigger selection register (EXTI_RTSTR)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									TR22	TR21	TR20	TR19	TR18	TR17	TR16
									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value (0).

Bits 22:0 **TRx**: Rising edge trigger event configuration bit of line x

0: Rising edge trigger disabled (for Event and Interrupt) for input line x

1: Rising edge trigger enabled (for Event and Interrupt) for input line x

Note: *The external wakeup lines are edge triggered, no glitch must be generated on these lines. If a rising edge on the external interrupt line occurs while writing to the EXTI_RTSTR register, the pending bit will not be set.*

Rising and falling edge triggers can be set for the same interrupt line. In this configuration, both generate a trigger condition.

7.3.4 Falling edge trigger selection register (EXTI_FTSR)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									TR22	TR21	TR20	TR19	TR18	TR17	TR16
									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value (0).

Bits 22:0 **TRx**: Falling edge trigger event configuration bit of line x

0: Falling edge trigger disabled (for Event and Interrupt) for input line x

1: Falling edge trigger enabled (for Event and Interrupt) for input line x

Note: The external wakeup lines are edge triggered, no glitch must be generated on these lines. If a falling edge on the external interrupt line occurs while writing to the EXTI_FTSR register, the pending bit will not be set.

Rising and falling edge triggers can be set for the same interrupt line. In this configuration, both generate a trigger condition.

7.3.5 EXTI software interrupt event register (EXTI_SWIER)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									SWIER 22	SWIER 21	SWIER 20	SWIER 19	SWIER 18	SWIER 17	SWIER 16
									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWIER 15	SWIER 14	SWIER 13	SWIER 12	SWIER 11	SWIER 10	SWIER 9	SWIER 8	SWIER 7	SWIER 6	SWIER 5	SWIER 4	SWIER 3	SWIER 2	SWIER 1	SWIER 0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value (0).

Bits 22:0 **SWIERx**: Software interrupt on line x

Writing a 1 to this bit when it is at 0 sets the corresponding pending bit in EXTI_PR. If the interrupt is enabled on this line in EXTI_IMR and EXTI_EMR, an interrupt request is generated.

This bit is cleared by clearing the corresponding bit in EXTI_PR (by writing a 1 to this bit).

7.3.6 EXTI pending register (EXTI_PR)

Address offset: 0x14

Reset value: undefined

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									PR22	PR21	PR20	PR19	PR18	PR17	PR16
									rw	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PR15	PR14	PR13	PR12	PR11	PR10	PR9	PR8	PR7	PR6	PR5	PR4	PR3	PR2	PR1	PR0
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Bits 31:23 Reserved, must be kept at reset value (0).

Bits 22:0 **PRx**: Pending bit

0: No trigger request occurred

1: The selected trigger request occurred

This bit is set when the selected edge event arrives on the external interrupt line. This bit is cleared by writing it to 1 or by changing the sensitivity of the edge detector.

7.3.7 EXTI register map

The following table gives the EXTI register map and the reset values.

Table 29. External interrupt/event controller register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	EXTI_IMR	Reserved										MR[22:0]																					
	Reset value																															0	0
0x04	EXTI_EMR	Reserved										MR[22:0]																					
	Reset value																															0	0
0x08	EXTI_RTSTR	Reserved										TR[22:0]																					
	Reset value																															0	0
0x0C	EXTI_FTSTR	Reserved										TR[22:0]																					
	Reset value																															0	0
0x10	EXTI_SWIER	Reserved										SWIER[22:0]																					
	Reset value																															0	0
0x14	EXTI_PR	Reserved										PR[22:0]																					
	Reset value																															0	0

Refer to [Table 1 on page 32](#) for the register boundary addresses.

9 Analog-to-digital converter (ADC)

9.1 ADC introduction

The 12-bit ADC is a successive approximation analog-to-digital converter. It has up to 26 multiplexed channels allowing it measure signals from 24 external and two internal sources. The A/D conversion of the channels can be performed in single, continuous, scan or discontinuous mode. The result of the ADC is stored into a left- or right-aligned 16-bit data register.

The analog watchdog feature allows the application to detect if the input voltage goes beyond the user-defined, higher or lower thresholds.

Conversions are always performed at maximum speed to have the highest possible conversion rate for a given system clock frequency. The automatic power control dramatically reduces the consumption by powering-on the ADC only during conversions.

9.2 ADC main features

- 12-bit, 10-bit, 8-bit or 6-bit configurable resolution
- Interrupt generation at the end of regular conversions, end of injected conversions, and in case of analog watchdog or overrun events (for regular conversions)
- Single and continuous conversion modes
- Scan mode for automatic conversions in a fully programmable order
- Programmable data alignment with in-built data coherency
- Programmable and individual sampling time for each ADC channel
- External trigger option with configurable edge detection for both regular and injected conversions
- Discontinuous mode
- ADC conversion time: 1 μ s at full speed (ADC clocked at 16 MHz) down to 4 μ s at low speed (ADC clocked at 4 MHz), independent of the APB clock
- Automatic power-up/power-down to reduce the power consumption
- ADC supply requirements:
 - 2.4 V to 3.6 V at full speed or with reference zooming ($V_{REF+} < V_{DDA}$)
 - down to 1.8 V at slower speeds
- ADC input range: $V_{REF-} \leq V_{IN} \leq V_{REF+}$
- Automatic programmable hardware delay insertion between conversions
- DMA request generation during regular channel conversion

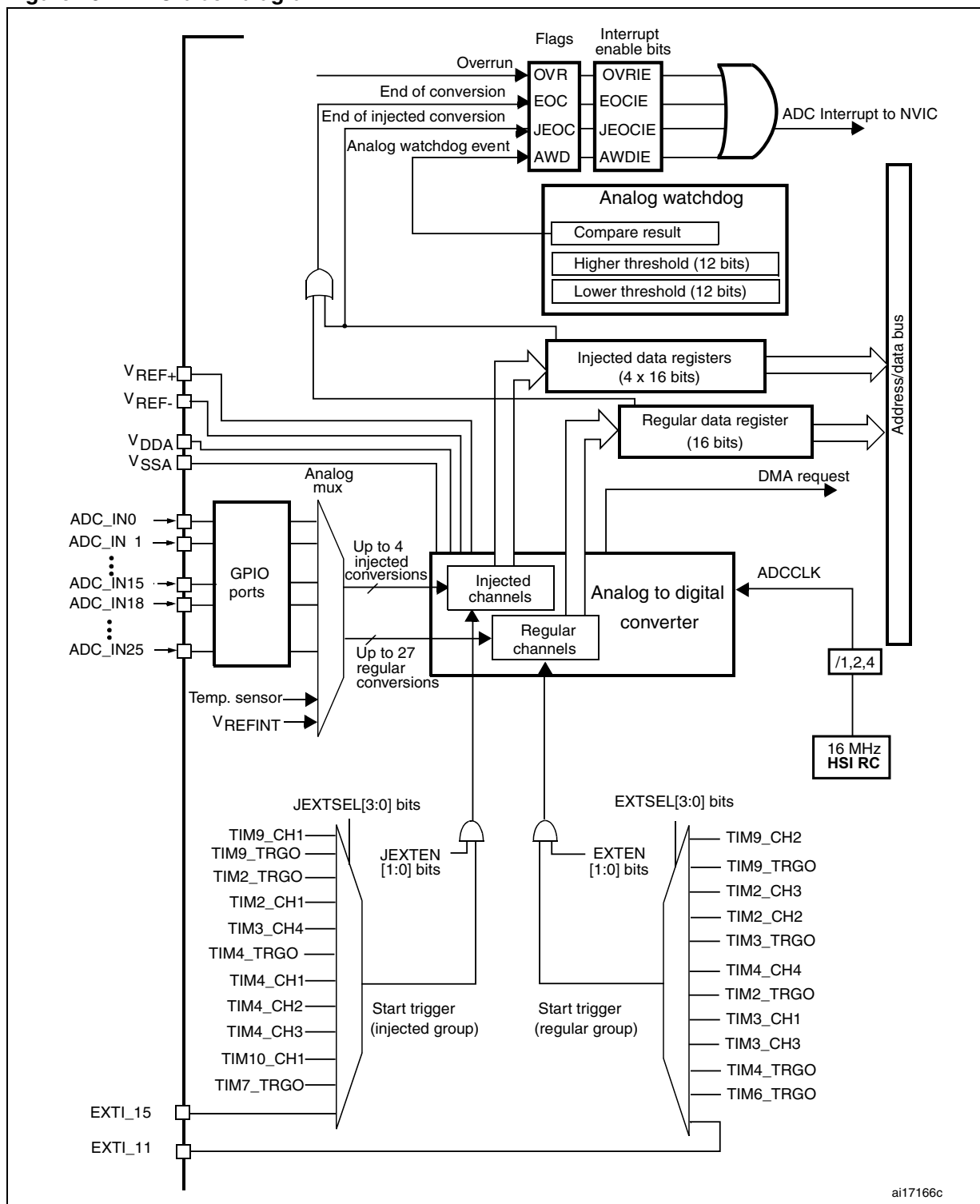
Figure 26 shows the block diagram of the ADC.

Note: V_{REF-} , if available (depending on package), must be tied to V_{SSA} .

9.3 ADC functional description

Figure 26 shows the ADC block diagram, *Table 34* gives the pin description.

Figure 26. ADC block diagram



Note: Due to internal connections (ADC multiplexer switches), ADC channels 4, 5, 22, 23, 24 and 25 are considered as fast channels (up to 1 Msample/s) and the other channels are slower (they do not exceed 800 ksamples/s). For more details, refer to [Figure 20: Routing interface \(RI\) block diagram on page 127](#).

Table 34. ADC pins

Name	Signal type	Remarks
V_{REF+}	Input, analog reference positive	The higher/positive reference voltage for the ADC is: $2.4V \leq V_{REF+} = V_{DDA}$ for full speed (ADCCLK = 16 MHz, 1 Msps) $1.8V \leq V_{REF+} = V_{DDA}$ for medium speed (ADCCLK = 8 MHz, 500 Ksps) $2.4V \leq V_{REF+} \neq V_{DDA}$ for medium speed (ADCCLK = 8 MHz, 500 Ksps) $1.8V \leq V_{REF+} < V_{DDA}$ for low speed (ADCCLK = 4 MHz, 250 Ksps) When product voltage range 3 is selected ($V_{CORE} = 1.2 V$), the ADC is low speed (ADCCLK = 4 MHz, 250 Ksps)
V_{DDA}	Input, analog supply	Analog power supply equal to V_{DD} and $2.4 V \leq V_{DDA} \leq V_{DD}$ (3.6 V) for full speed $1.8 V \leq V_{DDA} \leq V_{DD}$ (3.6 V) for medium and low speed
V_{REF-}	Input, analog reference negative	The lower/negative reference voltage for the ADC, $V_{REF-} = V_{SSA}$
V_{SSA}	Input, analog supply ground	Ground for analog power supply equal to V_{SS}
ADC_IN[15:0] and ADC_IN[25:18]	Analog input signals	24 analog input channels

9.3.1 ADC power on-off control

The ADC is powered on by setting the ADON bit in the ADC_CR2 register. When the ADON bit is set for the first time, it wakes up the ADC from the Power-down mode.

Conversion starts when either the SWSTART or the JSWSTART bit is set, or in response to an external trigger. These software or hardware triggers must be enabled only when the ADC is ready to convert (ADONS=1).

Resetting the ADON bit stops the conversions and put the ADC in power down mode. In this mode the ADC consumes almost no power. ADONS is cleared after ADON has been synchronized to the ADCCLK clock domain.

Note: *Due to the latency introduced by the synchronization between the two clock domains, ADON must be set only when ADONS=0 and it must be cleared only when the ADC is ready to convert (ADONS=1).*

Power down configurations (PDI and PDD)

In order to reduce the consumption when the ADC is ready to convert (ADONS=1), the ADC can be automatically powered off when it is not converting, until the next conversion starts depending on the PDI and PDD bits in the ADC_CR1 register. Refer to [Section 9.10: Power saving on page 188](#) for more details.

Using the PDI bit, the user can determine whether the ADC is powered up or down when it is not converting (waiting for a hardware or software trigger event).

Using the PDD bit, the user can determine whether the ADC is powered up or down between 2 conversions (or sequences of conversions) when a delay is inserted (DELS bits).

When PDI=1, ADONS is the image of ADON (same value) as viewed from the ADCCLK clock.

Conversion starts after the ADC power-up time (t_{STAB}) when either the SWSTART or the JSWSTART bit is set, or in response to an external trigger. These software or hardware triggers must be enabled only when the ADC is ready to convert (ADONS=1).

Resetting the ADON bit stops the conversions and places the ADC in a mode where it is no longer supplied.

Note: Due to the latency introduced by the synchronization between the two clock domains, ADON must be set only when ADONS=0 and it must be cleared only when ADONS=1.

9.3.2 ADC clock

To avoid unnecessary consumption while not converting, the ADC digital interface has been designed to operate in a completely independent manner, at its maximum speed using an internal 16 MHz clock source (HSI), whatever the CPU operating frequency (which can range from a few sub-kHz up to 32 MHz).

Note: When entering Stop mode, the ADC analog and digital interfaces remain inactive as the HSI and PCLK2 are disabled. Since the HSI is still deactivated after resuming from Stop mode, the user must enable the HSI as the ADC analog interface clock source and continue using ADC conversions.

The ADCCLK clock is provided by the clock controller. It is generated from the HSI oscillator after a clock divider:

- by 1 for full speed ($f_{ADCCLK} = 16 \text{ MHz}$)
- by 2 for medium speed and by 4 for low speed ($f_{ADCCLK} = 4 \text{ MHz}$)

Depending on the APB clock (PCLK) frequency, the ADCCLK clock frequency can be higher or lower than PCLK. In particular, when the APB becomes too low, it can become difficult to get the results of conversions at full speed without losing any data (because the data flow is higher than what the CPU or the DMA can handle). This problem can be solved by inserting a delay between 2 conversions or between 2 sequences of conversions in order to give the system enough time to read and save the converted data before the next data arrive. Refer to [Section 9.9: Hardware freeze and delay insertion modes for slow conversions on page 185](#) for more details.

9.3.3 Channel selection

There are 26 multiplexed channels. It is possible to organize the conversions in two groups: regular and injected. A group consists of a sequence of conversions that can be done on any channel and in any order. For instance, it is possible to implement the conversion sequence in the following order: Ch3, Ch8, Ch2, Ch2, Ch0, Ch2, Ch2, Ch15.

- A **regular group** is composed of up to 27 conversions. The regular channels and their order in the conversion sequence must be selected in the ADC_SQRx registers. The total number of conversions, which can be up to 27 in the regular group must be written in the L[4:0] bits in the ADC_SQR1 register.

- An **injected group** is composed of up to 4 conversions. The injected channels and their order in the conversion sequence must be selected in the ADC_JSQR register. The total number of conversions, which can be up to 4 in the injected group must be written in the L[1:0] bits in the ADC_JSQR register.

Note: If the ADC_SQRx register is modified during a regular conversion or the ADC_JSQR register is modified during an injected conversion, the current conversion is reset and the ADC waits for a new start pulse. If the conversion that is reset is an injected conversion that had interrupted a regular conversion, then the regular conversion is resumed.

Temperature sensor, V_{REFINT} internal channels

The temperature sensor is connected to channel ADCx_IN16 and the internal reference voltage V_{REFINT} is connected to ADCx_IN17. These two internal channels can be selected and converted as injected or regular channels.

9.3.4 Single conversion mode

In Single conversion mode the ADC does one conversion. This mode is started with the CONT bit in the ADC_CR2 at 0 by either:

- setting the SWSTART bit in the ADC_CR2 register (for a regular channel only)
- setting the JSWSTART bit (for an injected channel)
- external trigger (for a regular or injected channel)

Once the conversion of the selected channel is complete:

- If a regular channel was converted (converted channel is selected by the SQ1[4:0] bits in the SQR5 register):
 - The converted data are stored into the 16-bit ADC_DR register
 - The EOC (end of conversion) flag is set
 - An interrupt is generated if the EOCIE bit is set
- If an injected channel was converted (converted channel is selected by the JSQ1[4:0] bits in the JSQR register):
 - The converted data are stored into the 16-bit ADC_JDR1 register
 - The JEOC (end of conversion injected) flag is set
 - An interrupt is generated if the JEOCIE bit is set

Then the ADC stops.

9.3.5 Continuous conversion mode

In continuous conversion mode, the ADC starts a new conversion as soon as it finishes one. This mode is started with the CONT bit at 1 either by external trigger or by setting the SWSTART bit in the ADC_CR2 register (for regular channels only).

After each conversion:

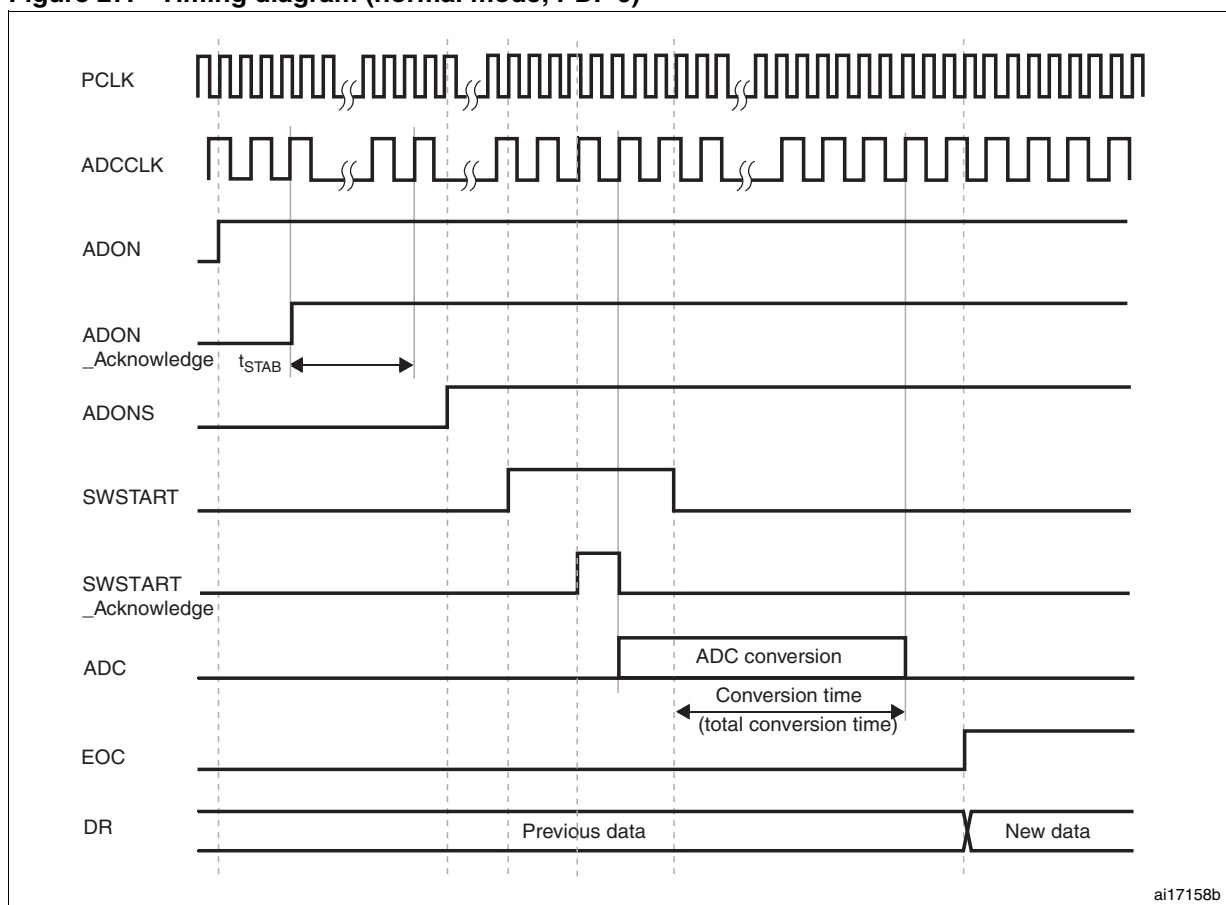
- If a regular channel was converted (converted channel is selected by the SQ1[4:0] bits in the SQR5 register):
 - The last converted data are stored into the 16-bit ADC_DR register
 - The EOC (end of conversion) flag is set
 - An interrupt is generated if the EOCIE bit is set

Note: *Injected channels cannot be converted continuously. The only exception is when an injected channel is configured to be converted automatically after regular channels in continuous mode (using JAUTO bit), refer to [Auto-injected conversion](#) section).*

9.3.6 Timing diagram

As shown in [Figure 27](#), the ADC needs a stabilization time (t_{STAB}) before it can actually convert. The ADON bit is set when a conversion can be triggered. A conversion is launched when the SWSTART bit is set (or when an external trigger is detected). After the conversion time (programmable sampling time + 12 ADCCLK clock cycles for 12-bit data), the EOC flag is set and the ADC data register contains the result of the conversion. Note that some delays are needed to resynchronize the different signals from one clock domain to the other.

Figure 27. Timing diagram (normal mode, PDI=0)



9.3.7 Analog watchdog

The AWD analog watchdog status bit is set if the analog voltage converted by the ADC is below a lower threshold or above a higher threshold. These thresholds are programmed in the 12 least significant bits of the ADC_HTR and ADC_LTR 16-bit registers. An interrupt can be enabled by using the AWDIE bit in the ADC_CR1 register.

The threshold value is independent of the alignment selected by the ALIGN bit in the ADC_CR2 register. The analog voltage is compared to the lower and higher thresholds before alignment.

Table 35 shows how the ADC_CR1 register should be configured to enable the analog watchdog on one or more channels.

Figure 28. Analog watchdog's guarded area

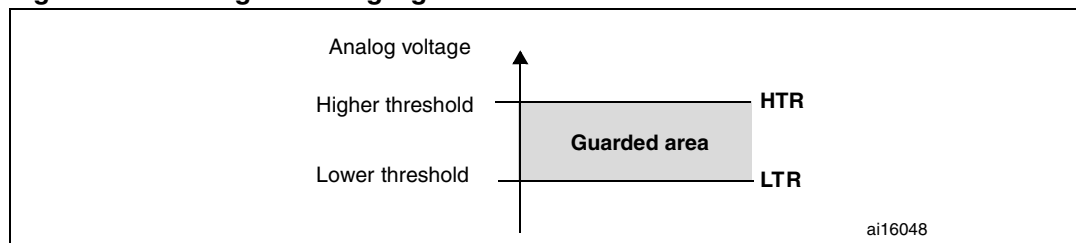


Table 35. Analog watchdog channel selection

Channels guarded by the analog watchdog	ADC_CR1 register control bits (x = don't care)		
	AWDSGL bit	AWDEN bit	JAWDEN bit
None	x	0	0
All injected channels	0	0	1
All regular channels	0	1	0
All regular and injected channels	0	1	1
Single ⁽¹⁾ injected channel	1	0	1
Single ⁽¹⁾ regular channel	1	1	0
Single ⁽¹⁾ regular or injected channel	1	1	1

1. Selected by the AWDCH[4:0] bits

9.3.8 Scan mode

This mode is used to scan a group of analog channels.

The Scan mode is selected by setting the SCAN bit in the ADC_CR1 register. Once this bit has been set, the ADC scans all the channels selected in the ADC_SQRx registers (for regular channels) or in the ADC_JSQR register (for injected channels). A single conversion is performed for each channel of the group. After each end of conversion, the next channel in the group is converted automatically. If the CONT bit in the ADC_CR2 register is set, regular channel conversion does not stop at the last selected channel in the group but continues again from the first selected channel.

If the DMA bit is set, the direct memory access (DMA) controller is used to transfer the data converted from the regular group of channels (stored in the ADC_DR register) to memory after each regular channel conversion.

The EOC bit is set in the ADC_SR register if:

- At the end of each regular group sequence the EOCS bit is cleared to 0
- At the end of each regular channel conversion the EOCS bit is set to 1

The data converted from an injected channel is always stored into the ADC_JDRx registers.

9.3.9 Injected channel management

Triggered injected conversion

To use triggered injection, the JAUTO bit must be cleared in the ADC_CR1 register.

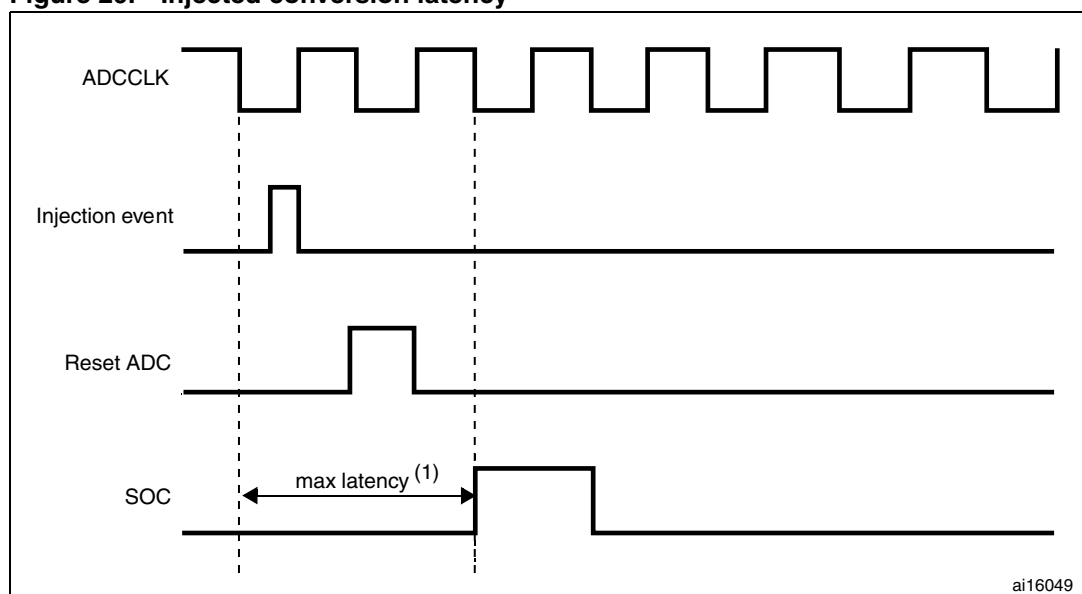
1. Start the conversion of a group of injected channels either by external trigger or by setting the JSWSTART bit in the ADC_CR2 register.
2. If an external injected trigger occurs or if the JSWSTART bit is set during the conversion of a regular group of channels, the current conversion is reset and the injected channel sequence switches to Scan-once mode.
3. Then, the regular conversion of the regular group of channels is resumed from the last interrupted regular conversion.

If a regular event occurs during an injected conversion, the injected conversion is not interrupted but the regular sequence is executed at the end of the injected sequence.

Figure 29 shows the corresponding timing diagram.

Note: When using triggered injection, one must ensure that the interval between trigger events is longer than the injection sequence. For instance, if the sequence length is 30 ADC clock cycles (that is two conversions with a sampling time of 3 clock periods), the minimum interval between triggers must be 31 ADC clock cycles.

Figure 29. Injected conversion latency



1. The maximum latency value can be found in the electrical characteristics of the STM32L15xxx datasheet.

Auto-injected conversion

If the JAUTO bit is set, then the channels in the injected group are automatically converted after the regular group of channels. This can be used to convert a sequence of up to 31 conversions programmed in the ADC_SQRx and ADC_JSQR registers.

In this mode, external trigger on injected channels must be disabled.

If the CONT bit is also set in addition to the JAUTO bit, regular channels followed by injected channels are continuously converted.

Note: It is not possible to use both the auto-injected and discontinuous modes simultaneously.

9.3.10 Discontinuous mode

Regular group

This mode is enabled by setting the DISCEN bit in the ADC_CR1 register. It can be used to convert a short sequence of n conversions ($n \leq 8$) that is part of the sequence of conversions selected in the ADC_SQRx registers. The value of n is specified by writing to the DISCNUM[2:0] bits in the ADC_CR1 register.

When an external trigger occurs, it starts the next n conversions selected in the ADC_SQRx registers until all the conversions in the sequence are done. The total sequence length is defined by the L[4:0] bits in the ADC_SQR1 register.

Example:

$n = 3$, regular channels to be converted = 0, 1, 2, 3, 6, 7, 9, 10
 1st trigger: sequence converted 0, 1, 2
 2nd trigger: sequence converted 3, 6, 7
 3rd trigger: sequence converted 9, 10 and an EOC event generated
 4th trigger: sequence converted 0, 1, 2

Note: *When a regular group is converted in discontinuous mode, no rollover occurs.*
When all subgroups are converted, the next trigger starts the conversion of the first subgroup. In the example above, the 4th trigger reconverts the channels 0, 1 and 2 in the 1st subgroup.

Injected group

This mode is enabled by setting the JDISCEN bit in the ADC_CR1 register. It can be used to convert a short sequence of n conversions ($n \leq 3$) part of the sequence of conversions selected in the ADC_JSQR registers. The value of n is specified by writing to the DISCNUM[2:0] bits in the ADC_CR1 register.

When an external trigger occurs, it starts the next channel conversions selected in the ADC_JSQR registers until all the conversions in the sequence are done. The total sequence length is defined by the JL[1:0] bits in the ADC_JSQR register.

Example:

$n = 1$, injected channels to be converted = 1, 2, 3
 1st trigger: channel 1 converted
 2nd trigger: channel 2 converted
 3rd trigger: channel 3 converted and EOC and JEOC events generated
 4th trigger: channel 1

- Note:**
- 1 *When all injected channels are converted, the next trigger starts the conversion of the first injected channel. In the example above, the 4th trigger reconverts the 1st injected channel 1.*
 - 2 *It is not possible to use both the auto-injected and discontinuous modes simultaneously.*
 - 3 *Discontinuous mode must not be set for regular and injected groups at the same time.*

9.4 Data alignment

The ALIGN bit in the ADC_CR2 register selects the alignment of the data stored after conversion. Data can be right- or left-aligned as shown in [Figure 30](#) and [Figure 31](#).

The converted data value from the injected group of channels is decreased by the user-defined offset written in the ADC_JOFRx registers so the result can be a negative value. The SEXT bit represents the extended sign value.

For channels in a regular group, no offset is subtracted so only twelve bits are significant.

Figure 30. Right alignment of 12-bit data

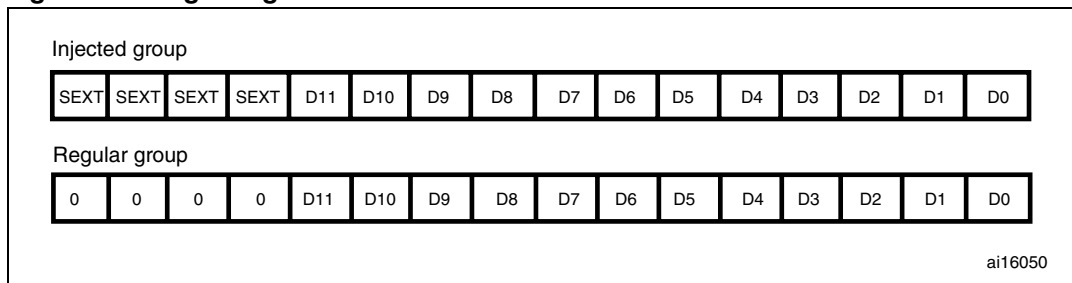
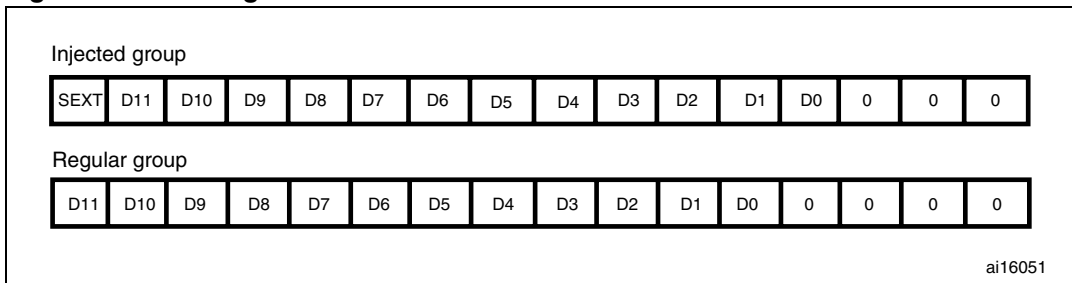
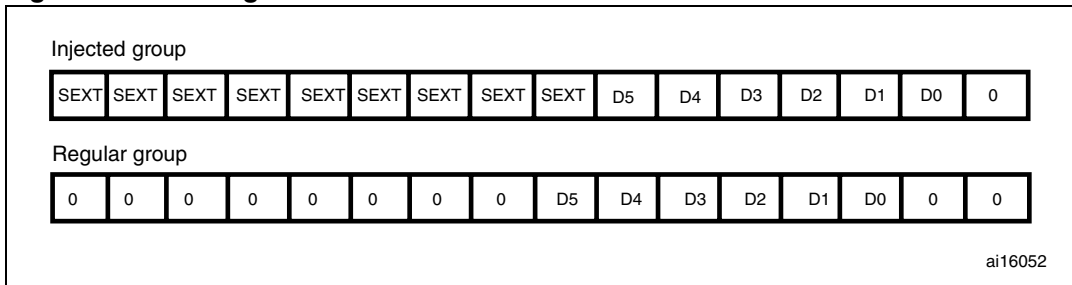


Figure 31. Left alignment of 12-bit data



Special case: when left-aligned, the data are aligned on a half-word basis except when the resolution is set to 6-bit. In that case, the data are aligned on a byte basis as shown in [Figure 32](#).

Figure 32. Left alignment of 6-bit data



9.5 Channel-wise programmable sampling time

The ADC samples the input voltage for a number of ADCCLK cycles that can be modified using the SMP[2:0] bits in the ADC_SMPRx registers (x = 1 to 3). Each channel can be sampled with a different sampling time.

The total conversion time is calculated as follows:

$$T_{\text{conv}} = \text{Sampling time} + \text{channel conversion time}$$

Example:

With ADCCLK = 16 MHz and sampling time = 4 cycles:

$T_{\text{conv}} = 4 + 12 = 16$ cycles = 1 μs (for 12-bit conversion)

$T_{\text{conv}} = 4 + 7 = 11$ cycles = 685 ns (for 6-bit conversion)

9.6 Conversion on external trigger

Conversion can be triggered by an external event (e.g. timer capture, EXTI line). If the EXTEN[1:0] control bits (for a regular conversion) or JEXTEN[1:0] bits (for an injected conversion) are different from “0b00”, then external events are able to trigger a conversion with the selected edge. [Table 36](#) provides the correspondence between the EXTEN[1:0] and JEXTEN[1:0] values and the trigger edge.

Table 36. Configuring the trigger edge detection

Source	EXTEN[1:0] / JEXTEN[1:0]
Trigger detection disabled	00
Detection on the rising edge	01
Detection on the falling edge	10
Detection on both the rising and falling edges	11

Note: The edge detection of the external trigger can be changed on the fly.

The EXTSEL[3:0] and JEXTSEL[3:0] control bits are used to select which out of 16 possible events can trigger conversion for the regular and injected groups.

[Table 37](#) gives the possible external trigger for regular conversion.

Table 37. External trigger for regular channels

Source	Type	EXTSEL[3:0]
TIM9_CC2 event	Internal signal from on-chip timers	0000
TIM9_TRGO event		0001
TIM2_CC3 event		0010
TIM2_CC2 event		0011
TIM3_TRGO event		0100
TIM4_CC4 event		0101
TIM2_TRGO event		0110
TIM3_CC1 event		0111
TIM3_CC3 event		1000
TIM4_TRGO event		1001
TIM6_TRGO event		1010

Table 37. External trigger for regular channels (continued)

Source	Type	EXTSEL[3:0]
Reserved	NA	1011
Reserved		1100
Reserved		1101
Reserved		1110
EXTI line11	External pin	1111

[Table 38](#) gives the possible external trigger for injected conversion.

Table 38. External trigger for injected channels

Source	Type	EXTSEL[3:0]
TIM9_CC1 event	Internal signal from on-chip timers	0000
TIM9_TRGO event		0001
TIM2_TRGO event		0010
TIM2_CC1 event		0011
TIM3_CC4 event		0100
TIM4_TRGO event		0101
TIM4_CC1 event		0110
TIM4_CC2 event		0111
TIM4_CC3 event		1000
TIM10_CC1 event		1001
TIM7_TRGO event		1010
Reserved	NA	1011
Reserved		1100
Reserved		1101
Reserved		1110
EXTI line15	External pin	1111

A regular group conversion can be interrupted by an injected trigger.

Note: 1 *The trigger selection can be changed on the fly. When this is done, however, trigger detection is disabled for a period of 2 PCLK cycles. This is to avoid spurious detections during the transition.*

2 *The interval between trigger events must be longer than:*

- *the sequence for regular conversions*
- *the sequence + 1 ADCCLK cycle for injected conversions*

For instance, if the sequence length is 32 ADC clock cycles (that is two conversions with a 4 clock-period sampling time), the minimum interval between regular triggers must be greater

than 32 ADC clock cycles and the interval between injected triggers must be greater than 33 ADC clock cycles.

9.7 Aborting a conversion

9.7.1 Injected channels

An injected conversion or a sequence of conversions can be stopped by writing to the JSQR register (the JL[1:0] bitfield has to be written with its current value). Then any ongoing injected conversion aborts and any pending trigger is reset. A new injected conversion can start when a new hardware or software trigger occurs.

After aborting an injected conversion, the system requires a few clock cycles before a new injected conversion can start (3 to 5 ADC clock cycles + 2 to 5 APB clock cycles). To meet this requirement, JSWSTART should not be set before JCNR=0.

9.7.2 Regular channels

A regular conversion or a sequence of conversions can be stopped by writing to any of the SQR1 to SQR5 registers (if SQR1 is written, the L[4:0] bitfield has to be written with its current value). The ADC then behaves in the same way as in the case of injected conversions (see [Section 9.7.2: Regular channels](#)).

If several of the SQRi registers have to be written in order to configure a new sequence, no conversion should be launched between the different write accesses. In this case, the following sequence must be applied:

1. Disable the external triggers by writing the EXTEN bits to 00 (when external triggers are used)
2. Change the sequence configuration (by writing to the SQRi registers)
3. Wait for RCNR=0 in the ADC_SR register
4. Enable the external trigger or set the SWSTART bit

9.8 Conversion resolution

It is possible to perform faster conversion by reducing the ADC resolution. The RES[1:0] bits are used to select the number of bits available in the data register. The minimal conversion time for each resolution, when the sampling time is 4 cycles, is then as follows:

- for 12-bit resolution : $12 + 4 = 16$ cycles
- for 10-bit resolution : $11 + 4 = 15$ cycles
- for 8-bit resolution : $9 + 4 = 13$ cycles
- for 6-bit resolution : $7 + 4 = 11$ cycles

9.9 Hardware freeze and delay insertion modes for slow conversions

When the APB clock is not fast enough to manage the data rate, a delay can be introduced between conversions to reduce this data rate. The delay is inserted after each regular

conversion and after each sequence of injected conversions as, during conversion, a trigger event (for the same group of conversions) occurring during this delay is ignored.

No delay is inserted between conversions of different groups (a regular conversion followed by an injected conversion or conversely):

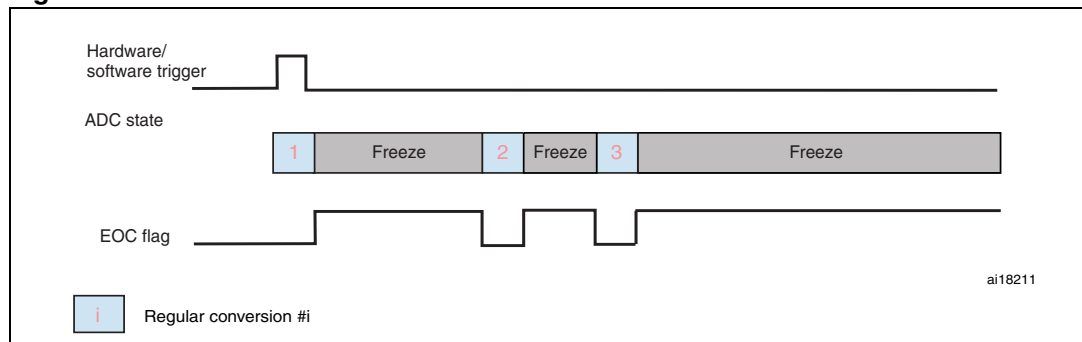
- If an injected trigger occurs during the delay of a regular conversion, the injected conversion starts immediately.
- If a regular conversion is to be resumed after being interrupted by an injected sequence, it starts as soon as the delay of the previous regular conversion is finished.

The behavior is slightly different in auto-injected mode where a new regular conversion can start only when the delay of the previous injected conversion has ended. This is to ensure that the software can read all the data of a given sequence before starting a new sequence. In this mode, a regular trigger is ignored if it occurs during the delay that follows a regular conversion. It is however considered pending if it occurs after this delay, even if it occurs during an injected sequence or the delay that follows it. The conversion then starts at the end of the delay of the injected sequence.

The length of the delay is configured using the DELS[2:0] bits in the ADC_CR2 register. Two cases should be considered:

- **ADC freeze mode:**
When DELS[2:0]=001, a new conversion can start only if all the previous data of the same group have been treated:
 - for a regular conversion: once the ADC_DR register has been read or if the EOC bit has been cleared
 - for an injected conversion: when the JEOC bit has been cleared
- **ADC delay insertion mode:**
When DELS[2:0]>001, a new conversion can start only after a given number of APB clock cycles after the end of the previous conversion in the same group.

Figure 33. ADC freeze mode

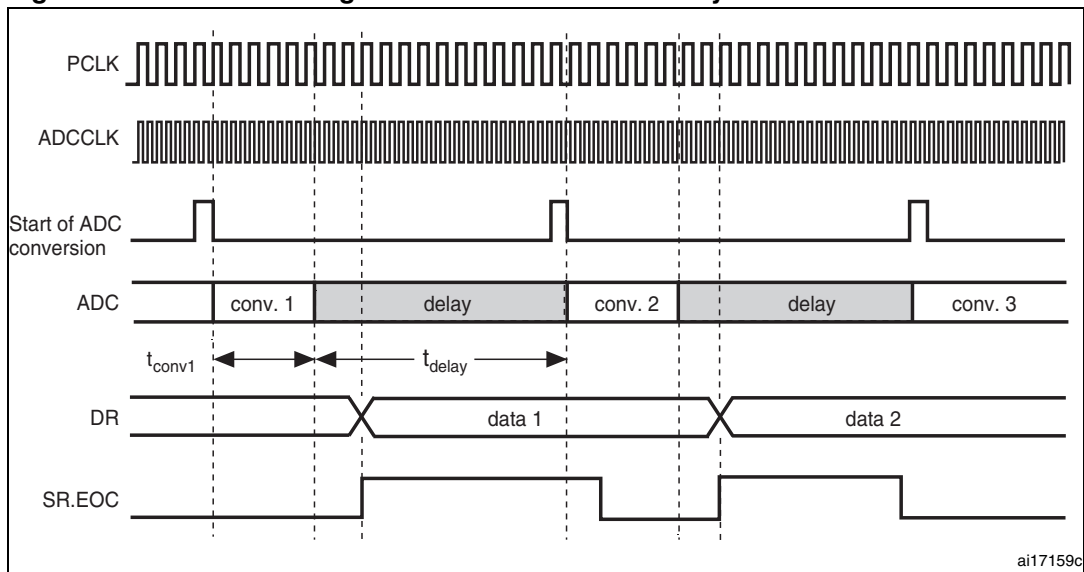


9.9.1 Inserting a delay after each regular conversion

When enabled, a delay is inserted at the end of each regular conversion before a new regular conversion can start. It gives time to read the converted data in the ADC_DR register before a new regular conversion is completed. The length of the delay is configured by the DELS[2:0] bits. [Figure 34](#) shows an example of continuous regular conversions where a 10 PCLK cycle delay is inserted after each conversion.

Note: When `ADC_CR2_EOCS = 1`, the delay is inserted after each sequence of regular group conversions.

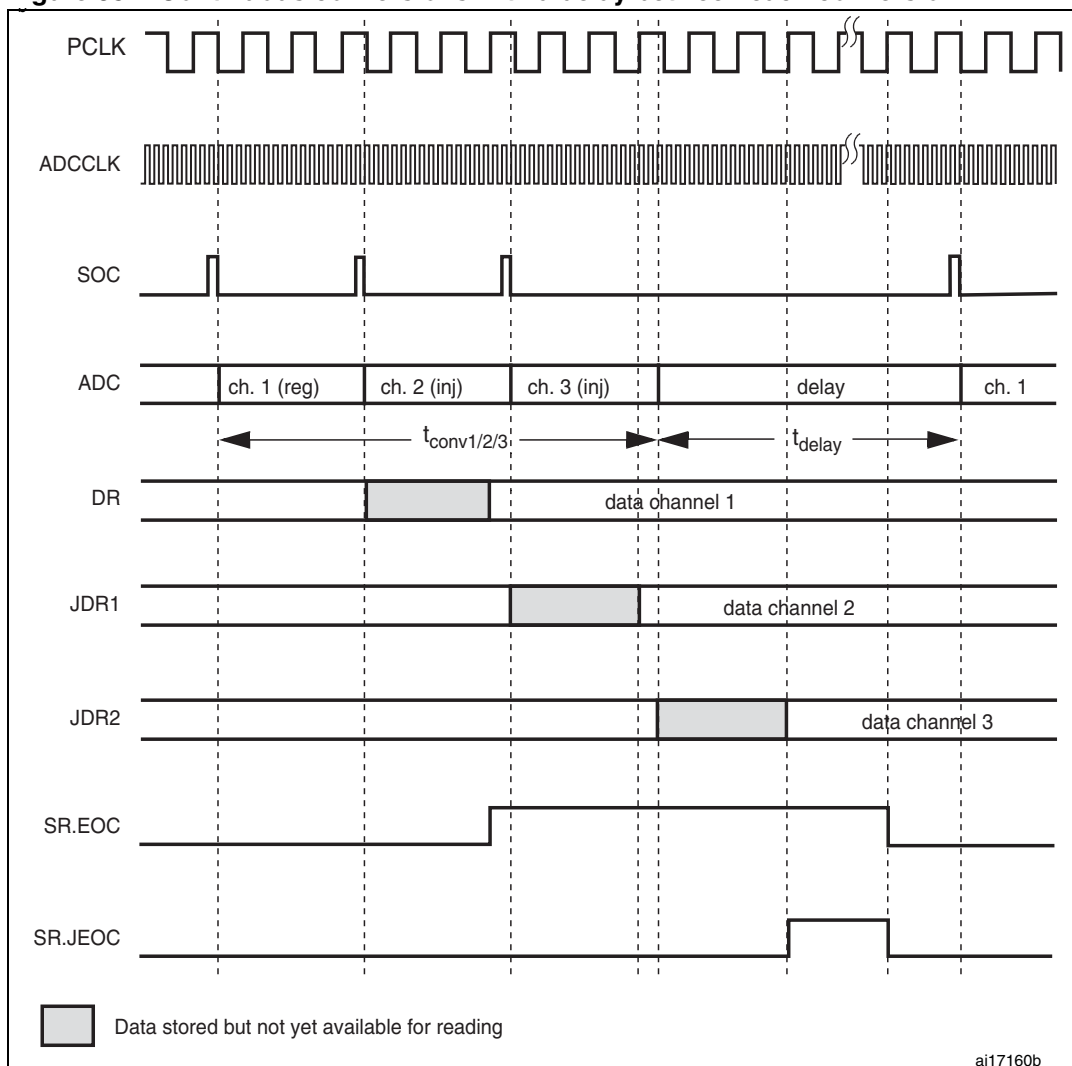
Figure 34. Continuous regular conversions with a delay



1. t_{conv1} : including sampling and conversion times (for instance 16 ADC clock cycles with the minimum sampling time)
2. t_{delay} : delay from the end of a conversion to the start of the next conversion (number of PCLK periods configured with the `DELS[2:0]` bits) + delay to synchronize the end of conversion (0 to 1 PCLK clock cycles) + delay to synchronize the end of delay (2 or 3 ADC clock cycles).

9.9.2 Inserting a delay after each sequence of auto-injected conversions

When enabled, a delay is inserted at the end of each sequence of injected conversions. Up to 5 conversion results can be stored into the `ADC_DR` and the `ADC_JDRx` registers. The length of the delay is configured by the `DELS[2:0]` bits. [Figure 35](#) shows an example of continuous conversions (the `CONT` bit is set) where a delay is inserted after each sequence of injected conversions. Here the `JAUTO` bit is set and the sequence ends after the last injected conversion (the sequence is made of 1 regular conversion + 2 injected conversions).

Figure 35. Continuous conversions with a delay between each conversion

1. $t_{\text{conv1/2/3}}$: including sampling and conversion times for channels 1, 2 and 3.
2. t_{delay} : delay from the end of the previous sequence to the start of the new sequence (number of PCLK periods configured with the DELS bits) + delay to synchronize the end of conversion (0 to 1 PCLK clock cycles) + delay to synchronize the end of delay (2 or 3 ADC clock cycles).

9.10 Power saving

ADC power-on and power-off can be managed by hardware to cut the consumption when the ADC is not converting. The ADC can be powered down:

- during the delay described above (when the PDD bit is set). Then the ADC is powered up again at the end of the delay and/or
- when the ADC is waiting for a trigger event (when the PDI bit is set). In this case the ADC is powered up at the next trigger event.

The ADC needs a certain time to start up before a conversion can actually be launched. This startup time must be taken into account before selecting the automatic power control modes or when configuring the delay. For this reason, it is also more efficient (from the

power point of view and when possible) when scanning several channels to launch a sequence of several conversions and stop the consumption after the sequence, than when launching each conversion one by one with a delay after each conversion.

For a given sequence of conversions, the ADCCLK clock must be enabled before launching the first conversion, and be present until the EOC bit (or the JEOC bit in case of injected channels) is set.

Figure 36, Figure 37 and Figure 38 show examples of power management in different configurations. ADON=1 in all these examples.

Figure 36. Automatic power-down control: example 1

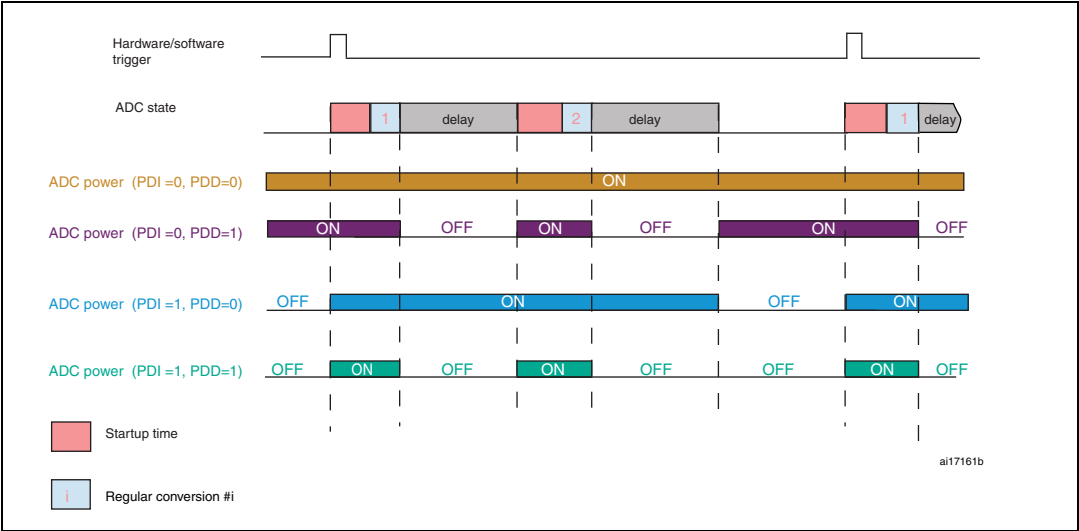


Figure 37. Automatic power-down control: example 2

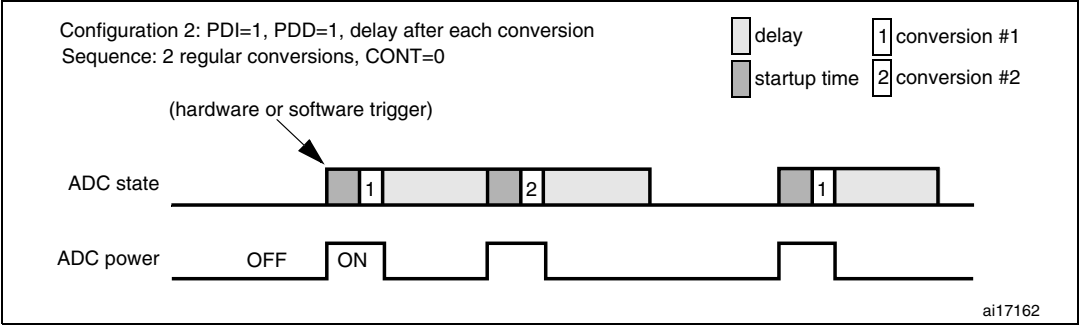
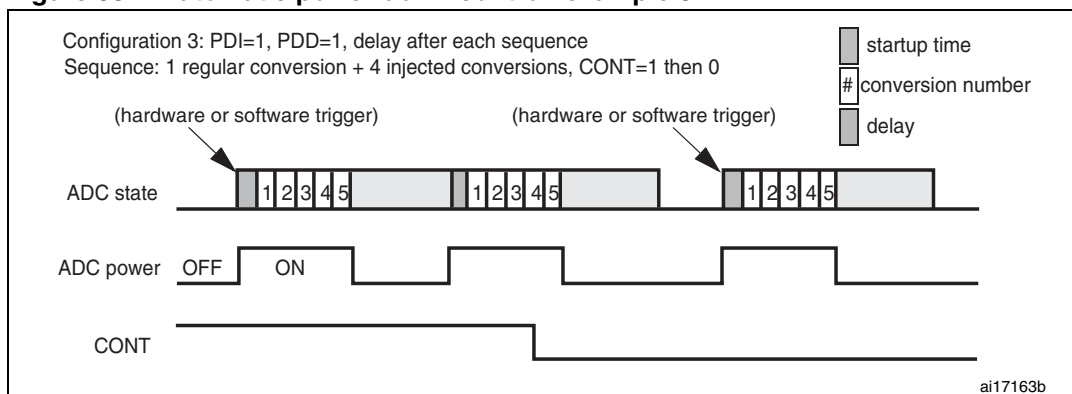


Figure 38. Automatic power-down control: example 3

9.11 Data management and overrun detection

9.11.1 Using the DMA

Since converted regular channel values are stored into a unique data register, it is useful to use DMA for conversion of more than one regular channel. This avoids the loss of the data already stored in the ADC_DR register.

When the DMA mode is enabled (DMA bit set to 1 in the ADC_CR2 register), after each conversion of a regular channel, a DMA request is generated. This allows the transfer of the converted data from the ADC_DR register to the destination location selected by the software.

Despite this, if data are lost (overrun), the OVR bit in the ADC_SR register is set and an interrupt is generated (if the OVRIE enable bit is set). DMA transfers are then disabled and DMA requests are no longer accepted. In this case, if a DMA request is made, the regular conversion in progress is aborted and further regular triggers are ignored. It is then necessary to clear the OVR flag and the DMAEN bit in the used DMA stream, and to re-initialize both the DMA and the ADC to have the wanted converted channel data transferred to the right memory location. Only then can the conversion be resumed and the data transfer, enabled again. Injected channel conversions are not impacted by overrun errors.

When OVR = 1 in DMA mode, the DMA requests are blocked after the last valid data have been transferred, which means that all the data transferred to the RAM can be considered as valid.

At the end of the last DMA transfer (number of transfers configured in the DMA controller's DMA_SxRTR register):

- No new DMA request is issued to the DMA controller if the DDS bit is cleared to 0 in the ADC_CR2 register (this avoids generating an overrun error). However the DMA bit is not cleared by hardware. It must be written to 0, then to 1 to start a new transfer.
- Requests can continue to be generated if the DDS bit is set to 1. This allows configuring the DMA in double-buffer circular mode.

9.11.2 Managing a sequence of conversions without using the DMA

If the conversions are slow enough, the conversion sequence can be handled by the software. In this case the EOCS bit must be set in the ADC_CR2 register for the EOC status

bit to be set at the end of each conversion, and not only at the end of the sequence. When EOCS = 1, overrun detection is automatically enabled. Thus, each time a conversion is complete, EOC is set and the ADC_DR register can be read. The overrun management is the same as when the DMA is used.

9.11.3 Conversions without reading all the data

It may be useful to let the ADC convert one or more channels without reading the data each time (if there is an analog watchdog for instance). For that, the DMA must be disabled (DMA = 0) and the EOC bit must be set at the end of a sequence only (EOCS = 0). In this configuration no overrun error is reported if a conversion finishes when the result of the previous conversion has not been read.

9.11.4 Overrun detection

Overrun detection is always enabled. It takes place before the data are synchronized to the APB clock.

Note: Only regular channel conversions generate overrun errors.

At the end of a conversion, the result is stored into an intermediate buffer (in the ADC clock domain) until it is transferred to the data register (ADC_DR, in the APB clock domain). If new data arrive before the previous data are transferred, the new data are lost and an overrun error is detected. The OVR bit is set in the ADC_SR register and an interrupt is generated if the OVRIE bit is set.

This may occur in two cases:

- either the delay is not properly set with respect to the APB clock frequency (the delay is too short to synchronize the data), or
- the previous data could not be synchronized to the APB clock because the ADC_DR register is not empty (when DMA=1 or EOCS=1). Indeed, in these modes, the contents of the ADC_DR register cannot be overwritten and so the register always contains the last valid data. ADC_DR is emptied by reading it or by clearing the EOC bit in the ADC_SR register.

- Note:*
- 1 An overrun may happen to be detected just after clearing the DMA (or EOCS) when the last data transferred by the DMA are read very late, which causes the next data to be lost.
 - 2 After clearing the OVR bit, the software should not launch a new regular conversion until RCNR=0 in the ADC_SR register.

9.12 Temperature sensor

The temperature sensor can be used to measure the internal temperature of the device. It is internally connected to the ADC TS (ADC channel 16: temperature sensor) input channel that is used to convert the sensor output voltage into a digital value.

Note: When it is not used, this sensor can be put in power-down mode.

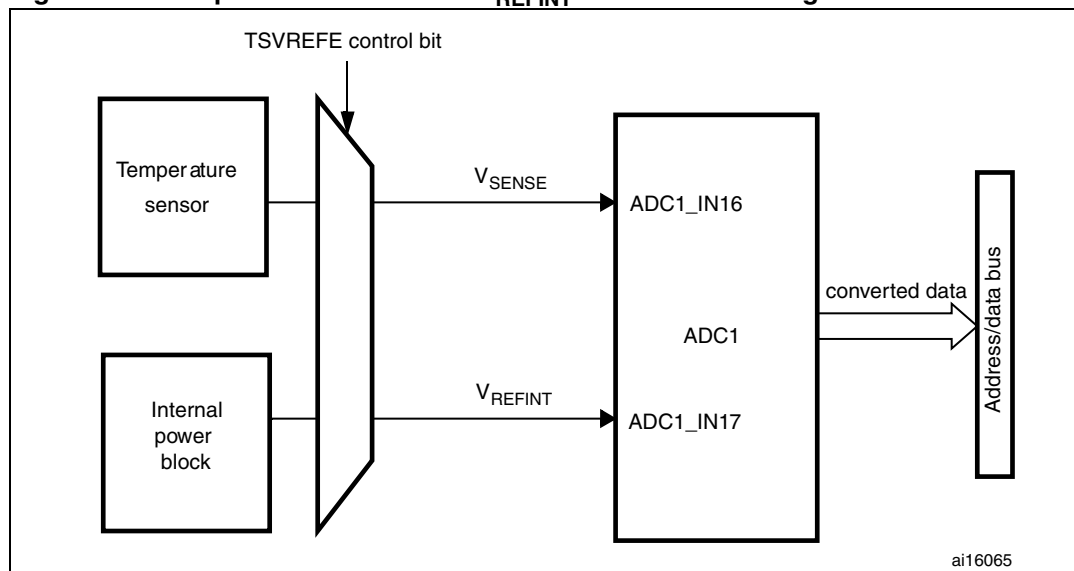
Note: The TSVREFE bit must be set to enable the conversion of both internal channels: ADC channel 16 (temperature sensor) and ADC channel 17 (VREFINT). If the temperature sensor conversion is required, this connection must be enabled.

The internal temperature sensor can also be used to detect temperature variations.

The temperature sensor is factory measured at high temperature and the result of the ADC conversion is stored in a specific data address : the TS_Factory_CONV_V90 data (result of the factory TS voltage conversion at 90°C; refer to the device datasheet for more details).

To reduce the temperature sensor error, the user can measure it at ambient temperature (25°C) to redefine more accurately the average slope (avg_slope) and the offset.

Figure 39. Temperature sensor and V_{REFINT} channel block diagram



9.12.1 How to read the temperature

To read the temperature from the sensor, use the following procedure:

1. Select the ADC TS (temperature sensor) input channel.
2. Select a sample time of 10 μ s.
3. Set the TSVREFE bit in the ADC_CCR register to wake up the temperature sensor from power-down mode.
4. Start the ADC conversion.
5. Read the resulting VSENSE data in the ADC data register.
6. Calculate the temperature using the following formulae:

$$T[^\circ\text{K}] = \frac{V_{\text{SENSE}}}{\text{AvgSlope}}$$

$$T[^\circ\text{C}] = \frac{V_{\text{SENSE}}}{\text{AvgSlope}} - 273.15$$

Avg_Slope = average slope of the "Temperature vs. VSENSE" curves (given in mV/°C).

Refer to the Electrical characteristics section for the Avg_Slope value.

Note: When the sensor wakes up from power-down mode, a stabilization time is required before a correct voltage can be output.

After power-on, the ADC also needs a stabilization time. To minimize this delay, the ADON and TSON bits should be set at the same time.

9.13 Internal reference voltage (V_{REFINT}) conversion

The internal reference voltage is internally connected to the VREFINT channel. This analog input channel is used to convert the internal reference voltage into a digital value.

The TSVREFE bit in the ADC_CCR register must be set to enable the internal reference voltage (and also the Temperature sensor). This reference voltage must be enabled only if its conversion is required.

The internal reference voltage is factory measured and the result of the ADC conversion is stored in a specific data address : the VREFINT_Factory_CONV byte.

9.14 ADC interrupts

An interrupt can be produced on the end of conversion for regular and injected groups, when the analog watchdog status bit is set and when the overrun status bit is set. Separate interrupt enable bits are available for flexibility.

Five other flags are present in the ADC_SR register, but there is no interrupt associated with them:

- JCNr (injected channel not ready)
- RCNR (regular channel not ready)
- ADONS (ADON status)
- JSTRT (Start of conversion for channels of an injected group)
- STRT (Start of conversion for channels of a regular group)

Figure 40. ADC flags and interrupts

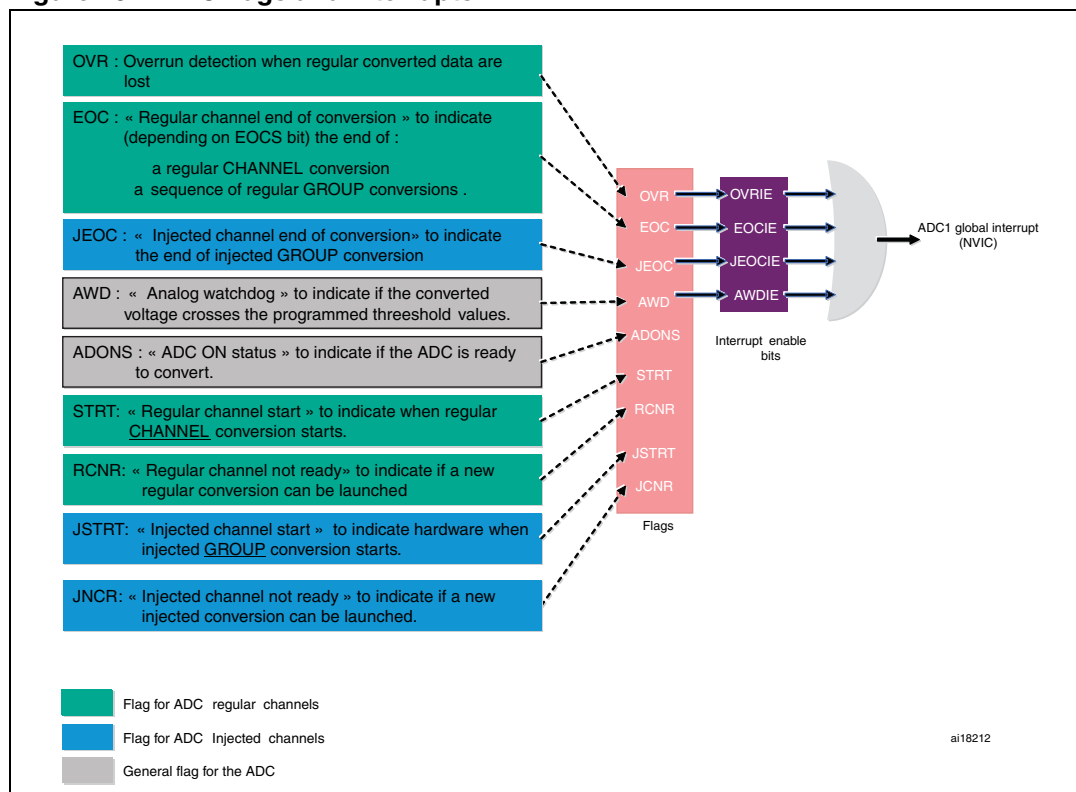


Table 39. ADC interrupts

Interrupt event	Event flag	Enable control bit
End of conversion of a regular group	EOC	EOCIE
End of conversion of an injected group	JEOC	JEOCIE
Analog watchdog status bit is set	AWD	AWDIE
Overrun	OVR	OVRIE

9.15 ADC registers

Refer to [Section 1.1 on page 29](#) for a list of abbreviations used in register descriptions.

The peripheral registers have to be accessed by words (32-bit).

9.15.1 ADC status register (ADC_SR)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						JCNR	RCNR	Reserved	ADONS	OVR	STRT	JSTRT	JEOC	EOC	AWD
						r	r		r	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0

Bits 31:10 Reserved, must be kept cleared.

Bit 9 **JCNR**: Injected channel not ready

This bit is set and cleared by hardware after the JSQR register is written. It indicates if a new injected conversion can be launched (by setting the JSWSTART bit).

0: Injected channel ready

1: Injected channel not ready, JSWSTART must not be set

Bit 8 **RCNR**: Regular channel not ready

This bit is set and cleared by hardware after one of the SQRx register is written or after the OVR bit is cleared. It indicates if a new regular conversion can be launched (by setting the SWSTART bit).

0: Regular channel ready

1: Regular channel not ready, SWSTART must not be set

Bit 7 Reserved, must be kept cleared.

Bit 6 **ADONS**: ADC ON status

This bit is set and cleared by hardware to indicate if the ADC is ready to convert.

0: The ADC is not ready

1: The ADC is ready to convert. External triggers can be enabled, the SWSTART and JSWSTART bits can be set.

Bit 5 OVR: Overrun

This bit is set by hardware when regular conversion data are lost. It is cleared by software. Overrun detection is enabled only when DMA = 1 or EOCS = 1.

0: No overrun occurred
1: Overrun has occurred

Bit 4 STRT: Regular channel start flag

This bit is set by hardware when regular channel conversion starts. It is cleared by software.

0: No regular channel conversion started
1: Regular channel conversion has started

Bit 3 JSTRT: Injected channel start flag

This bit is set by hardware when injected group conversion starts. It is cleared by software.

0: No injected group conversion started
1: Injected group conversion has started

Bit 2 JEOC: Injected channel end of conversion

This bit is set by hardware at the end of the conversion of all injected channels in the group. It is cleared by software.

0: Conversion is not complete
1: Conversion complete

Bit 1 EOC: Regular channel end of conversion

This bit is set by hardware at the end of the conversion of a regular group of channels. It is cleared by software or by reading the ADC_DR register.

0: Conversion not complete (EOCS=0), or sequence of conversions not complete (EOCS=1)
1: Conversion complete (EOCS=0), or sequence of conversions complete (EOCS=1)

Bit 0 AWD: Analog watchdog flag

This bit is set by hardware when the converted voltage crosses the values programmed in the ADC_LTR and ADC_HTR registers. It is cleared by software.

0: No analog watchdog event occurred
1: Analog watchdog event occurred

9.15.2 ADC control register 1 (ADC_CR1)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
Reserved					OVRIE	RES[1:0]		AWDEN	JAWDEN	Reserved					PDI	PDD
					rw	rw	rw	rw	rw							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
DISCNUM[2:0]			JDISC N	DISC EN	JAUTO	AWDSG L	SCAN	JEOCIE	AWDIE	EOCIE	AWDCH[4:0]					
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	

Bits 31:27 Reserved, must be kept cleared.

Bit 26 **OVRIE**: Overrun interrupt enable

This bit is set and cleared by software to enable/disable the Overrun interrupt.

0: Overrun interrupt disabled

1: Overrun interrupt enabled. An interrupt is generated when the OVR bit is set.

Bits 25:24 **RES[1:0]**: Resolution

These bits are written by software to select the resolution of the conversion.

00: 12-bit ($T_{CONV} = 12 \text{ ADCCLK cycles}$)

01: 10-bit ($T_{CONV} = 11 \text{ ADCCLK cycles}$)

10: 8-bit ($T_{CONV} = 9 \text{ ADCCLK cycles}$)

11: 6-bit ($T_{CONV} = 7 \text{ ADCCLK cycles}$)

This bit must be written only when ADON=0.

Bit 23 **AWDEN**: Analog watchdog enable on regular channels

This bit is set and cleared by software.

0: Analog watchdog disabled on regular channels

1: Analog watchdog enabled on regular channels

Bit 22 **JAWDEN**: Analog watchdog enable on injected channels

This bit is set and cleared by software.

0: Analog watchdog disabled on injected channels

1: Analog watchdog enabled on injected channels

Bits 21:18 Reserved, must be kept cleared.

Bit 17 **PDI**: Power down during the idle phase

This bit is written and cleared by software. When ADON=1, it determines whether the ADC is powered up or down when not converting (waiting for a hardware or software trigger event).

0: The ADC is powered up when waiting for a start event

1: The ADC is powered down when waiting for a start event

Note: This bit must be written only when ADON=0.

Bit 16 **PDD**: Power down during the delay phase

This bit is written and cleared by software. When ADON=1, it determines whether the ADC is powered up or down between 2 conversions (or sequences of conversions) when a delay is inserted (DELS bits).

0: The ADC is powered up during the delay

1: The ADC is powered down during the delay

Note: This bit must be written only when ADON=0.

Bits 15:13 **DISCNUM[2:0]**: Discontinuous mode channel count

These bits are written by software to define the number of channels to be converted in discontinuous mode, after receiving an external trigger.

000: 1 channel

001: 2 channels

...

111: 8 channels

Note: This bit must be written only when ADON=0.

Bit 12 **JDISEN**: Discontinuous mode on injected channels

This bit is set and cleared by software to enable/disable discontinuous mode on the injected channels of a group.

0: Discontinuous mode on injected channels disabled

1: Discontinuous mode on injected channels enabled

Note: This bit must be written only when ADON=0.

Bit 11 **DISCEN**: Discontinuous mode on regular channels

This bit is set and cleared by software to enable/disable Discontinuous mode on regular channels.

0: Discontinuous mode on regular channels disabled

1: Discontinuous mode on regular channels enabled

Note: This bit must be written only when ADON=0.

Bit 10 **JAUTO**: Automatic injected group conversion

This bit is set and cleared by software to enable/disable automatic injected group conversion after regular group conversion.

0: Automatic injected group conversion disabled

1: Automatic injected group conversion enabled

Note: This bit must be written only when ADON=0.

Bit 9 **AWDSGL**: Enable the watchdog on a single channel in scan mode

This bit is set and cleared by software to enable/disable the analog watchdog on the channel identified by the AWDCH[4:0] bits.

0: Analog watchdog enabled on all channels

1: Analog watchdog enabled on a single channel

Bit 8 **SCAN**: Scan mode

This bit is set and cleared by software to enable/disable the Scan mode. In the Scan mode, the inputs selected through the ADC_SQRx or ADC_JSQRx registers are converted.

0: Scan mode disabled

1: Scan mode enabled

Note: This bit must be written only when ADON=0.

Bit 7 **JEOCIE**: Interrupt enable for injected channels

This bit is set and cleared by software to enable/disable the end of conversion interrupt for injected channels.

0: JEOC interrupt disabled

1: JEOC interrupt enabled. An interrupt is generated when the JEOC bit is set.

Bit 6 **AWDIE**: Analog watchdog interrupt enable

This bit is set and cleared by software to enable/disable the analog watchdog interrupt. In Scan mode if the watchdog thresholds are crossed, scan is aborted only if this bit is enabled.

0: Analog watchdog interrupt disabled
1: Analog watchdog interrupt enabled

Bit 5 **EOCIE**: Interrupt enable for EOC

This bit is set and cleared by software to enable/disable the end of conversion interrupt.

0: EOC interrupt disabled
1: EOC interrupt enabled. An interrupt is generated when the EOC bit is set.

Bits 4:0 **AWDCH[4:0]**: Analog watchdog channel select bits

These bits are set and cleared by software. They select the input channel to be guarded by the analog watchdog.

00000: ADC analog input Channel0

00001: ADC analog input Channel1

...

11000: ADC analog input Channel24

11001: ADC analog input Channel25

11010: ADC analog input Channel26

Other values reserved.

Note: ADC1 analog inputs Channel16, Channel 17 and Channel26 are internally connected to the temperature sensor, to V_{REFINT} and to V_{COMP} respectively.

9.15.3 ADC control register 2 (ADC_CR2)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved	SWST ART	EXTEN		EXTSEL[3:0]				Reserved	JSWST ART	JEXTEN		JEXTSEL[3:0]			
	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				ALIGN	EOCS	DDS	DMA	Res.	DELS			Reserved		CONT	ADON
				rw	rw	rw	rw		rw	rw	rw			rw	rw

Bit 31 Reserved, must be kept cleared.

Bit 30 **SWSTART**: Start conversion of regular channels

This bit is set by software to start conversion and cleared by hardware as soon as the conversion starts.

0: Reset state
1: Starts conversion of regular channels

Note: This bit must be set only when ADONS=1 and RCNR=0.

Bits 29:28 **EXTEN**: External trigger enable for regular channels

These bits are set and cleared by software to select the external trigger polarity and enable the trigger of a regular group.

00: Trigger detection disabled

01: Trigger detection on the rising edge

10: Trigger detection on the falling edge

11: Trigger detection on both the rising and falling edges

Note: The external trigger must be enabled only when ADONS=1.

Bits 27:24 **EXTSEL[3:0]**: External event select for regular group

These bits select the external event used to trigger the start of conversion of a regular group:

0000: TIM9_CC2 event

0001: TIM9_TRGO event

0010: TIM2_CC3 event

0011: TIM2_CC2 event

0100: TIM3_TRGO event

0101: TIM4_CC4 event

0110: TIM2_TRGO event

0111: TIM3_CC1 event

1000: TIM3_CC3 event

1001: TIM4_TRGO event

1010: TIM6_TRGO event

1011: Reserved

1100: Reserved

1101: Reserved

1110: Reserved

1111: EXTI line11

Bit 23 Reserved, must be kept cleared.

Bit 22 **JSWSTART**: Start conversion of injected channels

This bit is set by software and cleared by hardware as soon as the conversion starts.

0: Reset state

1: Starts conversion of injected channels

Note: This bit must be set only when ADONS=1 and JCNR=0.

Bits 21:20 **JEXTEN**: External trigger enable for injected channels

These bits are set and cleared by software to select the external trigger polarity and enable the trigger of an injected group.

00: Trigger detection disabled

01: Trigger detection on the rising edge

10: Trigger detection on the falling edge

11: Trigger detection on both the rising and falling edges

Note: The external trigger must be enabled only when ADONS=1.

Bits 19:16 **JEXTSEL[3:0]**: External event select for injected group

These bits select the external event used to trigger the start of conversion of an injected group.

0000: TIM9_CC1 event
0001: TIM9_TRGO event
0010: TIM2_TRGO event
0011: TIM2_CC1 event
0100: TIM3_CC4 event
0101: TIM4_TRGO event
0110: TIM4_CC1 event
0111: TIM4_CC2 event
1000: TIM4_CC3 event
1001: TIM10_CC1 event
1010: TIM7_TRGO event
1011: Reserved
1100: Reserved
1101: Reserved
1110: Reserved
1111: EXTI line15

Bits 15:12 Reserved, must be kept cleared.

Bit 11 **ALIGN**: Data alignment

This bit is set and cleared by software. Refer to [Figure 30](#) and [Figure 31](#).

0: Right alignment
1: Left alignment

Bit 10 **EOCS**: End of conversion selection

This bit is set and cleared by software.

0: The EOC bit is set at the end of each sequence of regular conversions
1: The EOC bit is set at the end of each regular conversion

Bit 9 **DDS**: DMA disable selection

This bit is set and cleared by software.

0: No new DMA request is issued after the last transfer (as configured in the DMA controller)
1: DMA requests are issued as long as data are converted and DMA=1

Bit 8 **DMA**: Direct memory access mode

This bit is set and cleared by software. Refer to the DMA controller chapter for more details.

0: DMA mode disabled
1: DMA mode enabled

Bit 7 Reserved, must be kept cleared.

Bit 6:4 **DELS**: Delay selection

These bits are set and cleared by software. They define the length of the delay which is applied after a conversion or a sequence of conversions.

000: No delay

001: Until the converted data have been read (DR read or EOC=0 for regular conversions, JEOP=0 for injected conversions)

010: 7 APB clock cycles after the end of conversion

011: 15 APB clock cycles after the end of conversion

100: 31 APB clock cycles after the end of conversion

101: 63 APB clock cycles after the end of conversion

110: 127 APB clock cycles after the end of conversion

111: 255 APB clock cycles after the end of conversion

Note: 1- This bit must be written only when ADON=0.

2- Due to clock domain crossing, a latency of 2 or 3 ADC clock cycles is added to the delay before a new conversion can start.

3- The delay required for a given frequency ratio between the APB clock and the ADC clock depends on the activity on the AHB and APB busses. If the ADC is the only peripheral that needs to transfer data, then a minimum delay should be configured: 15 APB clock cycles if $f_{APB} < f_{ADCCLK}/2$ or else 7 APB clock cycles if $f_{APB} < f_{ADCCLK}$, otherwise no delay is needed.

Bits 3:2 Reserved, must be kept cleared.

Bit 1 **CONT**: Continuous conversion

This bit is set and cleared by software. If it is set, conversion takes place continuously until it is cleared.

0: Single conversion mode

1: Continuous conversion mode

Bit 0 **ADON**: A/D Converter ON / OFF

This bit is set and cleared by software.

0: Disable ADC conversion and go to power down mode

1: Enable ADC: conversions can start as soon as a start event (hardware or software) is received. When not converting, the ADC goes to the power up or power down mode depending on the PDI and PDD bits.

Note: This bit must be set only when ADONS=0 and cleared only when ADONS=1.

9.15.4 ADC sample time register 1 (ADC_SMPR1)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved														SMP25[2:1]	
														rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMP25[0]	SMP24[2:0]			SMP23[2:0]			SMP22[2:0]			SMP21[2:0]			SMP20[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31: 21 Reserved, must be kept cleared.

Bits 20:0 **SMPx[2:0]**: Channel x sampling time selection

These bits are written by software to select the sampling time individually for each channel.
During sampling cycles, the channel selection bits must remain unchanged.

000: 4 cycles

001: 9 cycles

010: 16 cycles

011: 24 cycles

100: 48 cycles

101: 96 cycles

110: 192 cycles

111: 384 cycles

Note: These bits must be written only when ADON=0.

9.15.5 ADC sample time register 2 (ADC_SMPR2)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		SMP19[2:0]			SMP18[2:0]			SMP17[2:0]			SMP16[2:0]			SMP15[2:1]	
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMP15[0]	SMP14[2:0]			SMP13[2:0]			SMP12[2:0]			SMP11[2:0]			SMP10[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept cleared.

Bits 29:0 **SMPx[2:0]**: Channel x sampling time selection

These bits are written by software to select the sampling time individually for each channel. During sample cycles, the channel selection bits must remain unchanged.

000: 4 cycles

001: 9 cycles

010: 16 cycles

011: 24 cycles

100: 48 cycles

101: 96 cycles

110: 192 cycles

111: 384 cycles

Note: These bits must be written only when ADON=0.

9.15.6 ADC sample time register 3 (ADC_SMPR3)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		SMP9[2:0]			SMP8[2:0]			SMP7[2:0]			SMP6[2:0]			SMP5[2:1]	
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMP5[0]	SMP4[2:0]			SMP3[2:0]			SMP2[2:0]			SMP1[2:0]			SMP0[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept cleared.

Bits 29:0 **SMPx[2:0]**: Channel x Sample time selection

These bits are written by software to select the sampling time individually for each channel. During the sampling cycles, the channel selection bits must remain unchanged.

000: 4 cycles

001: 9 cycles

010: 16 cycles

011: 24 cycles

100: 48 cycles

101: 96 cycles

110: 192 cycles

111: 384 cycles

Note: These bits must be written only when ADON=0.

9.15.7 ADC injected channel data offset register x (ADC_JOFRx)(x=1..4)

Address offset: 0x18-0x24

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				JOFFSETx[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:12 Reserved, must be kept cleared.

Bits 11:0 **JOFFSETx[11:0]**: Data offset for injected channel x

These bits are written by software to define the offset to be subtracted from the raw converted data when converting injected channels. The conversion result can be read from in the ADC_JDRx registers.

9.15.8 ADC watchdog higher threshold register (ADC_HTR)

Address offset: 0x28

Reset value: 0x0000 0FFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				HT[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:12 Reserved, must be kept cleared.

Bits 11:0 **HT[11:0]**: Analog watchdog higher threshold

These bits are written by software to define the higher threshold for the analog watchdog.

9.15.9 ADC watchdog lower threshold register (ADC_LTR)

Address offset: 0x2C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				LT[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:12 Reserved, must be kept cleared.

Bits 11:0 **LT[11:0]**: Analog watchdog lower threshold

These bits are written by software to define the lower threshold for the analog watchdog.

9.15.10 ADC regular sequence register 1 (ADC_SQR1)

Address offset: 0x30

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved							L[4:0]					Reserved			
							rw	rw	rw	rw	rw				
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserv- ed	SQ27[4:0]					SQ26[4:0]					SQ25[4:0]				
	rw	rw	rw	rw	rw	rw	rw				rw	rw	rw	rw	rw

Bits 31:25 Reserved, must be kept cleared.

Bits 24:20 **L[4:0]**: Regular channel sequence length

These bits are written by software to define the total number of conversions in the regular channel conversion sequence.

00000: 1 conversion

00001: 2 conversions

...

11010: 27 conversions

Bits 19:15 Reserved, must be kept cleared.

Bits 14:10 **SQ27[4:0]**: 27th conversion in regular sequence

These bits are written by software with the channel number (0..26) assigned as the 27th in the conversion sequence.

Bits 9:5 **SQ26[4:0]**: 26th conversion in regular sequence

Bits 4:0 **SQ25[4:0]**: 25th conversion in regular sequence

9.15.11 ADC regular sequence register 2 (ADC_SQR2)

Address offset: 0x34

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		SQ24[4:0]					SQ23[4:0]					SQ22[4:1]			
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ22[0]	SQ21[4:0]					SQ20[4:0]					SQ19[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept cleared.

Bits 29:26 **SQ24[4:0]**: 24th conversion in regular sequence

These bits are written by software with the channel number (0..26) assigned as the 24th in the sequence to be converted.

Bits 24:20 **SQ23[4:0]**: 23rd conversion in regular sequence

Bits 19:15 **SQ22[4:0]**: 22nd conversion in regular sequence

Bits 14:10 **SQ21[4:0]**: 21st conversion in regular sequence

Bits 9:5 **SQ20[4:0]**: 20th conversion in regular sequence

Bits 4:0 **SQ19[4:0]**: 19th conversion in regular sequence

9.15.12 ADC regular sequence register 3 (ADC_SQR3)

Address offset: 0x38

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		SQ18[4:0]					SQ17[4:0]					SQ16[4:1]			
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ16[0]	SQ15[4:0]					SQ14[4:0]					SQ13[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept cleared.

Bits 29:25 **SQ18[4:0]**: 18th conversion in regular sequence

These bits are written by software with the channel number (0..26) assigned as the 18th in the sequence to be converted.

Bits 24:20 **SQ17[4:0]**: 17th conversion in regular sequence

Bits 19:15 **SQ16[4:0]**: 16th conversion in regular sequence

Bits 14:10 **SQ15[4:0]**: 15th conversion in regular sequence

Bits 9:5 **SQ14[4:0]**: 14th conversion in regular sequence

Bits 4:0 **SQ13[4:0]**: 13th conversion in regular sequence

9.15.13 ADC regular sequence register 4 (ADC_SQR4)

Address offset: 0x3C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		SQ12[4:0]					SQ11[4:0]					SQ10[4:1]			
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ10[0]	SQ9[4:0]					SQ8[4:0]					SQ7[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept cleared.

Bits 29:26 **SQ12[4:0]**: 12th conversion in regular sequence

These bits are written by software with the channel number (0..26) assigned as the 12th in the sequence to be converted.

Bits 24:20 **SQ11[4:0]**: 11th conversion in regular sequence

Bits 19:15 **SQ10[4:0]**: 10th conversion in regular sequence

Bits 14:10 **SQ9[4:0]**: 9th conversion in regular sequence

Bits 9:5 **SQ8[4:0]**: 8th conversion in regular sequence

Bits 4:0 **SQ7[4:0]**: 7th conversion in regular sequence

9.15.14 ADC regular sequence register 5 (ADC_SQR5)

Address offset: 0x40

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		SQ6[4:0]					SQ5[4:0]					SQ4[4:1]			
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ4_0	SQ3[4:0]					SQ2[4:0]					SQ1[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept cleared.

Bits 29:25 **SQ6[4:0]**: 6th conversion in regular sequence

These bits are written by software with the channel number (0..26) assigned as the 6th in the sequence to be converted.

Bits 24:20 **SQ5[4:0]**: 5th conversion in regular sequence

Bits 19:15 **SQ4[4:0]**: 4th conversion in regular sequence

Bits 14:10 **SQ3[4:0]**: 3rd conversion in regular sequence

Bits 9:5 **SQ2[4:0]**: 2nd conversion in regular sequence

Bits 4:0 **SQ1[4:0]**: 1st conversion in regular sequence

9.15.15 ADC injected sequence register (ADC_JSQR)

Address offset: 0x44

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved										JL[1:0]		JSQ4[4:1]			
										rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
JSQ4[0]	JSQ3[4:0]					JSQ2[4:0]					JSQ1[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:22 Reserved, must be kept cleared.

Bits 21:20 **JL[1:0]**: Injected sequence length

These bits are written by software to define the total number of conversions in the injected channel conversion sequence.

00: 1 conversion

01: 2 conversions

10: 3 conversions

11: 4 conversions

Bits 19:15 **JSQ4[4:0]**: 4th conversion in injected sequence (when JL[1:0]=3, see note below)

These bits are written by software with the channel number (0..18) assigned as the 4th in the sequence to be converted.

Bits 14:10 **JSQ3[4:0]**: 3rd conversion in injected sequence (when JL[1:0]=3, see note below)Bits 9:5 **JSQ2[4:0]**: 2nd conversion in injected sequence (when JL[1:0]=3, see note below)Bits 4:0 **JSQ1[4:0]**: 1st conversion in injected sequence (when JL[1:0]=3, see note below)

Note: When JL[1:0]=3 (4 injected conversions in the sequencer), the ADC converts the channels in the following order: JSQ1[4:0], JSQ2[4:0], JSQ3[4:0], and JSQ4[4:0].

When JL=2 (3 injected conversions in the sequencer), the ADC converts the channels in the following order: JSQ2[4:0], JSQ3[4:0], and JSQ4[4:0].

When JL=1 (2 injected conversions in the sequencer), the ADC converts the channels in starting from JSQ3[4:0], and then JSQ4[4:0].

When JL=0 (1 injected conversion in the sequencer), the ADC converts only JSQ4[4:0] channel.

9.15.16 ADC injected data register x (ADC_JDRx) (x= 1..4)

Address offset: 0x48 - 0x54

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
JDATA[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept cleared.

Bits 15:0 **JDATA[15:0]**: Injected data

These bits are read-only. They contain the conversion result from injected channel x. The data are left -or right-aligned as shown in [Figure 30](#) and [Figure 31](#).

9.15.17 ADC regular data register (ADC_DR)

Address offset: 0x58

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved.

Bits 15:0 **DATA[15:0]**: Regular data

These bits are read-only. They contain the conversion result from the regular channels. The data are left- or right-aligned as shown in [Figure 30](#) and [Figure 31](#).

9.15.18 ADC common status register (ADC_CSR)

Address offset: 0x00 (this offset address is relative to the base address of ADC common registers, i.e. 0x300)

Reset value: 0x0000 0000

This register provides an image of the status bits of the different ADCs. Nevertheless it is read-only and does not allow to clear the different status bits. Instead each status bit must be cleared by writing it to 0 in the corresponding ADC_SR register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved									ADONS1	OVR1	STRT1	JSTRT1	JEOC 1	EOC1	AWD1
									r	r	r	r	r	r	r

Bits 31:7 Reserved, must be kept cleared.

Bit 6 **ADONS1**: *ADON Status of ADC1*

This bit is a copy of the ADONS bit in the ADC_SR register.

Bit 5 **OVR1**: Overrun flag of the ADC

This bit is a copy of the OVR bit in the ADC_SR register.

Bit 4 **STRT1**: Regular channel Start flag of the ADC

This bit is a copy of the STRT bit in the ADC_SR register.

- Bit 3 **JSTRT1**: Injected channel Start flag of the ADC
This bit is a copy of the JSTRT bit in the ADC_SR register.
- Bit 2 **JEOC1**: Injected channel end of conversion of the ADC
This bit is a copy of the JEOC bit in the ADC_SR register.
- Bit 1 **EOC1**: End of conversion of the ADC
This bit is a copy of the EOC bit in the ADC_SR register.
- Bit 0 **AWD1**: Analog watchdog flag of the ADC
This bit is a copy of the AWD bit in the ADC_SR register.

9.15.19 ADC common control register (ADC_CCR)

Address offset: 0x04 (this offset address is relative to the base address of ADC common registers, i.e. 0x300)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								TSVREFE	Reserved					ADCPRE[1:0]	
								rw						rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															

Bits 31:24 Reserved, must be kept cleared.

Bit 23 **TSVREFE**: Temperature sensor and V_{REFINT} enable

This bit is set and cleared by software to enable/disable the temperature sensor and the V_{REFINT} channel.

0: Temperature sensor and V_{REFINT} channel disabled

1: Temperature sensor and V_{REFINT} channel enabled

Bits 22:18 Reserved, must be kept cleared.

Bits 17:16 **ADCPRE**: ADC prescaler

Set and cleared by software to select the frequency of the clock to the ADC.

00: HSI divided by 1

01: HSI divided by 2

10: HSI divided by 4

11: Reserved

Bits 15:0 Reserved, must be kept cleared.

The following table summarizes the ADC registers.

Offset	Register
0x000 - 0x058	ADC
0x05C - 0x2FC	Reserved
0x300 - 0x304	Common registers

[illegible]

[illegible]

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	ADC_CSR																									ADONS	OVR	STRT	JSTRT	JEOC	EOC	AWD	
	Reset value																									0	0	0	0	0	0	0	
																										ADC1							
0x04	ADC_CCR	Reserved								TSVREFE	Reserved						ADCPRE		Reserved														
	Reset value									0							0	0															

Refer to [Table 1 on page 32](#) for the *Register boundary addresses* table.

10 Digital-to-analog converter (DAC)

10.1 DAC introduction

The DAC module is a 12-bit, voltage output digital-to-analog converter. The DAC can be configured in 8- or 12-bit mode and may be used in conjunction with the DMA controller. In 12-bit mode, the data could be left- or right-aligned. The DAC has two output channels, each with its own converter. In dual DAC channel mode, conversions could be done independently or simultaneously when both channels are grouped together for synchronous update operations. An input reference pin, V_{REF+} (shared with ADC) is available for better resolution.

10.2 DAC main features

- Two DAC converters: one output channel each
- Left or right data alignment in 12-bit mode
- Synchronized update capability
- Noise-wave generation
- Triangular-wave generation
- Dual DAC channel for independent or simultaneous conversions
- DMA capability for each channel
- DMA underrun error detection
- External triggers for conversion
- Input voltage reference, V_{REF+}

[Figure 41](#) shows the block diagram of a DAC channel and [Table 43](#) gives the pin description.

Figure 41. DAC channel block diagram

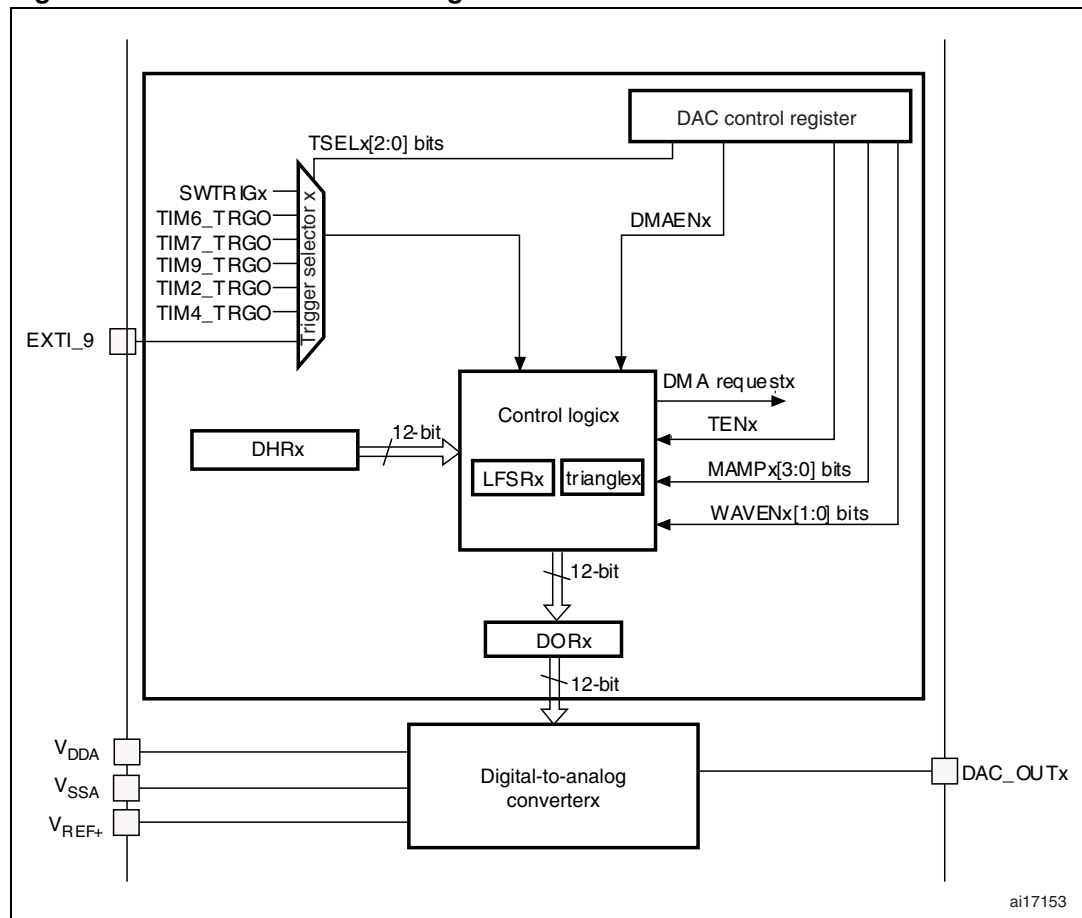


Table 43. DAC pins

Name	Signal type	Remarks
V _{REF+}	Input, analog reference positive	The higher/positive reference voltage for the DAC, $1.8\text{ V} \leq V_{\text{REF+}} \leq V_{\text{DDA}}$
V _{DDA}	Input, analog supply	Analog power supply
V _{SSA}	Input, analog supply ground	Ground for analog power supply
DAC_OUTx	Analog output signal	DAC channelx analog output

Note: Once the DAC channelx is enabled, the corresponding GPIO pin (PA4 or PA5) is automatically connected to the analog converter output (DAC_OUTx). In order to avoid parasitic consumption, the PA4 or PA5 pin should first be configured to analog (AIN).

10.3 DAC functional description

10.3.1 DAC channel enable

Each DAC channel can be powered on by setting its corresponding ENx bit in the DAC_CR register. The DAC channel is then enabled after a startup time t_{WAKEUP} .

Note: The ENx bit enables the analog DAC Channelx macrocell only. The DAC Channelx digital interface is enabled even if the ENx bit is reset.

10.3.2 DAC output buffer enable

The DAC integrates two output buffers that can be used to reduce the output impedance, and to drive external loads directly without having to add an external operational amplifier. Each DAC channel output buffer can be enabled and disabled using the corresponding BOFFx bit in the DAC_CR register.

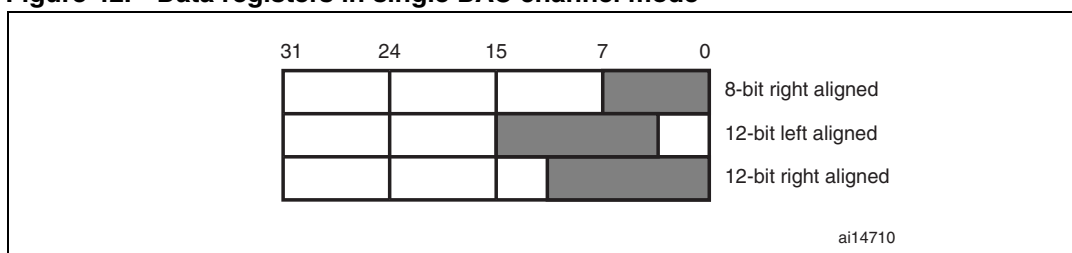
10.3.3 DAC data format

Depending on the selected configuration mode, the data have to be written into the specified register as described below:

- Single DAC channelx, there are three possibilities:
 - 8-bit right alignment: the software has to load data into the DAC_DHR8Rx [7:0] bits (stored into the DHRx[11:4] bits)
 - 12-bit left alignment: the software has to load data into the DAC_DHR12Lx [15:4] bits (stored into the DHRx[11:0] bits)
 - 12-bit right alignment: the software has to load data into the DAC_DHR12Rx [11:0] bits (stored into the DHRx[11:0] bits)

Depending on the loaded DAC_DHRyyyx register, the data written by the user is shifted and stored into the corresponding DHRx (data holding registerx, which are internal non-memory-mapped registers). The DHRx register is then loaded into the DORx register either automatically, by software trigger or by an external event trigger.

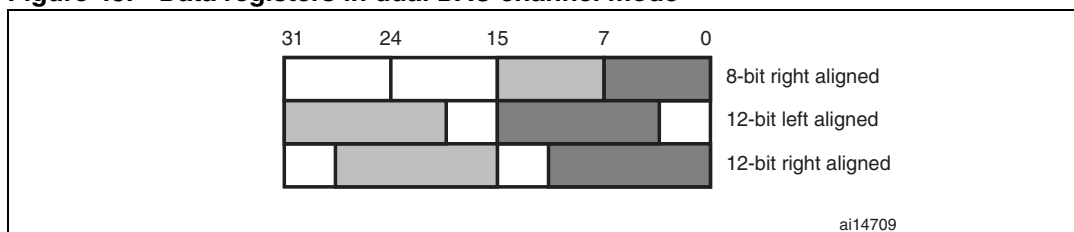
Figure 42. Data registers in single DAC channel mode



- Dual DAC channels, there are three possibilities:
 - 8-bit right alignment: data for DAC channel1 to be loaded into the DAC_DHR8RD [7:0] bits (stored into the DHR1[11:4] bits) and data for DAC channel2 to be loaded into the DAC_DHR8RD [15:8] bits (stored into the DHR2[11:4] bits)
 - 12-bit left alignment: data for DAC channel1 to be loaded into the DAC_DHR12LD [15:4] bits (stored into the DHR1[11:0] bits) and data for DAC channel2 to be loaded into the DAC_DHR12LD [31:20] bits (stored into the DHR2[11:0] bits)
 - 12-bit right alignment: data for DAC channel1 to be loaded into the DAC_DHR12RD [11:0] bits (stored into the DHR1[11:0] bits) and data for DAC channel2 to be loaded into the DAC_DHR12LD [27:16] bits (stored into the DHR2[11:0] bits)

Depending on the loaded DAC_DHRyyyD register, the data written by the user is shifted and stored into DHR1 and DHR2 (data holding registers, which are internal non-memory-mapped registers). The DHR1 and DHR2 registers are then loaded into the DOR1 and DOR2 registers, respectively, either automatically, by software trigger or by an external event trigger.

Figure 43. Data registers in dual DAC channel mode



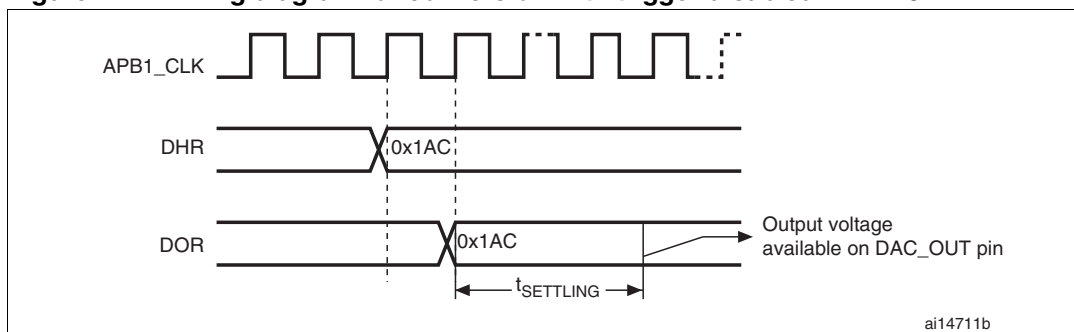
10.3.4 DAC conversion

The DAC_DORx cannot be written directly and any data transfer to the DAC channelx must be performed by loading the DAC_DHRx register (write to DAC_DHR8Rx, DAC_DHR12Lx, DAC_DHR12Rx, DAC_DHR8RD, DAC_DHR12LD or DAC_DHR12LD).

Data stored in the DAC_DHRx register are automatically transferred to the DAC_DORx register after one APB1 clock cycle, if no hardware trigger is selected (TENx bit in DAC_CR register is reset). However, when a hardware trigger is selected (TENx bit in DAC_CR register is set) and a trigger occurs, the transfer is performed three APB1 clock cycles later.

When DAC_DORx is loaded with the DAC_DHRx contents, the analog output voltage becomes available after a time t_{SETTLING} that depends on the power supply voltage and the analog output load.

Figure 44. Timing diagram for conversion with trigger disabled TEN = 0



10.3.5 DAC output voltage

Digital inputs are converted to output voltages on a linear conversion between 0 and V_{REF+} .

The analog output voltages on each DAC channel pin are determined by the following equation:

$$DAC_{output} = V_{REF} \times \frac{DOR}{4095}$$

10.3.6 DAC trigger selection

If the $TENx$ control bit is set, conversion can then be triggered by an external event (timer counter, external interrupt line). The $TSELx[2:0]$ control bits determine which out of 8 possible events will trigger conversion as shown in [Table 44](#).

Table 44. External triggers

Source	Type	TSEL[2:0]
Timer 6 TRGO event	Internal signal from on-chip timers	000
Reserved		001
Timer 7 TRGO event		010
Timer 9 TRGO event		011
Timer 2 TRGO event		100
Timer 4 TRGO event		101
EXTI line9	External pin	110
SWTRIG	Software control bit	111

Each time a DAC interface detects a rising edge on the selected timer TRGO output, or on the selected external interrupt line 9, the last data stored into the DAC_DHRx register are transferred into the DAC_DORx register. The DAC_DORx register is updated three APB1 cycles after the trigger occurs.

If the software trigger is selected, the conversion starts once the SWTRIG bit is set. SWTRIG is reset by hardware once the DAC_DORx register has been loaded with the DAC_DHRx register contents.

- Note:*
- 1 $TSELx[2:0]$ bit cannot be changed when the ENx bit is set.
 - 2 When software trigger is selected, the transfer from the DAC_DHRx register to the DAC_DORx register takes only one APB1 clock cycle.

10.3.7 DMA request

Each DAC channel has a DMA capability. Two DMA channels are used to service DAC channel DMA requests.

A DAC DMA request is generated when an external trigger (but not a software trigger) occurs while the $DMAENx$ bit is set. The value of the DAC_DHRx register is then transferred into the DAC_DORx register.

In dual mode, if both $DMAENx$ bits are set, two DMA requests are generated. If only one DMA request is needed, you should set only the corresponding $DMAENx$ bit. In this way, the application can manage both DAC channels in dual mode by using one DMA request and a unique DMA channel.

DMA underrun

The DAC DMA request is not queued so that if a second external trigger arrives before the acknowledgement for the first external trigger is received (first request), then no new request is issued and the DMA channelx underrun flag DMAUDRx in the DAC_SR register is set, reporting the error condition. DMA data transfers are then disabled and no further DMA request is treated. The DAC channelx continues to convert old data.

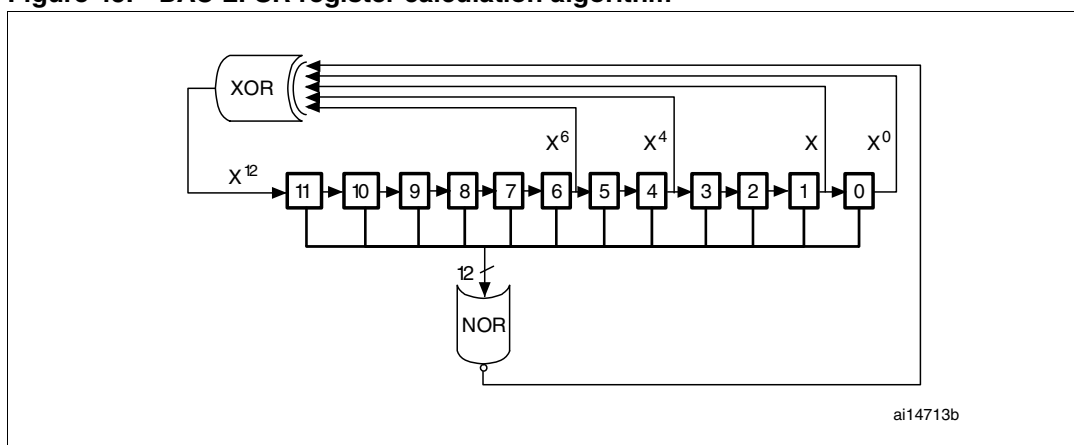
The software should clear the DMAUDRx flag by writing “1”, clear the DMAEN bit of the used DMA stream and re-initialize both DMA and DAC channelx to restart the transfer correctly. The software should modify the DAC trigger conversion frequency or lighten the DMA workload to avoid a new DMA underrun. Finally, the DAC conversion could be resumed by enabling both DMA data transfer and conversion trigger.

For each DAC channelx, an interrupt is also generated if its corresponding DMAUDRIEx bit in the DAC_CR register is enabled.

10.3.8 Noise generation

In order to generate a variable-amplitude pseudonoise, an LFSR (linear feedback shift register) is available. DAC noise generation is selected by setting WAVEx[1:0] to “01”. The preloaded value in LFSR is 0xAAA. This register is updated three APB1 clock cycles after each trigger event, following a specific calculation algorithm.

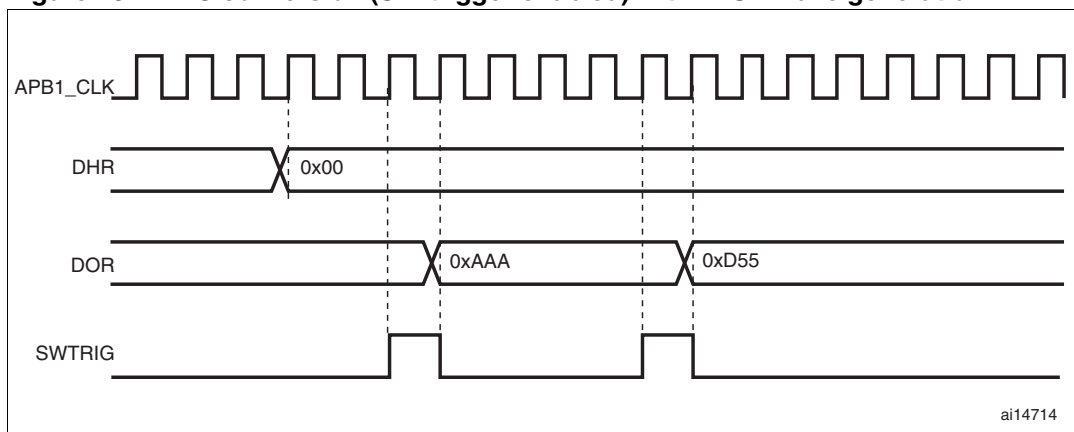
Figure 45. DAC LFSR register calculation algorithm



The LFSR value, that may be masked partially or totally by means of the MAMPx[3:0] bits in the DAC_CR register, is added up to the DAC_DHRx contents without overflow and this value is then stored into the DAC_DORx register.

If LFSR is 0x0000, a ‘1’ is injected into it (antilock-up mechanism).

It is possible to reset LFSR wave generation by resetting the WAVEx[1:0] bits.

Figure 46. DAC conversion (SW trigger enabled) with LFSR wave generation

Note: The DAC trigger must be enabled for noise generation by setting the *TENx* bit in the *DAC_CR* register.

10.3.9 Triangle-wave generation

It is possible to add a small-amplitude triangular waveform on a DC or slowly varying signal. DAC triangle-wave generation is selected by setting *WAVEx*[1:0] to “10”. The amplitude is configured through the *MAMPx*[3:0] bits in the *DAC_CR* register. An internal triangle counter is incremented three APB1 clock cycles after each trigger event. The value of this counter is then added to the *DAC_DHRx* register without overflow and the sum is stored into the *DAC_DORx* register. The triangle counter is incremented as long as it is less than the maximum amplitude defined by the *MAMPx*[3:0] bits. Once the configured amplitude is reached, the counter is decremented down to 0, then incremented again and so on.

It is possible to reset triangle wave generation by resetting the *WAVEx*[1:0] bits.

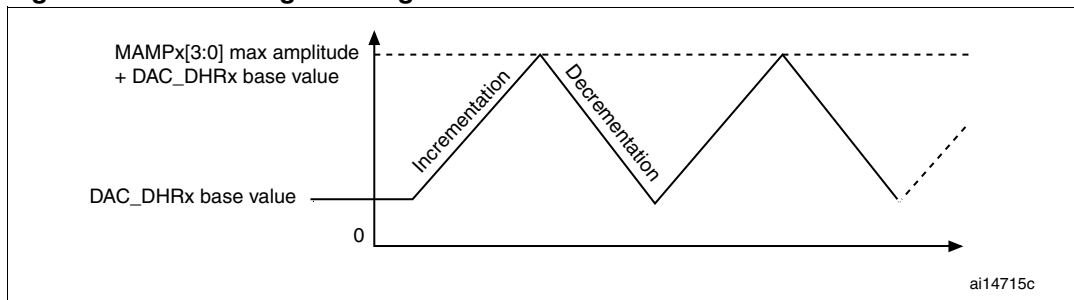
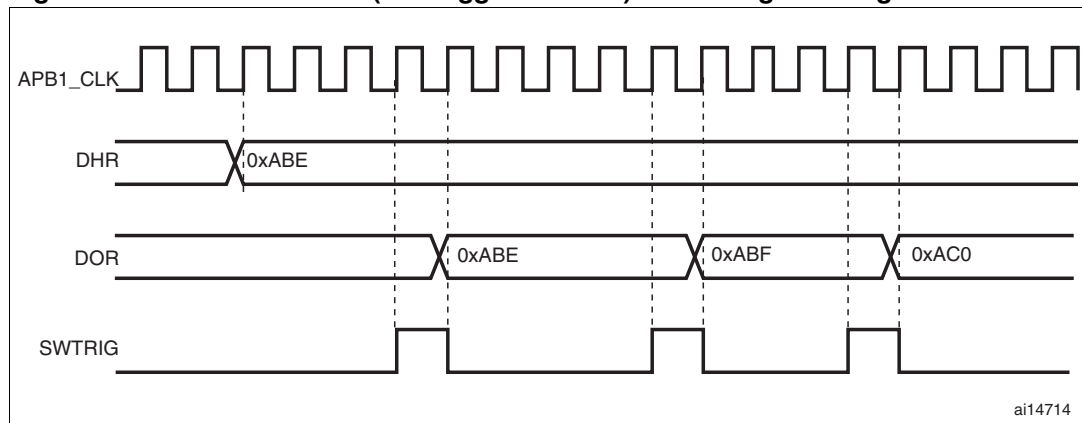
Figure 47. DAC triangle wave generation

Figure 48. DAC conversion (SW trigger enabled) with triangle wave generation

- Note:**
- 1 The DAC trigger must be enabled for noise generation by setting the *TENx* bit in the *DAC_CR* register.
 - 2 The *MAMPx[3:0]* bits must be configured before enabling the DAC, otherwise they cannot be changed.

10.4 Dual DAC channel conversion

To efficiently use the bus bandwidth in applications that require the two DAC channels at the same time, three dual registers are implemented: DHR8RD, DHR12RD and DHR12LD. A unique register access is then required to drive both DAC channels at the same time.

Eleven possible conversion modes are possible using the two DAC channels and these dual registers. All the conversion modes can nevertheless be obtained using separate DHRx registers if needed.

All modes are described in the paragraphs below.

10.4.1 Independent trigger without wave generation

To configure the DAC in this conversion mode, the following sequence is required:

- Set the two DAC channel trigger enable bits *TEN1* and *TEN2*
- Configure different trigger sources by setting different values in the *TSEL1[2:0]* and *TSEL2[2:0]* bits
- Load the dual DAC channel data into the desired DHR register (DAC_DHR12RD, DAC_DHR12LD or DAC_DHR8RD)

When a DAC channel1 trigger arrives, the DHR1 register is transferred into DAC_DOR1 (three APB1 clock cycles later).

When a DAC channel2 trigger arrives, the DHR2 register is transferred into DAC_DOR2 (three APB1 clock cycles later).

10.4.2 Independent trigger with single LFSR generation

To configure the DAC in this conversion mode, the following sequence is required:

- Set the two DAC channel trigger enable bits TEN1 and TEN2
- Configure different trigger sources by setting different values in the TSEL1[2:0] and TSEL2[2:0] bits
- Configure the two DAC channel WAVEx[1:0] bits as “01” and the same LFSR mask value in the MAMPx[3:0] bits
- Load the dual DAC channel data into the desired DHR register (DHR12RD, DHR12LD or DHR8RD)

When a DAC channel1 trigger arrives, the LFSR1 counter, with the same mask, is added to the DHR1 register and the sum is transferred into DAC_DOR1 (three APB1 clock cycles later). Then the LFSR1 counter is updated.

When a DAC channel2 trigger arrives, the LFSR2 counter, with the same mask, is added to the DHR2 register and the sum is transferred into DAC_DOR2 (three APB1 clock cycles later). Then the LFSR2 counter is updated.

10.4.3 Independent trigger with different LFSR generation

To configure the DAC in this conversion mode, the following sequence is required:

- Set the two DAC channel trigger enable bits TEN1 and TEN2
- Configure different trigger sources by setting different values in the TSEL1[2:0] and TSEL2[2:0] bits
- Configure the two DAC channel WAVEx[1:0] bits as “01” and set different LFSR masks values in the MAMP1[3:0] and MAMP2[3:0] bits
- Load the dual DAC channel data into the desired DHR register (DAC_DHR12RD, DAC_DHR12LD or DAC_DHR8RD)

When a DAC channel1 trigger arrives, the LFSR1 counter, with the mask configured by MAMP1[3:0], is added to the DHR1 register and the sum is transferred into DAC_DOR1 (three APB1 clock cycles later). Then the LFSR1 counter is updated.

When a DAC channel2 trigger arrives, the LFSR2 counter, with the mask configured by MAMP2[3:0], is added to the DHR2 register and the sum is transferred into DAC_DOR2 (three APB1 clock cycles later). Then the LFSR2 counter is updated.

10.4.4 Independent trigger with single triangle generation

To configure the DAC in this conversion mode, the following sequence is required:

- Set the two DAC channel trigger enable bits TEN1 and TEN2
- Configure different trigger sources by setting different values in the TSEL1[2:0] and TSEL2[2:0] bits
- Configure the two DAC channel WAVEx[1:0] bits as “1x” and the same maximum amplitude value in the MAMPx[3:0] bits
- Load the dual DAC channel data into the desired DHR register (DAC_DHR12RD, DAC_DHR12LD or DAC_DHR8RD)

When a DAC channel1 trigger arrives, the DAC channel1 triangle counter, with the same triangle amplitude, is added to the DHR1 register and the sum is transferred into

DAC_DOR1 (three APB1 clock cycles later). The DAC channel1 triangle counter is then updated.

When a DAC channel2 trigger arrives, the DAC channel2 triangle counter, with the same triangle amplitude, is added to the DHR2 register and the sum is transferred into DAC_DOR2 (three APB1 clock cycles later). The DAC channel2 triangle counter is then updated.

10.4.5 Independent trigger with different triangle generation

To configure the DAC in this conversion mode, the following sequence is required:

- Set the two DAC channel trigger enable bits TEN1 and TEN2
- Configure different trigger sources by setting different values in the TSEL1[2:0] and TSEL2[2:0] bits
- Configure the two DAC channel WAVEx[1:0] bits as “1x” and set different maximum amplitude values in the MAMP1[3:0] and MAMP2[3:0] bits
- Load the dual DAC channel data into the desired DHR register (DAC_DHR12RD, DAC_DHR12LD or DAC_DHR8RD)

When a DAC channel1 trigger arrives, the DAC channel1 triangle counter, with a triangle amplitude configured by MAMP1[3:0], is added to the DHR1 register and the sum is transferred into DAC_DOR1 (three APB1 clock cycles later). The DAC channel1 triangle counter is then updated.

When a DAC channel2 trigger arrives, the DAC channel2 triangle counter, with a triangle amplitude configured by MAMP2[3:0], is added to the DHR2 register and the sum is transferred into DAC_DOR2 (three APB1 clock cycles later). The DAC channel2 triangle counter is then updated.

10.4.6 Simultaneous software start

To configure the DAC in this conversion mode, the following sequence is required:

- Load the dual DAC channel data to the desired DHR register (DAC_DHR12RD, DAC_DHR12LD or DAC_DHR8RD)

In this configuration, one APB1 clock cycle later, the DHR1 and DHR2 registers are transferred into DAC_DOR1 and DAC_DOR2, respectively.

10.4.7 Simultaneous trigger without wave generation

To configure the DAC in this conversion mode, the following sequence is required:

- Set the two DAC channel trigger enable bits TEN1 and TEN2
- Configure the same trigger source for both DAC channels by setting the same value in the TSEL1[2:0] and TSEL2[2:0] bits
- Load the dual DAC channel data to the desired DHR register (DAC_DHR12RD, DAC_DHR12LD or DAC_DHR8RD)

When a trigger arrives, the DHR1 and DHR2 registers are transferred into DAC_DOR1 and DAC_DOR2, respectively (after three APB1 clock cycles).

10.4.8 Simultaneous trigger with single LFSR generation

To configure the DAC in this conversion mode, the following sequence is required:

- Set the two DAC channel trigger enable bits TEN1 and TEN2
- Configure the same trigger source for both DAC channels by setting the same value in the TSEL1[2:0] and TSEL2[2:0] bits
- Configure the two DAC channel WAVEx[1:0] bits as “01” and the same LFSR mask value in the MAMPx[3:0] bits
- Load the dual DAC channel data to the desired DHR register (DHR12RD, DHR12LD or DHR8RD)

When a trigger arrives, the LFSR1 counter, with the same mask, is added to the DHR1 register and the sum is transferred into DAC_DOR1 (three APB1 clock cycles later). The LFSR1 counter is then updated. At the same time, the LFSR2 counter, with the same mask, is added to the DHR2 register and the sum is transferred into DAC_DOR2 (three APB1 clock cycles later). The LFSR2 counter is then updated.

10.4.9 Simultaneous trigger with different LFSR generation

To configure the DAC in this conversion mode, the following sequence is required:

- Set the two DAC channel trigger enable bits TEN1 and TEN2
- Configure the same trigger source for both DAC channels by setting the same value in the TSEL1[2:0] and TSEL2[2:0] bits
- Configure the two DAC channel WAVEx[1:0] bits as “01” and set different LFSR mask values using the MAMP1[3:0] and MAMP2[3:0] bits
- Load the dual DAC channel data into the desired DHR register (DAC_DHR12RD, DAC_DHR12LD or DAC_DHR8RD)

When a trigger arrives, the LFSR1 counter, with the mask configured by MAMP1[3:0], is added to the DHR1 register and the sum is transferred into DAC_DOR1 (three APB1 clock cycles later). The LFSR1 counter is then updated.

At the same time, the LFSR2 counter, with the mask configured by MAMP2[3:0], is added to the DHR2 register and the sum is transferred into DAC_DOR2 (three APB1 clock cycles later). The LFSR2 counter is then updated.

10.4.10 Simultaneous trigger with single triangle generation

To configure the DAC in this conversion mode, the following sequence is required:

- Set the two DAC channel trigger enable bits TEN1 and TEN2
- Configure the same trigger source for both DAC channels by setting the same value in the TSEL1[2:0] and TSEL2[2:0] bits
- Configure the two DAC channel WAVEx[1:0] bits as “1x” and the same maximum amplitude value using the MAMPx[3:0] bits
- Load the dual DAC channel data into the desired DHR register (DAC_DHR12RD, DAC_DHR12LD or DAC_DHR8RD)

When a trigger arrives, the DAC channel1 triangle counter, with the same triangle amplitude, is added to the DHR1 register and the sum is transferred into DAC_DOR1 (three APB1 clock cycles later). The DAC channel1 triangle counter is then updated.

At the same time, the DAC channel2 triangle counter, with the same triangle amplitude, is

added to the DHR2 register and the sum is transferred into DAC_DOR2 (three APB1 clock cycles later). The DAC channel2 triangle counter is then updated.

10.4.11 Simultaneous trigger with different triangle generation

To configure the DAC in this conversion mode, the following sequence is required:

- Set the two DAC channel trigger enable bits TEN1 and TEN2
- Configure the same trigger source for both DAC channels by setting the same value in the TSEL1[2:0] and TSEL2[2:0] bits
- Configure the two DAC channel WAVEx[1:0] bits as “1x” and set different maximum amplitude values in the MAMP1[3:0] and MAMP2[3:0] bits
- Load the dual DAC channel data into the desired DHR register (DAC_DHR12RD, DAC_DHR12LD or DAC_DHR8RD)

When a trigger arrives, the DAC channel1 triangle counter, with a triangle amplitude configured by MAMP1[3:0], is added to the DHR1 register and the sum is transferred into DAC_DOR1 (three APB1 clock cycles later). Then the DAC channel1 triangle counter is updated.

At the same time, the DAC channel2 triangle counter, with a triangle amplitude configured by MAMP2[3:0], is added to the DHR2 register and the sum is transferred into DAC_DOR2 (three APB1 clock cycles later). Then the DAC channel2 triangle counter is updated.

10.5 DAC registers

Refer to [Section 1.1 on page 29](#) for a list of abbreviations used in register descriptions.

The peripheral registers have to be accessed by words (32-bit).

10.5.1 DAC control register (DAC_CR)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		DMAU DRIE2	DMA EN2	MAMP2[3:0]				WAVE2[1:0]		TSEL2[2:0]			TEN2	BOFF2	EN2
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved		DMAU DRIE1	DMA EN1	MAMP1[3:0]				WAVE1[1:0]		TSEL1[2:0]			TEN1	BOFF1	EN1
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved.

Bits 29 **DMAUDRIE2**: DAC channel2 DMA underrun interrupt enable

This bit is set and cleared by software.

0: DAC channel2 DMA underrun interrupt disabled

1: DAC channel2 DMA underrun interrupt enabled

Bit 28 **DMAEN2**: DAC channel2 DMA enable

This bit is set and cleared by software.

0: DAC channel2 DMA mode disabled

1: DAC channel2 DMA mode enabled

Bit 27:24 **MAMP2[3:0]**: DAC channel2 mask/amplitude selector

These bits are written by software to select mask in wave generation mode or amplitude in triangle generation mode.

0000: Unmask bit0 of LFSR/ triangle amplitude equal to 1
 0001: Unmask bits[1:0] of LFSR/ triangle amplitude equal to 3
 0010: Unmask bits[2:0] of LFSR/ triangle amplitude equal to 7
 0011: Unmask bits[3:0] of LFSR/ triangle amplitude equal to 15
 0100: Unmask bits[4:0] of LFSR/ triangle amplitude equal to 31
 0101: Unmask bits[5:0] of LFSR/ triangle amplitude equal to 63
 0110: Unmask bits[6:0] of LFSR/ triangle amplitude equal to 127
 0111: Unmask bits[7:0] of LFSR/ triangle amplitude equal to 255
 1000: Unmask bits[8:0] of LFSR/ triangle amplitude equal to 511
 1001: Unmask bits[9:0] of LFSR/ triangle amplitude equal to 1023
 1010: Unmask bits[10:0] of LFSR/ triangle amplitude equal to 2047
 ≥ 1011: Unmask bits[11:0] of LFSR/ triangle amplitude equal to 4095

Bit 23:22 **WAVE2[1:0]**: DAC channel2 noise/triangle wave generation enable

These bits are set/reset by software.

00: wave generation disabled
 01: Noise wave generation enabled
 1x: Triangle wave generation enabled

Note: Only used if bit TEN2 = 1 (DAC channel2 trigger enabled)

Bits 21:19 **TSEL2[2:0]**: DAC channel2 trigger selection

These bits select the external event used to trigger DAC channel2

000: Timer 6 TRGO event
 001: Reserved
 010: Timer 7 TRGO event
 011: Timer 9 TRGO event
 100: Timer 2 TRGO event
 101: Timer 4 TRGO event
 110: External line9
 111: Software trigger

Note: Only used if bit TEN2 = 1 (DAC channel2 trigger enabled).

Bit 18 **TEN2**: DAC channel2 trigger enable

This bit is set and cleared by software to enable/disable DAC channel2 trigger

0: DAC channel2 trigger disabled and data written into the DAC_DHRx register are transferred one APB1 clock cycle later to the DAC_DOR2 register
 1: DAC channel2 trigger enabled and data from the DAC_DHRx register are transferred three APB1 clock cycles later to the DAC_DOR2 register

Note: When software trigger is selected, the transfer from the DAC_DHRx register to the DAC_DOR2 register takes only one APB1 clock cycle.

Bit 17 **BOFF2**: DAC channel2 output buffer disable

This bit is set and cleared by software to enable/disable DAC channel2 output buffer.

0: DAC channel2 output buffer enabled
 1: DAC channel2 output buffer disabled

Bit 16 **EN2**: DAC channel2 enable

This bit is set and cleared by software to enable/disable DAC channel2.

0: DAC channel2 disabled
 1: DAC channel2 enabled

Bits 15:14 Reserved.

Bit 13 **DMAUDRIE1**: DAC channel1 DMA Underrun Interrupt enable

This bit is set and cleared by software.

0: DAC channel1 DMA Underrun Interrupt disabled

1: DAC channel1 DMA Underrun Interrupt enabled

Bit 12 **DMAEN1**: DAC channel1 DMA enable

This bit is set and cleared by software.

0: DAC channel1 DMA mode disabled

1: DAC channel1 DMA mode enabled

Bits 11:8 **MAMP1[3:0]**: DAC channel1 mask/amplitude selector

These bits are written by software to select mask in wave generation mode or amplitude in triangle generation mode.

0000: Unmask bit0 of LFSR/ triangle amplitude equal to 1

0001: Unmask bits[1:0] of LFSR/ triangle amplitude equal to 3

0010: Unmask bits[2:0] of LFSR/ triangle amplitude equal to 7

0011: Unmask bits[3:0] of LFSR/ triangle amplitude equal to 15

0100: Unmask bits[4:0] of LFSR/ triangle amplitude equal to 31

0101: Unmask bits[5:0] of LFSR/ triangle amplitude equal to 63

0110: Unmask bits[6:0] of LFSR/ triangle amplitude equal to 127

0111: Unmask bits[7:0] of LFSR/ triangle amplitude equal to 255

1000: Unmask bits[8:0] of LFSR/ triangle amplitude equal to 511

1001: Unmask bits[9:0] of LFSR/ triangle amplitude equal to 1023

1010: Unmask bits[10:0] of LFSR/ triangle amplitude equal to 2047

≥ 1011: Unmask bits[11:0] of LFSR/ triangle amplitude equal to 4095

Bits 7:6 **WAVE1[1:0]**: DAC channel1 noise/triangle wave generation enable

These bits are set and cleared by software.

00: wave generation disabled

01: Noise wave generation enabled

1x: Triangle wave generation enabled

Note: Only used if bit TEN1 = 1 (DAC channel1 trigger enabled).

Bits 5:3 **TSEL1[2:0]**: DAC channel1 trigger selection

These bits select the external event used to trigger DAC channel1.

000: Timer 6 TRGO event

001: Reserved

010: Timer 7 TRGO event

011: Timer 9 TRGO event

100: Timer 2 TRGO event

101: Timer 4 TRGO event

110: External line9

111: Software trigger

Note: Only used if bit TEN1 = 1 (DAC channel1 trigger enabled).

Bit 2 **TEN1**: DAC channel1 trigger enable

This bit is set and cleared by software to enable/disable DAC channel1 trigger.

0: DAC channel1 trigger disabled and data written into the DAC_DHRx register are transferred one APB1 clock cycle later to the DAC_DOR1 register

1: DAC channel1 trigger enabled and data from the DAC_DHRx register are transferred three APB1 clock cycles later to the DAC_DOR1 register

Note: When software trigger is selected, the transfer from the DAC_DHRx register to the DAC_DOR1 register takes only one APB1 clock cycle.

Bit 1 **BOFF1**: DAC channel1 output buffer disable

This bit is set and cleared by software to enable/disable DAC channel1 output buffer.

0: DAC channel1 output buffer enabled

1: DAC channel1 output buffer disabled

Bit 0 **EN1**: DAC channel1 enable

This bit is set and cleared by software to enable/disable DAC channel1.

0: DAC channel1 disabled

1: DAC channel1 enabled

10.5.2 DAC software trigger register (DAC_SWTRIGR)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved													SWTRIG2		SWTRIG1
													w		w

Bits 31:2 Reserved.

Bit 1 **SWTRIG2**: DAC channel2 software trigger

This bit is set and cleared by software to enable/disable the software trigger.

0: Software trigger disabled

1: Software trigger enabled

Note: This bit is cleared by hardware (one APB1 clock cycle later) once the DAC_DHR2 register value has been loaded into the DAC_DOR2 register.

Bit 0 **SWTRIG1**: DAC channel1 software trigger

This bit is set and cleared by software to enable/disable the software trigger.

0: Software trigger disabled

1: Software trigger enabled

Note: This bit is cleared by hardware (one APB1 clock cycle later) once the DAC_DHR1 register value has been loaded into the DAC_DOR1 register.

10.5.3 DAC channel1 12-bit right-aligned data holding register (DAC_DHR12R1)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				DACC1DHR[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:12 Reserved.

Bit 11:0 **DACC1DHR[11:0]**: DAC channel1 12-bit right-aligned data

These bits are written by software which specifies 12-bit data for DAC channel1.

10.5.4 DAC channel1 12-bit left aligned data holding register (DAC_DHR12L1)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DACC1DHR[11:0]												Reserved			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw				

Bits 31:16 Reserved.

Bit 15:4 **DACC1DHR[11:0]**: DAC channel1 12-bit left-aligned data

These bits are written by software which specifies 12-bit data for DAC channel1.

Bits 3:0 Reserved.

10.5.5 DAC channel1 8-bit right aligned data holding register (DAC_DHR8R1)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								DACC1DHR[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:8 Reserved.

Bits 7:0 **DACC1DHR[7:0]**: DAC channel1 8-bit right-aligned data

These bits are written by software which specifies 8-bit data for DAC channel1.

10.5.6 DAC channel2 12-bit right aligned data holding register (DAC_DHR12R2)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				DACC2DHR[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:12 Reserved.

Bits 11:0 **DACC2DHR[11:0]**: DAC channel2 12-bit right-aligned data

These bits are written by software which specifies 12-bit data for DAC channel2.

10.5.7 DAC channel2 12-bit left aligned data holding register (DAC_DHR12L2)

Address offset: 0x18

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DACC2DHR[11:0]												Reserved			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw				

Bits 31:16 Reserved.

Bits 15:4 **DACC2DHR[11:0]**: DAC channel2 12-bit left-aligned data

These bits are written by software which specify 12-bit data for DAC channel2.

Bits 3:0 Reserved.

10.5.8 DAC channel2 8-bit right-aligned data holding register (DAC_DHR8R2)

Address offset: 0x1C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								DACC2DHR[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:8 Reserved.

Bits 7:0 **DACC2DHR[7:0]**: DAC channel2 8-bit right-aligned data

These bits are written by software which specifies 8-bit data for DAC channel2.

10.5.9 Dual DAC 12-bit right-aligned data holding register (DAC_DHR12RD)

Address offset: 0x20

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				DACC2DHR[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				DACC1DHR[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:28 Reserved.

Bits 27:16 **DACC2DHR[11:0]**: DAC channel2 12-bit right-aligned data

These bits are written by software which specifies 12-bit data for DAC channel2.

Bits 15:12 Reserved.

Bits 11:0 **DACC1DHR[11:0]**: DAC channel1 12-bit right-aligned data

These bits are written by software which specifies 12-bit data for DAC channel1.

10.5.10 DUAL DAC 12-bit left aligned data holding register (DAC_DHR12LD)

Address offset: 0x24

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DACC2DHR[11:0]												Reserved			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw				
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DACC1DHR[11:0]												Reserved			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw				

Bits 31:20 **DACC2DHR[11:0]**: DAC channel2 12-bit left-aligned data

These bits are written by software which specifies 12-bit data for DAC channel2.

Bits 19:16 Reserved.

Bits 15:4 **DACC1DHR[11:0]**: DAC channel1 12-bit left-aligned data

These bits are written by software which specifies 12-bit data for DAC channel1.

Bits 3:0 Reserved.

10.5.11 DUAL DAC 8-bit right aligned data holding register (DAC_DHR8RD)

Address offset: 0x28

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DACC2DHR[7:0]								DACC1DHR[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved.

Bits 15:8 **DACC2DHR[7:0]**: DAC channel2 8-bit right-aligned data

These bits are written by software which specifies 8-bit data for DAC channel2.

Bits 7:0 **DACC1DHR[7:0]**: DAC channel1 8-bit right-aligned data

These bits are written by software which specifies 8-bit data for DAC channel1.

10.5.12 DAC channel1 data output register (DAC_DOR1)

Address offset: 0x2C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				DACC1DOR[11:0]											
				r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:12 Reserved.

Bit 11:0 **DACC1DOR[11:0]**: DAC channel1 data output

These bits are read-only, they contain data output for DAC channel1.

10.5.13 DAC channel2 data output register (DAC_DOR2)

Address offset: 0x30

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				DACC2DOR[11:0]											
				r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:12 Reserved.

Bit 11:0 **DACC2DOR[11:0]**: DAC channel2 data output

These bits are read-only, they contain data output for DAC channel2.

10.5.14 DAC status register (DAC_SR)

Address offset: 0x34

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		DMAUDR2	Reserved												
		rc_w1													
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved		DMAUDR1	Reserved												
		rc_w1													

Bits 31:30 Reserved.

Bit 29 **DMAUDR2**: DAC channel2 DMA underrun flag

This bit is set by hardware and cleared by software (by writing it to 1).

0: No DMA underrun error condition occurred for DAC channel2

1: DMA underrun error condition occurred for DAC channel2 (the currently selected trigger is driving DAC channel2 conversion at a frequency higher than the DMA service capability rate)

Bits 28:14 Reserved.

Bit 13 **DMAUDR1**: DAC channel1 DMA underrun flag

This bit is set by hardware and cleared by software (by writing it to 1).

0: No DMA underrun error condition occurred for DAC channel1

1: DMA underrun error condition occurred for DAC channel1 (the currently selected trigger is driving DAC channel1 conversion at a frequency higher than the DMA service capability rate)

Bits 12:0 Reserved.

10.5.15 DAC register map

[Table 45](#) summarizes the DAC registers.

Table 45. DAC register map

Address offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	DAC_CR	Reserved		DMAUDRIE2	DMAEN2	MAMP2[3:0]				WAVE2[2:0]		TSEL2[2:0]			TEN2	BOFF2	EN2	Reserved		DMAUDRIE1	DMAEN1	MAMP1[3:0]				WAVE1[2:0]		TSEL1[2:0]			TEN1	BOFF1	EN1
0x04	DAC_SWT_RIGR	Reserved																												SWTRIG2	SWTRIG1		
0x08	DAC_DHR1_2R1	Reserved																		DACC1DHR[11:0]													
0x0C	DAC_DHR1_2L1	Reserved														DACC1DHR[11:0]										Reserved							
0x10	DAC_DHR8_R1	Reserved																						DACC1DHR[7:0]									
0x14	DAC_DHR1_2R2	Reserved																		DACC2DHR[11:0]													
0x18	DAC_DHR1_2L2	Reserved														DACC2DHR[11:0]										Reserved							
0x1C	DAC_DHR8_R2	Reserved																						DACC2DHR[7:0]									
0x20	DAC_DHR1_2RD	Reserved			DACC2DHR[11:0]												Reserved				DACC1DHR[11:0]												
0x24	DAC_DHR1_2LD	DACC2DHR[11:0]										Reserved				DACC1DHR[11:0]										Reserved							
0x28	DAC_DHR8_RD	Reserved														DACC2DHR[7:0]						DACC1DHR[7:0]											
0x2C	DAC_DOR1	Reserved																		DACC1DOR[11:0]													
0x30	DAC_DOR2	Reserved																		DACC2DOR[11:0]													
0x34	DAC_SR	Reserved	DMAUDR2	Reserved														DMAUDR1	Reserved														

Refer to [Table 1 on page 32](#) for the register boundary addresses.

13 General-purpose timers (TIM2 to TIM4)

13.1 TIM2 to TIM4 introduction

The general-purpose timers consist of a 16-bit auto-reload counter driven by a programmable prescaler.

They may be used for a variety of purposes, including measuring the pulse lengths of input signals (*input capture*) or generating output waveforms (*output compare and PWM*).

Pulse lengths and waveform periods can be modulated from a few microseconds to several milliseconds using the timer prescaler and the RCC clock controller prescalers.

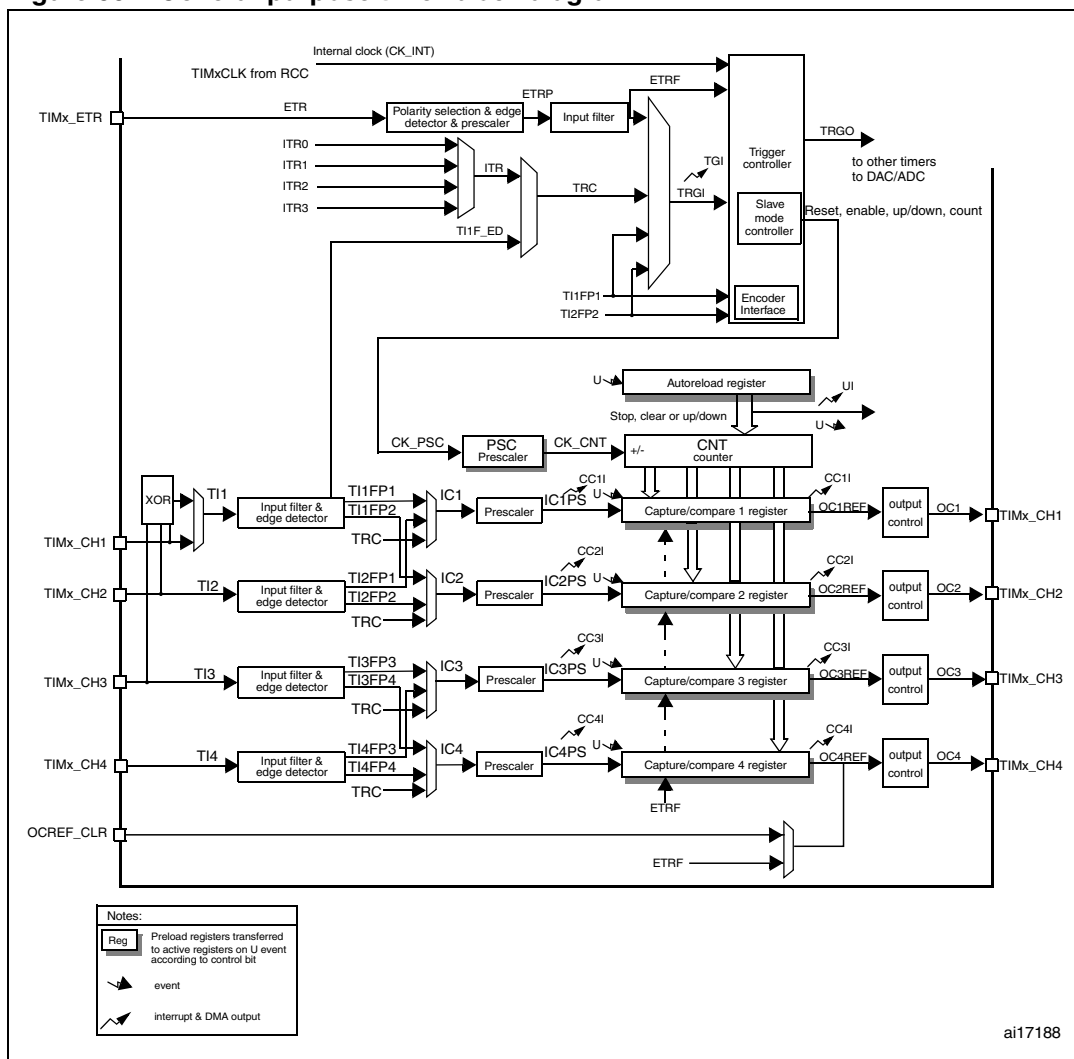
The timers are completely independent, and do not share any resources. They can be synchronized together as described in [Section 13.3.15](#).

13.2 TIM2 to TIM4 main features

General-purpose TIMx timer features include:

- 16-bit up, down, up/down auto-reload counter.
- 16-bit programmable prescaler used to divide (also “on the fly”) the counter clock frequency by any factor between 1 and 65535.
- Up to 4 independent channels for:
 - Input capture
 - Output compare
 - PWM generation (Edge- and Center-aligned modes)
 - One-pulse mode output
- Synchronization circuit to control the timer with external signals and to interconnect several timers.
- Interrupt/DMA generation on the following events:
 - Update: counter overflow/underflow, counter initialization (by software or internal/external trigger)
 - Trigger event (counter start, stop, initialization or count by internal/external trigger)
 - Input capture
 - Output compare
- Supports incremental (quadrature) encoder and hall-sensor circuitry for positioning purposes
- Trigger input for external clock or cycle-by-cycle current management

Figure 66. General-purpose timer block diagram



13.3 TIM2 to TIM4 functional description

13.3.1 Time-base unit

The main block of the programmable timer is a 16-bit counter with its related auto-reload register. The counter can count up but also down or both up and down. The counter clock can be divided by a prescaler.

The counter, the auto-reload register and the prescaler register can be written or read by software. This is true even when the counter is running.

The time-base unit includes:

- Counter Register (TIMx_CNT)
- Prescaler Register (TIMx_PSC):
- Auto-Reload Register (TIMx_ARR)

The auto-reload register is preloaded. Writing to or reading from the auto-reload register accesses the preload register. The content of the preload register are transferred into the shadow register permanently or at each update event (UEV), depending on the auto-reload preload enable bit (ARPE) in TIMx_CR1 register. The update event is sent when the counter reaches the overflow (or underflow when downcounting) and if the UDIS bit equals 0 in the TIMx_CR1 register. It can also be generated by software. The generation of the update event is described in detail for each configuration.

The counter is clocked by the prescaler output CK_CNT, which is enabled only when the counter enable bit (CEN) in TIMx_CR1 register is set (refer also to the slave mode controller description to get more details on counter enabling).

Note that the actual counter enable signal CNT_EN is set 1 clock cycle after CEN.

Prescaler description

The prescaler can divide the counter clock frequency by any factor between 1 and 65536. It is based on a 16-bit counter controlled through a 16-bit register (in the TIMx_PSC register). It can be changed on the fly as this control register is buffered. The new prescaler ratio is taken into account at the next update event.

Figure 67 and Figure 68 give some examples of the counter behavior when the prescaler ratio is changed on the fly:

Figure 67. Counter timing diagram with prescaler division change from 1 to 2

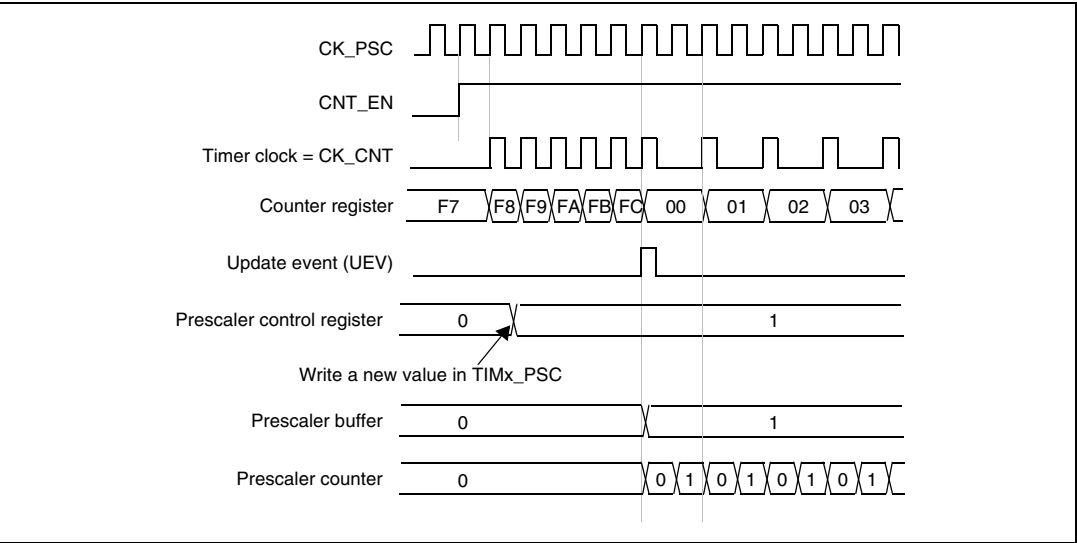
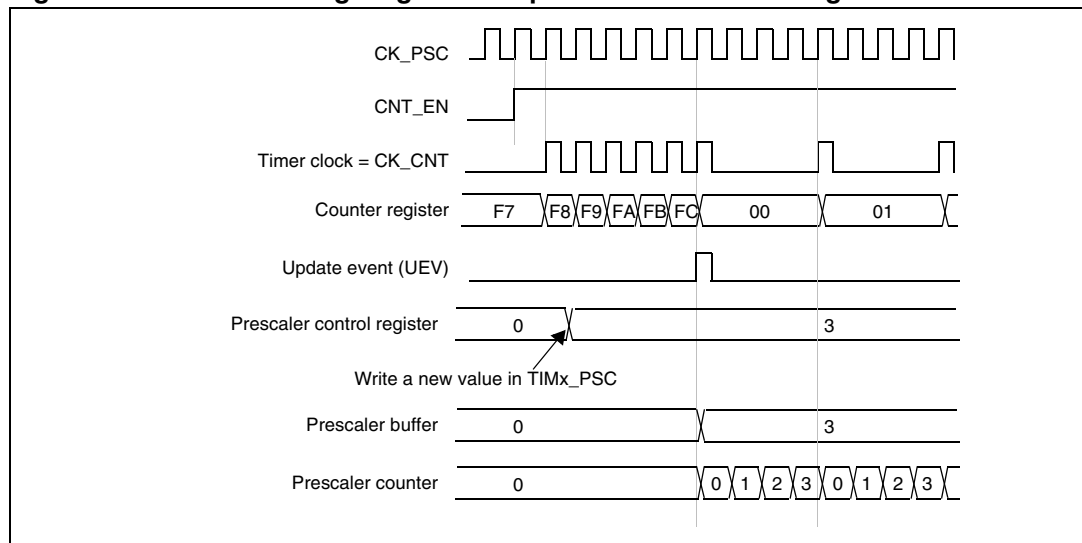


Figure 68. Counter timing diagram with prescaler division change from 1 to 4

13.3.2 Counter modes

Upcounting mode

In upcounting mode, the counter counts from 0 to the auto-reload value (content of the TIMx_ARR register), then restarts from 0 and generates a counter overflow event.

An Update event can be generated at each counter overflow or by setting the UG bit in the TIMx_EGR register (by software or by using the slave mode controller).

The UEV event can be disabled by software by setting the UDIS bit in TIMx_CR1 register. This is to avoid updating the shadow registers while writing new values in the preload registers. Then no update event occurs until the UDIS bit has been written to 0. However, the counter restarts from 0, as well as the counter of the prescaler (but the prescale rate does not change). In addition, if the URS bit (update request selection) in TIMx_CR1 register is set, setting the UG bit generates an update event UEV but without setting the UIF flag (thus no interrupt or DMA request is sent). This is to avoid generating both update and capture interrupts when clearing the counter on the capture event.

When an update event occurs, all the registers are updated and the update flag (UIF bit in TIMx_SR register) is set (depending on the URS bit):

- The buffer of the prescaler is reloaded with the preload value (content of the TIMx_PSC register)
- The auto-reload shadow register is updated with the preload value (TIMx_ARR)

The following figures show some examples of the counter behavior for different clock frequencies when TIMx_ARR=0x36.

Figure 69. Counter timing diagram, internal clock divided by 1

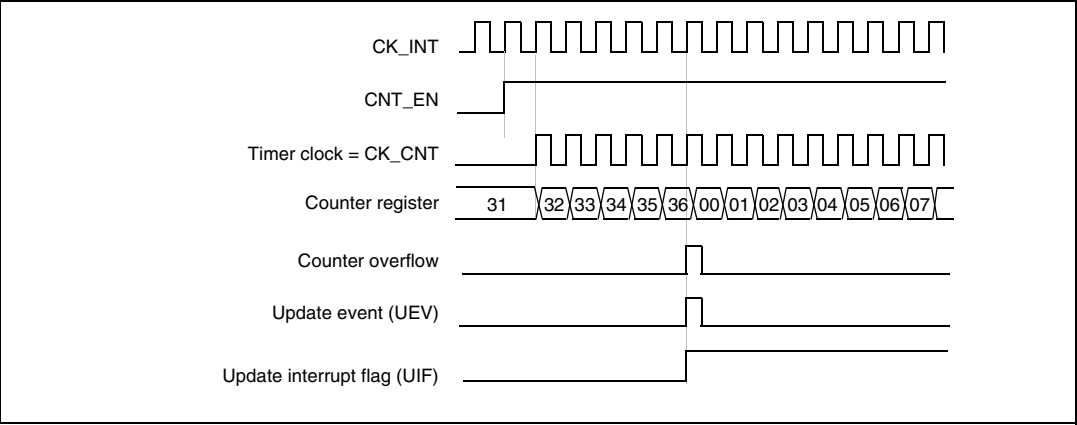


Figure 70. Counter timing diagram, internal clock divided by 2

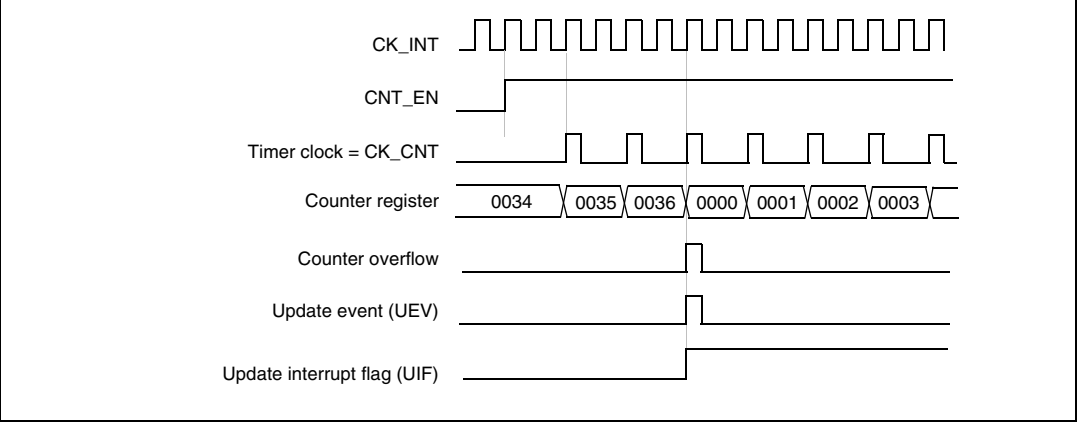


Figure 71. Counter timing diagram, internal clock divided by 4

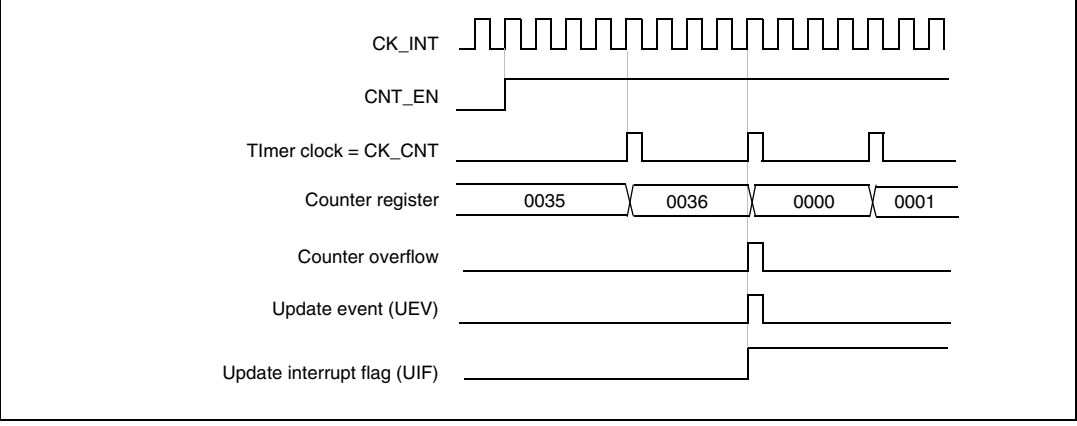


Figure 72. Counter timing diagram, internal clock divided by N

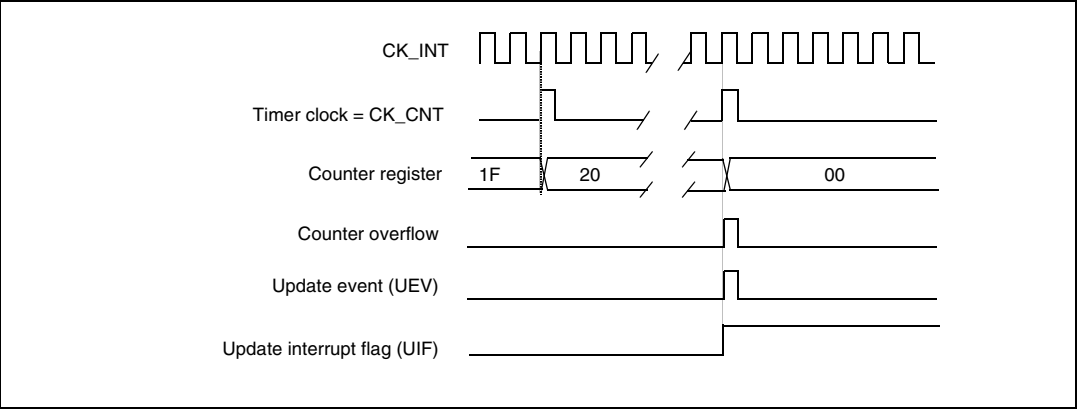


Figure 73. Counter timing diagram, Update event when ARPE=0 (TIMx_ARR not preloaded)

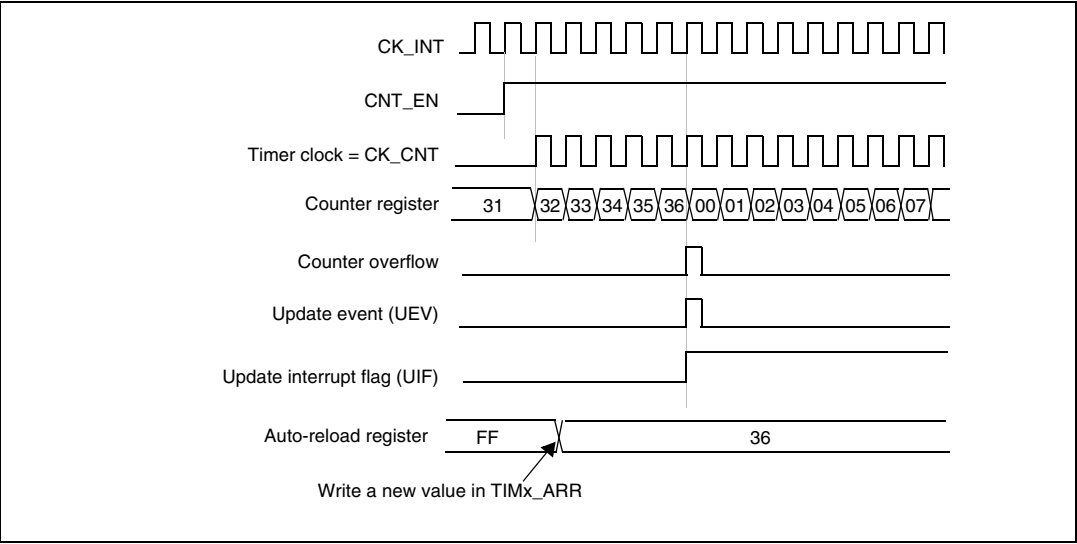
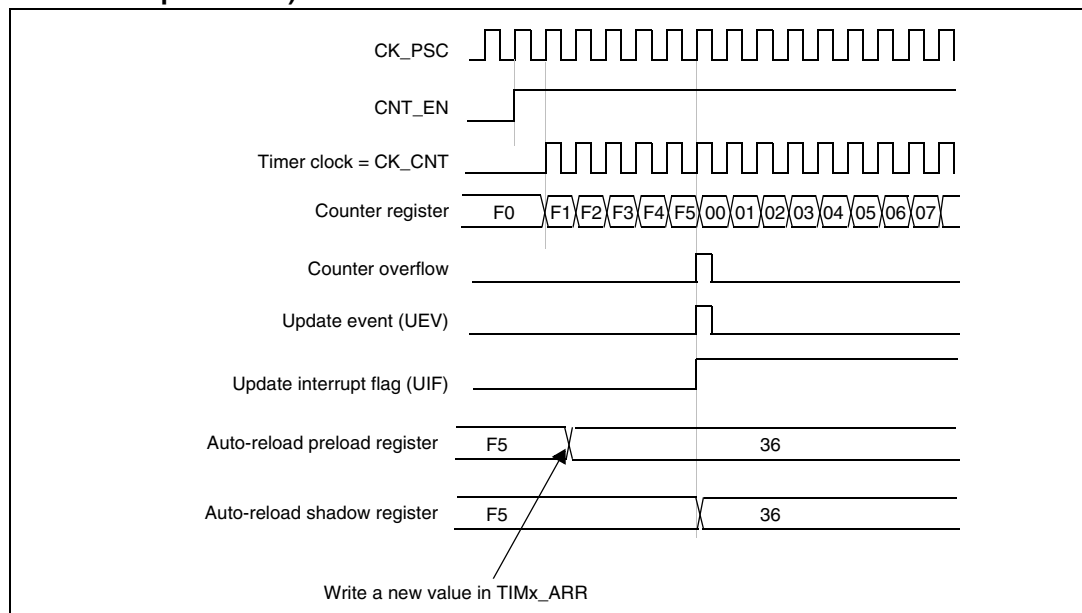


Figure 74. Counter timing diagram, Update event when ARPE=1 (TIMx_ARR preloaded)

Downcounting mode

In downcounting mode, the counter counts from the auto-reload value (content of the TIMx_ARR register) down to 0, then restarts from the auto-reload value and generates a counter underflow event.

An Update event can be generated at each counter underflow or by setting the UG bit in the TIMx_EGR register (by software or by using the slave mode controller).

The UEV update event can be disabled by software by setting the UDIS bit in TIMx_CR1 register. This is to avoid updating the shadow registers while writing new values in the preload registers. Then no update event occurs until UDIS bit has been written to 0. However, the counter restarts from the current auto-reload value, whereas the counter of the prescaler restarts from 0 (but the prescale rate doesn't change).

In addition, if the URS bit (update request selection) in TIMx_CR1 register is set, setting the UG bit generates an update event UEV but without setting the UIF flag (thus no interrupt or DMA request is sent). This is to avoid generating both update and capture interrupts when clearing the counter on the capture event.

When an update event occurs, all the registers are updated and the update flag (UIF bit in TIMx_SR register) is set (depending on the URS bit):

- The buffer of the prescaler is reloaded with the preload value (content of the TIMx_PSC register).
- The auto-reload active register is updated with the preload value (content of the TIMx_ARR register). Note that the auto-reload is updated before the counter is reloaded, so that the next period is the expected one.

The following figures show some examples of the counter behavior for different clock frequencies when TIMx_ARR=0x36.

Figure 75. Counter timing diagram, internal clock divided by 1

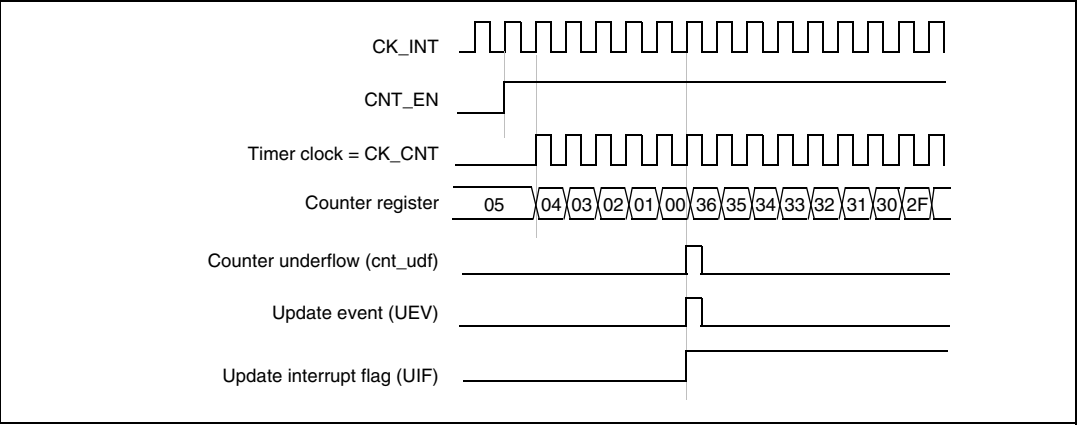


Figure 76. Counter timing diagram, internal clock divided by 2

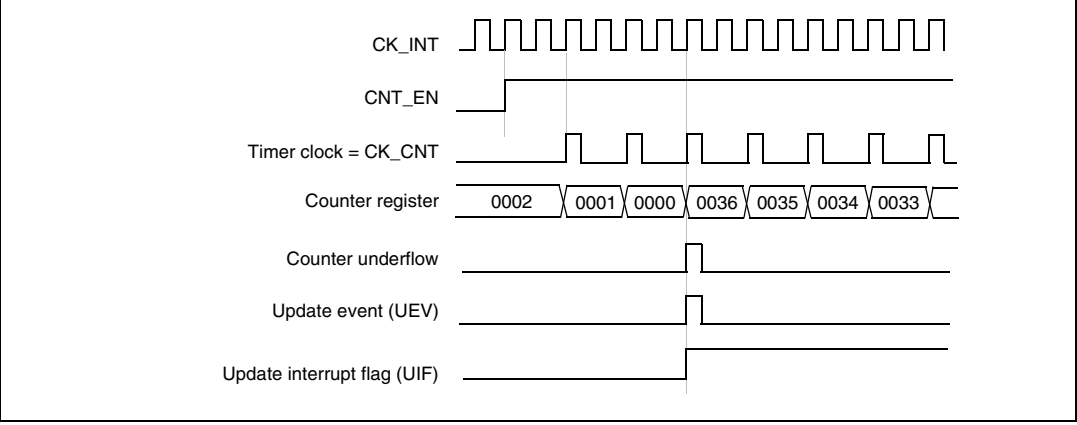


Figure 77. Counter timing diagram, internal clock divided by 4

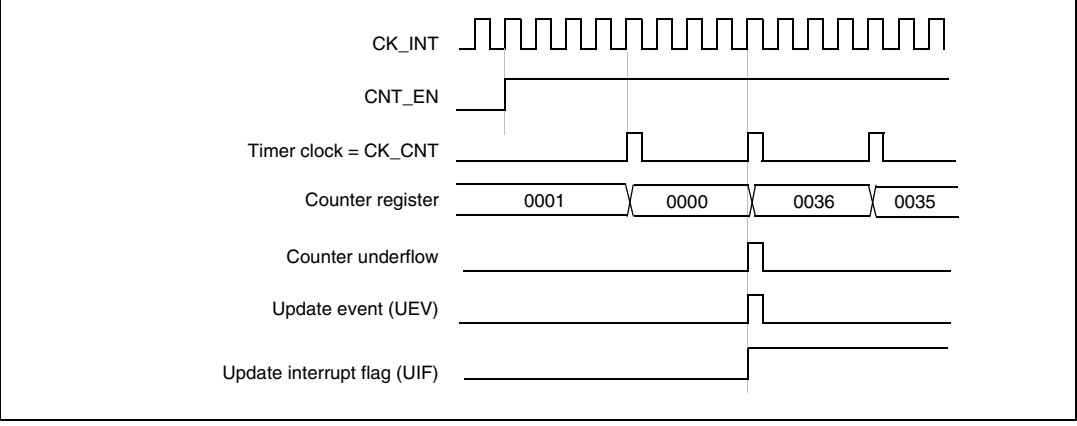
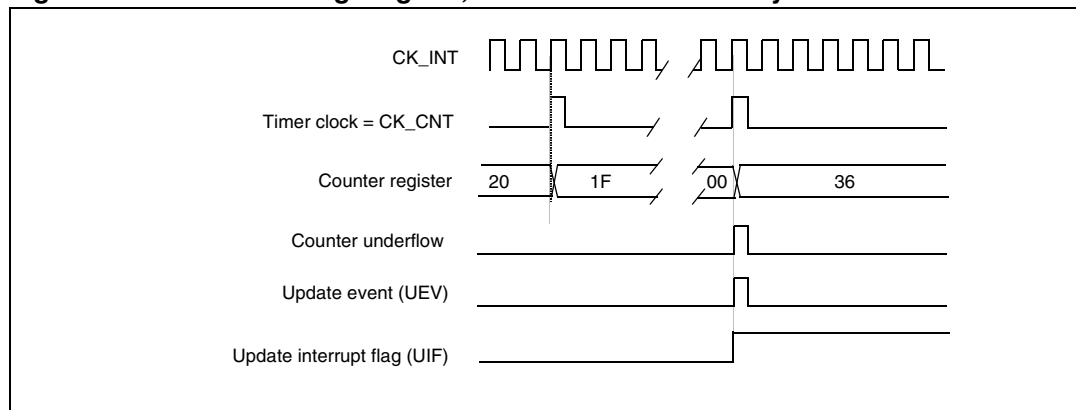
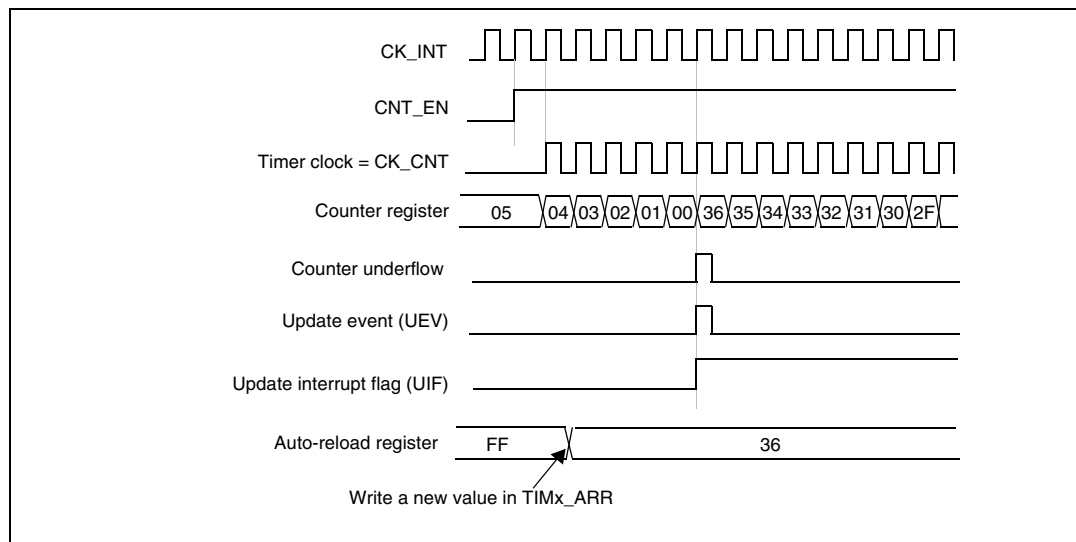


Figure 78. Counter timing diagram, internal clock divided by N**Figure 79. Counter timing diagram, Update event when repetition counter is not used****Center-aligned mode (up/down counting)**

In center-aligned mode, the counter counts from 0 to the auto-reload value (content of the TIMx_ARR register) – 1, generates a counter overflow event, then counts from the auto-reload value down to 1 and generates a counter underflow event. Then it restarts counting from 0.

Center-aligned mode is active when the CMS bits in TIMx_CR1 register are not equal to '00'. The Output compare interrupt flag of channels configured in output is set when: the counter counts down (Center aligned mode 1, CMS = "01"), the counter counts up (Center aligned mode 2, CMS = "10") the counter counts up and down (Center aligned mode 3, CMS = "11").

In this mode, the direction bit (DIR from TIMx_CR1 register) cannot be written. It is updated by hardware and gives the current direction of the counter.

The update event can be generated at each counter overflow and at each counter underflow or by setting the UG bit in the TIMx_EGR register (by software or by using the slave mode controller) also generates an update event. In this case, the counter restarts counting from 0, as well as the counter of the prescaler.

The UEV update event can be disabled by software by setting the UDIS bit in TIMx_CR1 register. This is to avoid updating the shadow registers while writing new values in the preload registers. Then no update event occurs until the UDIS bit has been written to 0. However, the counter continues counting up and down, based on the current auto-reload value.

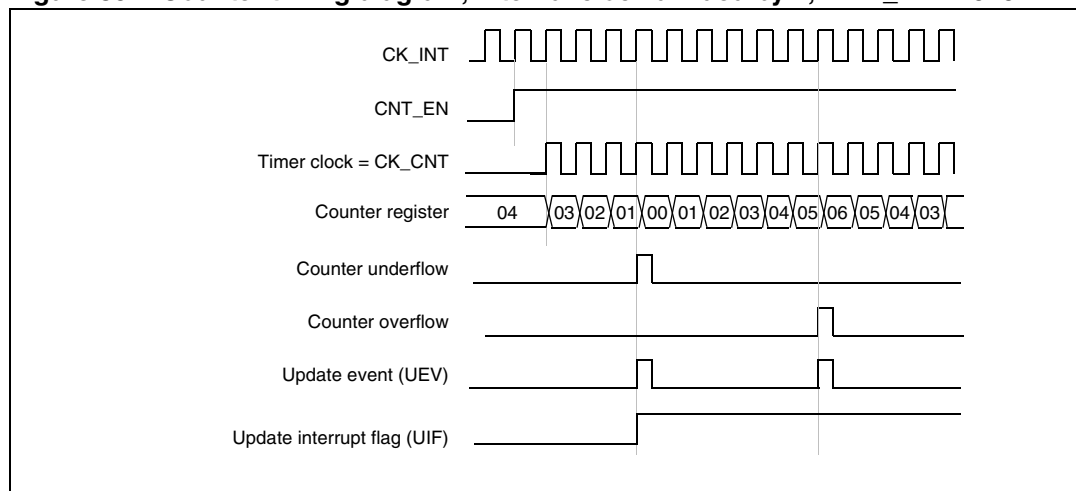
In addition, if the URS bit (update request selection) in TIMx_CR1 register is set, setting the UG bit generates an update event UEV but without setting the UIF flag (thus no interrupt or DMA request is sent). This is to avoid generating both update and capture interrupt when clearing the counter on the capture event.

When an update event occurs, all the registers are updated and the update flag (UIF bit in TIMx_SR register) is set (depending on the URS bit):

- The buffer of the prescaler is reloaded with the preload value (content of the TIMx_PSC register).
- The auto-reload active register is updated with the preload value (content of the TIMx_ARR register). Note that if the update source is a counter overflow, the auto-reload is updated before the counter is reloaded, so that the next period is the expected one (the counter is loaded with the new value).

The following figures show some examples of the counter behavior for different clock frequencies.

Figure 80. Counter timing diagram, internal clock divided by 1, TIMx_ARR=0x6



1. Here, center-aligned mode 1 is used (for more details refer to [Section 13.4.1: TIMx control register 1 \(TIMx_CR1\) on page 306](#)).

Figure 81. Counter timing diagram, internal clock divided by 2

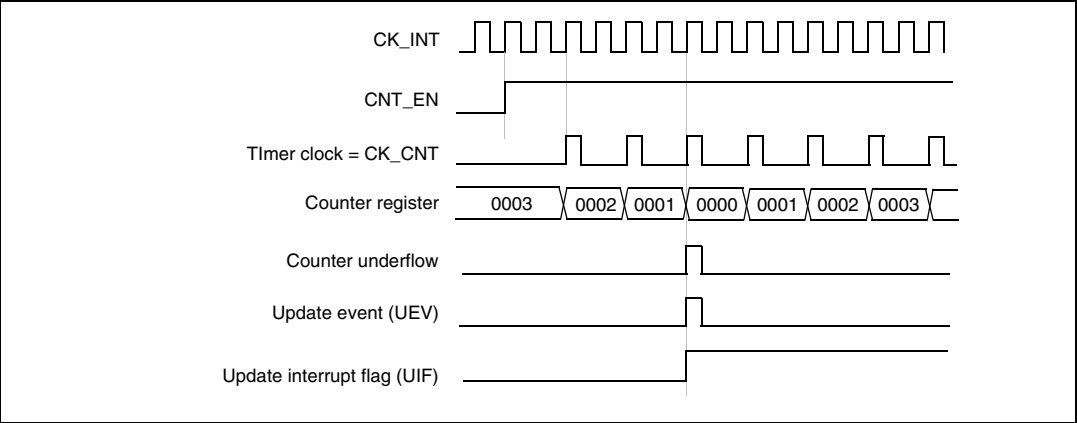
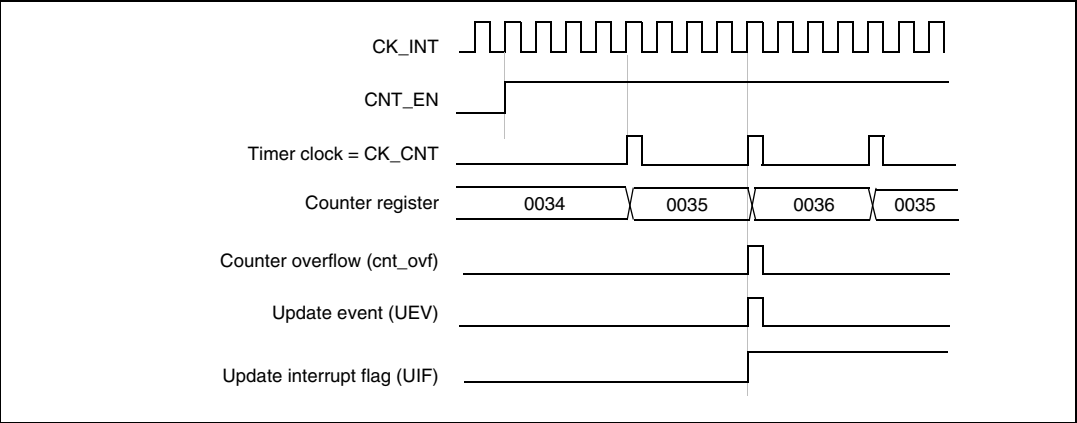


Figure 82. Counter timing diagram, internal clock divided by 4, TIMx_ARR=0x36



1. Center-aligned mode 2 or 3 is used with an UIF on overflow.

Figure 83. Counter timing diagram, internal clock divided by N

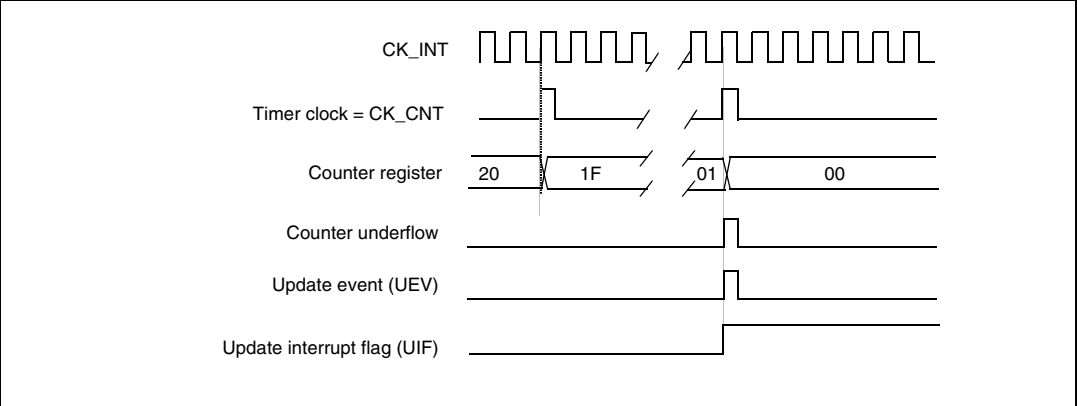


Figure 84. Counter timing diagram, Update event with ARPE=1 (counter underflow)

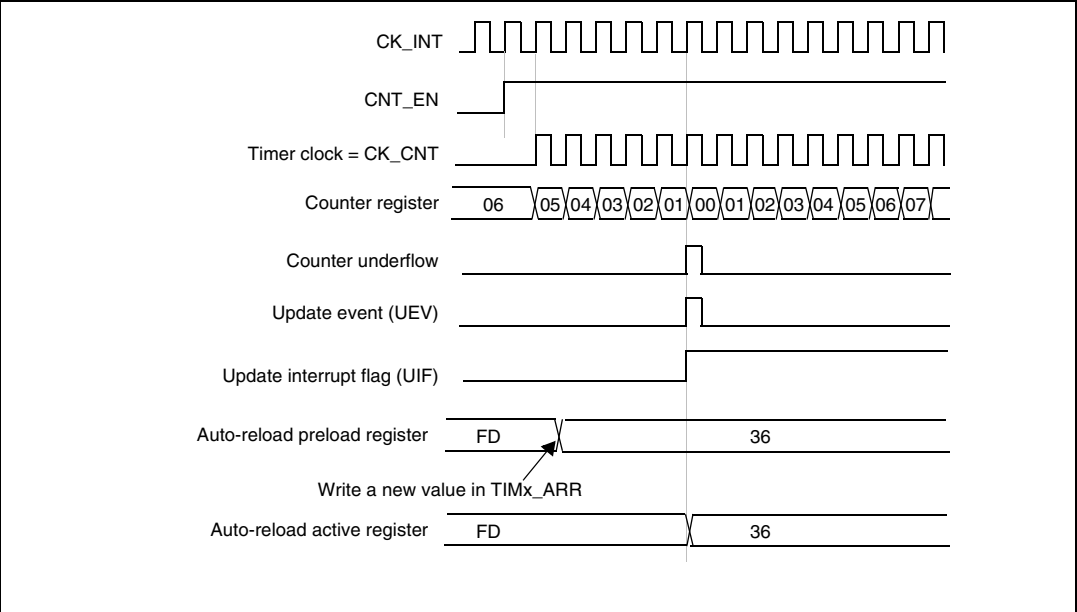
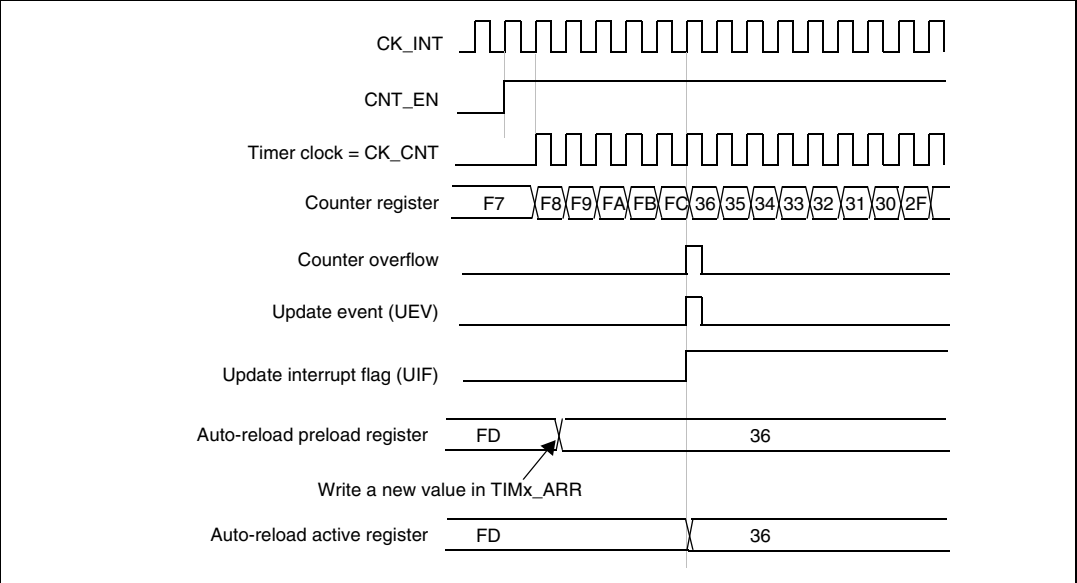


Figure 85. Counter timing diagram, Update event with ARPE=1 (counter overflow)



13.3.3 Clock selection

The counter clock can be provided by the following clock sources:

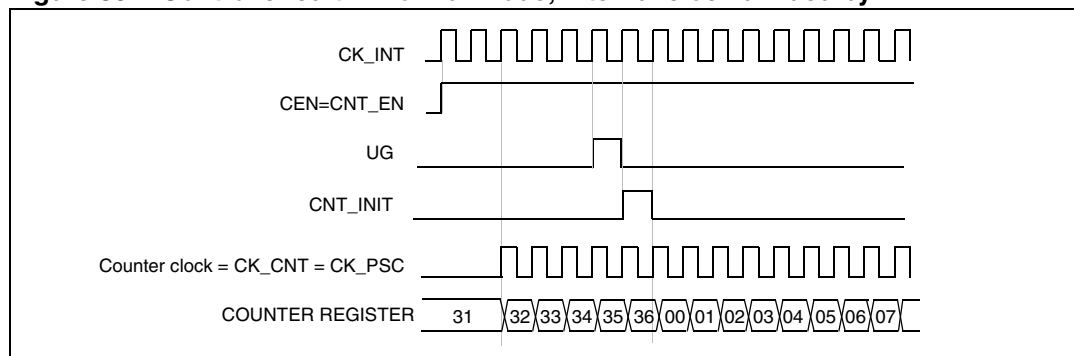
- Internal clock (CK_INT)
- External clock mode1: external input pin (TIx)
- External clock mode2: external trigger input (ETR)
- Internal trigger inputs (ITRx): using one timer as prescaler for another timer, for example, you can configure Timer 3 to act as a prescaler for Timer 2. Refer to : [Using one timer as prescaler for another on page 301](#) for more details.

Internal clock source (CK_INT)

If the slave mode controller is disabled (SMS=000 in the TIMx_SMCR register), then the CEN, DIR (in the TIMx_CR1 register) and UG bits (in the TIMx_EGR register) are actual control bits and can be changed only by software (except UG which remains cleared automatically). As soon as the CEN bit is written to 1, the prescaler is clocked by the internal clock CK_INT.

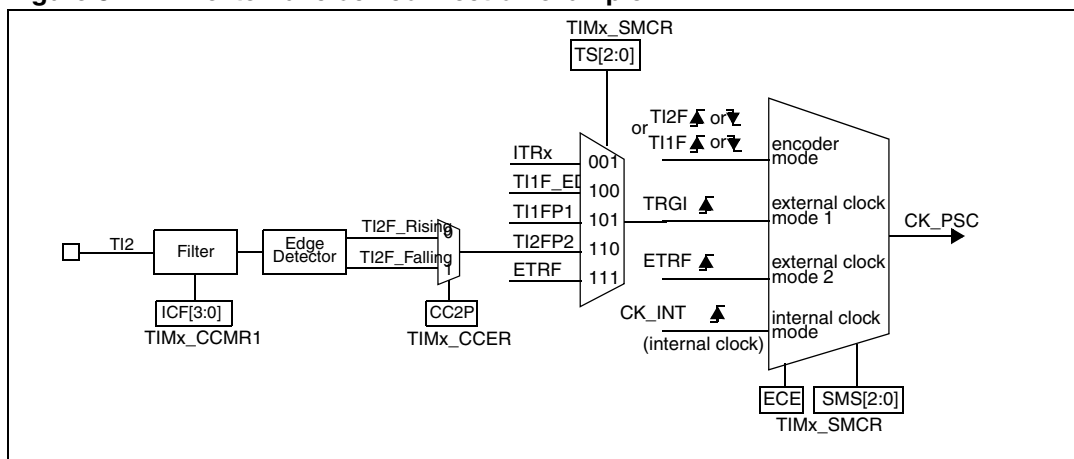
[Figure 86](#) shows the behavior of the control circuit and the upcounter in normal mode, without prescaler.

Figure 86. Control circuit in normal mode, internal clock divided by 1



External clock source mode 1

This mode is selected when SMS=111 in the TIMx_SMCR register. The counter can count at each rising or falling edge on a selected input.

Figure 87. TI2 external clock connection example

For example, to configure the upcounter to count in response to a rising edge on the TI2 input, use the following procedure:

For example, to configure the upcounter to count in response to a rising edge on the TI2 input, use the following procedure:

1. Configure channel 2 to detect rising edges on the TI2 input by writing CC2S= '01 in the TIMx_CCMR1 register.
2. Configure the input filter duration by writing the IC2F[3:0] bits in the TIMx_CCMR1 register (if no filter is needed, keep IC2F=0000).

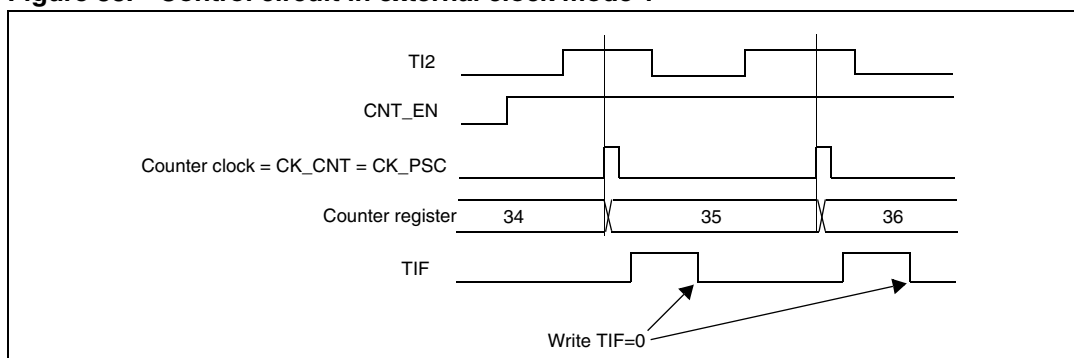
Note:

The capture prescaler is not used for triggering, so you don't need to configure it.

3. Select rising edge polarity by writing CC2P=0 and CC2NP=0 in the TIMx_CCER register.
4. Configure the timer in external clock mode 1 by writing SMS=111 in the TIMx_SMCR register.
5. Select TI2 as the input source by writing TS=110 in the TIMx_SMCR register.
6. Enable the counter by writing CEN=1 in the TIMx_CR1 register.

When a rising edge occurs on TI2, the counter counts once and the TIF flag is set.

The delay between the rising edge on TI2 and the actual clock of the counter is due to the resynchronization circuit on TI2 input.

Figure 88. Control circuit in external clock mode 1

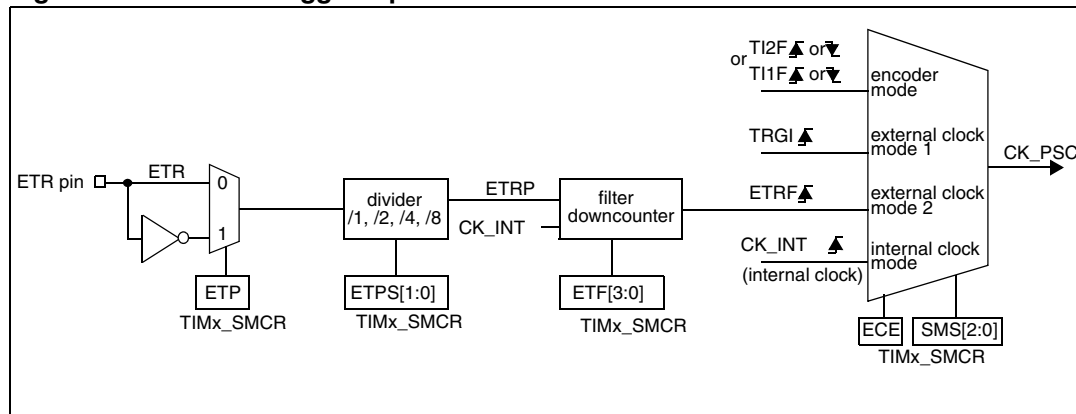
External clock source mode 2

This mode is selected by writing ECE=1 in the TIMx_SMCR register.

The counter can count at each rising or falling edge on the external trigger input ETR.

The [Figure 89](#) gives an overview of the external trigger input block.

Figure 89. External trigger input block



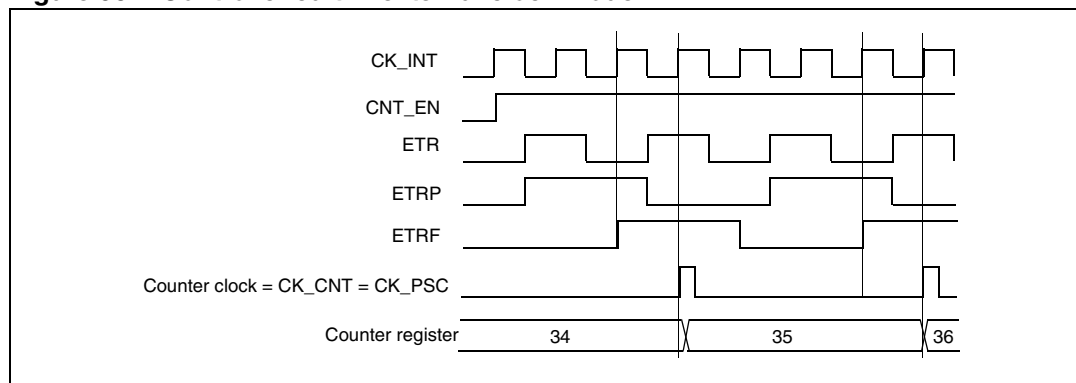
For example, to configure the upcounter to count each 2 rising edges on ETR, use the following procedure:

1. As no filter is needed in this example, write ETF[3:0]=0000 in the TIMx_SMCR register.
2. Set the prescaler by writing ETPS[1:0]=01 in the TIMx_SMCR register
3. Select rising edge detection on the ETR pin by writing ETP=0 in the TIMx_SMCR register
4. Enable external clock mode 2 by writing ECE=1 in the TIMx_SMCR register.
5. Enable the counter by writing CEN=1 in the TIMx_CR1 register.

The counter counts once each 2 ETR rising edges.

The delay between the rising edge on ETR and the actual clock of the counter is due to the resynchronization circuit on the ETRP signal.

Figure 90. Control circuit in external clock mode 2



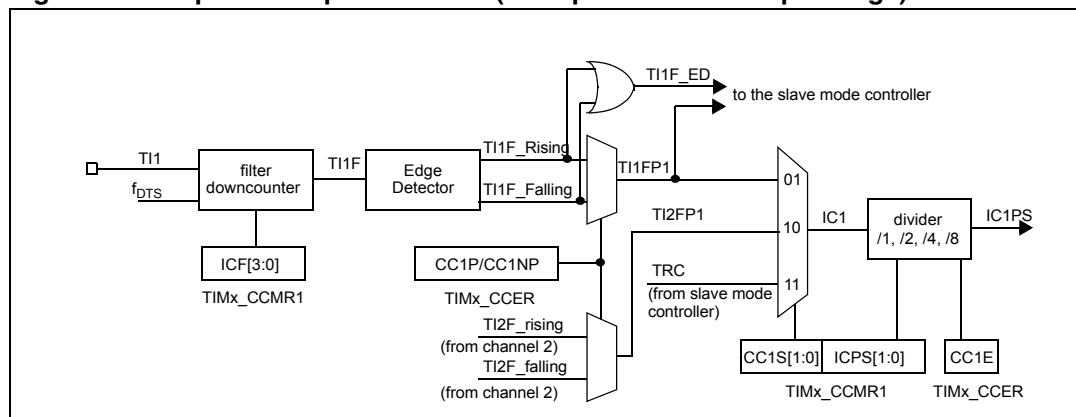
13.3.4 Capture/compare channels

Each Capture/Compare channel is built around a capture/compare register (including a shadow register), an input stage for capture (with digital filter, multiplexing and prescaler) and an output stage (with comparator and output control).

The following figure gives an overview of one Capture/Compare channel.

The input stage samples the corresponding TIMx input to generate a filtered signal TIxF. Then, an edge detector with polarity selection generates a signal (TIxFPx) which can be used as trigger input by the slave mode controller or as the capture command. It is prescaled before the capture register (ICxPS).

Figure 91. Capture/compare channel (example: channel 1 input stage)



The output stage generates an intermediate waveform which is then used for reference: OCxRef (active high). The polarity acts at the end of the chain.

Figure 92. Capture/compare channel 1 main circuit

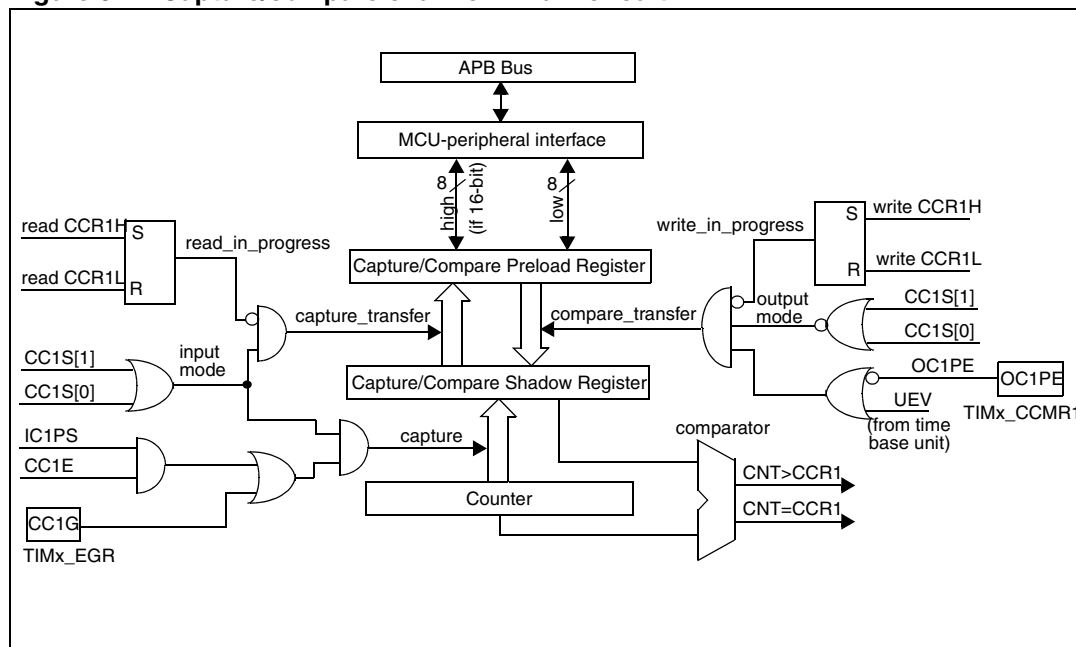
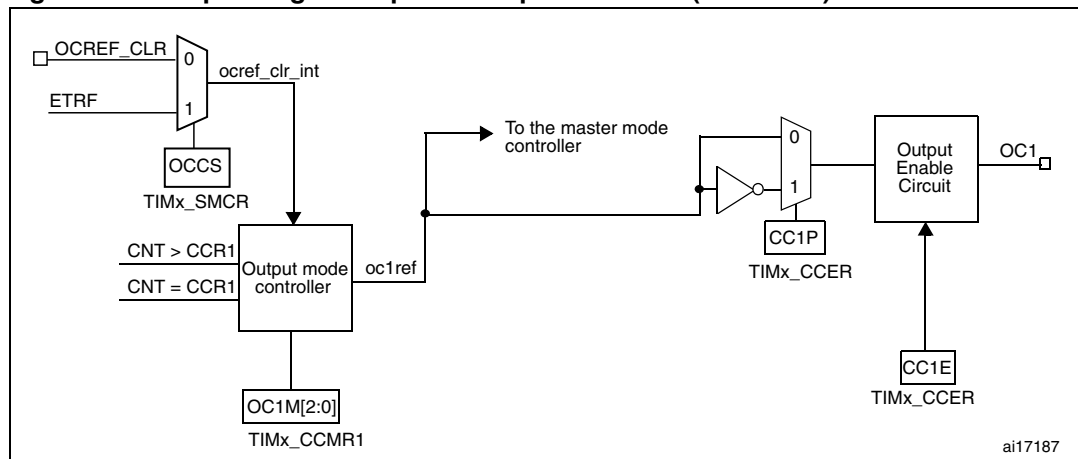


Figure 93. Output stage of capture/compare channel (channel 1)

The capture/compare block is made of one preload register and one shadow register. Write and read always access the preload register.

In capture mode, captures are actually done in the shadow register, which is copied into the preload register.

In compare mode, the content of the preload register is copied into the shadow register which is compared to the counter.

13.3.5 Input capture mode

In Input capture mode, the Capture/Compare Registers (TIMx_CCRx) are used to latch the value of the counter after a transition detected by the corresponding ICx signal. When a capture occurs, the corresponding CCxIF flag (TIMx_SR register) is set and an interrupt or a DMA request can be sent if they are enabled. If a capture occurs while the CCxIF flag was already high, then the over-capture flag CCxOF (TIMx_SR register) is set. CCxIF can be cleared by software by writing it to 0 or by reading the captured data stored in the TIMx_CCRx register. CCxOF is cleared when you write it to 0.

The following example shows how to capture the counter value in TIMx_CCR1 when TI1 input rises. To do this, use the following procedure:

- Select the active input: TIMx_CCR1 must be linked to the TI1 input, so write the CC1S bits to 01 in the TIMx_CCMR1 register. As soon as CC1S becomes different from 00, the channel is configured in input and the TIMx_CCR1 register becomes read-only.
- Program the input filter duration you need with respect to the signal you connect to the timer (when the input is one of the TIx (ICxF bits in the TIMx_CCMRx register). Let's imagine that, when toggling, the input signal is not stable during at most 5 internal clock cycles. We must program a filter duration longer than these 5 clock cycles. We can validate a transition on TI1 when 8 consecutive samples with the new level have been detected (sampled at f_{DTS} frequency). Then write IC1F bits to 0011 in the TIMx_CCMR1 register.
- Select the edge of the active transition on the TI1 channel by writing the CC1P and CC1NP bits to 00 in the TIMx_CCER register (rising edge in this case).
- Program the input prescaler. In our example, we wish the capture to be performed at each valid transition, so the prescaler is disabled (write IC1PS bits to 00 in the TIMx_CCMR1 register).
- Enable capture from the counter into the capture register by setting the CC1E bit in the TIMx_CCER register.
- If needed, enable the related interrupt request by setting the CC1IE bit in the TIMx_DIER register, and/or the DMA request by setting the CC1DE bit in the TIMx_DIER register.

When an input capture occurs:

- The TIMx_CCR1 register gets the value of the counter on the active transition.
- CC1IF flag is set (interrupt flag). CC1OF is also set if at least two consecutive captures occurred whereas the flag was not cleared.
- An interrupt is generated depending on the CC1IE bit.
- A DMA request is generated depending on the CC1DE bit.

In order to handle the overcapture, it is recommended to read the data before the overcapture flag. This is to avoid missing an overcapture which could happen after reading the flag and before reading the data.

Note: IC interrupt and/or DMA requests can be generated by software by setting the corresponding CCxG bit in the TIMx_EGR register.

13.3.6 PWM input mode

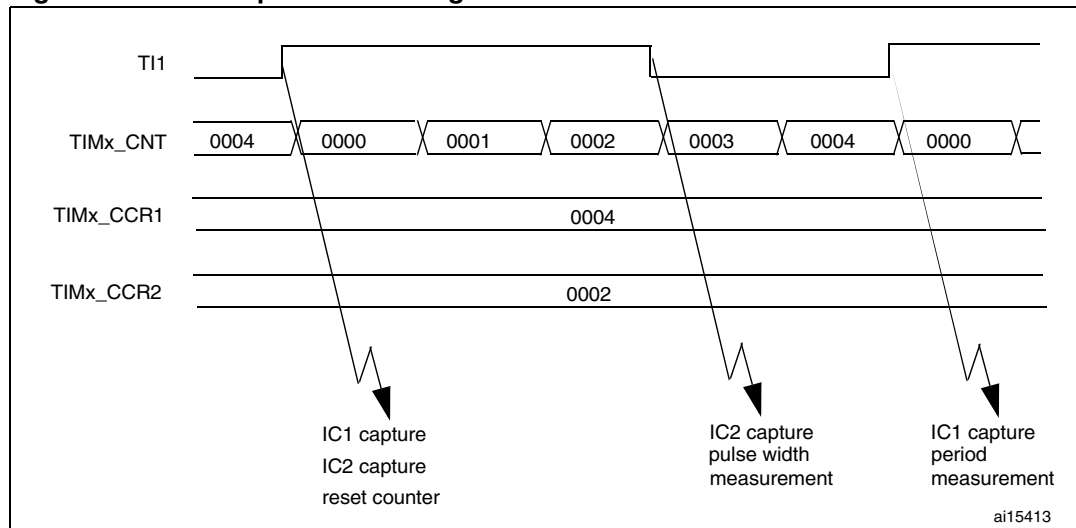
This mode is a particular case of input capture mode. The procedure is the same except:

- Two ICx signals are mapped on the same TIx input.
- These 2 ICx signals are active on edges with opposite polarity.
- One of the two TIxFP signals is selected as trigger input and the slave mode controller is configured in reset mode.

For example, you can measure the period (in TIMx_CCR1 register) and the duty cycle (in TIMx_CCR2 register) of the PWM applied on TI1 using the following procedure (depending on CK_INT frequency and prescaler value):

- Select the active input for TIMx_CCR1: write the CC1S bits to 01 in the TIMx_CCMR1 register (TI1 selected).
- Select the active polarity for TI1FP1 (used both for capture in TIMx_CCR1 and counter clear): write the CC1P to '0' and the CC1NP bit to '0' (active on rising edge).
- Select the active input for TIMx_CCR2: write the CC2S bits to 10 in the TIMx_CCMR1 register (TI1 selected).
- Select the active polarity for TI1FP2 (used for capture in TIMx_CCR2): write the CC2P bit to '1' and the CC2NP bit to '0' (active on falling edge).
- Select the valid trigger input: write the TS bits to 101 in the TIMx_SMCR register (TI1FP1 selected).
- Configure the slave mode controller in reset mode: write the SMS bits to 100 in the TIMx_SMCR register.
- Enable the captures: write the CC1E and CC2E bits to '1' in the TIMx_CCER register.

Figure 94. PWM input mode timing



13.3.7 Forced output mode

In output mode (CCxS bits = 00 in the TIMx_CCMRx register), each output compare signal (OCxREF and then OCx) can be forced to active or inactive level directly by software, independently of any comparison between the output compare register and the counter.

To force an output compare signal (ocxref/OCx) to its active level, you just need to write 101 in the OCxM bits in the corresponding TIMx_CCMRx register. Thus ocxref is forced high (OCxREF is always active high) and OCx get opposite value to CCxP polarity bit.

e.g.: CCxP=0 (OCx active high) => OCx is forced to high level.

ocxref signal can be forced low by writing the OCxM bits to 100 in the TIMx_CCMRx register.

Anyway, the comparison between the TIMx_CCRx shadow register and the counter is still performed and allows the flag to be set. Interrupt and DMA requests can be sent accordingly. This is described in the Output Compare Mode section.

13.3.8 Output compare mode

This function is used to control an output waveform or indicating when a period of time has elapsed.

When a match is found between the capture/compare register and the counter, the output compare function:

- Assigns the corresponding output pin to a programmable value defined by the output compare mode (OCxM bits in the TIMx_CCMRx register) and the output polarity (CCxP bit in the TIMx_CCER register). The output pin can keep its level (OCxM=000), be set active (OCxM=001), be set inactive (OCxM=010) or can toggle (OCxM=011) on match.
- Sets a flag in the interrupt status register (CCxIF bit in the TIMx_SR register).
- Generates an interrupt if the corresponding interrupt mask is set (CCxIE bit in the TIMx_DIER register).
- Sends a DMA request if the corresponding enable bit is set (CCxDE bit in the TIMx_DIER register, CCDS bit in the TIMx_CR2 register for the DMA request selection).

The TIMx_CCRx registers can be programmed with or without preload registers using the OCxPE bit in the TIMx_CCMRx register.

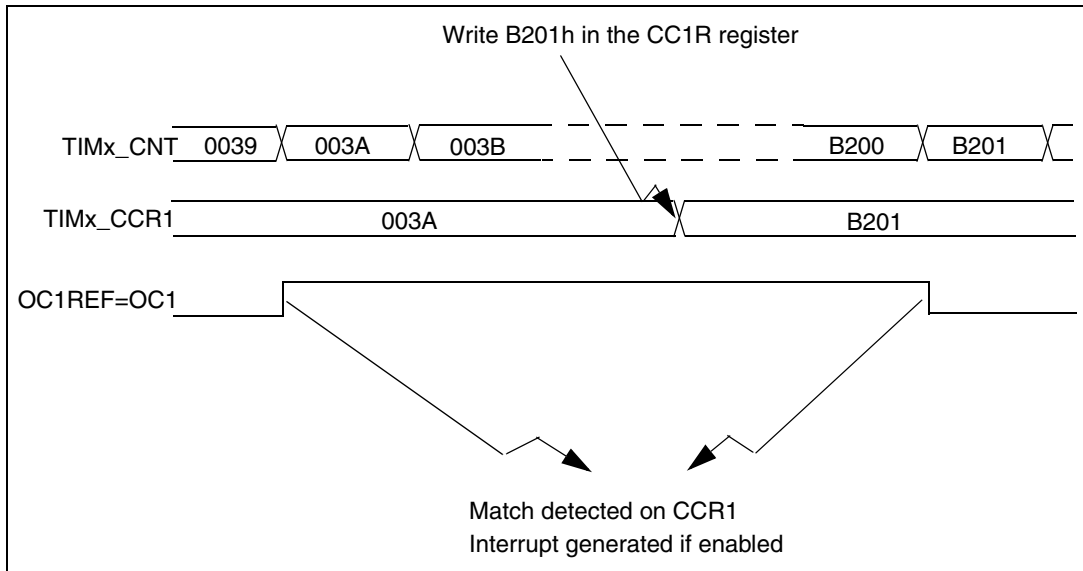
In output compare mode, the update event UEV has no effect on ocxref and OCx output. The timing resolution is one count of the counter. Output compare mode can also be used to output a single pulse (in One-pulse mode).

Procedure:

1. Select the counter clock (internal, external, prescaler).
2. Write the desired data in the TIMx_ARR and TIMx_CCRx registers.
3. Set the CCxIE and/or CCxDE bits if an interrupt and/or a DMA request is to be generated.
4. Select the output mode. For example, you must write OCxM=011, OCxPE=0, CCxP=0 and CCxE=1 to toggle OCx output pin when CNT matches CCRx, CCRx preload is not used, OCx is enabled and active high.
5. Enable the counter by setting the CEN bit in the TIMx_CR1 register.

The TIMx_CCRx register can be updated at any time by software to control the output waveform, provided that the preload register is not enabled (OCxPE=0, else TIMx_CCRx shadow register is updated only at the next update event UEV). An example is given in [Figure 95](#).

Figure 95. Output compare mode, toggle on OC1.



13.3.9 PWM mode

Pulse width modulation mode allows you to generate a signal with a frequency determined by the value of the TIMx_ARR register and a duty cycle determined by the value of the TIMx_CCRx register.

The PWM mode can be selected independently on each channel (one PWM per OCx output) by writing 110 (PWM mode 1) or '111 (PWM mode 2) in the OCxM bits in the TIMx_CCMRx register. You must enable the corresponding preload register by setting the OCxPE bit in the TIMx_CCMRx register, and eventually the auto-reload preload register (in upcounting or center-aligned modes) by setting the ARPE bit in the TIMx_CR1 register.

As the preload registers are transferred to the shadow registers only when an update event occurs, before starting the counter, you have to initialize all the registers by setting the UG bit in the TIMx_EGR register.

OCx polarity is software programmable using the CCxP bit in the TIMx_CCER register. It can be programmed as active high or active low. OCx output is enabled by the CCxE bit in the TIMx_CCER register. Refer to the TIMx_CCERx register description for more details.

In PWM mode (1 or 2), TIMx_CNT and TIMx_CCRx are always compared to determine whether $TIMx_CCRx \leq TIMx_CNT$ or $TIMx_CNT \leq TIMx_CCRx$ (depending on the direction of the counter). However, to comply with the OCREF_CLR functionality (OCREF can be cleared by an external event through the ETR signal until the next PWM period), the OCREF signal is asserted only:

- When the result of the comparison changes, or
- When the output compare mode (OCxM bits in TIMx_CCMRx register) switches from the "frozen" configuration (no comparison, OCxM='000) to one of the PWM modes (OCxM='110 or '111).

This forces the PWM by software while the timer is running.

The timer is able to generate PWM in edge-aligned mode or center-aligned mode depending on the CMS bits in the TIMx_CR1 register.

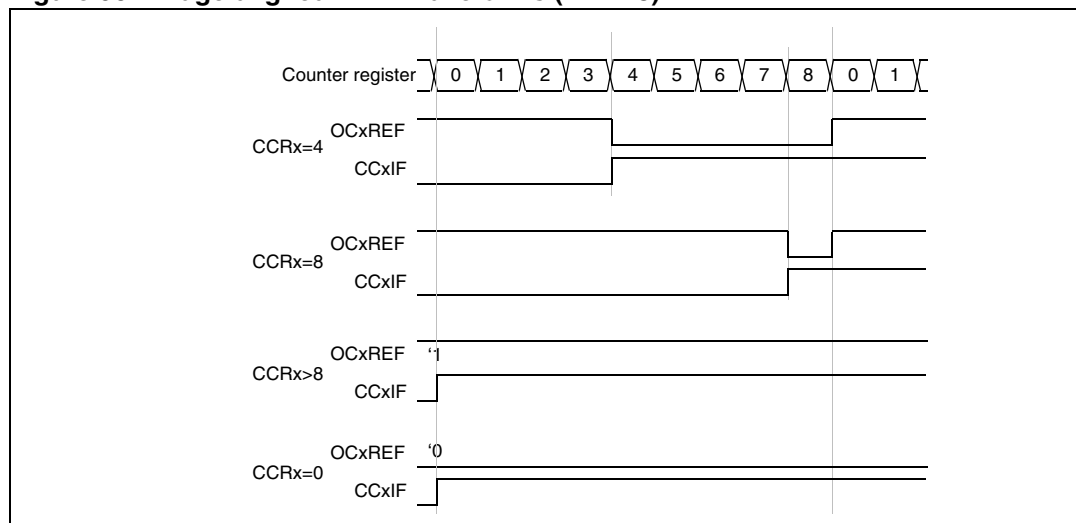
PWM edge-aligned mode

Upcounting configuration

Upcounting is active when the DIR bit in the TIMx_CR1 register is low. Refer to the [Section : Upcounting mode on page 273](#).

In the following example, we consider PWM mode 1. The reference PWM signal OCxREF is high as long as $TIMx_CNT < TIMx_CCRx$ else it becomes low. If the compare value in TIMx_CCRx is greater than the auto-reload value (in TIMx_ARR) then OCxREF is held at '1'. If the compare value is 0 then OCxREF is held at '0'. [Figure 96](#) shows some edge-aligned PWM waveforms in an example where $TIMx_ARR=8$.

Figure 96. Edge-aligned PWM waveforms (ARR=8)



Downcounting configuration

Downcounting is active when DIR bit in TIMx_CR1 register is high. Refer to [Downcounting mode on page 276](#)

In PWM mode 1, the reference signal ocxref is low as long as $TIMx_CNT > TIMx_CCRx$ else it becomes high. If the compare value in TIMx_CCRx is greater than the auto-reload value in TIMx_ARR, then ocxref is held at '1'. 0% PWM is not possible in this mode.

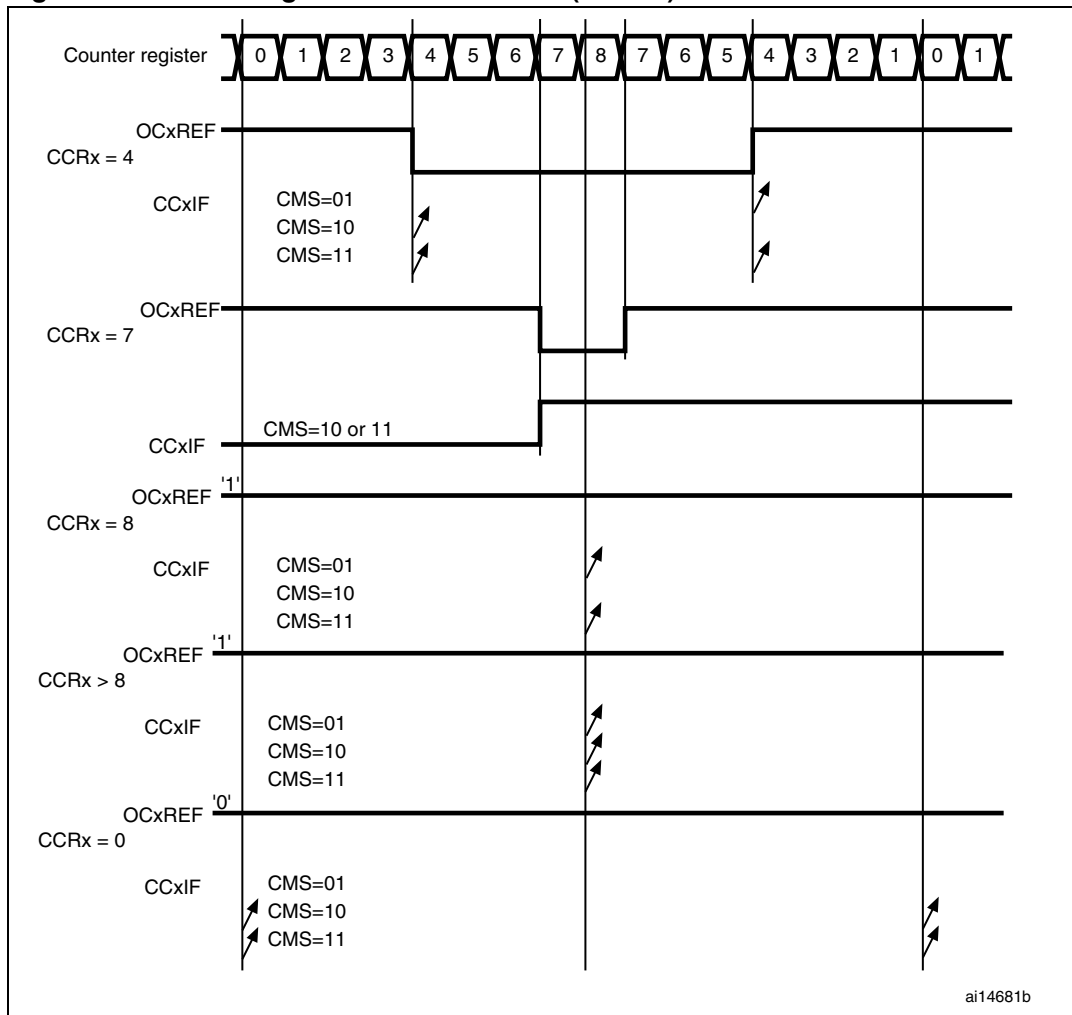
PWM center-aligned mode

Center-aligned mode is active when the CMS bits in TIMx_CR1 register are different from '00' (all the remaining configurations having the same effect on the ocxref/OCx signals). The compare flag is set when the counter counts up, when it counts down or both when it counts up and down depending on the CMS bits configuration. The direction bit (DIR) in the TIMx_CR1 register is updated by hardware and must not be changed by software. Refer to the [Center-aligned mode \(up/down counting\) on page 278](#).

Figure 97 shows some center-aligned PWM waveforms in an example where:

- TIMx_ARR=8,
- PWM mode is the PWM mode 1,
- The flag is set when the counter counts down corresponding to the center-aligned mode 1 selected for CMS=01 in TIMx_CR1 register.

Figure 97. Center-aligned PWM waveforms (ARR=8)



Hints on using center-aligned mode:

- When starting in center-aligned mode, the current up-down configuration is used. It means that the counter counts up or down depending on the value written in the DIR bit in the TIMx_CR1 register. Moreover, the DIR and CMS bits must not be changed at the same time by the software.
- Writing to the counter while running in center-aligned mode is not recommended as it can lead to unexpected results. In particular:
 - The direction is not updated if you write a value in the counter that is greater than the auto-reload value (TIMx_CNT > TIMx_ARR). For example, if the counter was counting up, it continues to count up.
 - The direction is updated if you write 0 or write the TIMx_ARR value in the counter but no Update Event UEV is generated.

- The safest way to use center-aligned mode is to generate an update by software (setting the UG bit in the TIMx_EGR register) just before starting the counter and not to write the counter while it is running.

13.3.10 One-pulse mode

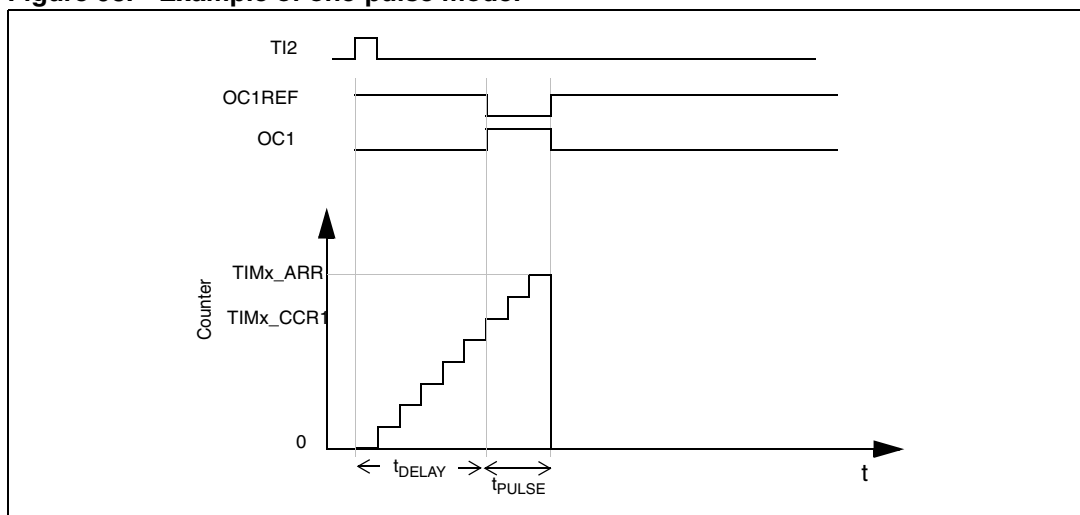
One-pulse mode (OPM) is a particular case of the previous modes. It allows the counter to be started in response to a stimulus and to generate a pulse with a programmable length after a programmable delay.

Starting the counter can be controlled through the slave mode controller. Generating the waveform can be done in output compare mode or PWM mode. You select One-pulse mode by setting the OPM bit in the TIMx_CR1 register. This makes the counter stop automatically at the next update event UEV.

A pulse can be correctly generated only if the compare value is different from the counter initial value. Before starting (when the timer is waiting for the trigger), the configuration must be:

- In upcounting: $CNT < CCRx \leq ARR$ (in particular, $0 < CCRx$),
- In downcounting: $CNT > CCRx$.

Figure 98. Example of one-pulse mode.



For example you may want to generate a positive pulse on OC1 with a length of t_{PULSE} and after a delay of t_{DELAY} as soon as a positive edge is detected on the TI2 input pin.

Let's use TI2FP2 as trigger 1:

- Map TI2FP2 on TI2 by writing IC2S=01 in the TIMx_CCMR1 register.
- TI2FP2 must detect a rising edge, write CC2P=0 and CC2NP='0' in the TIMx_CCER register.
- Configure TI2FP2 as trigger for the slave mode controller (TRGI) by writing TS=110 in the TIMx_SMCR register.
- TI2FP2 is used to start the counter by writing SMS to '110 in the TIMx_SMCR register (trigger mode).

The OPM waveform is defined by writing the compare registers (taking into account the clock frequency and the counter prescaler).

- The t_{DELAY} is defined by the value written in the TIMx_CCR1 register.
- The t_{PULSE} is defined by the difference between the auto-reload value and the compare value (TIMx_ARR - TIMx_CCR1).
- Let's say you want to build a waveform with a transition from '0 to '1 when a compare match occurs and a transition from '1 to '0 when the counter reaches the auto-reload value. To do this you enable PWM mode 2 by writing OC1M=111 in the TIMx_CCMR1 register. You can optionally enable the preload registers by writing OC1PE=1 in the TIMx_CCMR1 register and ARPE in the TIMx_CR1 register. In this case you have to write the compare value in the TIMx_CCR1 register, the auto-reload value in the TIMx_ARR register, generate an update by setting the UG bit and wait for external trigger event on TI2. CC1P is written to '0 in this example.

In our example, the DIR and CMS bits in the TIMx_CR1 register should be low.

You only want 1 pulse(Single mode), so you write '1 in the OPM bit in the TIMx_CR1 register to stop the counter at the next update event (when the counter rolls over from the auto-reload value back to 0). When OPM bit in the TIMx_CR1 register is set to '0', so the Repetitive Mode is selected.

Particular case: OCx fast enable:

In One-pulse mode, the edge detection on TIx input set the CEN bit which enables the counter. Then the comparison between the counter and the compare value makes the output toggle. But several clock cycles are needed for these operations and it limits the minimum delay t_{DELAY} min we can get.

If you want to output a waveform with the minimum delay, you can set the OCxFE bit in the TIMx_CCMRx register. Then OCxRef (and OCx) is forced in response to the stimulus, without taking in account the comparison. Its new level is the same as if a compare match had occurred. OCxFE acts only if the channel is configured in PWM1 or PWM2 mode.

13.3.11 Clearing the OCxREF signal on an external event

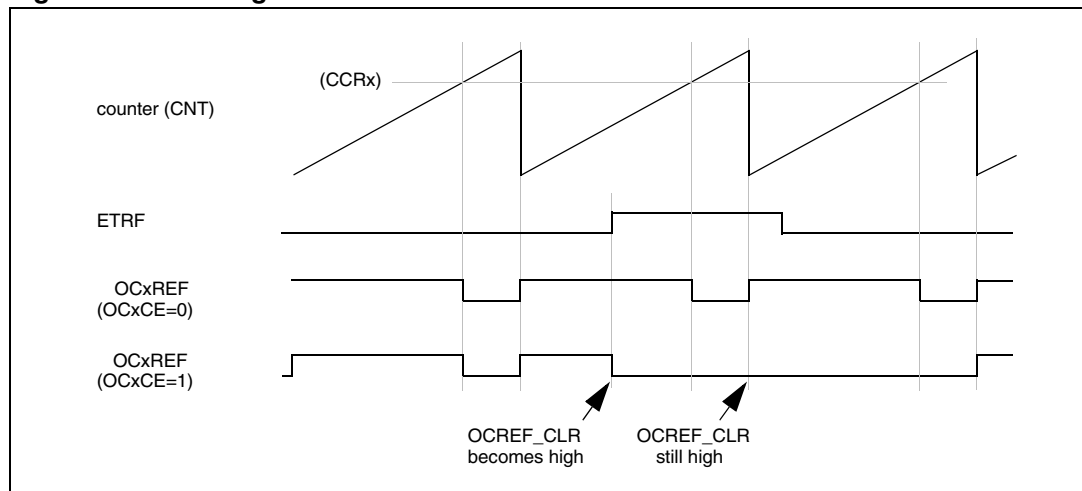
The OCxREF signal of a given channel can be cleared when a high level is applied on OCREF_CLR_INPUT (OCxCE enable bit in the corresponding TIMx_CCMRx register set to 1). OCxREF remains low until the next update event UEV occurs. This function can only be used in the output compare and PWM modes. It does not work in forced mode.

OCREF_CLR_INPUT can be selected between the OCREF_CLR input and ETRF (ETR after the filter) by configuring the OCCS bit in the TIMx_SMCR register.

When ETRF is chosen, ETR must be configured as follows:

1. The external trigger prescaler should be kept off: bits ETPS[1:0] in the TIMx_SMCR register are cleared to 00.
2. The external clock mode 2 must be disabled: bit ECE in the TIM1_SMCR register is cleared to 0.
3. The external trigger polarity (ETP) and the external trigger filter (ETF) can be configured according to the application's needs.

Figure 99 shows the behavior of the OCxREF signal when the ETRF input becomes high, for both values of the OCxCE enable bit. In this example, the timer TIMx is programmed in PWM mode.

Figure 99. Clearing TIMx OCxREF

Note: In case of a PWM with a 100% duty cycle (if $CCR_x > ARR$), OCxREF is enabled again at the next counter overflow.

13.3.12 Encoder interface mode

To select Encoder Interface mode write SMS='001 in the TIMx_SMCR register if the counter is counting on TI2 edges only, SMS=010 if it is counting on TI1 edges only and SMS=011 if it is counting on both TI1 and TI2 edges.

Select the TI1 and TI2 polarity by programming the CC1P and CC2P bits in the TIMx_CCER register. CC1NP and CC2NP must be kept cleared. When needed, you can program the input filter as well.

The two inputs TI1 and TI2 are used to interface to an incremental encoder. Refer to [Table 52](#). The counter is clocked by each valid transition on TI1FP1 or TI2FP2 (TI1 and TI2 after input filter and polarity selection, TI1FP1=TI1 if not filtered and not inverted, TI2FP2=TI2 if not filtered and not inverted) assuming that it is enabled (CEN bit in TIMx_CR1 register written to '1'). The sequence of transitions of the two inputs is evaluated and generates count pulses as well as the direction signal. Depending on the sequence the counter counts up or down, the DIR bit in the TIMx_CR1 register is modified by hardware accordingly. The DIR bit is calculated at each transition on any input (TI1 or TI2), whatever the counter is counting on TI1 only, TI2 only or both TI1 and TI2.

Encoder interface mode acts simply as an external clock with direction selection. This means that the counter just counts continuously between 0 and the auto-reload value in the TIMx_ARR register (0 to ARR or ARR down to 0 depending on the direction). So you must configure TIMx_ARR before starting. In the same way, the capture, compare, prescaler, trigger output features continue to work as normal.

In this mode, the counter is modified automatically following the speed and the direction of the incremental encoder and its content, therefore, always represents the encoder's position. The count direction correspond to the rotation direction of the connected sensor. The table summarizes the possible combinations, assuming TI1 and TI2 don't switch at the same time.

Table 52. Counting direction versus encoder signals

Active edge	Level on opposite signal (TI1FP1 for TI2, TI2FP2 for TI1)	TI1FP1 signal		TI2FP2 signal	
		Rising	Falling	Rising	Falling
Counting on TI1 only	High	Down	Up	No Count	No Count
	Low	Up	Down	No Count	No Count
Counting on TI2 only	High	No Count	No Count	Up	Down
	Low	No Count	No Count	Down	Up
Counting on TI1 and TI2	High	Down	Up	Up	Down
	Low	Up	Down	Down	Up

An external incremental encoder can be connected directly to the MCU without external interface logic. However, comparators are normally be used to convert the encoder's differential outputs to digital signals. This greatly increases noise immunity. The third encoder output which indicate the mechanical zero position, may be connected to an external interrupt input and trigger a counter reset.

The [Figure 100](#) gives an example of counter operation, showing count signal generation and direction control. It also shows how input jitter is compensated where both edges are selected. This might occur if the sensor is positioned near to one of the switching points. For this example we assume that the configuration is the following:

- CC1S= 01 (TIMx_CCMR1 register, TI1FP1 mapped on TI1)
- CC2S= 01 (TIMx_CCMR2 register, TI2FP2 mapped on TI2)
- CC1P=0, CC1NP = '0 (TIMx_CCER register, TI1FP1 noninverted, TI1FP1=TI1)
- CC2P=0, CC2NP = '0 (TIMx_CCER register, TI2FP2 noninverted, TI2FP2=TI2)
- SMS= 011 (TIMx_SMCR register, both inputs are active on both rising and falling edges)
- CEN= 1 (TIMx_CR1 register, Counter is enabled)

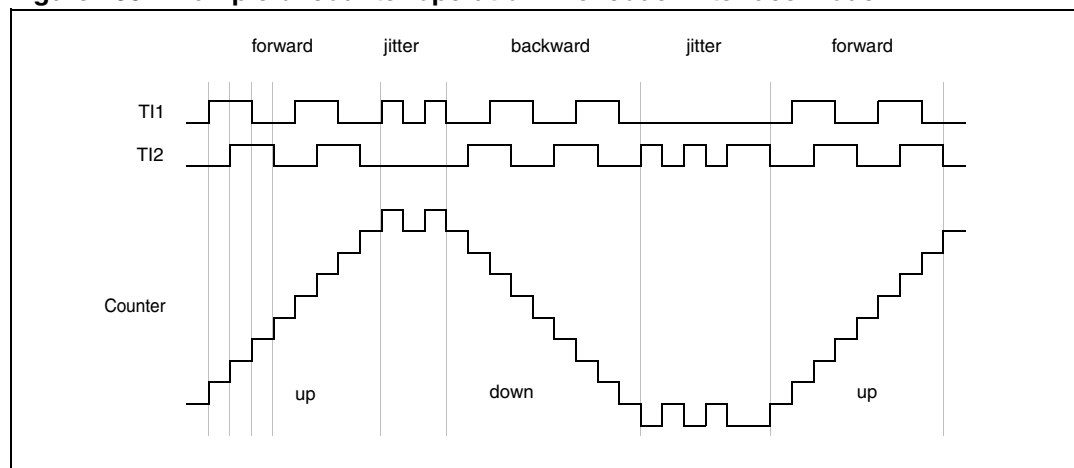
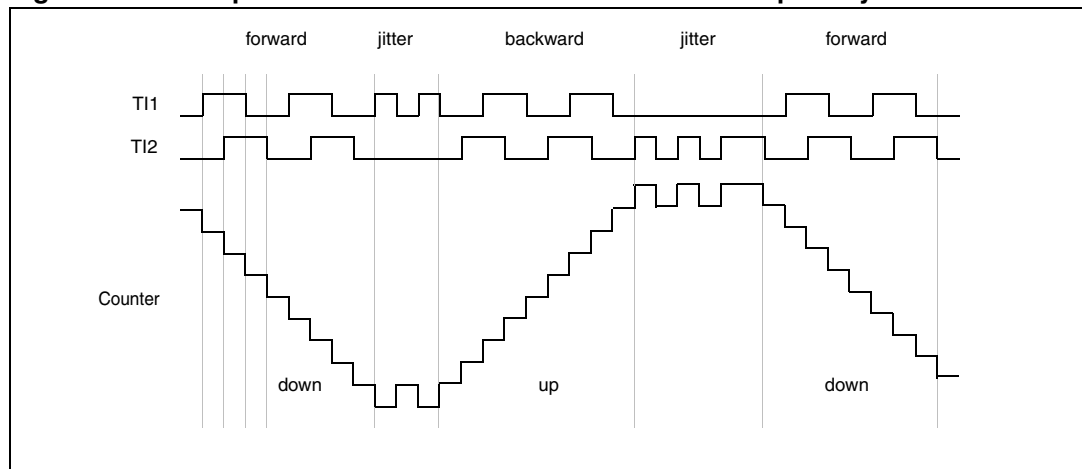
Figure 100. Example of counter operation in encoder interface mode.

Figure 101 gives an example of counter behavior when IC1FP1 polarity is inverted (same configuration as above except CC1P=1).

Figure 101. Example of encoder interface mode with IC1FP1 polarity inverted.



The timer, when configured in Encoder Interface mode provides information on the sensor's current position. You can obtain dynamic information (speed, acceleration, deceleration) by measuring the period between two encoder events using a second timer configured in capture mode. The output of the encoder which indicates the mechanical zero can be used for this purpose. Depending on the time between two events, the counter can also be read at regular times. You can do this by latching the counter value into a third input capture register if available (then the capture signal must be periodic and can be generated by another timer). when available, it is also possible to read its value through a DMA request generated by a Real-Time clock.

13.3.13 Timer input XOR function

The TI1S bit in the TIMx_CR2 register, allows the input filter of channel 1 to be connected to the output of a XOR gate, combining the three input pins TIMx_CH1 to TIMx_CH3.

The XOR output can be used with all the timer input functions such as trigger or input capture.

13.3.14 Timers and external trigger synchronization

The TIMx Timers can be synchronized with an external trigger in several modes: Reset mode, Gated mode and Trigger mode.

Slave mode: Reset mode

The counter and its prescaler can be reinitialized in response to an event on a trigger input. Moreover, if the URS bit from the TIMx_CR1 register is low, an update event UEV is generated. Then all the preloaded registers (TIMx_ARR, TIMx_CCRx) are updated.

In the following example, the upcounter is cleared in response to a rising edge on TI1 input:

- Configure the channel 1 to detect rising edges on TI1. Configure the input filter duration (in this example, we don't need any filter, so we keep IC1F=0000). The capture prescaler is not used for triggering, so you don't need to configure it. The CC1S bits select the input capture source only, CC1S = 01 in the TIMx_CCMR1 register. Write

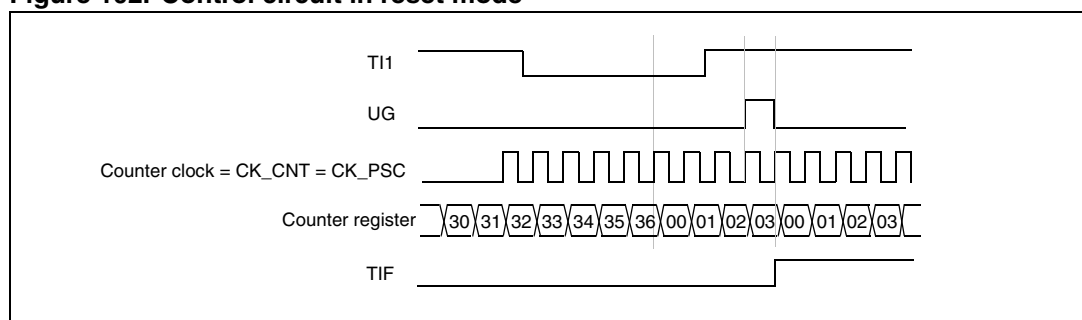
CC1P=0 and CC1NP=0 in TIMx_CCER register to validate the polarity (and detect rising edges only).

- Configure the timer in reset mode by writing SMS=100 in TIMx_SMCR register. Select TI1 as the input source by writing TS=101 in TIMx_SMCR register.
- Start the counter by writing CEN=1 in the TIMx_CR1 register.

The counter starts counting on the internal clock, then behaves normally until TI1 rising edge. When TI1 rises, the counter is cleared and restarts from 0. In the meantime, the trigger flag is set (TIF bit in the TIMx_SR register) and an interrupt request, or a DMA request can be sent if enabled (depending on the TIE and TDE bits in TIMx_DIER register).

The following figure shows this behavior when the auto-reload register TIMx_ARR=0x36. The delay between the rising edge on TI1 and the actual reset of the counter is due to the resynchronization circuit on TI1 input.

Figure 102. Control circuit in reset mode



Slave mode: Gated mode

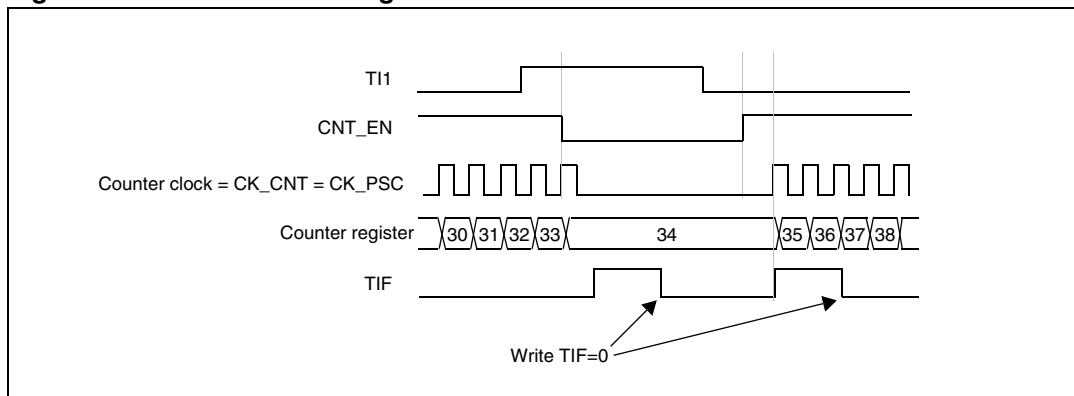
The counter can be enabled depending on the level of a selected input.

In the following example, the upcounter counts only when TI1 input is low:

- Configure the channel 1 to detect low levels on TI1. Configure the input filter duration (in this example, we don't need any filter, so we keep IC1F=0000). The capture prescaler is not used for triggering, so you don't need to configure it. The CC1S bits select the input capture source only, CC1S=01 in TIMx_CCMR1 register. Write CC1P=1 and CC1NP=0 in TIMx_CCER register to validate the polarity (and detect low level only).
- Configure the timer in gated mode by writing SMS=101 in TIMx_SMCR register. Select TI1 as the input source by writing TS=101 in TIMx_SMCR register.
- Enable the counter by writing CEN=1 in the TIMx_CR1 register (in gated mode, the counter doesn't start if CEN=0, whatever is the trigger input level).

The counter starts counting on the internal clock as long as TI1 is low and stops as soon as TI1 becomes high. The TIF flag in the TIMx_SR register is set both when the counter starts or stops.

The delay between the rising edge on TI1 and the actual stop of the counter is due to the resynchronization circuit on TI1 input.

Figure 103. Control circuit in gated mode

Note: The configuration “CCxP=CCxNP=1” (detection of both rising and falling edges) does not have any effect in gated mode because gated mode acts on a level and not on an edge.

Slave mode: Trigger mode

The counter can start in response to an event on a selected input.

In the following example, the upcounter starts in response to a rising edge on TI2 input:

- Configure the channel 2 to detect rising edges on TI2. Configure the input filter duration (in this example, we don't need any filter, so we keep IC2F=0000). The capture prescaler is not used for triggering, so you don't need to configure it. CC2S bits are selecting the input capture source only, CC2S=01 in TIMx_CCMR1 register. Write CC2P=1 and CC2NP=0 in TIMx_CCER register to validate the polarity (and detect low level only).
- Configure the timer in trigger mode by writing SMS=110 in TIMx_SMCR register. Select TI2 as the input source by writing TS=110 in TIMx_SMCR register.

When a rising edge occurs on TI2, the counter starts counting on the internal clock and the TIF flag is set.

The delay between the rising edge on TI2 and the actual start of the counter is due to the resynchronization circuit on TI2 input.

Figure 104. Control circuit in trigger mode

Slave mode: External Clock mode 2 + trigger mode

The external clock mode 2 can be used in addition to another slave mode (except external clock mode 1 and encoder mode). In this case, the ETR signal is used as external clock input, and another input can be selected as trigger input when operating in reset mode,

gated mode or trigger mode. It is recommended not to select ETR as TRGI through the TS bits of TIMx_SMCR register.

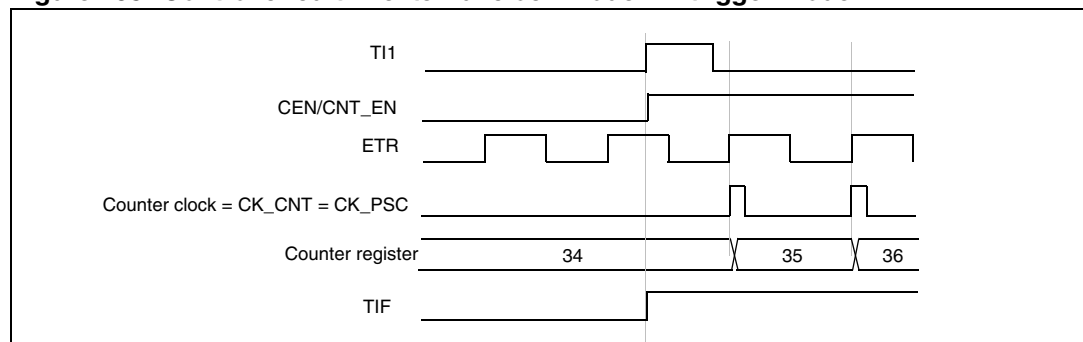
In the following example, the upcounter is incremented at each rising edge of the ETR signal as soon as a rising edge of TI1 occurs:

1. Configure the external trigger input circuit by programming the TIMx_SMCR register as follows:
 - ETF = 0000: no filter
 - ETPS=00: prescaler disabled
 - ETP=0: detection of rising edges on ETR and ECE=1 to enable the external clock mode 2.
2. Configure the channel 1 as follows, to detect rising edges on TI1:
 - IC1F=0000: no filter.
 - The capture prescaler is not used for triggering and does not need to be configured.
 - CC1S=01 in TIMx_CCMR1 register to select only the input capture source
 - CC1P=0 and CC1NP=0 in TIMx_CCER register to validate the polarity (and detect rising edge only).
3. Configure the timer in trigger mode by writing SMS=110 in TIMx_SMCR register. Select TI1 as the input source by writing TS=101 in TIMx_SMCR register.

A rising edge on TI1 enables the counter and sets the TIF flag. The counter then counts on ETR rising edges.

The delay between the rising edge of the ETR signal and the actual reset of the counter is due to the resynchronization circuit on ETRP input.

Figure 105. Control circuit in external clock mode 2 + trigger mode



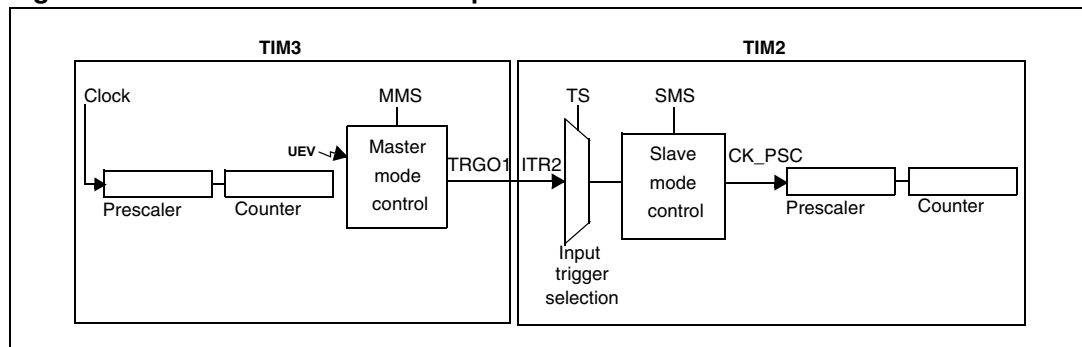
13.3.15 Timer synchronization

The TIMx timers are linked together internally for timer synchronization or chaining. When one Timer is configured in Master Mode, it can reset, start, stop or clock the counter of another Timer configured in Slave Mode.

Figure 106: Master/Slave timer example presents an overview of the trigger selection and the master mode selection blocks.

Using one timer as prescaler for another

Figure 106. Master/Slave timer example



For example, you can configure TIM3 to act as a prescaler for TIM2. Refer to [Figure 106](#). To do this:

- Configure TIM3 in master mode so that it outputs a periodic trigger signal on each update event UEV. If you write MMS=010 in the TIM3_CR2 register, a rising edge is output on TRGO1 each time an update event is generated.
- To connect the TRGO1 output of TIM3 to TIM2, TIM2 must be configured in slave mode using ITR2 as internal trigger. You select this through the TS bits in the TIM2_SMCR register (writing TS=000).
- Then you put the slave mode controller in external clock mode 1 (write SMS=111 in the TIM2_SMCR register). This causes TIM2 to be clocked by the rising edge of the periodic TIM3 trigger signal (which correspond to the TIM3 counter overflow).
- Finally both timers must be enabled by setting their respective CEN bits (TIMx_CR1 register).

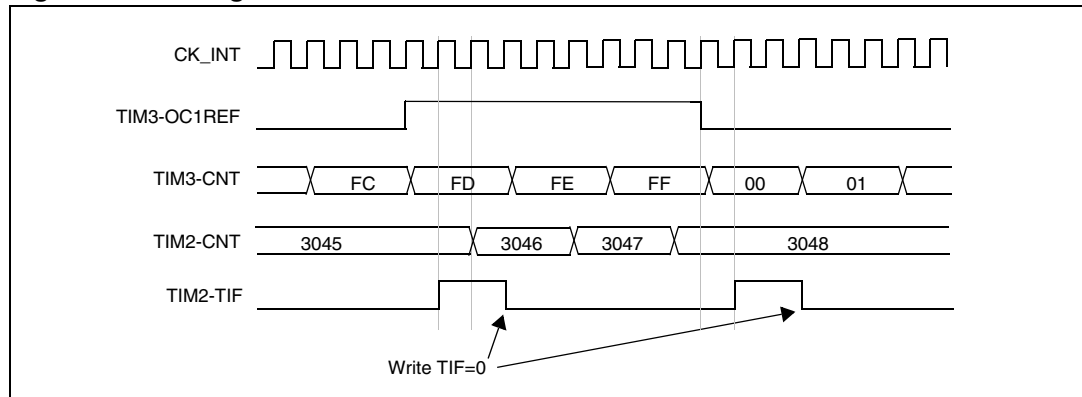
Note: If OCx is selected on TIM3 as the trigger output (MMS=1xx), its rising edge is used to clock the counter of TIM2.

Using one timer to enable another timer

In this example, we control the enable of TIM2 with the output compare 1 of Timer 3. Refer to [Figure 106](#) for connections. TIM2 counts on the divided internal clock only when OC1REF of TIM3 is high. Both counter clock frequencies are divided by 3 by the prescaler compared to CK_INT ($f_{CK_CNT} = f_{CK_INT}/3$).

- Configure TIM3 master mode to send its Output Compare 1 Reference (OC1REF) signal as trigger output (MMS=100 in the TIM3_CR2 register).
- Configure the TIM3 OC1REF waveform (TIM3_CCMR1 register).
- Configure TIM2 to get the input trigger from TIM3 (TS=000 in the TIM2_SMCR register).
- Configure TIM2 in gated mode (SMS=101 in TIM2_SMCR register).
- Enable TIM2 by writing '1 in the CEN bit (TIM2_CR1 register).
- Start TIM3 by writing '1 in the CEN bit (TIM3_CR1 register).

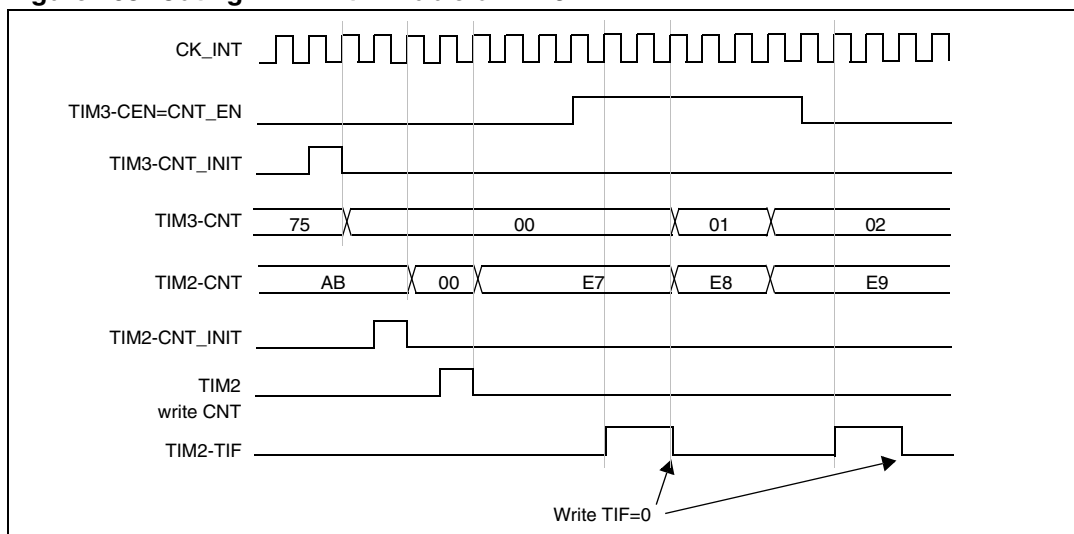
Note: The counter 2 clock is not synchronized with counter 1, this mode only affects the TIM2 counter enable signal.

Figure 107. Gating TIM2 with OC1REF of TIM3

In the example in [Figure 107](#), the TIM2 counter and prescaler are not initialized before being started. So they start counting from their current value. It is possible to start from a given value by resetting both timers before starting TIM3. You can then write any value you want in the timer counters. The timers can easily be reset by software using the UG bit in the TIMx_EGR registers.

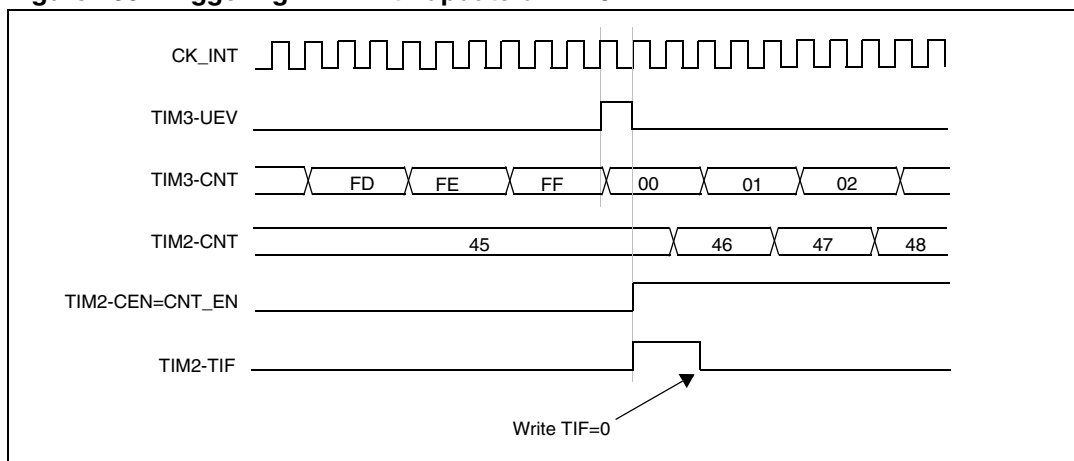
In the next example, we synchronize TIM3 and TIM2. TIM3 is the master and starts from 0. TIM2 is the slave and starts from 0xE7. The prescaler ratio is the same for both timers. TIM2 stops when TIM3 is disabled by writing '0 to the CEN bit in the TIM3_CR1 register:

- Configure TIM3 master mode to send its Output Compare 1 Reference (OC1REF) signal as trigger output (MMS=100 in the TIM3_CR2 register).
- Configure the TIM3 OC1REF waveform (TIM3_CCMR1 register).
- Configure TIM2 to get the input trigger from TIM3 (TS=000 in the TIM2_SMCR register).
- Configure TIM2 in gated mode (SMS=101 in TIM2_SMCR register).
- Reset TIM3 by writing '1 in UG bit (TIM3_EGR register).
- Reset TIM2 by writing '1 in UG bit (TIM2_EGR register).
- Initialize TIM2 to 0xE7 by writing '0xE7' in the TIM2 counter (TIM2_CNT).
- Enable TIM2 by writing '1 in the CEN bit (TIM2_CR1 register).
- Start TIM3 by writing '1 in the CEN bit (TIM3_CR1 register).
- Stop TIM3 by writing '0 in the CEN bit (TIM3_CR1 register).

Figure 108. Gating TIM2 with Enable of TIM3**Using one timer to start another timer**

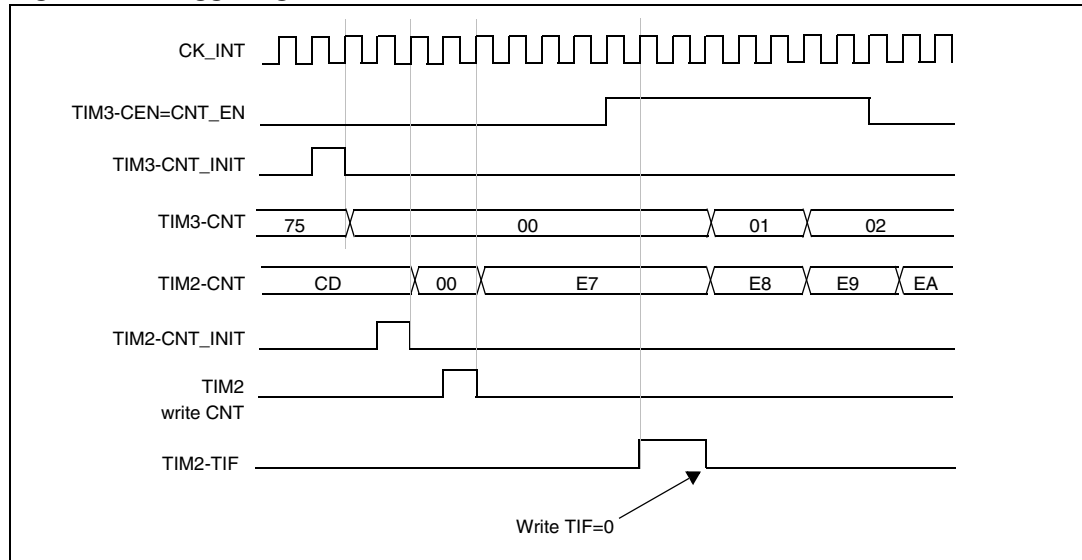
In this example, we set the enable of Timer 2 with the update event of Timer 3. Refer to [Figure 106](#) for connections. Timer 2 starts counting from its current value (which can be non-zero) on the divided internal clock as soon as the update event is generated by Timer 1. When Timer 2 receives the trigger signal its CEN bit is automatically set and the counter counts until we write '0 to the CEN bit in the TIM2_CR1 register. Both counter clock frequencies are divided by 3 by the prescaler compared to CK_INT ($f_{CK_CNT} = f_{CK_INT}/3$).

- Configure TIM3 master mode to send its Update Event (UEV) as trigger output (MMS=010 in the TIM3_CR2 register).
- Configure the TIM3 period (TIM3_ARR registers).
- Configure TIM2 to get the input trigger from TIM3 (TS=000 in the TIM2_SMCR register).
- Configure TIM2 in trigger mode (SMS=110 in TIM2_SMCR register).
- Start TIM3 by writing '1 in the CEN bit (TIM3_CR1 register).

Figure 109. Triggering TIM2 with update of TIM3

As in the previous example, you can initialize both counters before starting counting. [Figure 110](#) shows the behavior with the same configuration as in [Figure 109](#) but in trigger mode instead of gated mode (SMS=110 in the TIM2_SMCR register).

Figure 110. Triggering TIM2 with Enable of TIM3



Using one timer as prescaler for another timer

For example, you can configure TIM3 to act as a prescaler for TIM2. Refer to [Figure 106](#) for connections. To do this:

- Configure TIM3 master mode to send its Update Event (UEV) as trigger output (MMS=010 in the TIM3_CR2 register). then it outputs a periodic signal on each counter overflow.
- Configure the TIM3 period (TIM3_ARR registers).
- Configure TIM2 to get the input trigger from TIM3 (TS=000 in the TIM2_SMCR register).
- Configure TIM2 in external clock mode 1 (SMS=111 in TIM2_SMCR register).
- Start TIM2 by writing '1 in the CEN bit (TIM2_CR1 register).
- Start TIM3 by writing '1 in the CEN bit (TIM3_CR1 register).

Starting 2 timers synchronously in response to an external trigger

In this example, we set the enable of TIM3 when its TI1 input rises, and the enable of TIM2 with the enable of TIM3. Refer to [Figure 106](#) for connections. To ensure the counters are aligned, TIM3 must be configured in Master/Slave mode (slave with respect to TI1, master with respect to TIM2):

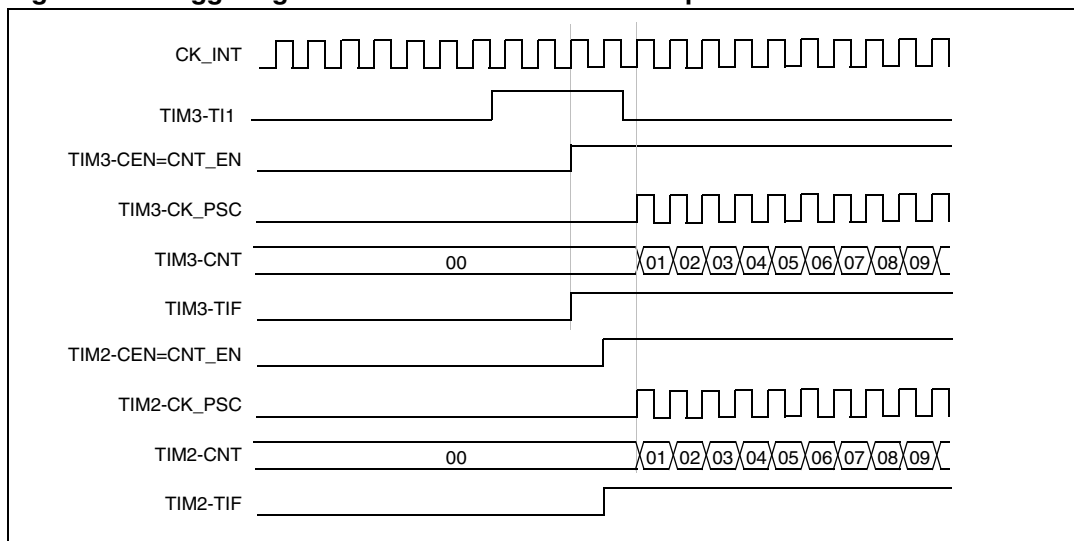
- Configure TIM3 master mode to send its Enable as trigger output (MMS=001 in the TIM3_CR2 register).
- Configure TIM3 slave mode to get the input trigger from TI1 (TS=100 in the TIM3_SMCR register).
- Configure TIM3 in trigger mode (SMS=110 in the TIM3_SMCR register).
- Configure the TIM3 in Master/Slave mode by writing MSM=1 (TIM3_SMCR register).

- Configure TIM2 to get the input trigger from TIM3 (TS=000 in the TIM2_SMCR register).
- Configure TIM2 in trigger mode (SMS=110 in the TIM2_SMCR register).

When a rising edge occurs on TI1 (TIM3), both counters starts counting synchronously on the internal clock and both TIF flags are set.

Note: *In this example both timers are initialized before starting (by setting their respective UG bits). Both counters starts from 0, but you can easily insert an offset between them by writing any of the counter registers (TIMx_CNT). You can see that the master/slave mode insert a delay between CNT_EN and CK_PSC on TIM3.*

Figure 111. Triggering TIM3 and TIM2 with TIM3 TI1 input



13.3.16 Debug mode

When the microcontroller enters debug mode (Cortex-M3 core - halted), the TIMx counter either continues to work normally or stops, depending on DBG_TIMx_STOP configuration bit in DBGMCU module. For more details, refer to [Section 24.16.2: Debug support for timers, watchdog and I2C](#).

13.4 TIMx registers

Refer to for a list of abbreviations used in register descriptions.

The peripheral registers can be accessed by half-words (16-bit) or words (32-bit).

13.4.1 TIMx control register 1 (TIMx_CR1)

Address offset: 0x00

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CKD[1:0]		ARPE	CMS		DIR	OPM	URS	UDIS	CEN
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:10 Reserved, always read as 0

Bits 9:8 **CKD**: Clock division

This bit-field indicates the division ratio between the timer clock (CK_INT) frequency and sampling clock used by the digital filters (ETR, Tlx),

00: $t_{DTS} = t_{CK_INT}$

01: $t_{DTS} = 2 \times t_{CK_INT}$

10: $t_{DTS} = 4 \times t_{CK_INT}$

11: Reserved

Bit 7 **ARPE**: Auto-reload preload enable

0: TIMx_ARR register is not buffered

1: TIMx_ARR register is buffered

Bits 6:5 **CMS**: Center-aligned mode selection

00: Edge-aligned mode. The counter counts up or down depending on the direction bit (DIR).

01: Center-aligned mode 1. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIMx_CCMRx register) are set only when the counter is counting down.

10: Center-aligned mode 2. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIMx_CCMRx register) are set only when the counter is counting up.

11: Center-aligned mode 3. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIMx_CCMRx register) are set both when the counter is counting up or down.

Note: It is not allowed to switch from edge-aligned mode to center-aligned mode as long as the counter is enabled (CEN=1)

Bit 4 **DIR**: Direction

0: Counter used as upcounter

1: Counter used as downcounter

Note: This bit is read only when the timer is configured in Center-aligned mode or Encoder mode.

Bit 3 **OPM**: One-pulse mode

0: Counter is not stopped at update event

1: Counter stops counting at the next update event (clearing the bit CEN)

Bit 2 URS: Update request source

This bit is set and cleared by software to select the UEV event sources.

0: Any of the following events generate an update interrupt or DMA request if enabled.

These events can be:

- Counter overflow/underflow
- Setting the UG bit
- Update generation through the slave mode controller

1: Only counter overflow/underflow generates an update interrupt or DMA request if enabled.

Bit 1 UDIS: Update disable

This bit is set and cleared by software to enable/disable UEV event generation.

0: UEV enabled. The Update (UEV) event is generated by one of the following events:

- Counter overflow/underflow
- Setting the UG bit
- Update generation through the slave mode controller

Buffered registers are then loaded with their preload values.

1: UEV disabled. The Update event is not generated, shadow registers keep their value (ARR, PSC, CCRx). However the counter and the prescaler are reinitialized if the UG bit is set or if a hardware reset is received from the slave mode controller.

Bit 0 CEN: Counter enable

0: Counter disabled

1: Counter enabled

Note: External clock, gated mode and encoder mode can work only if the CEN bit has been previously set by software. However trigger mode can set the CEN bit automatically by hardware.

CEN is cleared automatically in one-pulse mode, when an update event occurs.

13.4.2 TIMx control register 2 (TIMx_CR2)

Address offset: 0x04

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								TI1S	MMS[2:0]				CCDS	Reserved	
								rw	rw	rw	rw	rw			

Bits 15:8 Reserved, always read as 0.

Bit 7 **TI1S**: TI1 selection

0: The TIMx_CH1 pin is connected to TI1 input

1: The TIMx_CH1, CH2 and CH3 pins are connected to the TI1 input (XOR combination)

Bits 6:4 **MMS**: Master mode selection

These bits allow to select the information to be sent in master mode to slave timers for synchronization (TRGO). The combination is as follows:

000: **Reset** - the UG bit from the TIMx_EGR register is used as trigger output (TRGO). If the reset is generated by the trigger input (slave mode controller configured in reset mode) then the signal on TRGO is delayed compared to the actual reset.

001: **Enable** - the Counter enable signal, CNT_EN, is used as trigger output (TRGO). It is useful to start several timers at the same time or to control a window in which a slave timer is enabled. The Counter Enable signal is generated by a logic OR between CEN control bit and the trigger input when configured in gated mode.

When the Counter Enable signal is controlled by the trigger input, there is a delay on TRGO, except if the master/slave mode is selected (see the MSM bit description in TIMx_SMCR register).

010: **Update** - The update event is selected as trigger output (TRGO). For instance a master timer can then be used as a prescaler for a slave timer.

011: **Compare Pulse** - The trigger output send a positive pulse when the CC1IF flag is to be set (even if it was already high), as soon as a capture or a compare match occurred. (TRGO)

100: **Compare** - OC1REF signal is used as trigger output (TRGO)

101: **Compare** - OC2REF signal is used as trigger output (TRGO)

110: **Compare** - OC3REF signal is used as trigger output (TRGO)

111: **Compare** - OC4REF signal is used as trigger output (TRGO)

Bit 3 **CCDS**: Capture/compare DMA selection

0: CCx DMA request sent when CCx event occurs

1: CCx DMA requests sent when update event occurs

Bits 2:0 Reserved, always read as 0

13.4.3 TIMx slave mode control register (TIMx_SMCR)

Address offset: 0x08

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETP	ECE	ETPS[1:0]		ETF[3:0]				MSM	TS[2:0]			OCCS	SMS[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 15 **ETP**: External trigger polarity

This bit selects whether ETR or $\overline{\text{ETR}}$ is used for trigger operations

0: ETR is non-inverted, active at high level or rising edge

1: ETR is inverted, active at low level or falling edge

Bit 14 **ECE**: External clock enable

This bit enables External clock mode 2

0: External clock mode 2 disabled

1: External clock mode 2 enabled. The counter is clocked by any active edge on the ETRF signal.

1: Setting the ECE bit has the same effect as selecting external clock mode 1 with TRGI connected to ETRF (SMS=111 and TS=111).

2: It is possible to simultaneously use external clock mode 2 with the following slave modes: reset mode, gated mode and trigger mode. Nevertheless, TRGI must not be connected to ETRF in this case (TS bits must not be 111).

3: If external clock mode 1 and external clock mode 2 are enabled at the same time, the external clock input is ETRF.

Bits 13:12 **ETPS**: External trigger prescaler

External trigger signal ETRP frequency must be at most 1/4 of CK_INT frequency. A prescaler can be enabled to reduce ETRP frequency. It is useful when inputting fast external clocks.

00: Prescaler OFF

01: ETRP frequency divided by 2

10: ETRP frequency divided by 4

11: ETRP frequency divided by 8

Bits 11:8 **ETF[3:0]**: External trigger filter

This bit-field then defines the frequency used to sample ETRP signal and the length of the digital filter applied to ETRP. The digital filter is made of an event counter in which N events are needed to validate a transition on the output:

0000: No filter, sampling is done at f_{DTS}

0001: $f_{\text{SAMPLING}} = f_{\text{CK_INT}}$, N=2

0010: $f_{\text{SAMPLING}} = f_{\text{CK_INT}}$, N=4

0011: $f_{\text{SAMPLING}} = f_{\text{CK_INT}}$, N=8

0100: $f_{\text{SAMPLING}} = f_{\text{DTS}}/2$, N=6

0101: $f_{\text{SAMPLING}} = f_{\text{DTS}}/2$, N=8

0110: $f_{\text{SAMPLING}} = f_{\text{DTS}}/4$, N=6

0111: $f_{\text{SAMPLING}} = f_{\text{DTS}}/4$, N=8

1000: $f_{\text{SAMPLING}} = f_{\text{DTS}}/8$, N=6

1001: $f_{\text{SAMPLING}} = f_{\text{DTS}}/8$, N=8

1010: $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$, N=5

1011: $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$, N=6

1100: $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$, N=8

1101: $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$, N=5

1110: $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$, N=6

1111: $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$, N=8

Bit 7 **MSM**: Master/Slave mode

0: No action

1: The effect of an event on the trigger input (TRGI) is delayed to allow a perfect synchronization between the current timer and its slaves (through TRGO). It is useful if we want to synchronize several timers on a single external event.

Bits 6:4 **TS**: Trigger selection

This bit-field selects the trigger input to be used to synchronize the counter.

000: Internal Trigger 0 (ITR0).

001: Internal Trigger 1 (ITR1).

010: Internal Trigger 2 (ITR2).

011: Internal Trigger 3 (ITR3).

100: TI1 Edge Detector (TI1F_ED)

101: Filtered Timer Input 1 (TI1FP1)

110: Filtered Timer Input 2 (TI2FP2)

111: External Trigger input (ETRF)

See [Table 53: TIMx internal trigger connection on page 311](#) for more details on ITRx meaning for each Timer.*Note: These bits must be changed only when they are not used (e.g. when SMS=000) to avoid wrong edge detections at the transition.*Bit 3 **OCSS**: OCREF clear selection

This bit is used to select the OCREF clear source

0: OCREF_CLR_INT is connected to the OCREF_CLR input

1: OCREF_CLR_INT is connected to ETRF

Bits 2:0 **SMS**: Slave mode selection

When external signals are selected the active edge of the trigger signal (TRGI) is linked to the polarity selected on the external input (see Input Control register and Control Register description).

000: Slave mode disabled - if CEN = '1 then the prescaler is clocked directly by the internal clock.

001: Encoder mode 1 - Counter counts up/down on TI2FP2 edge depending on TI1FP1 level.

010: Encoder mode 2 - Counter counts up/down on TI1FP1 edge depending on TI2FP2 level.

011: Encoder mode 3 - Counter counts up/down on both TI1FP1 and TI2FP2 edges depending on the level of the other input.

100: Reset Mode - Rising edge of the selected trigger input (TRGI) reinitializes the counter and generates an update of the registers.

101: Gated Mode - The counter clock is enabled when the trigger input (TRGI) is high. The counter stops (but is not reset) as soon as the trigger becomes low. Both start and stop of the counter are controlled.

110: Trigger Mode - The counter starts at a rising edge of the trigger TRGI (but it is not reset). Only the start of the counter is controlled.

111: External Clock Mode 1 - Rising edges of the selected trigger (TRGI) clock the counter.

Note: The gated mode must not be used if TI1F_ED is selected as the trigger input (TS=100). Indeed, TI1F_ED outputs 1 pulse for each transition on TI1F, whereas the gated mode checks the level of the trigger signal.

Table 53. TIMx internal trigger connection

Slave TIM	ITR0 (TS = 000)	ITR1 (TS = 001)	ITR2 (TS = 010)	ITR3 (TS = 011)
TIM2	TIM9	TIM10	TIM3	TIM4
TIM3	TIM9	TIM2	TIM11	TIM4
TIM4	TIM10	TIM2	TIM3	TIM9

13.4.4 TIMx DMA/Interrupt enable register (TIMx_DIER)

Address offset: 0x0C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	TDE	Res.	CC4DE	CC3DE	CC2DE	CC1DE	UDE	Res.	TIE	Res.	CC4IE	CC3IE	CC2IE	CC1IE	UIE
	rw		rw	rw	rw	rw	rw		rw		rw	rw	rw	rw	rw

Bit 15 Reserved, always read as 0.

Bit 14 **TDE**: Trigger DMA request enable
 0: Trigger DMA request disabled
 1: Trigger DMA request enabled

Bit 13 Reserved, always read as 0

Bit 12 **CC4DE**: Capture/Compare 4 DMA request enable
 0: CC4 DMA request disabled
 1: CC4 DMA request enabled

Bit 11 **CC3DE**: Capture/Compare 3 DMA request enable
 0: CC3 DMA request disabled
 1: CC3 DMA request enabled

Bit 10 **CC2DE**: Capture/Compare 2 DMA request enable
 0: CC2 DMA request disabled
 1: CC2 DMA request enabled

Bit 9 **CC1DE**: Capture/Compare 1 DMA request enable
 0: CC1 DMA request disabled
 1: CC1 DMA request enabled

Bit 8 **UDE**: Update DMA request enable
 0: Update DMA request disabled
 1: Update DMA request enabled

Bit 7 Reserved, always read as 0.

Bit 6 **TIE**: Trigger interrupt enable
 0: Trigger interrupt disabled
 1: Trigger interrupt enabled

Bit 5 Reserved, always read as 0.

Bit 4 **CC4IE**: Capture/Compare 4 interrupt enable
 0: CC4 interrupt disabled
 1: CC4 interrupt enabled

Bit 3 **CC3IE**: Capture/Compare 3 interrupt enable
 0: CC3 interrupt disabled
 1: CC3 interrupt enabled

Bit 2 **CC2IE**: Capture/Compare 2 interrupt enable
 0: CC2 interrupt disabled
 1: CC2 interrupt enabled

Bit 1 **CC1IE**: Capture/Compare 1 interrupt enable
 0: CC1 interrupt disabled
 1: CC1 interrupt enabled

Bit 0 **UIE**: Update interrupt enable
 0: Update interrupt disabled
 1: Update interrupt enabled

13.4.5 TIMx status register (TIMx_SR)

Address offset: 0x10

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CC4OF	CC3OF	CC2OF	CC1OF	Reserved		TIF	Res	CC4IF	CC3IF	CC2IF	CC1IF	UIF
			rc_w0	rc_w0	rc_w0	rc_w0			rc_w0		rc_w0	rc_w0	rc_w0	rc_w0	

Bit 15:13 Reserved, always read as 0.

Bit 12 **CC4OF**: Capture/Compare 4 overcapture flag
 refer to CC1OF description

Bit 11 **CC3OF**: Capture/Compare 3 overcapture flag
 refer to CC1OF description

Bit 10 **CC2OF**: Capture/compare 2 overcapture flag
 refer to CC1OF description

Bit 9 **CC1OF**: Capture/Compare 1 overcapture flag
 This flag is set by hardware only when the corresponding channel is configured in input capture mode. It is cleared by software by writing it to '0'.
 0: No overcapture has been detected
 1: The counter value has been captured in TIMx_CCR1 register while CC1IF flag was already set

Bits 8:7 Reserved, always read as 0.

Bit 6 **TIF**: Trigger interrupt flag
 This flag is set by hardware on trigger event (active edge detected on TRGI input when the slave mode controller is enabled in all modes but gated mode. It is set when the counter starts or stops when gated mode is selected. It is cleared by software.
 0: No trigger event occurred
 1: Trigger interrupt pending

Bit 5 Reserved, always read as 0

Bit 4 **CC4IF**: Capture/Compare 4 interrupt flag
 refer to CC1IF description

Bit 3 **CC3IF**: Capture/Compare 3 interrupt flag
refer to CC1IF description

Bit 2 **CC2IF**: Capture/Compare 2 interrupt flag
refer to CC1IF description

Bit 1 **CC1IF**: Capture/compare 1 interrupt flag

If channel CC1 is configured as output:

This flag is set by hardware when the counter matches the compare value, with some exception in center-aligned mode (refer to the CMS bits in the TIMx_CR1 register description). It is cleared by software.

0: No match

1: The content of the counter TIMx_CNT matches the content of the TIMx_CCR1 register.

When the contents of TIMx_CCR1 are greater than the contents of TIMx_ARR, the CC1IF bit goes high on the counter overflow (in upcounting and up/down-counting modes) or underflow (in downcounting mode)

If channel CC1 is configured as input:

This bit is set by hardware on a capture. It is cleared by software or by reading the TIMx_CCR1 register.

0: No input capture occurred

1: The counter value has been captured in TIMx_CCR1 register (An edge has been detected on IC1 which matches the selected polarity)

Bit 0 **UIF**: Update interrupt flag

- This bit is set by hardware on an update event. It is cleared by software.
0: No update occurred
1: Update interrupt pending. This bit is set by hardware when the registers are updated:
- At overflow or underflow (for TIM2 to TIM4) and if UDIS=0 in the TIMx_CR1 register.
- When CNT is reinitialized by software using the UG bit in TIMx_EGR register, if URS=0 and UDIS=0 in the TIMx_CR1 register.
When CNT is reinitialized by a trigger event (refer to the synchro control register description), if URS=0 and UDIS=0 in the TIMx_CR1 register.

13.4.6 TIMx event generation register (TIMx_EGR)

Address offset: 0x14

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved									TG	Res.	CC4G	CC3G	CC2G	CC1G	UG
									w		w	w	w	w	w

Bits 15:7 Reserved, always read as 0.

Bit 6 **TG**: Trigger generation

This bit is set by software in order to generate an event, it is automatically cleared by hardware.

0: No action

1: The TIF flag is set in TIMx_SR register. Related interrupt or DMA transfer can occur if enabled.

Bit 5 Reserved, always read as 0.

Bit 4 **CC4G**: Capture/compare 4 generation

refer to CC1G description

Bit 3 **CC3G**: Capture/compare 3 generation

refer to CC1G description

Bit 2 **CC2G**: Capture/compare 2 generation

refer to CC1G description

Bit 1 **CC1G**: Capture/compare 1 generation

This bit is set by software in order to generate an event, it is automatically cleared by hardware.

0: No action

1: A capture/compare event is generated on channel 1:

If channel CC1 is configured as output:

CC1IF flag is set, Corresponding interrupt or DMA request is sent if enabled.

If channel CC1 is configured as input:

The current value of the counter is captured in TIMx_CCR1 register. The CC1IF flag is set, the corresponding interrupt or DMA request is sent if enabled. The CC1OF flag is set if the CC1IF flag was already high.

Bit 0 **UG**: Update generation

This bit can be set by software, it is automatically cleared by hardware.

0: No action

1: Re-initialize the counter and generates an update of the registers. Note that the prescaler counter is cleared too (anyway the prescaler ratio is not affected). The counter is cleared if the center-aligned mode is selected or if DIR=0 (upcounting), else it takes the auto-reload value (TIMx_ARR) if DIR=1 (downcounting).

13.4.7 TIMx capture/compare mode register 1 (TIMx_CCMR1)

Address offset: 0x18

Reset value: 0x0000

The channels can be used in input (capture mode) or in output (compare mode). The direction of a channel is defined by configuring the corresponding CCxS bits. All the other bits of this register have a different function in input and in output mode. For a given bit, OCxx describes its function when the channel is configured in output, ICxx describes its function when the channel is configured in input. So you must take care that the same bit can have a different meaning for the input stage and for the output stage.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC2CE	OC2M[2:0]			OC2PE	OC2FE	CC2S[1:0]		OC1CE	OC1M[2:0]			OC1PE	OC1FE	CC1S[1:0]	
IC2F[3:0]				IC2PSC[1:0]				IC1F[3:0]			IC1PSC[1:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Output compare mode

Bit 15 **OC2CE**: Output compare 2 clear enable

Bits 14:12 **OC2M[2:0]**: Output compare 2 mode

Bit 11 **OC2PE**: Output compare 2 preload enable

Bit 10 **OC2FE**: Output compare 2 fast enable

Bits 9:8 **CC2S[1:0]**: Capture/Compare 2 selection

This bit-field defines the direction of the channel (input/output) as well as the used input.

00: CC2 channel is configured as output

01: CC2 channel is configured as input, IC2 is mapped on TI2

10: CC2 channel is configured as input, IC2 is mapped on TI1

11: CC2 channel is configured as input, IC2 is mapped on TRC. This mode is working only if an internal trigger input is selected through the TS bit (TIMx_SMCR register)

Note: CC2S bits are writable only when the channel is OFF (CC2E = 0 in TIMx_CCER).

Bit 7 **OC1CE**: Output compare 1 clear enable

OC1CE: Output Compare 1 Clear Enable

0: OC1Ref is not affected by the ETRF input

1: OC1Ref is cleared as soon as a High level is detected on ETRF input

Bits 6:4 **OC1M**: Output compare 1 mode

These bits define the behavior of the output reference signal OC1REF from which OC1 and OC1N are derived. OC1REF is active high whereas OC1 and OC1N active level depends on CC1P and CC1NP bits.

000: Frozen - The comparison between the output compare register TIMx_CCR1 and the counter TIMx_CNT has no effect on the outputs.(this mode is used to generate a timing base).

001: Set channel 1 to active level on match. OC1REF signal is forced high when the counter TIMx_CNT matches the capture/compare register 1 (TIMx_CCR1).

010: Set channel 1 to inactive level on match. OC1REF signal is forced low when the counter TIMx_CNT matches the capture/compare register 1 (TIMx_CCR1).

011: Toggle - OC1REF toggles when TIMx_CNT=TIMx_CCR1.

100: Force inactive level - OC1REF is forced low.

101: Force active level - OC1REF is forced high.

110: PWM mode 1 - In upcounting, channel 1 is active as long as TIMx_CNT<TIMx_CCR1 else inactive. In downcounting, channel 1 is inactive (OC1REF=0) as long as TIMx_CNT>TIMx_CCR1 else active (OC1REF=1).

111: PWM mode 2 - In upcounting, channel 1 is inactive as long as TIMx_CNT<TIMx_CCR1 else active. In downcounting, channel 1 is active as long as TIMx_CNT>TIMx_CCR1 else inactive.

Note: 1: These bits can not be modified as long as LOCK level 3 has been programmed (LOCK bits in TIMx_BDTR register) and CC1S=00 (the channel is configured in output).

2: In PWM mode 1 or 2, the OCREF level changes only when the result of the comparison changes or when the output compare mode switches from "frozen" mode to "PWM" mode.

Bit 3 **OC1PE**: Output compare 1 preload enable

0: Preload register on TIMx_CCR1 disabled. TIMx_CCR1 can be written at anytime, the new value is taken in account immediately.

1: Preload register on TIMx_CCR1 enabled. Read/Write operations access the preload register. TIMx_CCR1 preload value is loaded in the active register at each update event.

Note: 1: These bits can not be modified as long as LOCK level 3 has been programmed (LOCK bits in TIMx_BDTR register) and CC1S=00 (the channel is configured in output).

2: The PWM mode can be used without validating the preload register only in one-pulse mode (OPM bit set in TIMx_CR1 register). Else the behavior is not guaranteed.

Bit 2 **OC1FE**: Output compare 1 fast enable

This bit is used to accelerate the effect of an event on the trigger in input on the CC output.

0: CC1 behaves normally depending on counter and CCR1 values even when the trigger is ON. The minimum delay to activate CC1 output when an edge occurs on the trigger input is 5 clock cycles.

1: An active edge on the trigger input acts like a compare match on CC1 output. Then, OC is set to the compare level independently from the result of the comparison. Delay to sample the trigger input and to activate CC1 output is reduced to 3 clock cycles. OC1FE acts only if the channel is configured in PWM1 or PWM2 mode.

Bits 1:0 **CC1S**: Capture/Compare 1 selection

This bit-field defines the direction of the channel (input/output) as well as the used input.

00: CC1 channel is configured as output.

01: CC1 channel is configured as input, IC1 is mapped on TI1.

10: CC1 channel is configured as input, IC1 is mapped on TI2.

11: CC1 channel is configured as input, IC1 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIMx_SMCR register)

Note: CC1S bits are writable only when the channel is OFF (CC1E = 0 in TIMx_CCER).

Input capture mode

Bits 15:12 **IC2F**: Input capture 2 filter

Bits 11:10 **IC2PSC[1:0]**: Input capture 2 prescaler

Bits 9:8 **CC2S**: Capture/compare 2 selection

This bit-field defines the direction of the channel (input/output) as well as the used input.

00: CC2 channel is configured as output.

01: CC2 channel is configured as input, IC2 is mapped on TI2.

10: CC2 channel is configured as input, IC2 is mapped on TI1.

11: CC2 channel is configured as input, IC2 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIMx_SMCR register)

Note: CC2S bits are writable only when the channel is OFF (CC2E = 0 in TIMx_CCER).

Bits 7:4 **IC1F**: Input capture 1 filter

This bit-field defines the frequency used to sample TI1 input and the length of the digital filter applied to TI1. The digital filter is made of an event counter in which N events are needed to validate a transition on the output:

0000: No filter, sampling is done at f_{DTS}

0001: $f_{SAMPLING}=f_{CK_INT}$, N=2

0010: $f_{SAMPLING}=f_{CK_INT}$, N=4

0011: $f_{SAMPLING}=f_{CK_INT}$, N=8

0100: $f_{SAMPLING}=f_{DTS}/2$, N=6

0101: $f_{SAMPLING}=f_{DTS}/2$, N=8

0110: $f_{SAMPLING}=f_{DTS}/4$, N=6

0111: $f_{SAMPLING}=f_{DTS}/4$, N=8

1000: $f_{SAMPLING}=f_{DTS}/8$, N=6

1001: $f_{SAMPLING}=f_{DTS}/8$, N=8

1010: $f_{SAMPLING}=f_{DTS}/16$, N=5

1011: $f_{SAMPLING}=f_{DTS}/16$, N=6

1100: $f_{SAMPLING}=f_{DTS}/16$, N=8

1101: $f_{SAMPLING}=f_{DTS}/32$, N=5

1110: $f_{SAMPLING}=f_{DTS}/32$, N=6

1111: $f_{SAMPLING}=f_{DTS}/32$, N=8

Note: In current silicon revision, f_{DTS} is replaced in the formula by CK_INT when ICx F[3:0]= 1, 2 or 3.

Bits 3:2 **IC1PSC**: Input capture 1 prescaler

This bit-field defines the ratio of the prescaler acting on CC1 input (IC1).

The prescaler is reset as soon as CC1E=0 (TIMx_CCER register).

00: no prescaler, capture is done each time an edge is detected on the capture input

01: capture is done once every 2 events

10: capture is done once every 4 events

11: capture is done once every 8 events

Bits 1:0 **CC1S**: Capture/Compare 1 selection

This bit-field defines the direction of the channel (input/output) as well as the used input.

00: CC1 channel is configured as output

01: CC1 channel is configured as input, IC1 is mapped on TI1

10: CC1 channel is configured as input, IC1 is mapped on TI2

11: CC1 channel is configured as input, IC1 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIMx_SMCR register)

Note: CC1S bits are writable only when the channel is OFF (CC1E = 0 in TIMx_CCER).

13.4.8 TIMx capture/compare mode register 2 (TIMx_CCMR2)

Address offset: 0x1C

Reset value: 0x0000

Refer to the above CCMR1 register description.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC4CE	OC4M[2:0]			OC4PE	OC4FE	CC4S[1:0]		OC3CE	OC3M[2:0]			OC3PE	OC3FE	CC3S[1:0]	
IC4F[3:0]				IC4PSC[1:0]				IC3F[3:0]			IC3PSC[1:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Output compare mode

Bit 15 **OC4CE**: Output compare 4 clear enable

Bits 14:12 **OC4M**: Output compare 4 mode

Bit 11 **OC4PE**: Output compare 4 preload enable

Bit 10 **OC4FE**: Output compare 4 fast enable

Bits 9:8 **CC4S**: Capture/Compare 4 selection

This bit-field defines the direction of the channel (input/output) as well as the used input.

00: CC4 channel is configured as output

01: CC4 channel is configured as input, IC4 is mapped on TI4

10: CC4 channel is configured as input, IC4 is mapped on TI3

11: CC4 channel is configured as input, IC4 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIMx_SMCR register)

Note: CC4S bits are writable only when the channel is OFF (CC4E = 0 in TIMx_CCER).

Bit 7 **OC3CE**: Output compare 3 clear enable

Bits 6:4 **OC3M**: Output compare 3 mode

Bit 3 **OC3PE**: Output compare 3 preload enable

Bit 2 **OC3FE**: Output compare 3 fast enable

Bits 1:0 **CC3S**: Capture/Compare 3 selection

This bit-field defines the direction of the channel (input/output) as well as the used input.

00: CC3 channel is configured as output

01: CC3 channel is configured as input, IC3 is mapped on TI3

10: CC3 channel is configured as input, IC3 is mapped on TI4

11: CC3 channel is configured as input, IC3 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIMx_SMCR register)

Note: CC3S bits are writable only when the channel is OFF (CC3E = 0 in TIMx_CCER).

Input capture mode

Bits 15:12 **IC4F**: Input capture 4 filter

Bits 11:10 **IC4PSC**: Input capture 4 prescaler

Bits 9:8 **CC4S**: Capture/Compare 4 selection

This bit-field defines the direction of the channel (input/output) as well as the used input.

00: CC4 channel is configured as output

01: CC4 channel is configured as input, IC4 is mapped on TI4

10: CC4 channel is configured as input, IC4 is mapped on TI3

11: CC4 channel is configured as input, IC4 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIMx_SMCR register)

Note: CC4S bits are writable only when the channel is OFF (CC4E = 0 in TIMx_CCER).

Bits 7:4 **IC3F**: Input capture 3 filter

Bits 3:2 **IC3PSC**: Input capture 3 prescaler

Bits 1:0 **CC3S**: Capture/Compare 3 selection

This bit-field defines the direction of the channel (input/output) as well as the used input.

00: CC3 channel is configured as output

01: CC3 channel is configured as input, IC3 is mapped on TI3

10: CC3 channel is configured as input, IC3 is mapped on TI4

11: CC3 channel is configured as input, IC3 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIMx_SMCR register)

Note: CC3S bits are writable only when the channel is OFF (CC3E = 0 in TIMx_CCER).

13.4.9 TIMx capture/compare enable register (TIMx_CCER)

Address offset: 0x20

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CC4NP	Res.	CC4P	CC4E	CC3NP	Res.	CC3P	CC3E	CC2NP	Res.	CC2P	CC2E	CC1NP	Res.	CC1P	CC1E
rw		rw	rw	rw		rw	rw	rw		rw	rw	rw		rw	rw

Bit 15 **CC4NP**: Capture/Compare 4 output Polarity.

Refer to CC1NP description

Bit 14 Reserved, always read as 0.

Bit 13 **CC4P**: Capture/Compare 4 output Polarity.

refer to CC1P description

Bit 12 **CC4E**: Capture/Compare 4 output enable.

refer to CC1E description

Bit 11 **CC3NP**: Capture/Compare 3 output Polarity.

refer to CC1NP description

Bit 10 Reserved, always read as 0.

Bits 9:8 Reserved, always read as 0.

Bit 9 **CC3P**: Capture/Compare 3 output Polarity.

refer to CC1P description

- Bit 8 **CC3E**: *Capture/Compare 3 output enable.*
refer to CC1E description
- Bit 7 **CC2NP**: *Capture/Compare 2 output Polarity.*
refer to CC1NP description
- Bit 6 Reserved, always read as 0.
- Bit 5 **CC2P**: *Capture/Compare 2 output Polarity.*
refer to CC1P description
- Bit 4 **CC2E**: *Capture/Compare 2 output enable.*
refer to CC1E description
- Bit 3 **CC1NP**: *Capture/Compare 1 output Polarity.*
CC1 channel configured as output:
 CC1NP must be kept cleared in this case.
CC1 channel configured as input:
 This bit is used in conjunction with CC1P to define TI1FP1/TI2FP1 polarity. refer to CC1P description.
- Bit 2 Reserved, always read as 0.
- Bit 1 **CC1P**: *Capture/Compare 1 output Polarity.*
CC1 channel configured as output:
 0: OC1 active high
 1: OC1 active low
CC1 channel configured as input:
 CC1NP/CC1P bits select TI1FP1 and TI2FP1 polarity for trigger or capture operations.
 00: noninverted/rising edge : circuit is sensitive to TIxFP1 rising edge (capture, trigger in reset, external clock or trigger mode), TIxFP1 is not inverted (trigger in gated mode, encoder mode).
 01: inverted/falling edge : circuit is sensitive to TIxFP1 falling edge (capture, trigger in reset, external clock or trigger mode), TIxFP1 is inverted (trigger in gated mode, encoder mode).
 10: reserved, do not use this configuration.
 11: noninverted/both edges: circuit is sensitive to both TIxFP1 rising and falling edges (capture, trigger in reset, external clock or trigger mode), TIxFP1 is not inverted (trigger in gated mode). This configuration must not be used for encoder mode.
- Bit 0 **CC1E**: *Capture/Compare 1 output enable.*
CC1 channel configured as output:
 0: Off - OC1 is not active
 1: On - OC1 signal is output on the corresponding output pin
CC1 channel configured as input:
 This bit determines if a capture of the counter value can actually be done into the input capture/compare register 1 (TIMx_CCR1) or not.
 0: Capture disabled
 1: Capture enabled

Table 54. Output control bit for standard OCx channels

CCxE bit	OCx output state
0	Output Disabled (OCx=0, OCx_EN=0)
1	OCx=OCxREF + Polarity, OCx_EN=1

Note: The state of the external IO pins connected to the standard OCx channels depends on the OCx channel state and the GPIO and AFIO registers.

13.4.10 TIMx counter (TIMx_CNT)

Address offset: 0x24

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **CNT[15:0]**: Counter value

13.4.11 TIMx prescaler (TIMx_PSC)

Address offset: 0x28

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **PSC[15:0]**: Prescaler value

The counter clock frequency CK_CNT is equal to $f_{CK_PSC} / (PSC[15:0] + 1)$.

PSC contains the value to be loaded in the active prescaler register at each update event.

13.4.12 TIMx auto-reload register (TIMx_ARR)

Address offset: 0x2C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **ARR[15:0]**: Auto-reload value

ARR is the value to be loaded in the actual auto-reload register.

Refer to the [Section 13.3.1: Time-base unit on page 271](#) for more details about ARR update and behavior.

The counter is blocked while the auto-reload value is null.

13.4.13 TIMx capture/compare register 1 (TIMx_CCR1)

Address offset: 0x34

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR1[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **CCR1[15:0]**: Capture/Compare 1 value

If channel CC1 is configured as output:

CCR1 is the value to be loaded in the actual capture/compare 1 register (preload value).

It is loaded permanently if the preload feature is not selected in the TIMx_CCMR1 register (bit OC1PE). Else the preload value is copied in the active capture/compare 1 register when an update event occurs.

The active capture/compare register contains the value to be compared to the counter TIMx_CNT and signaled on OC1 output.

If channel CC1 is configured as input:

CCR1 is the counter value transferred by the last input capture 1 event (IC1).

13.4.14 TIMx capture/compare register 2 (TIMx_CCR2)

Address offset: 0x38

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR2[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **CCR2[15:0]**: Capture/Compare 2 value

If channel CC2 is configured as output:

CCR2 is the value to be loaded in the actual capture/compare 2 register (preload value).

It is loaded permanently if the preload feature is not selected in the TIMx_CCMR1 register (bit OC2PE). Else the preload value is copied in the active capture/compare 2 register when an update event occurs.

The active capture/compare register contains the value to be compared to the counter TIMx_CNT and signalled on OC2 output.

If channel CC2 is configured as input:

CCR2 is the counter value transferred by the last input capture 2 event (IC2).

13.4.15 TIMx capture/compare register 3 (TIMx_CCR3)

Address offset: 0x3C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR3[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **CCR3[15:0]**: Capture/Compare value

If channel CC3 is configured as output:

CCR3 is the value to be loaded in the actual capture/compare 3 register (preload value).

It is loaded permanently if the preload feature is not selected in the TIMx_CCMR2 register (bit OC3PE). Else the preload value is copied in the active capture/compare 3 register when an update event occurs.

The active capture/compare register contains the value to be compared to the counter TIMx_CNT and signalled on OC3 output.

If channel CC3 is configured as input:

CCR3 is the counter value transferred by the last input capture 3 event (IC3).

13.4.16 TIMx capture/compare register 4 (TIMx_CCR4)

Address offset: 0x40

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR4[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **CCR4[15:0]**: Capture/Compare value

1/ if CC4 channel is configured as output (CC4S bits):

CCR4 is the value to be loaded in the actual capture/compare 4 register (preload value).

It is loaded permanently if the preload feature is not selected in the TIMx_CCMR2 register (bit OC4PE). Else the preload value is copied in the active capture/compare 4 register when an update event occurs.

The active capture/compare register contains the value to be compared to the counter TIMx_CNT and signalled on OC4 output.

2/ if CC4 channel is configured as input (CC4S bits in TIMx_CCMR4 register):

CCR4 is the counter value transferred by the last input capture 4 event (IC4).

13.4.17 TIMx DMA control register (TIMx_DCR)

Address offset: 0x48

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			DBL[4:0]					Reserved			DBA[4:0]				
			rw	rw	rw	rw	rw				rw	rw	rw	rw	rw

Bits 15:13 Reserved, always read as 0

Bits 12:8 **DBL[4:0]**: DMA burst length

This 5-bit vector defines the number of DMA transfers (the timer recognizes a burst transfer when a read or a write access is done to the TIMx_DMAR address).

00000: 1 transfer,

00001: 2 transfers,

00010: 3 transfers,

...

10001: 18 transfers.

Bits 7:5 Reserved, always read as 0

Bits 4:0 **DBA[4:0]**: DMA base address

This 5-bit vector defines the base-address for DMA transfers (when read/write access are done through the TIMx_DMAR address). DBA is defined as an offset starting from the address of the TIMx_CR1 register.

Example:

00000: TIMx_CR1,

00001: TIMx_CR2,

00010: TIMx_SMCR,

...

Example: Let us consider the following transfer: DBL = 7 bytes & DBA = TIMx_CR1. In this case the transfer is done to/from 7 registers starting from the TIMx_CR1 address..

– If DBL = 7 bytes and DBA = TIM2_CR1 represents the address of the byte to be transferred, the address of the transfer should be given by the following equation:

(TIMx_CR1 address) + DBA + (DMA index), where DMA index = DBL

In this example, 7 bytes are added to (TIMx_CR1 address) + DBA, which gives us the address from/to which the data will be copied. In this case, the transfer is done to 7 registers starting from the following address: (TIMx_CR1 address) + DBA

According to the configuration of the DMA Data Size, several cases may occur:

– If you configure the DMA Data Size in half-words, 16-bit data will be transferred to each of the 7 registers.

If you configure the DMA Data Size in bytes, the data will also be transferred to 7 registers: the first register will contain the first MSB byte, the second register, the first LSB byte and so on. So with the transfer Timer, you also have to specify the size of data transferred by DMA.

13.4.18 TIMx DMA address for full transfer (TIMx_DMAR)

Address offset: 0x4C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DMAB[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **DMAB[15:0]**: DMA register for burst accesses

A read or write access to the DMAR register accesses the register located at the address:

“(TIMx_CR1 address) + DBA + (DMA index)” in which:

TIMx_CR1 address is the address of the control register 1,

DBA is the DMA base address configured in the TIMx_DCR register,

DMA index is the offset automatically controlled by the DMA transfer, depending on the length of the transfer DBL in the TIMx_DCR register.

Example of using the DMA burst feature:

In this example the Timer DMA burst feature is used to update the contents of the CCRx registers (x = 2, 3, 4) with the DMA transferring half words into the CCRx registers.

This is done in the following steps:

- Configure the corresponding DMA channel as follows:
 - DMA channel peripheral address is the DMAR register address
 - DMA channel memory address is the address of the buffer in the RAM containing the data to be transferred by DMA into CCRx registers.
 - Number of data to transfer = 3 (See note below).
 - Circular mode disabled.
- Configure the DCR register by configuring the DBA and DBL bit fields as follows:
DBL = 3 transfers, DBA = 0xE.
- Enable the TIMx update DMA request (set the UDE bit in the DIER register).
- Enable TIMx
- Enable the DMA channel

Note: This example is for the case where every CCRx register to to be updated once. If every CCRx register is to be updated twice for example, the number of data to transfer should be 6. Let's take the example of a buffer in the RAM containing data1, data2, data3, data4, data5 and data6. The data is transferred to the CCRx registers as follows: on the first update DMA request, data1 is transferred to CCR2, data2 is transferred to CCR3, data3 is transferred to CCR4 and on the second update DMA request, data4 is transferred to CCR2, data5 is transferred to CCR3 and data6 is transferred to CCR4.

13.4.19 TIMx register map

TIMx registers are mapped as 16-bit addressable registers as described in the table below:

Table 55. TIM2 to TIM4 register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																
0x00	TIMx_CR1 Reset value	Reserved																						CKD [1:0]		ARPE		CMS [1:0]		DIR		OPM		URS		UDIS		CEN											
0x04	TIMx_CR2 Reset value	Reserved																						TIS		MMS[2:0]		CCDS		Reserved																			
0x08	TIMx_SMCR Reset value	Reserved																ETP		ECE		ETPS [1:0]		ETF[3:0]		MSM		TS[2:0]		Reserved		SMS[2:0]																	
0x0C	TIMx_DIER Reset value	Reserved																TDE		COMDE		CC4DE		CC3DE		CC2DE		CC1DE		UDE		Reserved		TIE		Reserved		CC4IE		CC3IE		CC2IE		CC1IE		UIE			
0x10	TIMx_SR Reset value	Reserved																CC4OF		CC3OF		CC2OF		CC1OF		Reserved		TIF		Reserved		CC4IF		CC3IF		CC2IF		CC1IF		UIF									
0x14	TIMx_EGR Reset value	Reserved																						TG		Reserved		CC4G		CC3G		CC2G		CC1G		UG													
0x18	TIMx_CCMR1 <i>Output Compare mode</i> Reset value	Reserved																OC2CE		OC2M [2:0]		OC2PE		OC2FE		CC2S [1:0]		OC1CE		OC1M [2:0]		OC1PE		OC1FE		CC1S [1:0]													
	TIMx_CCMR1 <i>Input Capture mode</i> Reset value	Reserved																IC2F[3:0]		IC2PSC [1:0]		CC2S [1:0]		IC1F[3:0]		IC1PSC [1:0]		CC1S [1:0]																					
0x1C	TIMx_CCMR2 <i>Output Compare mode</i> Reset value	Reserved																OC4CE		OC4M [2:0]		OC4PE		OC4FE		CC4S [1:0]		OC3CE		OC3M [2:0]		OC3PE		OC3FE		CC3S [1:0]													
	TIMx_CCMR2 <i>Input Capture mode</i> Reset value	Reserved																IC4F[3:0]		IC4PSC [1:0]		CC4S [1:0]		IC3F[3:0]		IC3PSC [1:0]		CC3S [1:0]																					
0x20	TIMx_CCER Reset value	Reserved																CC4NP		Reserved		CC4P		CC4E		CC3NP		Reserved		CC3P		CC3E		CC2NP		Reserved		CC2P		CC2E		CC1NP		Reserved		CC1P		CC1E	
0x24	TIMx_CNT Reset value	Reserved																CNT[15:0]																															
0x28	TIMx_PSC Reset value	Reserved																PSC[15:0]																															
0x2C	TIMx_ARR Reset value	Reserved																ARR[15:0]																															
0x30	Reserved																																																
0x34	TIMx_CCR1 Reset value	Reserved																CCR1[15:0]																															

Table 55. TIM2 to TIM4 register map and reset values (continued)

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0x38	TIMx_CCR2 Reset value	Reserved																CCR2[15:0] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																	
0x3C	TIMx_CCR3 Reset value	Reserved																CCR3[15:0] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																	
0x40	TIMx_CCR4 Reset value	Reserved																CCR4[15:0] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																	
0x44	Reserved																																		
0x48	TIMx_DCR Reset value	Reserved																DBL[4:0] 0 0 0 0 0				Reserved				DBA[4:0] 0 0 0 0 0									
0x4C	TIMx_DMAR Reset value	Reserved																DMAB[15:0] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																	
0x50	Reserved																																		

Refer to [Table 1 on page 32](#) for the register boundary addresses.

HAL libraries (reduced)

Description of STM32L1 HAL and low-layer drivers

Introduction

STMCube™ is an STMicroelectronics original initiative to make developers' lives easier by reducing development effort, time and cost. STM32Cube covers the whole STM32 portfolio.

THIS IS A REDUCED VERSION FOR TEACHING PURPOSES

Contents

Introduction	2
UART STM32CubeMX	4
UART Mode and Configuration	4
UART IRQ Configuration	5
HAL UART Generic Driver	5
UART Firmware driver registers structures	5
UART_InitTypeDef	5
UART_HandleTypeDef	6
UART Firmware driver API description	7
How to use this driver	7
Initialization and Configuration functions	7
IO operation functions	8
Peripheral Control functions	8
Peripheral State and Errors functions	9
Detailed description of functions	9
UART Firmware driver defines	13
UART 13	
RTC STM32CubeMX	16
RTC Mode and Configuration	16
RTC Alarm Configuration	18
RTC IRQ Configuration	19
HAL RTC Generic Driver	19
RTC Firmware driver registers structures	19
RTC_InitTypeDef	19
RTC_DateTypeDef	20
RTC_HandleTypeDef	20
RTC Firmware driver API description	20
Backup Domain Operating Condition	20
Backup Domain Reset	21
Backup Domain Access	21
How to use this driver	21
RTC and low power modes	21
Initialization and de-initialization functions	21
RTC Time and Date functions	22
RTC Alarm functions	22
Peripheral State functions	22
Peripheral Control functions	23
Detailed description of functions	23
RTC Firmware driver defines	27

HAL I2C Generic Driver	33
I2C Firmware driver registers structures	33
I2C_InitTypeDef	33
I2C_HandleTypeDef	33
I2C Firmware driver API description	34
How to use this driver	34
Initialization and de-initialization functions	35
IO operation functions	36
Peripheral State, Mode and Error functions	36
Detailed description of functions	36
I2C Firmware driver defines	42
I2C 42	
HAL SPI Generic Driver	46
SPI Firmware driver registers structures	46
SPI_InitTypeDef	46
SPI_HandleTypeDef	46
SPI Firmware driver API description	47
How to use this driver	47
Initialization and de-initialization functions	47
IO operation functions	48
Peripheral State and Errors functions	48
Detailed description of functions	48
SPI Firmware driver defines	52
SPI 52	

UART STM32CubeMX

UART Mode and Configuration

Pinout & Configuration

Clock Configuration

Additional

Options

Categories

A-Z

System Core

Analog

Timers

Connectivity

I2C1

I2C2

SPI1

SPI2

SPI3

✓ USART1

USART2

USART3

USB

Multimedia

Computing

Middleware

USART1 Mode and Configuration

Mode

Mode Asynchronous

Hardware Flow Control (RS232) Disable

Configuration

Reset Configuration

✓ NVIC Settings

✓ DMA Settings

✓ GPIO Settings

✓ Parameter Settings

✓ User Constants

Configure the below parameters :

Search (Ctrl+F)

Basic Parameters

Baud Rate 115200 Bits/s

Word Length 8 Bits (including Parity)

Parity None

Stop Bits 1

Advanced Parameters

Data Direction Receive and Transmit

Over Sampling 16 Samples

UART IRQ Configuration

Pinout & Configuration

Clock Configuration

Additional

Options

Categories A-Z

System Core

Analog

Timers

Connectivity

I2C1

I2C2

SPI1

SPI2

SPI3

✓ USART1

USART2

USART3

USB

Multimedia

Computing

Middleware

USART1 Mode and Configuration

Mode

Mode Asynchronous

Hardware Flow Control (RS232) Disable

Configuration

Reset Configuration

✓ NVIC Settings

✓ DMA Settings

✓ GPIO Settings

✓ Parameter Settings

✓ User Constants

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
USART1 global interrupt	☐	0	0

HAL UART Generic Driver

UART Firmware driver registers structures

UART_InitTypeDef

Data Fields

uint32_t BaudRate

uint32_t WordLength

uint32_t StopBits

uint32_t Parity

uint32_t Mode

uint32_t HwFlowCtl

uint32_t OverSampling

Field Documentation

uint32_t UART_InitTypeDef::BaudRate

This member configures the UART communication baud rate. The baud rate is computed using the following formula: $\text{IntegerDivider} = ((\text{PCLKx}) / (8 * (\text{OVR8} + 1) * (\text{huart} > \text{Init.BaudRate})))$ $\text{FractionalDivider} = ((\text{IntegerDivider} - ((\text{uint32_t}) \text{IntegerDivider})) * 8 * (\text{OVR8} + 1)) + 0.5$ Where OVR8 is the "oversampling by 8 mode" configuration bit in the CR1 register.

uint32_t UART_InitTypeDef::WordLength

Specifies the number of data bits transmitted or received in a frame. This parameter can be a value of [**UART_Word_Length**](#)

uint32_t UART_InitTypeDef::StopBits

Specifies the number of stop bits transmitted. This parameter can be a value of [***UART_Stop_Bits***](#)

uint32_t UART_InitTypeDef::Parity

Specifies the parity mode. This parameter can be a value of [***UART_Parity***](#) **Note:**When parity is enabled, the computed parity is inserted at the MSB position of the transmitted data (9th bit when the word length is set to 9 data bits; 8th bit when the word length is set to 8 data bits).

uint32_t UART_InitTypeDef::Mode

Specifies whether the Receive or Transmit mode is enabled or disabled. This parameter can be a value of [***UART_Mode***](#)

uint32_t UART_InitTypeDef::HwFlowCtl

Specifies whether the hardware flow control mode is enabled or disabled. This parameter can be a value of [***UART_Hardware_Flow_Control***](#)

uint32_t UART_InitTypeDef::OverSampling

Specifies whether the Over sampling 8 is enabled or disabled, to achieve higher speed (up to fPCLK/8). This parameter can be a value of [***UART_Over_Sampling***](#)

UART_HandleTypeDef

Data Fields

USART_TypeDef * Instance

UART_InitTypeDef Init

uint8_t * pTxBuffPtr

uint16_t TxXferSize

uint16_t TxXferCount

uint8_t * pRxBuffPtr

uint16_t RxXferSize

uint16_t RxXferCount

DMA_HandleTypeDef * hdmatx

DMA_HandleTypeDef * hdmarx

HAL_LockTypeDef Lock

__IO HAL_UART_StateTypeDef State

__IO uint32_t ErrorCode

Field Documentation

USART_TypeDef* UART_HandleTypeDef::Instance

UART registers base address

UART_InitTypeDef UART_HandleTypeDef::Init

UART communication parameters

uint8_t* UART_HandleTypeDef::pTxBuffPtr

Pointer to UART Tx transfer Buffer

uint16_t UART_HandleTypeDef::TxXferSize

UART Tx Transfer size

uint16_t UART_HandleTypeDef::TxXferCount

UART Tx Transfer Counter

uint8_t* UART_HandleTypeDef::pRxBuffPtr

Pointer to UART Rx transfer Buffer

uint16_t UART_HandleTypeDef::RxXferSize

UART Rx Transfer size

uint16_t UART_HandleTypeDef::RxXferCount

UART Rx Transfer Counter

DMA_HandleTypeDef* UART_HandleTypeDef::hdmatx

UART Tx DMA Handle parameters

DMA_HandleTypeDef* UART_HandleTypeDef::hdmarx

UART Rx DMA Handle parameters

HAL_LockTypeDef UART_HandleTypeDef::Lock

Locking object

__IO HAL_UART_StateTypeDef UART_HandleTypeDef::State

UART communication state

__IO uint32_t UART_HandleTypeDef::ErrorCode

UART Error code

UART Firmware driver API description

How to use this driver

The UART HAL driver can be used as follows:

Declare a UART_HandleTypeDef handle structure.

Initialize the UART low level resources by implementing the HAL_UART_MspInit() API:

- Enable the USARTx interface clock.

- UART pins configuration:

 - Enable the clock for the UART GPIOs.

 - Configure the UART pins as alternate function pull-up.

- NVIC configuration if you need to use interrupt process

 - (HAL_UART_Transmit_IT() and HAL_UART_Receive_IT() APIs):

 - Configure the USARTx interrupt priority.

 - Enable the NVIC USART IRQ handle.

Program the Baud Rate, Word Length, Stop Bit, Parity, Hardware flow control and Mode(Receiver/Transmitter) in the UART Init structure.

For the UART asynchronous mode, initialize the UART registers by calling the HAL_UART_Init() API.

For the UART Half duplex mode, initialize the UART registers by calling the HAL_HalfDuplex_Init() API.

For the LIN mode, initialize the UART registers by calling the HAL_LIN_Init() API.

For the Multi-Processor mode, initialize the UART registers by calling the HAL_MultiProcessor_Init() API.

Three operation modes are available within this driver :

Polling mode IO operation

Send an amount of data in blocking mode using HAL_UART_Transmit()

Receive an amount of data in blocking mode using HAL_UART_Receive()

Interrupt mode IO operation

Send an amount of data in non blocking mode using HAL_UART_Transmit_IT()

At transmission end of transfer HAL_UART_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_UART_TxCpltCallback

Receive an amount of data in non blocking mode using HAL_UART_Receive_IT()

At reception end of transfer HAL_UART_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_UART_RxCpltCallback

In case of transfer Error, HAL_UART_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_UART_ErrorCallback

UART HAL driver macros list

Below the list of most used macros in UART HAL driver.

__HAL_UART_ENABLE: Enable the UART peripheral

__HAL_UART_DISABLE: Disable the UART peripheral

__HAL_UART_GET_FLAG : Check whether the specified UART flag is set or not

__HAL_UART_CLEAR_FLAG : Clear the specified UART pending flag

__HAL_UART_ENABLE_IT: Enable the specified UART interrupt

__HAL_UART_DISABLE_IT: Disable the specified UART interrupt

__HAL_UART_GET_IT_SOURCE: Check whether the specified UART interrupt has occurred or not

Initialization and Configuration functions

This subsection provides a set of functions allowing to initialize the USARTx or the UARTy in asynchronous mode.

For the asynchronous mode only these parameters can be configured:

- Baud Rate

Word Length
Stop Bit
Parity
Hardware flow control
Receiver/transmitter modes
Over Sampling Methode

The HAL_UART_Init(), HAL_HalfDuplex_Init(), HAL_LIN_Init() and HAL_MultiProcessor_Init() APIs follow respectively the UART asynchronous, UART Half duplex, LIN and Multi-Processor configuration procedures (details for the procedures are available in reference manual (RM0038)).

This section contains the following APIs:

[*HAL_UART_Init\(\)*](#)
[*HAL_HalfDuplex_Init\(\)*](#)
[*HAL_LIN_Init\(\)*](#)
[*HAL_MultiProcessor_Init\(\)*](#)
[*HAL_UART_DeInit\(\)*](#)
[*HAL_UART_MspInit\(\)*](#)
[*HAL_UART_MspDeInit\(\)*](#)

IO operation functions

This subsection provides a set of functions allowing to manage the UART asynchronous and Half duplex data transfers.

There are two modes of transfer:

Blocking mode: The communication is performed in polling mode. The HAL status of all data processing is returned by the same function after finishing transfer.

Non blocking mode: The communication is performed using Interrupts or DMA, these APIs return the HAL status. The end of the data processing will be indicated through the dedicated UART IRQ when using Interrupt mode or the DMA IRQ when using DMA mode. The HAL_UART_TxCpltCallback(), HAL_UART_RxCpltCallback() user callbacks will be executed respectively at the end of the transmit or receive process. The HAL_UART_ErrorCallback() user callback will be executed when a communication error is detected.

Blocking mode APIs are:

HAL_UART_Transmit()
HAL_UART_Receive()

Non Blocking mode APIs with Interrupt are:

HAL_UART_Transmit_IT()
HAL_UART_Receive_IT()
HAL_UART_IRQHandler()

A set of Transfer Complete Callbacks are provided in non blocking mode:

HAL_UART_TxHalfCpltCallback()
HAL_UART_TxCpltCallback()
HAL_UART_RxHalfCpltCallback()
HAL_UART_RxCpltCallback()
HAL_UART_ErrorCallback()

This section contains the following APIs:

[*HAL_UART_Transmit\(\)*](#)
[*HAL_UART_Receive\(\)*](#)
[*HAL_UART_Transmit_IT\(\)*](#)
[*HAL_UART_Receive_IT\(\)*](#)
[*HAL_UART_IRQHandler\(\)*](#)
[*HAL_UART_TxCpltCallback\(\)*](#)
[*HAL_UART_TxHalfCpltCallback\(\)*](#)
[*HAL_UART_RxCpltCallback\(\)*](#)
[*HAL_UART_RxHalfCpltCallback\(\)*](#)
[*HAL_UART_ErrorCallback\(\)*](#)

Peripheral Control functions

This subsection provides a set of functions allowing to control the UART:

HAL_LIN_SendBreak() API can be helpful to transmit the break character.
 HAL_MultiProcessor_EnterMuteMode() API can be helpful to enter the UART in mute mode.
 HAL_MultiProcessor_ExitMuteMode() API can be helpful to exit the UART mute mode by software.
 HAL_HalfDuplex_EnableTransmitter() API to enable the UART transmitter and disables the UART receiver in Half Duplex mode
 HAL_HalfDuplex_EnableReceiver() API to enable the UART receiver and disables the UART transmitter in Half Duplex mode

This section contains the following APIs:

[*HAL_LIN_SendBreak\(\)*](#)
[*HAL_MultiProcessor_EnterMuteMode\(\)*](#)
[*HAL_MultiProcessor_ExitMuteMode\(\)*](#)
[*HAL_HalfDuplex_EnableTransmitter\(\)*](#)
[*HAL_HalfDuplex_EnableReceiver\(\)*](#)

Peripheral State and Errors functions

This subsection provides a set of functions allowing to return the State of UART communication process, return Peripheral Errors occurred during communication process

HAL_UART_GetState() API can be helpful to check in run-time the state of the UART peripheral.
 HAL_UART_GetError() check in run-time errors that could be occurred during communication.

This section contains the following APIs:

[*HAL_UART_GetState\(\)*](#)
[*HAL_UART_GetError\(\)*](#)

Detailed description of functions

HAL_UART_Init

Function name	HAL_StatusTypeDef HAL_UART_Init (UART_HandleTypeDef *huart)
Function description	Initializes the UART mode according to the specified parameters in the UART_InitTypeDef and create the associated handle.
Parameters	huart: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
Return values	HAL: status

HAL_HalfDuplex_Init

Function name	HAL_StatusTypeDef HAL_HalfDuplex_Init (UART_HandleTypeDef *huart)
Function description	Initializes the half-duplex mode according to the specified parameters in the UART_InitTypeDef and create the associated handle.
Parameters	huart: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
Return values	HAL: status

HAL_LIN_Init

Function name	HAL_StatusTypeDef HAL_LIN_Init (UART_HandleTypeDef *huart, uint32_t BreakDetectLength)
---------------	---

Function description	Initializes the LIN mode according to the specified parameters in the UART_InitTypeDef and create the associated handle.
Parameters	huart: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module. BreakDetectLength: Specifies the LIN break detection length. This parameter can be one of the following values: UART_LINBREAKDETECTLENGTH_10B: 10-bit break detection UART_LINBREAKDETECTLENGTH_11B: 11-bit break detection

Return values **HAL:** status

HAL_MultiProcessor_Init

Function name	HAL_StatusTypeDef HAL_MultiProcessor_Init (UART_HandleTypeDef * huart, uint8_t Address, uint32_t WakeUpMethod)
Function description	Initializes the Multi-Processor mode according to the specified parameters in the UART_InitTypeDef and create the associated handle.
Parameters	huart: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module. Address: UART node address WakeUpMethod: specifies the UART wakeup method. This parameter can be one of the following values: UART_WAKEUPMETHOD_IDLELINE: Wakeup by an idle line detection UART_WAKEUPMETHOD_ADDRESSMARK: Wakeup by an address mark

Return values **HAL:** status

HAL_UART_DeInit

Function name	HAL_StatusTypeDef HAL_UART_DeInit (UART_HandleTypeDef * huart)
Function description	DeInitializes the UART peripheral.
Parameters	huart: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values **HAL:** status

HAL_UART_MspInit

Function name	void HAL_UART_MspInit (UART_HandleTypeDef * huart)
Function description	UART MSP Init.
Parameters	huart: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values **None:**

HAL_UART_MspDeInit

Function name	void HAL_UART_MspDeInit (UART_HandleTypeDef * huart)
Function description	UART MSP DeInit.

Parameters **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values **None:**

HAL_UART_Transmit

Function name **HAL_StatusTypeDef HAL_UART_Transmit**
(UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description Sends an amount of data in blocking mode.

Parameters **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
pData: Pointer to data buffer
Size: Amount of data to be sent
Timeout: Timeout duration

Return values **HAL:** status

HAL_UART_Receive

Function name **HAL_StatusTypeDef HAL_UART_Receive**
(UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description Receives an amount of data in blocking mode.

Parameters **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
pData: Pointer to data buffer
Size: Amount of data to be received
Timeout: Timeout duration

Return values **HAL:** status

HAL_UART_Transmit_IT

Function name **HAL_StatusTypeDef HAL_UART_Transmit_IT**
(UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size)

Function description Sends an amount of data in non blocking mode.

Parameters **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
pData: Pointer to data buffer
Size: Amount of data to be sent

Return values **HAL:** status

HAL_UART_Receive_IT

Function name **HAL_StatusTypeDef HAL_UART_Receive_IT**
(UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size)

Function description Receives an amount of data in non blocking mode.

Parameters **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
pData: Pointer to data buffer
Size: Amount of data to be received

Return values **HAL:** status

HAL_UART_IRQHandler

Function name	void HAL_UART_IRQHandler (UART_HandleTypeDef * huart)
Function description	This function handles UART interrupt request.
Parameters	huart: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
Return values	None:

HAL_UART_TxCpltCallback

Function name	void HAL_UART_TxCpltCallback (UART_HandleTypeDef * huart)
Function description	Tx Transfer completed callbacks.
Parameters	huart: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
Return values	None:

HAL_UART_RxCpltCallback

Function name	void HAL_UART_RxCpltCallback (UART_HandleTypeDef * huart)
Function description	Rx Transfer completed callbacks.
Parameters	huart: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
Return values	None:

HAL_UART_ErrorCallback

Function name	void HAL_UART_ErrorCallback (UART_HandleTypeDef * huart)
Function description	UART error callbacks.
Parameters	huart: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
Return values	None:

HAL_UART_GetState

Function name	HAL_UART_StateTypeDef HAL_UART_GetState (UART_HandleTypeDef * huart)
Function description	Returns the UART state.
Parameters	huart: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
Return values	HAL: state

HAL_UART_GetError

Function name	uint32_t HAL_UART_GetError (UART_HandleTypeDef * huart)
Function description	Return the UART error code.
Parameters	huart: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART.

UART Firmware driver defines

UART

UART Error Codes

HAL_UART_ERROR_NONE	No error
HAL_UART_ERROR_PE	Parity error
HAL_UART_ERROR_NE	Noise error
HAL_UART_ERROR_FE	frame error
HAL_UART_ERROR_ORE	Overrun error

UART Exported Macros

`__HAL_UART_RESET_HANDLE_STATE`

Description: Reset UART handle state.

Parameters:

__HANDLE : specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value: None

`__HAL_UART_FLUSH_DRREGISTER`

Description: Flush the UART DR register.

Parameters:

__HANDLE : specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

`__HAL_UART_GET_FLAG`

Description: Check whether the specified UART flag is set or not.

Parameters:

__HANDLE : specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

__FLAG : specifies the flag to check. This parameter can be one of the following values:
UART_FLAG_CTS: CTS Change flag (not available for

UART4 and UART5)
 UART_FLAG_LBD: LIN Break
 detection flag
 UART_FLAG_TXE: Transmit
 data register empty flag
 UART_FLAG_TC:
 Transmission Complete flag
 UART_FLAG_RXNE: Receive data
 register not empty flag
 UART_FLAG_IDLE: Idle Line
 detection flag
 UART_FLAG_ORE: OverRun
 Error flag
 UART_FLAG_NE: Noise Error flag
 UART_FLAG_FE: Framing
 Error flag
 UART_FLAG_PE: Parity Error flag

Return value: The: new state of __ FLAG_ _____(TRUE or FALSE).

`__HAL_UART_CLEAR_FLAG`

Description: Clear the specified UART pending flag.

Parameters:

__HANDLE : specifies the UART
 Handle. UART Handle selects the
 USARTx or UARTy peripheral
 (USART,UART availability and x,y
 values depending on device).
 __FLAG : specifies the flag to check.
 This parameter can be any
 combination of the following values:
 UART_FLAG_CTS: CTS Change
 flag (not available for UART4
 and UART5).
 UART_FLAG_LBD: LIN Break
 detection flag.
 UART_FLAG_TC:
 Transmission Complete flag.
 UART_FLAG_RXNE: Receive data
 register not empty flag.

Return value: None

Notes: PE (Parity error), FE (Framing error), NE (Noise error), ORE (OverRun error) and IDLE (Idle line detected)

flags are cleared by software sequence: a read
 operation to USART_SR register followed by a
 read operation to USART_DR register. RXNE
 flag can be also cleared by a read to the
 USART_DR register. TC flag can be also
 cleared by software sequence: a read operation
 to USART_SR register followed by a write
 operation to USART_DR register. TXE flag is
 cleared only by a write to the USART_DR
 register.

__HAL_UART_ENABLE_IT

Description: Enable the specified UART interrupt.

Parameters:

- __HANDLE : specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).
- __INTERRUPT : specifies the UART interrupt source to enable. This parameter can be one of the following values:
 - UART_IT_CTS: CTS change interrupt
 - UART_IT_LBD: LIN Break detection interrupt
 - UART_IT_TXE: Transmit Data Register empty interrupt
 - UART_IT_TC: Transmission complete interrupt
 - UART_IT_RXNE: Receive Data register not empty interrupt
 - UART_IT_IDLE: Idle line detection interrupt
 - UART_IT_PE: Parity Error interrupt
 - UART_IT_ERR: Error interrupt(Frame error, noise

error, overrun error)

Return value: None

__HAL_UART_DISABLE_IT

Description: Disable the specified UART interrupt.

Parameters:

- __HANDLE : specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).
- __INTERRUPT : specifies the UART interrupt source to disable. This parameter can be one of the following values:
 - UART_IT_CTS: CTS change interrupt
 - UART_IT_LBD: LIN Break detection interrupt
 - UART_IT_TXE: Transmit Data Register empty interrupt
 - UART_IT_TC: Transmission complete interrupt
 - UART_IT_RXNE: Receive Data register not empty interrupt
 - UART_IT_IDLE: Idle line detection interrupt
 - UART_IT_PE: Parity Error interrupt
 - UART_IT_ERR: Error interrupt(Frame error, noise error, overrun error)

Return value: None

__HAL_UART_GET_IT_SOURCE

Description: Check whether the specified UART interrupt has occurred or not.

Parameters:

- __HANDLE : specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).
- __IT : specifies the UART interrupt source to check. This parameter can be one of the following values:
 - UART_IT_CTS: CTS change

interrupt (not available for UART4 and UART5)
 UART_IT_LBD: LIN Break detection interrupt
 UART_IT_TXE: Transmit Data Register empty interrupt
 UART_IT_TC: Transmission complete interrupt
 UART_IT_RXNE: Receive Data register not empty interrupt
 UART_IT_IDLE: Idle line detection interrupt
 UART_IT_ERR: Error interrupt

value: The: new state of __ IT__ (TRUE or FALSE).

`__HAL_UART_ENABLE`

Description: Enable UART.

Parameters:

`__HANDLE` : specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).

Return value: None

`__HAL_UART_DISABLE`

Description: Disable UART. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).

Return value: None

UART Flags

`UART_FLAG_TXE`, `UART_FLAG_TC`, `UART_FLAG_RXNE`

UART Interrupt Definitions

`UART_IT_PE`, `UART_IT_TXE`, `UART_IT_TC`, `UART_IT_RXNE`,

UART Transfer Mode

`UART_MODE_RX`, `UART_MODE_TX`, `UART_MODE_TX_RX`

UART Over Sampling

`UART_OVERSAMPLING_16`, `UART_OVERSAMPLING_8`

UART Parity

`UART_PARITY_NONE`, `UART_PARITY_EVEN`, `UART_PARITY_ODD`

UART Number of Stop Bits

`UART_STOPBITS_1`, `UART_STOPBITS_2`

UART Word Length

`UART_WORDLENGTH_8B`, `UART_WORDLENGTH_9B`

RTC STM32CubeMX

RTC Mode and Configuration

Options



Categories

A->Z

System Core >

Analog >

Timers >

☒ RTC☐ TIM2
☐ TIM3
☐ TIM4
☐ TIM5
☐ TIM6
☐ TIM7
☐ TIM9
☐ TIM10
☐ TIM11

Connectivity >

Multimedia >

Computing >

Middleware >

RTC Mode and Configuration

Mode

☒ Activate Clock Source☒ Activate CalendarAlarm A Alarm B WakeUp ☐ Timestamp☐ Tamper 1☐ Tamper 2Calibration ☐ Reference clock detection

Configuration

Reset Configuration

☒ Parameter Settings☒ User Constants

Configure the below parameters :



General

Hour Format Hourformat 24

Asynchronous Predivider value 127

Synchronous Predivider value 255

Calendar Time

Data Format BCD data format

Hours 0

Minutes 0

Seconds 0

Day Light Saving: value of hour adj... Daylightsaving None

Store Operation Storeoperation Reset

Calendar Date

Week Day Monday

Month January

Date 1

Year 0

RTC Alarm Configuration

Pinout & Configuration

Clock Configur

Additional

Options

Categories

A->Z

System Core

Analog

Timers

RTC

TIM2

TIM3

TIM4

TIM5

TIM6

TIM7

TIM9

TIM10

TIM11

Connectivity

Multimedia

Computing

Middleware

RTC Mode and Configuration

Mode

Activate Clock Source

Activate Calendar

Alarm A

Internal Alarm A

Alarm B

Disable

WakeUp

Disable

Timestamp

Tamper 1

Configuration

Reset Configuration

Parameter Settings

User Constants

NVIC Settings

Configure the below parameters :

Search (Ctrl+F)

Hours

Minutes

Seconds

Day Light Saving: value of hou...

Daylightsaving None

Store Operation

Storeoperation Reset

Calendar Date

Week Day

Monday

Month

January

Date

1

Year

0

Alarm A

Hours

Minutes

Seconds

Alarm Mask Date Week day

Disable

Alarm Mask Hours

Disable

Alarm Mask Minutes

Disable

Alarm Mask Seconds

Disable

Alarm Sub Second Mask

All Alarm SS fields are masked.

Alarm Date Week Day Sel

Date

Alarm Date

1

RTC IRQ Configuration

Pinout & Configuration

Clock Configuration

Additional

Options

Categories

A->Z

System Core

Analog

Timers

RTC

TIM2

TIM3

TIM4

TIM5

TIM6

TIM7

TIM9

TIM10

TIM11

Connectivity

Multimedia

Computing

Middleware

RTC Mode and Configuration

Mode

☒ Activate Clock Source

☒ Activate Calendar

Alarm AInternal Alarm A

Alarm BDisable

WakeUpDisable

☐ Timestamp

☐ Tamper 1

☐ Tamper 2

CalibrationDisable

☐ Reference clock detection

Configuration

Reset Configuration

☒ Parameter Settings

☒ User Constants

☒ NVIC Settings

NVIC Interrupt Table	Enabl...	Preemption Pri...	Sub Prio...
RTC alarms A and B interrupt through EXTI li...	☐	0	0

HAL RTC Generic Driver

RTC Firmware driver registers structures

RTC_InitTypeDef

Data Fields

uint32_t HourFormat

uint32_t AsynchPrediv

uint32_t SynchPrediv

uint32_t OutPut

uint32_t OutPutPolarity

uint32_t OutPutType

Field Documentation

uint32_t RTC_InitTypeDef::HourFormat

Specifies the RTC Hour Format. This parameter can be a value of

[RTC_Hour_Formats](#)

uint32_t RTC_InitTypeDef::AsynchPrediv

Specifies the RTC Asynchronous Predivider value. This parameter must be a number between Min_Data = 0x00 and Max_Data = 0x7F

uint32_t RTC_InitTypeDef::SynchPrediv

Specifies the RTC Synchronous Predivider value. This parameter must be a number between

Min_Data = 0x00 and Max_Data = 0x7FFF

uint32_t RTC_InitTypeDef::OutPut

Specifies which signal will be routed to the RTC output. This parameter can be a value of

[*RTCEx_Output_selection_Definitions*](#)

uint32_t RTC_InitTypeDef::OutPutPolarity

Specifies the polarity of the output signal. This parameter can be a value of

[*RTC_Output_Polarity_Definitions*](#)

uint32_t RTC_InitTypeDef::OutPutType

Specifies the RTC Output Pin mode. This parameter can be a value of

[*RTC_Output_Type_ALARM_OUT*](#)

RTC_DateTypeDef

Data Fields

uint8_t WeekDay

uint8_t Month

uint8_t Date

uint8_t Year

Field Documentation

uint8_t RTC_DateTypeDef::WeekDay

Specifies the RTC Date WeekDay. This parameter can be a value of

[*RTC_WeekDay_Definitions*](#)

uint8_t RTC_DateTypeDef::Month

Specifies the RTC Date Month (in BCD format). This parameter can be a value of

[*RTC_Month_Date_Definitions*](#)

uint8_t RTC_DateTypeDef::Date

Specifies the RTC Date. This parameter must be a number between Min_Data = 1 and Max_Data = 31

uint8_t RTC_DateTypeDef::Year

Specifies the RTC Date Year. This parameter must be a number between Min_Data = 0 and Max_Data = 99

RTC_HandleTypeDef

Data Fields

RTC_TypeDef * Instance

RTC_InitTypeDef Init

HAL_LockTypeDef Lock

__IO HAL_RTCStateTypeDef State

Field Documentation

RTC_TypeDef* RTC_HandleTypeDef::Instance

Register base address

RTC_InitTypeDef RTC_HandleTypeDef::Init

RTC required parameters

HAL_LockTypeDef RTC_HandleTypeDef::Lock

RTC locking object

__IO HAL_RTCStateTypeDef RTC_HandleTypeDef::State

Time communication state

RTC Firmware driver API description

Backup Domain Operating Condition

The real-time clock (RTC) and the RTC backup registers can be powered from the VBAT voltage when the main VDD supply is powered off. To retain the content of the RTC backup registers and supply the RTC when VDD is turned off, VBAT pin can be connected to an optional standby voltage supplied by a battery or by another source.

To allow the RTC operating even when the main digital supply (VDD) is turned off, the VBAT pin powers the following blocks:

The RTC

The LSE oscillator

PC13 to PC15 I/Os (when available)

When the backup domain is supplied by VDD (analog switch connected to VDD), the following pins are available:

PC14 and PC15 can be used as either GPIO or LSE pins
PC13 can be used as a GPIO or as the RTC_AF1 pin

When the backup domain is supplied by VBAT (analog switch connected to VBAT because VDD is not present), the following pins are available:

PC14 and PC15 can be used as LSE pins only
PC13 can be used as the RTC_AF1 pin

Backup Domain Reset

The backup domain reset sets all RTC registers and the RCC_BDCR register to their reset values.

A backup domain reset is generated when one of the following events occurs:

Software reset, triggered by setting the BDRST bit in the RCC Backup domain control register (RCC_BDCR).

VDD or VBAT power on, if both supplies have previously been powered off.

Backup Domain Access

After reset, the backup domain (RTC registers, RTC backup data registers and backup SRAM) is protected against possible unwanted write accesses.

To enable access to the RTC Domain and RTC registers, proceed as follows:

Enable the Power Controller (PWR) APB1 interface clock using the
__HAL_RCC_PWR_CLK_ENABLE() function.

Enable access to RTC domain using the HAL_PWR_EnableBkUpAccess() function.

Select the RTC clock source using the HAL_RCC_RTC_CONFIG() function.

Enable RTC Clock using the HAL_RCC_RTC_ENABLE() function.

How to use this driver

Enable the RTC domain access (see description in the section above).

Configure the RTC Prescaler (Asynchronous and Synchronous) and RTC hour format using the HAL_RTC_Init() function.

Time and Date configuration

To configure the RTC Calendar (Time and Date) use the HAL_RTC_SetTime() and HAL_RTC_SetDate() functions.

To read the RTC Calendar, use the HAL_RTC_GetTime() and HAL_RTC_GetDate() functions.

Alarm configuration

To configure the RTC Alarm use the HAL_RTC_SetAlarm() function. You can also configure the RTC Alarm with interrupt mode using the HAL_RTC_SetAlarm_IT() function.

To read the RTC Alarm, use the HAL_RTC_GetAlarm() function.

RTC and low power modes

The MCU can be woken up from a low power mode by an RTC alternate function.

The RTC alternate functions are the RTC alarms (Alarm A and Alarm B), RTC wakeup, RTC tamper event detection and RTC time stamp event detection. These RTC alternate functions can wake up the system from the Stop and Standby low power modes.

The system can also wake up from low power modes without depending on an external interrupt (Auto-wakeup mode), by using the RTC alarm or the RTC wakeup events.

The RTC provides a programmable time base for waking up from the Stop or Standby mode at regular intervals. Wakeup from STOP and STANDBY modes is possible only when the RTC clock source is LSE or LSI.

Initialization and de-initialization functions

This section provides functions allowing to initialize and configure the RTC Prescaler (Synchronous and Asynchronous), RTC Hour format, disable RTC registers Write protection, enter and exit the RTC initialization mode, RTC registers synchronization check and reference clock detection enable.

The RTC Prescaler is programmed to generate the RTC 1Hz time base. It is split into 2 programmable prescalers to minimize power consumption.

A 7-bit asynchronous prescaler and a 13-bit synchronous prescaler.

When both prescalers are used, it is recommended to configure the asynchronous prescaler to a high value to minimize power consumption.

All RTC registers are Write protected. Writing to the RTC registers is enabled by writing a key into the Write Protection register, RTC_WPR.

To configure the RTC Calendar, user application should enter initialization mode. In this mode, the calendar counter is stopped and its value can be updated. When the initialization sequence is complete, the calendar restarts counting after 4 RTCCLK cycles.

To read the calendar through the shadow registers after Calendar initialization, calendar update or after wakeup from low power modes the software must first clear the RSF flag. The software must then wait until it is set again before reading the calendar, which means that the calendar registers have been correctly copied into the RTC_TR and RTC_DR shadow registers. The HAL_RTC_WaitForSynchro() function implements the above software sequence (RSF clear and RSF check).

This section contains the following APIs:

[*HAL_RTC_Init\(\)*](#)
[*HAL_RTC_DeInit\(\)*](#)
[*HAL_RTC_MspInit\(\)*](#)
[*HAL_RTC_MspDeInit\(\)*](#)
[*HAL_RTC_SetTime\(\)*](#)
[*HAL_RTC_GetTime\(\)*](#)
[*HAL_RTC_SetDate\(\)*](#)
[*HAL_RTC_GetDate\(\)*](#)

RTC Time and Date functions

This section provides functions allowing to configure Time and Date features This section contains the following APIs:

[*HAL_RTC_SetTime\(\)*](#)
[*HAL_RTC_SetDate\(\)*](#)
[*HAL_RTC_GetDate\(\)*](#)
[*HAL_RTC_GetTime\(\)*](#)
[*HAL_RTC_SetAlarm\(\)*](#)
[*HAL_RTC_SetAlarm_IT\(\)*](#)
[*HAL_RTC_DeactivateAlarm\(\)*](#)
[*HAL_RTC_GetAlarm\(\)*](#)
[*HAL_RTC_AlarmIRQHandler\(\)*](#)
[*HAL_RTC_PollForAlarmAEvent\(\)*](#)
[*HAL_RTC_AlarmAEventCallback\(\)*](#)

RTC Alarm functions

This section provides functions allowing to configure Alarm feature

This section contains the following APIs:

[*HAL_RTC_DeactivateAlarm\(\)*](#)
[*HAL_RTC_AlarmIRQHandler\(\)*](#)
[*HAL_RTC_AlarmAEventCallback\(\)*](#)
[*HAL_RTC_PollForAlarmAEvent\(\)*](#)
[*HAL_RTC_SetAlarm\(\)*](#)
[*HAL_RTC_SetAlarm_IT\(\)*](#)
[*HAL_RTC_GetAlarm\(\)*](#)
[*HAL_RTC_WaitForSynchro\(\)*](#)

Peripheral State functions

This subsection provides functions allowing to

Get RTC state

This section contains the following APIs:

[**HAL_RTC_GetState\(\)**](#)

Peripheral Control functions

This subsection provides functions allowing to

Wait for RTC Time and Date Synchronization This

section contains the following APIs:

[**HAL_RTC_WaitForSynchro\(\)**](#)

Detailed description of functions

HAL_RTC_Init

Function name	HAL_StatusTypeDef HAL_RTC_Init (RTC_HandleTypeDef * hrtc)
Function description	Initializes the RTC peripheral.
Parameters	hrtc : pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
Return values	HAL : status

HAL_RTC_DeInit

Function name	HAL_StatusTypeDef HAL_RTC_DeInit (RTC_HandleTypeDef * hrtc)
Function description	Deinitializes the RTC peripheral.
Parameters	hrtc : pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC. hrtc : pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
Return values	HAL : status HAL : status
Notes	This function doesn't reset the RTC Backup Data registers. This function does not reset the RTC Backup Data registers.

HAL_RTC_MspInit

Function name	void HAL_RTC_MspInit (RTC_HandleTypeDef * hrtc)
Function description	Initializes the RTC MSP.
Parameters	hrtc : pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
Return values	None :

HAL_RTC_MspDeInit

Function name	void HAL_RTC_MspDeInit (RTC_HandleTypeDef * hrtc)
Function description	Deinitializes the RTC MSP.
Parameters	hrtc : pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
Return values	None :

HAL_RTC_SetTime

Function name	HAL_StatusTypeDef HAL_RTC_SetTime (RTC_HandleTypeDef * hrtc, RTC_TimeTypeDef * sTime, uint32_t Format)
Function description	Sets RTC current time.
Parameters	hrtc: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC. sTime: Pointer to Time structure Format: Specifies the format of the entered parameters. This parameter can be one of the following values: RTC_FORMAT_BIN: Binary data format RTC_FORMAT_BCD: BCD data format
Return values	HAL: status

HAL_RTC_GetTime

Function name	HAL_StatusTypeDef HAL_RTC_GetTime (RTC_HandleTypeDef * hrtc, RTC_TimeTypeDef * sTime, uint32_t Format)
Function description	Get RTC current time.
Parameters	hrtc: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC. sTime: Pointer to Time structure with Hours, Minutes and Seconds fields returned with input format (BIN or BCD), also SubSeconds field (if available) returning the RTC_SSR register content and SecondFraction field the Synchronous pre-scaler factor to be used for second fraction ratio computation. Format: Specifies the format of the entered parameters. This parameter can be one of the following values: RTC_FORMAT_BIN: Binary data format RTC_FORMAT_BCD: BCD data format
Return values	HAL: status
Notes	If available, you can use SubSeconds and SecondFraction (sTime structure fields returned) to convert SubSeconds value in second fraction ratio with time unit following generic formula: $\text{Second fraction ratio} * \text{time_unit} = \left[\frac{\text{SecondFraction} - \text{SubSeconds}}{\text{SecondFraction} + 1} \right] * \text{time_unit}$ This conversion can be performed only if no shift operation is pending (ie. SHFP=0) when PREDIV_S >= SS You must call HAL_RTC_GetDate() after HAL_RTC_GetTime() to unlock the values in the higher-order calendar shadow registers to ensure consistency between the time and date values. Reading RTC current time locks the values in calendar shadow registers until Current date is read to ensure consistency between the time and date values.

HAL_RTC_SetDate

Function name	HAL_StatusTypeDef HAL_RTC_SetDate (RTC_HandleTypeDef * hrtc, RTC_DateTypeDef * sDate, uint32_t Format)
Function description	Sets RTC current date.
Parameters	hrtc: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC. sDate: Pointer to date structure Format: specifies the format of the entered parameters. This parameter can be one of the following values: RTC_FORMAT_BIN: Binary data format RTC_FORMAT_BCD: BCD data format
Return values	HAL: status

HAL_RTC_GetDate

Function name	HAL_StatusTypeDef HAL_RTC_GetDate (RTC_HandleTypeDef * hrtc, RTC_DateTypeDef * sDate, uint32_t Format)
Function description	Gets RTC current date.
Parameters	hrtc : pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC. sDate : Pointer to Date structure Format : Specifies the format of the entered parameters. This parameter can be one of the following values: RTC_FORMAT_BIN: Binary data format RTC_FORMAT_BCD: BCD data format
Return values	HAL : status
Notes	You must call HAL_RTC_GetDate() after HAL_RTC_GetTime() to unlock the values in the higher-order calendar shadow registers to ensure consistency between the time and date values. Reading RTC current time locks the values in calendar shadow registers until Current date is read.

HAL_RTC_SetAlarm

Function name	HAL_StatusTypeDef HAL_RTC_SetAlarm (RTC_HandleTypeDef * hrtc, RTC_AlarmTypeDef * sAlarm, uint32_t Format)
Function description	Sets the specified RTC Alarm.
Parameters	hrtc : pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC. sAlarm : Pointer to Alarm structure Format : Specifies the format of the entered parameters. This parameter can be one of the following values: RTC_FORMAT_BIN: Binary data format RTC_FORMAT_BCD: BCD data format
Return values	HAL : status

HAL_RTC_SetAlarm_IT

Function name	HAL_StatusTypeDef HAL_RTC_SetAlarm_IT (RTC_HandleTypeDef * hrtc, RTC_AlarmTypeDef * sAlarm, uint32_t Format)
Function description	Sets the specified RTC Alarm with Interrupt.
Parameters	hrtc : pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC. sAlarm : Pointer to Alarm structure Format : Specifies the format of the entered parameters. This parameter can be one of the following values: RTC_FORMAT_BIN: Binary data format RTC_FORMAT_BCD: BCD data format
Return values	HAL : status
Notes	The Alarm register can only be written when the corresponding Alarm is disabled (Use the HAL_RTC_DeactivateAlarm()). The HAL_RTC_SetTime() must be called before enabling the Alarm feature.

HAL_RTC_DeactivateAlarm

Function name	HAL_StatusTypeDef HAL_RTC_DeactivateAlarm (RTC_HandleTypeDef * hrtc, uint32_t Alarm)
Function description	Deactive the specified RTC Alarm.
Parameters	hrtc: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC. Alarm: Specifies the Alarm. This parameter can be one of the following values: RTC_ALARM_A: AlarmA RTC_ALARM_B: AlarmB
Return values	HAL: status

HAL_RTC_GetAlarm

Function name	HAL_StatusTypeDef HAL_RTC_GetAlarm (RTC_HandleTypeDef * hrtc, RTC_AlarmTypeDef * sAlarm, uint32_t Alarm, uint32_t Format)
Function description	Gets the RTC Alarm value and masks.
Parameters	hrtc: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC. sAlarm: Pointer to Date structure Alarm: Specifies the Alarm. This parameter can be one of the following values: RTC_ALARM_A: AlarmA RTC_ALARM_B: AlarmB Format: Specifies the format of the entered parameters. This parameter can be one of the following values: RTC_FORMAT_BIN: Binary data format RTC_FORMAT_BCD: BCD data format
Return values	HAL: status

HAL_RTC_AlarmIRQHandler

Function name	void HAL_RTC_AlarmIRQHandler (RTC_HandleTypeDef * hrtc)
Function description	This function handles Alarm interrupt request.
Parameters	hrtc: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
Return values	None:

HAL_RTC_PollForAlarmAEvent

Function name	HAL_StatusTypeDef HAL_RTC_PollForAlarmAEvent (RTC_HandleTypeDef * hrtc, uint32_t Timeout)
Function description	This function handles AlarmA Polling request.
Parameters	hrtc: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC. Timeout: Timeout duration
Return values	HAL: status

HAL_RTC_AlarmAEventCallback

Function name	void HAL_RTC_AlarmAEventCallback (RTC_HandleTypeDef * hrtc)
Function description	Alarm A callback.
Parameters	hrtc: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values **None:**

HAL_RTC_GetState

Function name **HAL_RTCStateTypeDef HAL_RTC_GetState (RTC_HandleTypeDef * hrtc)**

Function description Returns the RTC state.

Parameters **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values **HAL:** state

RTC_EnterInitMode

Function name **HAL_StatusTypeDef RTC_EnterInitMode (RTC_HandleTypeDef * hrtc)**

Function description Enters the RTC Initialization mode.

Parameters **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values **HAL:** status

Notes The RTC Initialization mode is write protected, use the `__HAL_RTC_WRITEPROTECTION_DISABLE()` before calling this function.

RTC_ByteToBcd2

Function name **uint8_t RTC_ByteToBcd2 (uint8_t Value)**

Function description Converts a 2 digit decimal to BCD format.

Parameters **Value:** Byte to be converted

Return values **Converted:** byte

RTC_Bcd2ToByte

Function name **uint8_t RTC_Bcd2ToByte (uint8_t Value)**

Function description Converts from 2 digit BCD to Binary.

Parameters **Value:** BCD value to be converted

Return values **Converted:** word

RTC Firmware driver defines

RTC

AlarmDateWeekDay Definitions

RTC_ALARMDATEWEEKDAYSEL_DATE
RTC_ALARMDATEWEEKDAYSEL_WEEKDAY
IS_RTC_ALARM_DATE_WEEKDAY_SEL

Alarm Mask Definitions

RTC_ALARMMASK_NONE
RTC_ALARMMASK_DATEWEEKDAY
RTC_ALARMMASK_HOURS
RTC_ALARMMASK_MINUTES
RTC_ALARMMASK_SECONDS

RTC_ALARM_MASK_ALL

IS_RTC_ALARM_MASK

Alarms Definitions

RTC_ALARM_A

RTC_ALARM_B

IS_RTC_ALARM

Alarm Definitions

IS_RTC_ALARM_DATE_WEEKDAY_DATE

IS_RTC_ALARM_DATE_WEEKDAY_WEEKDAY

Alarm Sub Seconds Value

IS_RTC_ALARM_SUB_SECOND_VALUE

AM PM Definitions

RTC_HOURFORMAT12_AM

RTC_HOURFORMAT12_PM

IS_RTC_HOURFORMAT12

Asynchronous Predivider

IS_RTC_ASYNC_PREDIV

DayLightSaving

RTC_DAYLIGHTSAVING_SUB1H

RTC_DAYLIGHTSAVING_ADD1H

RTC_DAYLIGHTSAVING_NONE

IS_RTC_DAYLIGHT_SAVING

RTC Exported Macros

__HAL_RTC_RESET_HANDLE_STATE

Description: Reset RTC handle state.

Parameters:

__HANDLE : RTC handle.

Return value: None

__HAL_RTC_WRITEPROTECTION_DISABLE

Description: Disable the write protection for RTC registers.

Parameters:

__HANDLE : specifies the RTC handle.

Return value: None

__HAL_RTC_WRITEPROTECTION_ENABLE

Description: Enable the write protection for RTC registers.

Parameters:

__HANDLE : specifies the RTC handle.

Return value: None

__HAL_RTC_ALARM_ENABLE

Description: Enable the RTC ALARMA peripheral.

Parameters:

__HANDLE : specifies the RTC handle.

Return value: None

__HAL_RTC_ALARM_DISABLE

Description: Disable the RTC ALARMA peripheral.

Parameters:

__HANDLE : specifies the RTC handle.

Return value: None

__HAL_RTC_ALARMB_ENABLE

Description: Enable the RTC ALARMB peripheral.

Parameters:

__HANDLE : specifies the RTC handle.

Return value: None

__HAL_RTC_ALARMB_DISABLE

Description: Disable the RTC ALARMB peripheral.

Parameters:

__HANDLE : specifies the RTC handle.

Return value: None

__HAL_RTC_ALARM_ENABLE_IT

Description: Enable the RTC Alarm interrupt.

Parameters:

__HANDLE : specifies the RTC handle.

__INTERRUPT : specifies the RTC Alarm interrupt sources to be enabled or disabled. This parameter can be any combination of the following values:

RTC_IT_ALRA: Alarm A interrupt

RTC_IT_ALRB: Alarm B interrupt

Return value: None

__HAL_RTC_ALARM_DISABLE_IT

Description: Disable the RTC Alarm interrupt.

Parameters:

__HANDLE : specifies the RTC handle.

__INTERRUPT : specifies the RTC Alarm interrupt sources to be enabled or disabled. This parameter can be any combination of the following values:

RTC_IT_ALRA: Alarm A interrupt

RTC_IT_ALRB: Alarm B interrupt

Return value: None

__HAL_RTC_ALARM_GET_IT

Description: Check whether the specified RTC Alarm interrupt has occurred or not.

Parameters:

__HANDLE : specifies the RTC handle.

__INTERRUPT : specifies the RTC Alarm interrupt sources to check. This parameter can be:

RTC_IT_ALRA: Alarm A interrupt

RTC_IT_ALRB: Alarm B interrupt

Return value: None

__HAL_RTC_ALARM_GET_IT_SOURCE

Description: Check whether the specified RTC Alarm interrupt has been enabled or not.

Parameters:

__HANDLE : specifies the RTC handle.

__INTERRUPT : specifies the RTC Alarm interrupt sources to check. This parameter can be:

RTC_IT_ALRA: Alarm A interrupt

RTC_IT_ALRB: Alarm B interrupt

Return value: None

__HAL_RTC_ALARM_GET_FLAG

Description: Get the selected RTC Alarm's flag status.

Parameters:

__HANDLE : specifies the RTC handle.
__FLAG : specifies the RTC Alarm Flag sources to check. This parameter can be:
RTC_FLAG_ALRAF
RTC_FLAG_ALRBF
RTC_FLAG_ALRAWF
RTC_FLAG_ALRBWF

Return value: None

__HAL_RTC_ALARM_CLEAR_FLAG

Description: Clear the RTC Alarm's pending flags.

Parameters:

__HANDLE : specifies the RTC handle.
__FLAG : specifies the RTC Alarm Flag sources to clear. This parameter can be:
RTC_FLAG_ALRAF
RTC_FLAG_ALRBF

Return value: None

Flags Definitions

RTC_FLAG_RECALPF, RTC_FLAG_TAMP3F, RTC_FLAG_TAMP2F, RTC_FLAG_TAMP1F,
RTC_FLAG_TSOVF, RTC_FLAG_TSF, RTC_FLAG_WUTF, RTC_FLAG_ALRBF,
RTC_FLAG_ALRAF, RTC_FLAG_INITF, RTC_FLAG_RSF, RTC_FLAG_INITS,
RTC_FLAG_SHPF, RTC_FLAG_WUTWF, RTC_FLAG_ALRBWF, RTC_FLAG_ALRAWF

Hour Formats

RTC_HOURFORMAT_24
RTC_HOURFORMAT_12
IS_RTC_HOUR_FORMAT

Input Parameter Format

RTC_FORMAT_BIN
RTC_FORMAT_BCD
IS_RTC_FORMAT

Interrupts Definitions

RTC_IT_TS, RTC_IT_WUT, RTC_IT_ALRB, RTC_IT_ALRA, RTC_IT_TAMP1,
RTC_IT_TAMP2, RTC_IT_TAMP3

Month Definitions

RTC_MONTH_JANUARY, RTC_MONTH_FEBRUARY, RTC_MONTH_MARCH,
RTC_MONTH_APRIL, RTC_MONTH_MAY, RTC_MONTH_JUNE, RTC_MONTH_JULY,
RTC_MONTH_AUGUST, RTC_MONTH_SEPTEMBER, RTC_MONTH_OCTOBER,
RTC_MONTH_NOVEMBER, RTC_MONTH_DECEMBER, IS_RTC_MONTH, IS_RTC_DATE

Output Polarity

RTC_OUTPUT_POLARITY_HIGH

RTC_OUTPUT_POLARITY_LOW

IS_RTC_OUTPUT_POL

Alarm Output Type

RTC_OUTPUT_TYPE_OPENDRAIN

RTC_OUTPUT_TYPE_PUSH_PULL

IS_RTC_OUTPUT_TYPE

Synchronous Predivider

IS_RTC_SYNCH_PREDIV

Default Timeout Value

RTC_TIMEOUT_VALUE

Time Definitions

IS_RTC_HOUR12

IS_RTC_HOUR24

IS_RTC_MINUTES

IS_RTC_SECONDS

WeekDay Definitions

RTC_WEEKDAY_MONDAY, RTC_WEEKDAY_TUESDAY, RTC_WEEKDAY_WEDNESDAY,
RTC_WEEKDAY_THURSDAY, RTC_WEEKDAY_FRIDAY, RTC_WEEKDAY_SATURDAY,
RTC_WEEKDAY_SUNDAY, IS_RTC_WEEKDAY

Year Definitions

IS_RTC_YEAR

HAL I2C Generic Driver

I2C Firmware driver registers structures

I2C_InitTypeDef

Data Fields

uint32_t ClockSpeed
uint32_t DutyCycle
uint32_t OwnAddress1
uint32_t AddressingMode
uint32_t DualAddressMode
uint32_t OwnAddress2
uint32_t GeneralCallMode
uint32_t NoStretchMode

Field Documentation

uint32_t I2C_InitTypeDef::ClockSpeed

Specifies the clock frequency. This parameter must be set to a value lower than 400kHz

uint32_t I2C_InitTypeDef::DutyCycle

Specifies the I2C fast mode duty cycle. This parameter can be a value of [I2C_duty_cycle_in_fast_mode](#)

uint32_t I2C_InitTypeDef::OwnAddress1

Specifies the first device own address. This parameter can be a 7-bit or 10-bit address.

uint32_t I2C_InitTypeDef::AddressingMode

Specifies if 7-bit or 10-bit addressing mode is selected. This parameter can be a value of [I2C_addressing_mode](#)

uint32_t I2C_InitTypeDef::DualAddressMode

Specifies if dual addressing mode is selected. This parameter can be a value of [I2C_dual_addressing_mode](#)

uint32_t I2C_InitTypeDef::OwnAddress2

Specifies the second device own address if dual addressing mode is selected This parameter can be a 7-bit address.

uint32_t I2C_InitTypeDef::GeneralCallMode

Specifies if general call mode is selected. This parameter can be a value of [I2C_general_call_addressing_mode](#)

uint32_t I2C_InitTypeDef::NoStretchMode

Specifies if nostretch mode is selected. This parameter can be a value of [I2C_nostretch_mode](#)

I2C_HandleTypeDef

Data Fields

*I2C_TypeDef * Instance*
I2C_InitTypeDef Init
*uint8_t * pBuffPtr*
uint16_t XferSize
__IO uint16_t XferCount
__IO uint32_t XferOptions
__IO uint32_t PreviousState
*DMA_HandleTypeDef * hdmatx*
*DMA_HandleTypeDef * hdmarx*
HAL_LockTypeDef Lock
__IO HAL_I2C_StateTypeDef State
__IO HAL_I2C_ModeTypeDef Mode
__IO uint32_t ErrorCode
__IO uint32_t Devaddress
__IO uint32_t Memaddress
__IO uint32_t MemaddSize
__IO uint32_t EventCount

Field Documentation

I2C_TypeDef* I2C_HandleTypeDef::Instance
I2C registers base address

I2C_InitTypeDef I2C_HandleTypeDef::Init
I2C communication parameters

uint8_t* I2C_HandleTypeDef::pBuffPtr
Pointer to I2C transfer buffer

uint16_t I2C_HandleTypeDef::XferSize
I2C transfer size

__IO uint16_t I2C_HandleTypeDef::XferCount
I2C transfer counter

__IO uint32_t I2C_HandleTypeDef::XferOptions
I2C transfer options

__IO uint32_t I2C_HandleTypeDef::PreviousState
I2C communication Previous state and mode context for internal usage

DMA_HandleTypeDef* I2C_HandleTypeDef::hdmatx
I2C Tx DMA handle parameters

DMA_HandleTypeDef* I2C_HandleTypeDef::hdmarx
I2C Rx DMA handle parameters

HAL_LockTypeDef I2C_HandleTypeDef::Lock
I2C locking object

__IO HAL_I2C_StateTypeDef I2C_HandleTypeDef::State
I2C communication state

__IO HAL_I2C_ModeTypeDef I2C_HandleTypeDef::Mode
I2C communication mode

__IO uint32_t I2C_HandleTypeDef::ErrorCode
I2C Error code

__IO uint32_t I2C_HandleTypeDef::Devaddress
I2C Target device address

__IO uint32_t I2C_HandleTypeDef::Memaddress
I2C Target memory address

__IO uint32_t I2C_HandleTypeDef::MemaddSize
I2C Target memory address size

__IO uint32_t I2C_HandleTypeDef::EventCount
I2C Event counter

I2C Firmware driver API description

How to use this driver

The I2C HAL driver can be used as follows:

Declare a I2C_HandleTypeDef handle structure, for example: I2C_HandleTypeDef hi2c;
Initialize the I2C low level resources by implementing the HAL_I2C_MspInit() API:

- Enable the I2Cx interface clock

- I2C pins configuration

 - Enable the clock for the I2C GPIOs

 - Configure I2C pins as alternate function open-drain

- NVIC configuration if you need to use interrupt process

 - Configure the I2Cx interrupt priority

 - Enable the NVIC I2C IRQ Channel

Configure the Communication Speed, Duty cycle, Addressing mode, Own Address1, Dual Addressing mode, Own Address2, General call and Nostretch mode in the hi2c Init structure.

Initialize the I2C registers by calling the HAL_I2C_Init(), configures also the low level

- Hardware (GPIO, CLOCK, NVIC...etc) by calling the customized

- HAL_I2C_MspInit(&hi2c) API.

To check if target device is ready for communication, use the function

- HAL_I2C_IsDeviceReady()

For I2C IO and IO MEM operations, three operation modes are available within this driver :

Polling mode IO operation

Transmit in master mode an amount of data in blocking mode using

- HAL_I2C_Master_Transmit()

Receive in master mode an amount of data in blocking mode using
HAL_I2C_Master_Receive()

Polling mode IO MEM operation

Write an amount of data in blocking mode to a specific memory address using
HAL_I2C_Mem_Write()

Read an amount of data in blocking mode from a specific memory address using
HAL_I2C_Mem_Read()

Interrupt mode IO operation

Transmit in master mode an amount of data in non-blocking mode using
HAL_I2C_Master_Transmit_IT()

At transmission end of transfer, HAL_I2C_MasterTxCpltCallback() is executed and user can add his own code by customization of function pointer
HAL_I2C_MasterTxCpltCallback()

Receive in master mode an amount of data in non-blocking mode using
HAL_I2C_Master_Receive_IT()

At reception end of transfer, HAL_I2C_MasterRxCpltCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_MasterRxCpltCallback()

In case of transfer Error, HAL_I2C_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_I2C_ErrorCallback()

Abort a master I2C process communication with Interrupt using
HAL_I2C_Master_Abort_IT()

End of abort process, HAL_I2C_AbortCpltCallback() is executed and user can add his own code by customization of function pointer HAL_I2C_AbortCpltCallback()

I2C HAL driver macros list

Below the list of most used macros in I2C HAL driver.

__HAL_I2C_ENABLE: Enable the I2C peripheral

__HAL_I2C_DISABLE: Disable the I2C peripheral

__HAL_I2C_GENERATE_NACK: Generate a Non-Acknowledge I2C peripheral in Slave mode

__HAL_I2C_GET_FLAG: Check whether the specified I2C flag is set or not

__HAL_I2C_CLEAR_FLAG: Clear the specified I2C pending flag

__HAL_I2C_ENABLE_IT: Enable the specified I2C interrupt

__HAL_I2C_DISABLE_IT: Disable the specified I2C interrupt

Initialization and de-initialization functions

This subsection provides a set of functions allowing to initialize and deinitialize the I2Cx peripheral:

User must Implement HAL_I2C_MspInit() function in which he configures all related peripherals resources (CLOCK, GPIO, DMA, IT and NVIC).

Call the function HAL_I2C_Init() to configure the selected device with the selected configuration:

Communication Speed

Duty cycle

Addressing mode

Own Address 1

Dual Addressing mode

Own Address 2

General call mode

Nostretch mode

Call the function HAL_I2C_DeInit() to restore the default configuration of the selected I2Cx peripheral.

This section contains the following APIs:

[**HAL_I2C_Init\(\)**](#)

[**HAL_I2C_DeInit\(\)**](#)

[*HAL_I2C_MspInit\(\)*](#)
[*HAL_I2C_MspDeInit\(\)*](#)

IO operation functions

This subsection provides a set of functions allowing to manage the I2C data transfers.

There are two modes of transfer:

Blocking mode : The communication is performed in the polling mode. The status of all data processing is returned by the same function after finishing transfer.

No-Blocking mode : The communication is performed using Interrupts or DMA. These functions return the status of the transfer startup. The end of the data processing will be indicated through the dedicated I2C IRQ when using Interrupt mode or the DMA IRQ when using DMA mode.

Blocking mode functions are :

`HAL_I2C_Master_Transmit()`
`HAL_I2C_Master_Receive()`
`HAL_I2C_Mem_Write()`
`HAL_I2C_Mem_Read()`
`HAL_I2C_IsDeviceReady()`

No-Blocking mode functions with Interrupt are :

`HAL_I2C_Master_Transmit_IT()`
`HAL_I2C_Master_Receive_IT()`
`HAL_I2C_Master_Sequential_Transmit_IT()`
`HAL_I2C_Master_Sequential_Receive_IT()`
`HAL_I2C_Mem_Write_IT()`
`HAL_I2C_Mem_Read_IT()`

This section contains the following APIs:

[*HAL_I2C_Master_Transmit\(\)*](#)
[*HAL_I2C_Master_Receive\(\)*](#)
[*HAL_I2C_Master_Transmit_IT\(\)*](#)
[*HAL_I2C_Master_Receive_IT\(\)*](#)
[*HAL_I2C_Master_Sequential_Transmit_IT\(\)*](#)
[*HAL_I2C_Master_Sequential_Receive_IT\(\)*](#)
[*HAL_I2C_Master_Abort_IT\(\)*](#)
[*HAL_I2C_Mem_Write\(\)*](#)
[*HAL_I2C_Mem_Read\(\)*](#)
[*HAL_I2C_Mem_Write_IT\(\)*](#)
[*HAL_I2C_Mem_Read_IT\(\)*](#)
[*HAL_I2C_IsDeviceReady\(\)*](#)
[*HAL_I2C_EV_IRQHandler\(\)*](#)
[*HAL_I2C_ER_IRQHandler\(\)*](#)
[*HAL_I2C_MasterTxCpltCallback\(\)*](#)
[*HAL_I2C_MasterRxCpltCallback\(\)*](#)
[*HAL_I2C_ListenCpltCallback\(\)*](#)
[*HAL_I2C_MemTxCpltCallback\(\)*](#)
[*HAL_I2C_MemRxCpltCallback\(\)*](#)
[*HAL_I2C_ErrorCallback\(\)*](#)
[*HAL_I2C_AbortCpltCallback\(\)*](#)

Peripheral State, Mode and Error functions

This subsection permits to get in run-time the status of the peripheral and the data flow.

This section contains the following APIs:

[*HAL_I2C_GetState\(\)*](#)
[*HAL_I2C_GetMode\(\)*](#)
[*HAL_I2C_GetError\(\)*](#)

Detailed description of functions

HAL_I2C_Init

Function name	HAL_StatusTypeDef HAL_I2C_Init (I2C_HandleTypeDef * hi2c)
Function description	Initializes the I2C according to the specified parameters in the I2C_InitTypeDef and initialize the associated handle.
Parameters	hi2c: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
Return values	HAL: status

HAL_I2C_DeInit

Function name	HAL_StatusTypeDef HAL_I2C_DeInit (I2C_HandleTypeDef * hi2c)
Function description	DeInitialize the I2C peripheral.
Parameters	hi2c: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
Return values	HAL: status

HAL_I2C_MspInit

Function name	void HAL_I2C_MspInit (I2C_HandleTypeDef * hi2c)
Function description	Initialize the I2C MSP.
Parameters	hi2c: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
Return values	None:

HAL_I2C_MspDeInit

Function name	void HAL_I2C_MspDeInit (I2C_HandleTypeDef * hi2c)
Function description	DeInitialize the I2C MSP.
Parameters	hi2c: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
Return values	None:

HAL_I2C_Master_Transmit

Function name	HAL_StatusTypeDef HAL_I2C_Master_Transmit (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t Timeout)
Function description	Transmits in master mode an amount of data in blocking mode.
Parameters	hi2c: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C. DevAddress: Target device address: The device 7 bits address value in datasheet must be shift at right before call interface pData: Pointer to data buffer Size: Amount of data to be sent Timeout: Timeout duration
Return values	HAL: status

HAL_I2C_Master_Receive

Function name	HAL_StatusTypeDef HAL_I2C_Master_Receive (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t Timeout)
Function description	Receives in master mode an amount of data in blocking mode.

Parameters	hi2c: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C. DevAddress: Target device address: The device 7 bits address value in datasheet must be shift at right before call interface pData: Pointer to data buffer Size: Amount of data to be sent Timeout: Timeout duration
Return values	HAL: status

HAL_I2C_Mem_Write

Function name	HAL_StatusTypeDef HAL_I2C_Mem_Write (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size, uint32_t Timeout)
Function description	Write an amount of data in blocking mode to a specific memory address.
Parameters	hi2c: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C. DevAddress: Target device address: The device 7 bits address value in datasheet must be shift at right before call interface MemAddress: Internal memory address MemAddSize: Size of internal memory address pData: Pointer to data buffer Size: Amount of data to be sent Timeout: Timeout duration
Return values	HAL: status

HAL_I2C_Mem_Read

Function name	HAL_StatusTypeDef HAL_I2C_Mem_Read (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size, uint32_t Timeout)
Function description	Read an amount of data in blocking mode from a specific memory address.
Parameters	hi2c: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C. DevAddress: Target device address: The device 7 bits address value in datasheet must be shift at right before call interface MemAddress: Internal memory address MemAddSize: Size of internal memory address pData: Pointer to data buffer Size: Amount of data to be sent Timeout: Timeout duration
Return values	HAL: status

HAL_I2C_IsDeviceReady

Function name	HAL_StatusTypeDef HAL_I2C_IsDeviceReady (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint32_t Trials, uint32_t Timeout)
Function description	Checks if target device is ready for communication.

Parameters	hi2c: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C. DevAddress: Target device address: The device 7 bits address value in datasheet must be shift at right before call interface Trials: Number of trials Timeout: Timeout duration
Return values	HAL: status
Notes	This function is used with Memory devices

HAL_I2C_Master_Transmit_IT

Function name	HAL_StatusTypeDef HAL_I2C_Master_Transmit_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size)
Function description	Transmit in master mode an amount of data in non-blocking mode with Interrupt.
Parameters	hi2c: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C. DevAddress: Target device address: The device 7 bits address value in datasheet must be shift at right before call interface pData: Pointer to data buffer Size: Amount of data to be sent
Return values	HAL: status

HAL_I2C_Master_Receive_IT

Function name	HAL_StatusTypeDef HAL_I2C_Master_Receive_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size)
Function description	Receive in master mode an amount of data in non-blocking mode with Interrupt.
Parameters	hi2c: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C. DevAddress: Target device address: The device 7 bits address value in datasheet must be shift at right before call interface pData: Pointer to data buffer Size: Amount of data to be sent
Return values	HAL: status

HAL_I2C_Mem_Write_IT

Function name	HAL_StatusTypeDef HAL_I2C_Mem_Write_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size)
Function description	Write an amount of data in non-blocking mode with Interrupt to a specific memory address.
Parameters	hi2c: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C. DevAddress: Target device address: The device 7 bits address value in datasheet must be shift at right before call interface MemAddress: Internal memory address MemAddSize: Size of internal memory address

pData: Pointer to data buffer
Size: Amount of data to be sent

Return values **HAL:** status

HAL_I2C_Mem_Read_IT

Function name **HAL_StatusTypeDef HAL_I2C_Mem_Read_IT**
(I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t
MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t
Size)

Function description Read an amount of data in non-blocking mode with Interrupt from a specific memory address.

Parameters **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
DevAddress: Target device address: The device 7 bits address value in datasheet must be shift at right before call interface
MemAddress: Internal memory address
MemAddSize: Size of internal memory address
pData: Pointer to data buffer
Size: Amount of data to be sent

Return values **HAL:** status

HAL_I2C_Master_Abort_IT

Function name **HAL_StatusTypeDef HAL_I2C_Master_Abort_IT**
(I2C_HandleTypeDef * hi2c, uint16_t DevAddress)

Function description Abort a master I2C process communication with Interrupt.

Parameters **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
DevAddress: Target device address: The device 7 bits address value in datasheet must be shift at right before call interface

Return values **HAL:** status

Notes This abort can be called only if state is ready

HAL_I2C_EV_IRQHandler

Function name **void HAL_I2C_EV_IRQHandler (I2C_HandleTypeDef * hi2c)**

Function description This function handles I2C event interrupt request.

Parameters **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values **None:**

HAL_I2C_ER_IRQHandler

Function name **void HAL_I2C_ER_IRQHandler (I2C_HandleTypeDef * hi2c)**

Function description This function handles I2C error interrupt request.

Parameters **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values **None:**

HAL_I2C_MasterTxCpltCallback

Function name	void HAL_I2C_MasterTxCpltCallback (I2C_HandleTypeDef * hi2c)
Function description	Master Tx Transfer completed callback.
Parameters	hi2c : Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
Return values	None:

HAL_I2C_MasterRxCpltCallback

Function name	void HAL_I2C_MasterRxCpltCallback (I2C_HandleTypeDef * hi2c)
Function description	Master Rx Transfer completed callback.
Parameters	hi2c : Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
Return values	None:

HAL_I2C_MemTxCpltCallback

Function name	void HAL_I2C_MemTxCpltCallback (I2C_HandleTypeDef * hi2c)
Function description	Memory Tx Transfer completed callback.
Parameters	hi2c : Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
Return values	None:

HAL_I2C_MemRxCpltCallback

Function name	void HAL_I2C_MemRxCpltCallback (I2C_HandleTypeDef * hi2c)
Function description	Memory Rx Transfer completed callback.
Parameters	hi2c : Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
Return values	None:

HAL_I2C_ErrorCallback

Function name	void HAL_I2C_ErrorCallback (I2C_HandleTypeDef * hi2c)
Function description	I2C error callback.
Parameters	hi2c : Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
Return values	None:

HAL_I2C_AbortCpltCallback

Function name	void HAL_I2C_AbortCpltCallback (I2C_HandleTypeDef * hi2c)
Function description	I2C abort callback.
Parameters	hi2c : Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
Return values	None:

HAL_I2C_GetState

Function name	HAL_I2C_StateTypeDef HAL_I2C_GetState (I2C_HandleTypeDef * hi2c)
Function description	Return the I2C handle state.
Parameters	hi2c: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
Return values	HAL: state

HAL_I2C_GetMode

Function name	HAL_I2C_ModeTypeDef HAL_I2C_GetMode (I2C_HandleTypeDef * hi2c)
Function description	Return the I2C Master, Slave, Memory or no mode.
Parameters	hi2c: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for I2C module
Return values	HAL: mode

HAL_I2C_GetError

Function name	uint32_t HAL_I2C_GetError (I2C_HandleTypeDef * hi2c)
Function description	Return the I2C error code.
Parameters	hi2c: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
Return values	I2C: Error Code

I2C Firmware driver defines

I2C

I2C addressing mode

I2C_ADDRESSINGMODE_7BIT
I2C_ADDRESSINGMODE_10BIT

I2C duty cycle in fast mode

I2C_DUTYCYCLE_2
I2C_DUTYCYCLE_16_9

I2C Error Code definition

HAL_I2C_ERROR_NONE	No error
HAL_I2C_ERROR_BERR	BERR error
HAL_I2C_ERROR_ARLO	ARLO error
HAL_I2C_ERROR_AF	AF error
HAL_I2C_ERROR_OVR	OVR error
HAL_I2C_ERROR_TIMEOUT	Timeout Error

I2C Exported Macros

__HAL_I2C_RESET_HANDLE_STATE	Description: Reset I2C handle state.
	Parameters:
	__HANDLE__ : specifies the I2C Handle.
	Return value: None

__HAL_I2C_ENABLE_IT

Description: Enable the specified I2C interrupt.

Parameters:

__HANDLE__ : specifies the I2C Handle.
__INTERRUPT : specifies the interrupt source to enable. This parameter can be one of the following values:
I2C_IT_BUF: Buffer interrupt enable
I2C_IT_EVT: Event interrupt enable
I2C_IT_ERR: Error interrupt enable

Return value: None

__HAL_I2C_DISABLE_IT

Description: Disable the specified I2C interrupt.

Parameters:

__HANDLE__ : specifies the I2C Handle.
__INTERRUPT : specifies the interrupt source to disable. This parameter can be one of the following values:
I2C_IT_BUF: Buffer interrupt enable
I2C_IT_EVT: Event interrupt enable
I2C_IT_ERR: Error interrupt enable

Return value: None

__HAL_I2C_GET_IT_SOURCE

Description: Check whether the specified I2C interrupt source is enabled or not.

Parameters:

__HANDLE__ : specifies the I2C Handle.
__INTERRUPT : specifies the I2C interrupt source to check. This parameter can be one of the following values:
I2C_IT_BUF: Buffer interrupt enable
I2C_IT_EVT: Event interrupt enable
I2C_IT_ERR: Error interrupt enable

Return value: The: new state of __INTERRUPT. (TRUE or FALSE).

__HAL_I2C_GET_FLAG

Description: Check whether the specified I2C flag is set or not.

Parameters:

__HANDLE__ : specifies the I2C Handle.

__FLAG__ : specifies the flag to check. This parameter can be one of the following values:

I2C_FLAG_OVR: Overrun/Underrun flag

I2C_FLAG_AF: Acknowledge failure flag

I2C_FLAG_ARLO: Arbitration lost flag

I2C_FLAG_BERR: Bus error flag

I2C_FLAG_TXE: Data register empty flag

I2C_FLAG_RXNE: Data register not empty flag

I2C_FLAG_STOPF: Stop detection flag

I2C_FLAG_ADD10: 10-bit header sent flag

I2C_FLAG_BTF: Byte transfer finished flag

I2C_FLAG_ADDR: Address sent flag
Address matched flag

I2C_FLAG_SB: Start bit flag

I2C_FLAG_DUALF: Dual flag

I2C_FLAG_GENCALL: General call
header flag

I2C_FLAG_TRA: Transmitter/Receiver flag

I2C_FLAG_BUSY: Bus busy flag

I2C_FLAG_MSL: Master/Slave flag

Return value: The: new state of __FLAG__ (TRUE or FALSE).

__HAL_I2C_CLEAR_FLAG

Description: Clear the I2C pending flags which are cleared by writing 0 in a specific bit.

Parameters:

__HANDLE__ : specifies the I2C Handle.

__FLAG__ : specifies the flag to clear. This parameter can be any combination of the following values:

I2C_FLAG_OVR: Overrun/Underrun flag
(Slave mode)

I2C_FLAG_AF: Acknowledge failure flag

I2C_FLAG_ARLO: Arbitration lost flag
(Master mode)

I2C_FLAG_BERR: Bus error flag

Return value: None

__HAL_I2C_CLEAR_ADDRFLAG

Description: Clears the I2C ADDR pending flag.

Parameters:

__HANDLE__ : specifies the I2C Handle.

Return value: None

__HAL_I2C_CLEAR_STOPFLAG

Description: Clears the I2C STOPF pending flag.

Parameters:

__HANDLE__ : specifies the I2C Handle.

Return value: None

__HAL_I2C_ENABLE

Description:

Enable the I2C peripheral.

Parameters:

__HANDLE : specifies the I2C Handle. This parameter can be I2Cx where x: 1 or 2 to select the I2C peripheral.

Return value: None

__HAL_I2C_DISABLE

Description:

Disable the I2C peripheral.

Parameters:

__HANDLE : specifies the I2C Handle. This parameter can be I2Cx where x: 1 or 2 to select the I2C peripheral.

Return value: None

I2C Flag definition

I2C_FLAG_OVR, I2C_FLAG_AF, I2C_FLAG_ARLO, I2C_FLAG_BERR, I2C_FLAG_TXE, I2C_FLAG_RXNE, I2C_FLAG_STOPF, I2C_FLAG_ADD10, I2C_FLAG_BTF, I2C_FLAG_ADDR, I2C_FLAG_SB, I2C_FLAG_DUALF, I2C_FLAG_GENCALL, I2C_FLAG_TRA, I2C_FLAG_BUSY, I2C_FLAG_MSL

I2C general call addressing mode

I2C_GENERALCALL_DISABLE, I2C_GENERALCALL_ENABLE

I2C Interrupt configuration definition

I2C_IT_BUF, I2C_IT_EVT, I2C_IT_ERR

I2C Memory Address Size

I2C_MEMADD_SIZE_8BIT

I2C_MEMADD_SIZE_16BIT

I2C nostretch mode

I2C_NOSTRETCH_DISABLE

I2C_NOSTRETCH_ENABLE

I2C XferDirection definition Master Point of View

I2C_DIRECTION_RECEIVE

I2C_DIRECTION_TRANSMIT

I2C XferOptions definition

I2C_FIRST_FRAME

I2C_NEXT_FRAME

I2C_FIRST_AND_LAST_FRAME

I2C_LAST_FRAME

HAL SPI Generic Driver

SPI Firmware driver registers structures

SPI_InitTypeDef

Data Fields

uint32_t Mode
uint32_t Direction
uint32_t DataSize
uint32_t CLKPolarity
uint32_t CLKPhase
uint32_t NSS
uint32_t BaudRatePrescaler
uint32_t FirstBit
uint32_t TMode
uint32_t CRCCalculation
uint32_t CRCPolynomial

Field Documentation

uint32_t SPI_InitTypeDef::Mode

Specifies the SPI operating mode. This parameter can be a value of [SPI_mode](#)

uint32_t SPI_InitTypeDef::Direction

Specifies the SPI Directional mode state. This parameter can be a value of [SPI_Direction_mode](#)

uint32_t SPI_InitTypeDef::DataSize

Specifies the SPI data size. This parameter can be a value of [SPI_data_size](#)

uint32_t SPI_InitTypeDef::CLKPolarity

Specifies the serial clock steady state. This parameter can be a value of [SPI_Clock_Polarity](#)

uint32_t SPI_InitTypeDef::CLKPhase

Specifies the clock active edge for the bit capture. This parameter can be a value of [SPI_Clock_Phase](#)

uint32_t SPI_InitTypeDef::NSS

Specifies whether the NSS signal is managed by hardware (NSS pin) or by software using the SSI bit. This parameter can be a value of [SPI_Slave_Select_management](#)

uint32_t SPI_InitTypeDef::BaudRatePrescaler

Specifies the Baud Rate prescaler value which will be used to configure the transmit and receive SCK clock. This parameter can be a value of [SPI_BaudRate_Prescaler](#) **Note:**The communication clock is derived from the master clock. The slave clock does not need to be set

uint32_t SPI_InitTypeDef::FirstBit

Specifies whether data transfers start from MSB or LSB bit. This parameter can be a value of [SPI_MSB_LSB_transmission](#)

uint32_t SPI_InitTypeDef::TMode

Specifies if the TI mode is enabled or not. This parameter can be a value of [SPI_TI_mode](#)

uint32_t SPI_InitTypeDef::CRCCalculation

Specifies if the CRC calculation is enabled or not. This parameter can be a value of [SPI_CRC_Calculation](#)

uint32_t SPI_InitTypeDef::CRCPolynomial

Specifies the polynomial used for the CRC calculation. This parameter must be a number between Min_Data = 0 and Max_Data = 65535

SPI_HandleTypeDef

Data Fields

*SPI_TypeDef * Instance*
SPI_InitTypeDef Init
*uint8_t * pTxBuffPtr*
uint16_t TxXferSize
__IO uint16_t TxXferCount

```

uint8_t *pRxBuffPtr
uint16_t RxXferSize
__IO uint16_t RxXferCount
DMA_HandleTypeDef *hdmatx
DMA_HandleTypeDef *hdmarx
void(* RxISR
void(* TxISR
HAL_LockTypeDef Lock
__IO HAL_SPI_StateTypeDef State
__IO uint32_t ErrorCode

```

Field Documentation

```

SPI_TypeDef* SPI_HandleTypeDef::Instance
SPI_InitTypeDef SPI_HandleTypeDef::Init
uint8_t* __SPI_HandleTypeDef::pTxBuffPtr
uint16_t __SPI_HandleTypeDef::TxXferSize
__IO uint16_t SPI_HandleTypeDef::TxXferCount
uint8_t* __SPI_HandleTypeDef::pRxBuffPtr
uint16_t __SPI_HandleTypeDef::RxXferSize
__IO uint16_t SPI_HandleTypeDef::RxXferCount
DMA_HandleTypeDef* __SPI_HandleTypeDef::hdmatx
DMA_HandleTypeDef* __SPI_HandleTypeDef::hdmarx
void(* __SPI_HandleTypeDef::RxISR)(struct __SPI_HandleTypeDef *hspi)
void(* __SPI_HandleTypeDef::TxISR)(struct __SPI_HandleTypeDef *hspi)
HAL_LockTypeDef SPI_HandleTypeDef::Lock
__IO HAL_SPI_StateTypeDef SPI_HandleTypeDef::State
__IO uint32_t SPI_HandleTypeDef::ErrorCode

```

SPI Firmware driver API description

How to use this driver

The SPI HAL driver can be used as follows:

Declare a SPI_HandleTypeDef handle structure, for example: SPI_HandleTypeDef hspi;
 Initialize the SPI low level resources by implementing the HAL_SPI_MspInit ()API:

- Enable the SPIx interface clock
- SPI pins configuration
 - Enable the clock for the SPI GPIOs
 - Configure these SPI pins as alternate function push-pull
- NVIC configuration if you need to use interrupt process
 - Configure the SPIx interrupt priority
 - Enable the NVIC SPI IRQ handle

Program the Mode, Direction , Data size, Baudrate Prescaler, NSS management, Clock polarity and phase, FirstBit and CRC configuration in the hspi Init structure.

Initialize the SPI registers by calling the HAL_SPI_Init() API:

This API configures also the low level Hardware GPIO, CLOCK, CORTEX...etc) by calling the customized HAL_SPI_MspInit() API.

Initialization and de-initialization functions

This subsection provides a set of functions allowing to initialize and de-initialize the SPIx peripheral:

User must implement HAL_SPI_MspInit() function in which he configures all related peripherals resources (CLOCK, GPIO, DMA, IT and NVIC).

Call the function HAL_SPI_Init() to configure the selected device with the selected configuration:

- Mode
- Direction
- Data Size
- Clock Polarity and Phase
- NSS Management
- BaudRate Prescaler

FirstBit
TMode
CRC Calculation
CRC Polynomial if CRC enabled
Call the function HAL_SPI_DeInit() to restore the default configuration of the selected SPIx peripheral.

This section contains the following APIs:

[*HAL_SPI_Init\(\)*](#)
[*HAL_SPI_DeInit\(\)*](#)
[*HAL_SPI_MspInit\(\)*](#)
[*HAL_SPI_MspDeInit\(\)*](#)

IO operation functions

The SPI supports master and slave mode :

There are two modes of transfer:

Blocking mode: The communication is performed in polling mode. The HAL status of all data processing is returned by the same function after finishing transfer.

No-Blocking mode: The communication is performed using Interrupts or DMA, These APIs return the HAL status. The end of the data processing will be indicated through the dedicated SPI IRQ when using Interrupt mode or the DMA IRQ when using DMA mode. The HAL_SPI_TxCpltCallback(), HAL_SPI_RxCpltCallback() and HAL_SPI_TxRxCpltCallback() user callbacks will be executed respectively at the end of the transmit or Receive process The HAL_SPI_ErrorCallback() user callback will be executed when a communication error is detected

APIs provided for these 2 transfer modes (Blocking mode or Non blocking mode using either Interrupt or DMA) exist for 1Line (simplex) and 2Lines (full duplex) modes.

This section contains the following APIs:

[*HAL_SPI_Transmit\(\)*](#)
[*HAL_SPI_Receive\(\)*](#)
[*HAL_SPI_TransmitReceive\(\)*](#)
[*HAL_SPI_Transmit_IT\(\)*](#)
[*HAL_SPI_Receive_IT\(\)*](#)
[*HAL_SPI_TransmitReceive_IT\(\)*](#)
[*HAL_SPI_Transmit_DMA\(\)*](#)
[*HAL_SPI_Receive_DMA\(\)*](#)
[*HAL_SPI_TransmitReceive_DMA\(\)*](#)
[*HAL_SPI_IRQHandler\(\)*](#)
[*HAL_SPI_TxCpltCallback\(\)*](#)
[*HAL_SPI_RxCpltCallback\(\)*](#)
[*HAL_SPI_TxRxCpltCallback\(\)*](#)
[*HAL_SPI_TxHalfCpltCallback\(\)*](#)
[*HAL_SPI_RxHalfCpltCallback\(\)*](#)
[*HAL_SPI_TxRxHalfCpltCallback\(\)*](#)
[*HAL_SPI_ErrorCallback\(\)*](#)

Peripheral State and Errors functions

This subsection provides a set of functions allowing to control the SPI.

HAL_SPI_GetState() API can be helpful to check in run-time the state of the SPI peripheral

HAL_SPI_GetError() check in run-time Errors occurring during communication This section contains the following APIs:

[*HAL_SPI_GetState\(\)*](#)
[*HAL_SPI_GetError\(\)*](#)

Detailed description of functions

HAL_SPI_Init

Function name	HAL_StatusTypeDef HAL_SPI_Init (SPI_HandleTypeDef * hspi)
Function description	Initializes the SPI according to the specified parameters in the SPI_InitTypeDef and create the associated handle.
Parameters	hspi: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
Return values	HAL: status

HAL_SPI_DeInit

Function name	HAL_StatusTypeDef HAL_SPI_DeInit (SPI_HandleTypeDef * hspi)
Function description	DeInitializes the SPI peripheral.
Parameters	hspi: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
Return values	HAL: status

HAL_SPI_MspInit

Function name	void HAL_SPI_MspInit (SPI_HandleTypeDef * hspi)
Function description	SPI MSP Init.
Parameters	hspi: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
Return values	None:

HAL_SPI_MspDeInit

Function name	void HAL_SPI_MspDeInit (SPI_HandleTypeDef * hspi)
Function description	SPI MSP DeInit.
Parameters	hspi: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
Return values	None:

HAL_SPI_Transmit

Function name	HAL_StatusTypeDef HAL_SPI_Transmit (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size, uint32_t Timeout)
Function description	Transmit an amount of data in blocking mode.
Parameters	hspi: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module. pData: pointer to data buffer Size: amount of data to be sent Timeout: Timeout duration
Return values	HAL: status

HAL_SPI_Receive

Function name	HAL_StatusTypeDef HAL_SPI_Receive (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size, uint32_t Timeout)
Function description	Receive an amount of data in blocking mode.
Parameters	hspi: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module. pData: pointer to data buffer Size: amount of data to be sent Timeout: Timeout duration

Return values **HAL:** status

HAL_SPI_TransmitReceive

Function name **HAL_StatusTypeDef HAL_SPI_TransmitReceive**
(SPI_HandleTypeDef * hspi, uint8_t * pTxData, uint8_t *
pRxData, uint16_t Size, uint32_t Timeout)

Function description Transmit and Receive an amount of data in blocking mode.

Parameters **hspi:** pointer to a SPI_HandleTypeDef structure that contains the
configuration information for SPI module.
pTxData: pointer to transmission data buffer
pRxData: pointer to reception data buffer to be
Size: amount of data to be sent
Timeout: Timeout duration

Return values **HAL:** status

HAL_SPI_Transmit_IT

Function name **HAL_StatusTypeDef HAL_SPI_Transmit_IT**
(SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size)

Function description Transmit an amount of data in no-blocking mode with Interrupt.

Parameters **hspi:** pointer to a SPI_HandleTypeDef structure that contains the
configuration information for SPI module.
pData: pointer to data buffer
Size: amount of data to be sent

Return values **HAL:** status

HAL_SPI_Receive_IT

Function name **HAL_StatusTypeDef HAL_SPI_Receive_IT**
(SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size)

Function description Receive an amount of data in no-blocking mode with Interrupt.

Parameters **hspi:** pointer to a SPI_HandleTypeDef structure that contains the
configuration information for SPI module.
pData: pointer to data buffer
Size: amount of data to be sent

Return values **HAL:** status

HAL_SPI_TransmitReceive_IT

Function name **HAL_StatusTypeDef HAL_SPI_TransmitReceive_IT**
(SPI_HandleTypeDef * hspi, uint8_t * pTxData, uint8_t *
pRxData, uint16_t Size)

Function description Transmit and Receive an amount of data in no-blocking mode with
Interrupt.

Parameters **hspi:** pointer to a SPI_HandleTypeDef structure that contains the
configuration information for SPI module.
pTxData: pointer to transmission data buffer
pRxData: pointer to reception data buffer to be
Size: amount of data to be sent

Return values **HAL:** status

HAL_SPI_IRQHandler

Function name **void HAL_SPI_IRQHandler (SPI_HandleTypeDef * hspi)**

Function description	This function handles SPI interrupt request.
Parameters	hspi : pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
Return values	HAL : status

HAL_SPI_TxCpltCallback

Function name	void HAL_SPI_TxCpltCallback (SPI_HandleTypeDef * hspi)
Function description	Tx Transfer completed callbacks.
Parameters	hspi : pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
Return values	None :

HAL_SPI_RxCpltCallback

Function name	void HAL_SPI_RxCpltCallback (SPI_HandleTypeDef * hspi)
Function description	Rx Transfer completed callbacks.
Parameters	hspi : pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
Return values	None :

HAL_SPI_TxRxCpltCallback

Function name	void HAL_SPI_TxRxCpltCallback (SPI_HandleTypeDef * hspi)
Function description	Tx and Rx Transfer completed callbacks.
Parameters	hspi : pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
Return values	None :

HAL_SPI_ErrorCallback

Function name	void HAL_SPI_ErrorCallback (SPI_HandleTypeDef * hspi)
Function description	SPI error callbacks.
Parameters	hspi : pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
Return values	None :

HAL_SPI_GetState

Function name	HAL_SPI_StateTypeDef HAL_SPI_GetState (SPI_HandleTypeDef * hspi)
Function description	Return the SPI state.
Parameters	hspi : pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
Return values	HAL : state

HAL_SPI_GetError

Function name	uint32_t HAL_SPI_GetError (SPI_HandleTypeDef * hspi)
Function description	Return the SPI error code.
Parameters	hspi : pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
Return values	SPI : Error Code

SPI Firmware driver defines

SPI

SPI BaudRate Prescaler

SPI_BAUDRATEPRESCALER_2, SPI_BAUDRATEPRESCALER_4,
SPI_BAUDRATEPRESCALER_8, SPI_BAUDRATEPRESCALER_16,
SPI_BAUDRATEPRESCALER_32, SPI_BAUDRATEPRESCALER_64,
SPI_BAUDRATEPRESCALER_128, SPI_BAUDRATEPRESCALER_256,
IS_SPI_BAUDRATE_PRESCALER

SPI Clock Phase

SPI_PHASE_1EDGE
SPI_PHASE_2EDGE
IS_SPI_CPHA

SPI Clock Polarity

SPI_POLARITY_LOW
SPI_POLARITY_HIGH
IS_SPI_CPOL

SPI CRC Calculation

SPI_CRCCALCULATION_DISABLE
SPI_CRCCALCULATION_ENABLE
IS_SPI_CRC_CALCULATION
IS_SPI_CRC_POLYNOMIAL

SPI data size

SPI_DATASIZE_8BIT
SPI_DATASIZE_16BIT
IS_SPI_DATASIZE

SPI Direction mode

SPI_DIRECTION_2LINES
SPI_DIRECTION_2LINES_RXONLY
SPI_DIRECTION_1LINE
IS_SPI_DIRECTION_MODE
IS_SPI_DIRECTION_2LINES_OR_1LINE
IS_SPI_DIRECTION_2LINES

SPI Error Codes

HAL_SPI_ERROR_NONE	No error
HAL_SPI_ERROR_MODF	MODF error
HAL_SPI_ERROR_CRC	CRC error
HAL_SPI_ERROR_OVR	OVR error
HAL_SPI_ERROR_FRE	FRE error
HAL_SPI_ERROR_FLAG	Flag: RXNE, TXE, BSY

SPI Exported Macros

__HAL_SPI_RESET_HANDLE_STATE	Description: Reset SPI handle state.
	Parameters:
	__HANDLE : specifies the SPI handle.

This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

Return value: None

`__HAL_SPI_ENABLE_IT`

Description: Enable or disable the specified SPI interrupts.

Parameters:

`__HANDLE_` : specifies the SPI handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

`__INTERRUPT` : specifies the interrupt source to enable or disable. This parameter can be one of the following values:

SPI_IT_TXE: Tx buffer empty interrupt enable

SPI_IT_RXNE: RX buffer not empty interrupt enable

SPI_IT_ERR: Error interrupt enable

Return value: None

`__HAL_SPI_DISABLE_IT`

`__HAL_SPI_GET_IT_SOURCE`

Description: Check if the specified SPI interrupt source is enabled or disabled.

Parameters:

`__HANDLE_` : specifies the SPI handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

`__INTERRUPT` : specifies the SPI interrupt source to check. This parameter can be one of the following values:

SPI_IT_TXE: Tx buffer empty interrupt enable

SPI_IT_RXNE: RX buffer not empty interrupt enable

SPI_IT_ERR: Error interrupt enable

Return value: The: new state of `__IT____` (TRUE or FALSE).

`__HAL_SPI_GET_FLAG`

Description: Check whether the specified SPI flag is set or not.

Parameters:

`__HANDLE_` : specifies the SPI handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

`__FLAG` : specifies the flag to check. This parameter can be one of the following values:

SPI_FLAG_RXNE: Receive buffer not empty flag

SPI_FLAG_TXE: Transmit buffer empty flag

SPI_FLAG_CRCERR: CRC error flag

SPI_FLAG_MODF: Mode fault flag

SPI_FLAG_OVR: Overrun flag

SPI_FLAG_BSY: Busy flag

SPI_FLAG_FRE: Frame format error flag

Return value: The: new state of `__FLAG_` (TRUE or FALSE)

<code>__HAL_SPI_CLEAR_CRCERRFLAG</code>	Description: Clear the SPI CRCERR pending flag. Parameters: <code>__HANDLE_</code> : specifies the SPI handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral. Return value: None
<code>__HAL_SPI_CLEAR_MODFFLAG</code>	Description: Clear the SPI MODF pending flag. Parameters: <code>__HANDLE_</code> : specifies the SPI handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral. Return value: None
<code>__HAL_SPI_CLEAR_OVRFLAG</code>	Description: Clear the SPI OVR pending flag. Parameters: <code>__HANDLE_</code> : specifies the SPI handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral. Return value: None
<code>__HAL_SPI_CLEAR_FREFLAG</code>	Description: Clear the SPI FRE pending flag. Parameters: <code>__HANDLE_</code> : specifies the SPI handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral. Return value: None
<code>__HAL_SPI_ENABLE</code>	Description: Enables the SPI. Parameters: <code>__HANDLE_</code> : specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral. Return value: None
<code>__HAL_SPI_DISABLE</code>	Description: Disables the SPI. Parameters: <code>__HANDLE_</code> : specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral. Return value: None

SPI Flag definition

`SPI_FLAG_RXNE`, `SPI_FLAG_TXE`, `SPI_FLAG_CRCERR`, `SPI_FLAG_MODF`, `SPI_FLAG_OVR`, `SPI_FLAG_BSY`, `SPI_FLAG_FRE`

SPI Interrupt configuration definition

`SPI_IT_TXE`, `SPI_IT_RXNE`, `SPI_IT_ERR`

SPI mode

SPI_MODE_SLAVE

SPI_MODE_MASTER

IS_SPI_MODE

SPI MSB LSB transmission

SPI_FIRSTBIT_MSB

SPI_FIRSTBIT_LSB

IS_SPI_FIRST_BIT

SPI Slave Select management

SPI_NSS_SOFT

SPI_NSS_HARD_INPUT

SPI_NSS_HARD_OUTPUT

IS_SPI_NSS

SPI TI mode

SPI_TIMODE_DISABLE

SPI_TIMODE_ENABLE

IS_SPI_TIMODE

